

DATA MINING

MULTIMEDIA, SOFT COMPUTING,
AND BIOINFORMATICS

Sushmita Mitra • Tinku Acharya

Data Mining

This page intentionally left blank

Data Mining

Multimedia, Soft Computing, and Bioinformatics

SUSHMITA MITRA

Associate Professor
Machine Intelligence Unit
Indian Statistical Institute
Kolkata, India

TINKU ACHARYA

Senior Executive Vice President
Chief Science Officer
Avisere Inc.
Tucson, Arizona
and
Adjunct Professor
Department of Electrical Engineering
Arizona State University
Tempe, Arizona



A JOHN WILEY & SONS, INC., PUBLICATION

Copyright © 2003 by John Wiley & Sons, Inc. All rights reserved.

Published by John Wiley & Sons, Inc., Hoboken, New Jersey.

Published simultaneously in Canada.

No part of this publication may be reproduced, stored in a retrieval system or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning or otherwise, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 750-4744, or on the web at www.copyright.com. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, e-mail: permreq@wiley.com.

Limit of Liability/Disclaimer of Warranty: While the publisher and author have used their best efforts in preparing this book, they make no representation or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaim any implied warranties of merchantability or fitness for a particular purpose. No warranty may be created or extended by sales representatives or written sales materials. The advice and strategies contained herein may not be suitable for your situation. You should consult with a professional where appropriate. Neither the publisher nor author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

For general information on our other products and services please contact our Customer Care Department within the U.S. at 877-762-2974, outside the U.S. at 317-572-3993 or fax 317-572-4002.

Wiley also publishes its books in a variety of electronic formats. Some content that appears in print, however, may not be available in electronic format.

Library of Congress Cataloging-in-Publication Data:

Mitra, Sushmita

Data mining : multimedia, soft computing, and bioinformatics /
Sushmita Mitra and Tinku Acharya.

p. cm.

Includes bibliographical references and index.

ISBN 0-471-46054-0 (cloth)

1. Data mining. I. Acharya, Tinku. II. Title.

QA76.9.D343M8 2003

006.3—dc21

2003011394

Printed in the United States of America.

10 9 8 7 6 5 4 3 2 1

*To Ma, who made me what I am,
and to Somosmita, who let me feel like a supermom.
—Sushmita*

*To Lisa, Arita, and Arani
—Tinku Acharya*

This page intentionally left blank

Contents

Preface	xv
1 Introduction to Data Mining	1
1.1 Introduction	1
1.2 Knowledge Discovery and Data Mining	5
1.3 Data Compression	10
1.4 Information Retrieval	12
1.5 Text Mining	14
1.6 Web Mining	15
1.7 Image Mining	16
1.8 Classification	18
1.9 Clustering	19
1.10 Rule Mining	20
1.11 String Matching	21
1.12 Bioinformatics	23
1.13 Data Warehousing	24
1.14 Applications and Challenges	25
1.15 Conclusions and Discussion	28
References	30
	<i>vii</i>

2	Soft Computing	35
2.1	Introduction	35
2.2	What is Soft Computing?	37
2.2.1	Relevance	37
2.2.2	Fuzzy sets	39
2.2.3	Neural networks	44
2.2.4	Neuro-fuzzy computing	53
2.2.5	Genetic algorithms	55
2.2.6	Rough sets	59
2.2.7	Wavelets	61
2.3	Role of Fuzzy Sets in Data Mining	62
2.3.1	Clustering	63
2.3.2	Granular computing	63
2.3.3	Association rules	64
2.3.4	Functional dependencies	65
2.3.5	Data summarization	65
2.3.6	Image mining	66
2.4	Role of Neural Networks in Data Mining	67
2.4.1	Rule extraction	67
2.4.2	Rule evaluation	67
2.4.3	Clustering and self-organization	69
2.4.4	Regression	69
2.4.5	Information retrieval	69
2.5	Role of Genetic Algorithms in Data Mining	70
2.5.1	Regression	71
2.5.2	Association rules	71
2.6	Role of Rough Sets in Data Mining	72
2.7	Role of Wavelets in Data Mining	73
2.8	Role of Hybridizations in Data Mining	74
2.9	Conclusions and Discussion	77
	References	78
3	Multimedia Data Compression	89
3.1	Introduction	89
3.2	Information Theory Concepts	91
3.2.1	Discrete memoryless model and entropy	91
3.2.2	Noiseless Source Coding Theorem	92
3.3	Classification of Compression Algorithms	94

3.4	A Data Compression Model	95
3.5	Measures of Compression Performance	96
3.5.1	Compression ratio and bits per sample	97
3.5.2	Quality metric	97
3.5.3	Coding complexity	99
3.6	Source Coding Algorithms	99
3.6.1	Run-length coding	99
3.6.2	Huffman coding	100
3.7	Principal Component Analysis for Data Compression	103
3.8	Principles of Still Image Compression	105
3.8.1	Predictive coding	105
3.8.2	Transform coding	107
3.8.3	Wavelet coding	109
3.9	Image Compression Standard: JPEG	112
3.10	The JPEG Lossless Coding Algorithm	113
3.11	Baseline JPEG Compression	116
3.11.1	Color space conversion	116
3.11.2	Source image data arrangement	118
3.11.3	The baseline compression algorithm	119
3.11.4	Decompression process in baseline JPEG	126
3.11.5	JPEG2000: Next generation still picture coding standard	129
3.12	Text Compression	131
3.12.1	The LZ77 algorithm	132
3.12.2	The LZ78 algorithm	133
3.12.3	The LZW algorithm	136
3.12.4	Other applications of Lempel–Ziv coding	139
3.13	Conclusions and Discussion	140
	References	140
4	String Matching	143
4.1	Introduction	143
4.1.1	Some definitions and preliminaries	144
4.1.2	String matching problem	146
4.1.3	Brute force string matching	148
4.2	Linear-Order String Matching Algorithms	150
4.2.1	String matching with finite automata	150
4.2.2	Knuth–Morris–Pratt algorithm	152
4.2.3	Boyer–Moore algorithm	158

4.2.4	Boyer–Moore–Horspool algorithm	161
4.2.5	Karp–Rabin algorithm	165
4.3	String Matching in Bioinformatics	169
4.4	Approximate String Matching	171
4.4.1	Basic definitions	172
4.4.2	Wagner–Fischer algorithm for computation of string distance	173
4.4.3	Text search with k -differences	176
4.5	Compressed Pattern Matching	177
4.6	Conclusions and Discussion	179
	References	179
5	Classification in Data Mining	181
5.1	Introduction	181
5.2	Decision Tree Classifiers	184
5.2.1	ID3	187
5.2.2	IBM IntelligentMiner	189
5.2.3	Serial PaRallelizable INduction of decision Trees (SPRINT)	189
5.2.4	RainForest	192
5.2.5	Overfitting	192
5.2.6	PrUning and BuiLding Integrated in Classification (PUBLIC)	194
5.2.7	Extracting classification rules from trees	194
5.2.8	Fusion with neural networks	195
5.3	Bayesian Classifiers	196
5.3.1	Bayesian rule for minimum risk	196
5.3.2	Naive Bayesian classifier	196
5.3.3	Bayesian belief network	198
5.4	Instance-Based Learners	199
5.4.1	Minimum distance classifiers	199
5.4.2	k -nearest neighbor (k -NN) classifier	201
5.4.3	Locally weighted regression	201
5.4.4	Radial basis functions (RBFs)	202
5.4.5	Case-based reasoning (CBR)	203
5.4.6	Granular computing and CBR	203
5.5	Support Vector Machines	204
5.6	Fuzzy Decision Trees	205
5.6.1	Classification	207

5.6.2	Rule generation and evaluation	212
5.6.3	Mapping of rules to fuzzy neural network	214
5.6.4	Results	216
5.7	Conclusions and Discussion	220
	References	221
6	Clustering in Data Mining	227
6.1	Introduction	227
6.2	Distance Measures and Symbolic Objects	229
6.2.1	Numeric objects	229
6.2.2	Binary objects	229
6.2.3	Categorical objects	231
6.2.4	Symbolic objects	231
6.3	Clustering Categories	232
6.3.1	Partitional clustering	232
6.3.2	Hierarchical clustering	235
6.3.3	Leader clustering	237
6.4	Scalable Clustering Algorithms	237
6.4.1	Clustering large applications	238
6.4.2	Density-based clustering	239
6.4.3	Hierarchical clustering	241
6.4.4	Grid-based methods	243
6.4.5	Other variants	244
6.5	Soft Computing-Based Approaches	244
6.5.1	Fuzzy sets	244
6.5.2	Neural networks	246
6.5.3	Wavelets	248
6.5.4	Rough sets	249
6.5.5	Evolutionary algorithms	250
6.6	Clustering with Categorical Attributes	251
6.6.1	Sieving Through Iterated Relational Reinforcements (STIRR)	252
6.6.2	Robust Hierarchical Clustering with Links (ROCK)	252
6.6.3	c-modes algorithm	253
6.7	Hierarchical Symbolic Clustering	255
6.7.1	Conceptual clustering	255
6.7.2	Agglomerative symbolic clustering	256
6.7.3	Cluster validity indices	257
6.7.4	Results	259

6.8	Conclusions and Discussion	261
	References	262
7	Association Rules	267
7.1	Introduction	267
7.2	Candidate Generation and Test Methods	269
7.2.1	A priori algorithm	269
7.2.2	Partition algorithm	272
7.2.3	Some extensions	272
7.3	Depth-First Search Methods	273
7.4	Interesting Rules	275
7.5	Multilevel Rules	276
7.6	Online Generation of Rules	277
7.7	Generalized Rules	278
7.8	Scalable Mining of Rules	280
7.9	Other Variants	281
7.9.1	Quantitative association rules	281
7.9.2	Temporal association rules	281
7.9.3	Correlation rules	282
7.9.4	Localized associations	282
7.9.5	Optimized association rules	283
7.10	Fuzzy Association Rules	283
7.11	Conclusions and Discussion	288
	References	289
8	Rule Mining with Soft Computing	293
8.1	Introduction	293
8.2	Connectionist Rule Generation	294
8.2.1	Neural models	295
8.2.2	Neuro-fuzzy models	296
8.2.3	Using knowledge-based networks	297
8.3	Modular Hybridization	302
8.3.1	Rough fuzzy MLP	302
8.3.2	Modular knowledge-based network	305
8.3.3	Evolutionary design	308
8.3.4	Rule extraction	310
8.3.5	Results	311
8.4	Conclusions and Discussion	315

References	315
9 Multimedia Data Mining	319
9.1 Introduction	319
9.2 Text Mining	320
9.2.1 Keyword-based search and mining	321
9.2.2 Text analysis and retrieval	322
9.2.3 Mathematical modeling of documents	323
9.2.4 Similarity-based matching for documents and queries	325
9.2.5 Latent semantic analysis	326
9.2.6 Soft computing approaches	328
9.3 Image Mining	329
9.3.1 Content-Based Image Retrieval	330
9.3.2 Color features	332
9.3.3 Texture features	337
9.3.4 Shape features	338
9.3.5 Topology	340
9.3.6 Multidimensional indexing	342
9.3.7 Results of a simple CBIR system	343
9.4 Video Mining	345
9.4.1 MPEG-7: Multimedia content description interface	347
9.4.2 Content-based video retrieval system	348
9.5 Web Mining	350
9.5.1 Search engines	351
9.5.2 Soft computing approaches	353
9.6 Conclusions and Discussion	357
References	357
10 Bioinformatics: An Application	365
10.1 Introduction	365
10.2 Preliminaries from Biology	367
10.2.1 Deoxyribonucleic acid	367
10.2.2 Amino acids	368
10.2.3 Proteins	369
10.2.4 Microarray and gene expression	371
10.3 Information Science Aspects	371
10.3.1 Protein folding	372
10.3.2 Protein structure modeling	373

10.3.3	Genomic sequence analysis	374
10.3.4	Homology search	374
10.4	Clustering of Microarray Data	378
10.4.1	First-generation algorithms	379
10.4.2	Second-generation algorithms	380
10.5	Association Rules	381
10.6	Role of Soft Computing	381
10.6.1	Predicting protein secondary structure	382
10.6.2	Predicting protein tertiary structure	382
10.6.3	Determining binding sites	385
10.6.4	Classifying gene expression data	385
10.7	Conclusions and Discussion	386
	References	387
	Index	392
	About the Authors	399

Preface

The success of the digital revolution and the growth of the Internet have ensured that huge volumes of high-dimensional multimedia data are available all around us. This information is often mixed, involving different datatypes such as text, image, audio, speech, hypertext, graphics, and video components interspersed with each other. The World Wide Web has played an important role in making the data, even from geographically distant locations, easily accessible to users all over the world. However, often most of this data are not of much interest to most of the users. The problem is to *mine* useful information or patterns from the huge datasets. Data mining refers to this process of extracting knowledge that is of interest to the user.

Data mining is an evolving and growing area of research and development, both in academia as well as in industry. It involves interdisciplinary research and development encompassing diverse domains. In our view, this area is far from being saturated, with newer techniques and directions being proposed in the literature everyday. In this age of multimedia data exploration, data mining should no longer be restricted to the mining of knowledge from large volumes of high-dimensional datasets in traditional databases only. Researchers need to pay attention to the mining of different datatypes, including numeric and alphanumeric formats, text, images, video, voice, speech, graphics, and also their mixed representations. Efficient management of such high-dimensional very large databases also influence the performance of data mining systems. Data Compression technologies can play a significant role.

It is also important that special multimedia data compression techniques are explored especially suitable for data mining applications.

With the completion of the Human Genome Project, we have access to large databases of biological information. Proper analysis of such huge data, involving decoding of genes in the DNA and the three-dimensional protein structures, holds immense promise in Bioinformatics. The applicability of data mining in this domain cannot be denied, given the lifesaving prospects of effective drug design. This is also of practical interest to the pharmaceutical industry.

Different forms of ambiguity or uncertainty inherent in real-life data need to be handled appropriately using soft computing. The goal is to arrive at a low-cost, reasonably good solution, instead of a high-cost, best solution. Fuzzy sets provide the uncertainty handling capability, inherent in human reasoning, while artificial neural networks help incorporate learning to minimize error. Genetic algorithms introduce effective parallel searching in the high-dimensional problem space.

Since all these aspects are not covered in that elaborate form in current books available in the market, we wanted to emphasize them in this book. Along with the traditional concepts and functions of data mining, like classification, clustering, and rule mining, we wish to highlight the current and burning issues related to mining in multimedia applications and Bioinformatics. Storage of such huge datasets being more feasible in the compressed domain, we also devote a reasonable portion of the text to data mining in the compressed domain. Topics like text mining, image mining, and Web mining are covered specifically.

Current trends show that the advances in data mining need not be constrained to stochastic, combinatorial, and/or classical so-called hard optimization-based techniques. We dwell, in greater detail, on the state of the art of soft computing approaches, advanced signal processing techniques such as Wavelet Transformation, data compression principles for both lossless and lossy techniques, access of data using matching pursuits in both raw and compressed data domains, fundamentals and principles of classical string matching algorithms, and how all these areas possibly influence data mining and its future growth. We cover aspects of advanced image compression, string matching, content based image retrieval, etc., which can influence future developments in data mining, particularly for multimedia data mining.

There are 10 chapters in the book. The first chapter provides an introduction to the basics of data mining and outlines its major functions and applications. This is followed in the second chapter by a discussion on soft computing and its different tools, including fuzzy sets, artificial neural networks, genetic algorithms, wavelet transforms, rough sets, and their hybridizations, along with their roles in data mining.

We then present some advanced topics and new aspects of data mining related to the processing and retrieval of multimedia data. These have direct applications to information retrieval, Web mining, image mining, and

text mining. The huge volumes of data required to be retrieved, processed, and stored make compression techniques a promising area to explore, in the context of both images and texts. Chapter 3 introduces the readers to the fundamentals of multimedia data compression and some popular algorithms for data compression. We discuss the principles of string matching and some classical algorithms in Chapter 4. Results of string matching hold ample promise both in multimedia applications and in Bioinformatics.

Chapters 5 to 8 concentrate on classification, clustering, and rule mining. In each of these topics, in addition to the classical discussions that are usually available in the books currently in the market, we strive to incorporate new algorithms and results based on soft computing and advanced signal processing techniques with recent developments.

We deal with multimedia data mining in Chapter 9. In this chapter we have discussed text mining, image mining, and Web mining issues. Next we introduce the readers to issues from Bioinformatics, in Chapter 10. In each case we discuss the related algorithms, showing how these can be a growing area of study in the light of data mining in the near future.

Finally, we pose some research problems, issues, and new direction of thoughts for researchers and developers. We have kept the presentation concise and included an exhaustive bibliography at the end of each chapter. Because reported research articles in relevant domains are scarce and scattered, we have tried to make them collectively accessible from the unified framework of this book. Some portion of the material in this book also covers our published work, which has been presented and discussed in different seminars, conferences, and workshops.

The book may be used in a graduate-level course as a part of the subject of data mining, machine learning, information retrieval, and artificial intelligence, or it may be used as a reference book for professionals and researchers. It is assumed that the readers have adequate background in college-level mathematics and introductory knowledge of statistics and probability theory.

For the major part of this project we worked from the two ends of this world, often communicating via the Internet. We have collected a great deal of rich information from the Internet. Thereby, we were the true beneficiaries of today's information technology. Progress in data mining will further pave the way for usage of information technology in every walk of life in near future. We are glad that we could complete this project in a short time within the schedule.

We take this opportunity to thank Dr. Val Moliere of John Wiley & Sons, Inc., for her initiative and encouragement throughout this project. She was very helpful in every stage of compilation of this book. We are grateful to Mr. B. Uma Shankar, Mr. Sudip Chakraborty, and Ms. Maya Dey for their valuable assistance while preparing the camera-ready manuscript. We sincerely thank Dr. Ping-Sing Tsai, who assisted by reviewing a number of chapters of the book and who provided valuable suggestions to further enrich the content. We extend our gratitude to Dr. Amit K. Das of Bengal Engineering College

in India, who supplied some material on content-based image retrieval in a very short notice. Prof. Malay K. Kundu, Prof. Chaitali Chakraborti, Dr. Andrew J. Griffis, Dr. Dragos Arotaritei, Dr. Rajat K. De, Dr. Pabitra Mitra, Mr. Roger Undhagen, and Mr. Jose M. Rodriguez deserve special thanks for their continuous encouragement and support towards the compilation of this treatise. We would also like to thank the anonymous reviewers of our book proposal for their very constructive review and suggestions.

Finally, sincere gratitude goes to each member of our families for bearing with us, especially by putting up with our erratic schedules during the final phase of this project. We are truly indebted to them for their love, encouragement, dedication, and support.

April 2003

Sushmita Mitra
Tinku Acharya

1

Introduction to Data Mining

1.1 INTRODUCTION

The digital revolution has made digitized information easy to capture, process, store, distribute, and transmit [1]–[3]. With significant progress in computing and related technologies and their ever-expanding usage in different walks of life, huge amount of data of diverse characteristics continue to be collected and stored in databases. The rate at which such data are stored is growing phenomenally. We can draw an analogy between the popular Moore's law and the way data are increasing with the growth of information in this world of data processing applications. The advancement of data processing and the emergence of newer applications were possible, partially because of the growth of the semiconductor and subsequently the computer industry. According to Moore's law, the number of transistors in a single microchip is doubled every 18 months, and the growth of the semiconductor industry has so far followed the prediction. We can correlate this with a similar observation from the data and information domain. If the amount of information in the world doubles every 20 months, the size and number of databases probably increases at a similar pace. Discovery of knowledge from this huge volume of data is a challenge indeed. Data mining is an attempt to make sense of the information explosion embedded in this huge volume of data [4].

Today, data are no longer restricted to tuples of numeric or character representations only. The advanced database management technology of today is enabled to integrate different types of data, such as image, video, text, and other numeric as well as non-numeric data, in a provably single database

in order to facilitate multimedia processing. As a result, traditional *ad hoc* mixtures of statistical techniques and data management tools are no longer adequate for analyzing this vast collection of mixed data.

The current Internet technology and its growing demand necessitates the development of more advanced data mining technologies to interpret the information and knowledge from the data distributed all over the world. In the 21st century this demand will continue to grow, and the access of large volumes of multimedia data will become a major transforming theme in the global society. As an example, a report on the United States Administrations initiative in the "Information Technology for 21st Century" [5] indicated improved Internet and multimedia applications in World Wide Web encompassing information visualization, interpretation, processing, analysis, etc. Hence development of advanced data mining technology will continue to be an important area of study, and it is accordingly expected that lots of energy will be spent in this area of development in the coming years.

There exist several domains where large volumes of data are stored in centralized or distributed databases. Some of the examples include the following.

- **Digital library:** This is an organized collection of digital information stored in large databases in the form of text (encoded or raw) and possibly as a large collection of document imagery [6].
- **Image archive:** This consists of large database of images, in either compressed or raw form. Often the image data are interspersed with text and numeric data for proper indexing, retrieval, and storage management.
- **Bioinformatics:** The machinery of each human body is built and run with 50,000 to 100,000 different kinds of genes or protein molecules, and we have five and half billion population in this diverse world. Bioinformatics involves analyzing and interpreting this vast amount of data stored in these large genomic databases [7].
- **Medical imagery:** Large volumes of medical data are generated everyday in the form of digital images such as digital radiographs, EKG, MRI, CAT, SCAN, etc. They are stored in large centralized or distributed databases in medical management systems. Automatic mining of these data is important to the medical community.
- **Health care:** In addition of the above medical image data, other non-image datatypes are also generated everyday. This may include health insurance information, patient's personal care physician's information, specialist information, patient's medical history, etc. In addition to these, several diagnostic information are stored by hospital management systems [8] for ready reference or research.

- **Finance and investment:** Finance and investment is a big data domain of interest for data mining. It includes, but is not limited to, stock indices, stock prices, historical performance of each stock, information about the bonds, notes, treasury and other security investments, banking information, interest rates, loan information, credit card data, debit card data, ATM card information, credit history of an individual, and fraud detection [9].
- **Manufacturing and production:** A huge volume of manufacturing and production data is generated in different forms in factories. Efficient storage and access of these data and their analysis for process optimization and trouble shooting is very important in the manufacturing industry [10].
- **Business and marketing:** Data need to be analyzed for sales forecast, business planning, marketing trend, etc.
- **Telecommunication network:** There are different types of data generated and stored in this application domain. They may be used to analyze calling patterns, call tracing, network management, congestion control, error control, fault management, etc.
- **Scientific domain:** This consists of astronomical observations [11], genomic data, biological data, etc. There has been an exponential growth in the collection and storage of biological databases over the last couple of years, the human genome database being one such example.
- **The World Wide Web (WWW) [12]:** A huge volume of multimedia data of different types is distributed everywhere in the Internet. The World Wide Web can be considered as the largest distributed database that ever existed. It consists of data that are heterogeneous in nature, and it happens to be the most unorganized database management system known today.
- **Biometrics:** Because of the need of extraordinary security of human lives today, biometric applications will continue to grow for positive identification of persons. A huge volume of biometric data such as fingerprint, face imagery, etc., need to be stored and used, for access and search toward this end.

Raw data are rarely of direct benefit. Its true value is predicated on (a) the ability to extract information useful for decision support or exploration and (b) understanding the phenomenon governing the data source. In most domains, data analysis was traditionally a manual process. One or more analyst(s) would become intimately familiar with the data and, with the help of statistical techniques, provide summaries and generate reports. In effect, the analyst acted as a sophisticated query processor. However, such an approach

rapidly broke down as the size of data grew and the number of dimensions increased. Databases containing number of data on the order of 10^9 or above and dimension on the order of 10^3 are becoming increasingly common. When the scale of data manipulation, exploration and inferencing goes beyond human capacities, people need the aid of computing technologies for automating the process.

All these have prompted the need for intelligent data analysis methodologies, which could discover useful knowledge from data. The term KDD refers to the overall process of *knowledge discovery in databases*. While some people treat *data mining* as a synonym for KDD, some others view it as a particular step in this process involving the application of specific algorithms for extracting patterns (models) from data. The additional steps in the KDD process, such as data preparation, data selection, data cleaning, incorporation of appropriate prior knowledge, and proper interpretation of the results of mining, ensures that useful knowledge is derived from the data.

Data mining tasks can be descriptive, (i.e., discovering interesting patterns or relationships describing the data), and predictive (i.e., predicting or classifying the behavior of the model based on available data). In other words, it is an interdisciplinary field with a general goal of predicting outcomes and uncovering relationships in data [13]–[16]. It uses automated tools that (a) employ sophisticated algorithms to discover mainly hidden patterns, associations, anomalies, and/or structure from large amounts of data stored in *data warehouses* or other information repositories and (b) filter necessary information from this big dataset.

The subject of Knowledge Discovery in Databases (KDD) has evolved, and continues to evolve, from the intersection of research from such fields as databases, machine learning, pattern recognition, statistics, information theory, artificial intelligence, reasoning with uncertainties, knowledge acquisition for expert systems, data visualization, machine discovery, and high-performance computing. KDD systems incorporate theories, algorithms, and methods from all these fields. Many successful applications have been reported from varied sectors such as marketing, finance, banking, manufacturing, security, medicine, multimedia, telecommunications, etc. Database theories and tools provide the necessary infrastructure to store, access and manipulate data. A good overview of KDD can be found in Refs. [17] and [18].

Data warehousing [2] refers to the current business trends in collecting and cleaning transactional data and making them available for analysis and decision support. Data mining works hand in hand with warehouse data. Data warehousing is analogous to a mechanism that provides an enterprise with a memory, while its mining provides the enterprise with intelligence.

KDD focuses on the overall process of knowledge discovery from large volumes of data, including the storage and accessing of such data, scaling of algorithms to massive datasets, interpretation and visualization of results, and the modeling and support of the overall human machine interaction. Efficient storage of the data, and hence its structure, is very important for its

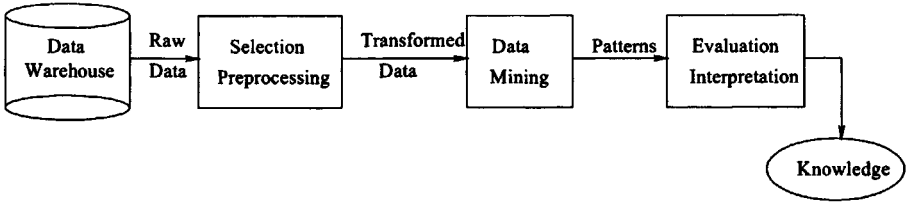


Fig. 1.1 The KDD process.

representation and access. Knowledge from modern data compression technologies should be utilized to explore how this storage mechanism can further be improved.

Data mining also overlaps with machine learning, statistics, artificial intelligence, databases, and visualization. However, the stress is more on the

- scalability of the number of features and instances,
- algorithms and architectures (while the foundations of methods and formulations are provided by statistics and machine learning), and
- automation for handling large volumes of heterogeneous data.

In the remaining part of this chapter we consider data mining from the perspective of machine learning, pattern recognition, image processing, and artificial intelligence. We begin by providing the basics of knowledge discovery and data mining in Section 1.2. Sections 1.3–1.7 deal with brief introductions to data compression, information retrieval, text mining, Web mining, and image mining. Their applicability to multimedia data are also highlighted. This is followed, in Sections 1.8–1.10, by a treatise on some of the major functions of data mining like classification, clustering, and rule mining. String matching, another important aspect of data mining with promising applications to Bioinformatics, is described in Section 1.11. An introduction to the research issues in Bioinformatics is provided in Section 1.12. The details on all these topics are provided in subsequent chapters of this book. In Section 1.13 we briefly present the concept of data warehousing. Section 1.14 highlights the applications of data mining and some existing challenges to future research. Finally, Section 1.15 concludes the chapter.

1.2 KNOWLEDGE DISCOVERY AND DATA MINING

Knowledge discovery in databases (KDD) is defined as *the nontrivial process of identifying valid, novel, potentially useful, and ultimately understandable patterns in data* [17, 19]. The overall process consists of turning low-level data into high-level knowledge. The KDD process is outlined in Fig. 1.1. It is interactive and iterative involving, more or less, the following steps [20]:

1. *Understanding the application domain:* This includes relevant prior knowledge and goals of the application.
2. *Extracting the target dataset:* This is nothing but selecting a dataset or focusing on a subset of variables, using feature ranking and selection techniques.
3. *Data preprocessing:* This is required to improve the quality of the actual data for mining. This also increases the mining efficiency by reducing the time required for mining the preprocessed data. Data preprocessing involves data cleaning, data transformation, data integration, data reduction or data compression for compact representation, etc.
 - (a) *Data cleaning:* It consists of some basic operations, such as normalization, noise removal and handling of missing data, reduction of redundancy, etc. Data from real-world sources are often erroneous, incomplete, and inconsistent, perhaps due to operational error or system implementation flaws. Such low-quality data needs to be cleaned prior to data mining.
 - (b) *Data integration:* Integration plays an important role in KDD. This operation includes integrating multiple, heterogeneous datasets generated from different sources.
 - (c) *Data reduction and projection:* This includes finding useful features to represent the data (depending on the goal of the task) and using dimensionality reduction, feature discretization, and feature extraction (or transformation) methods. Application of the principles of data compression can play an important role in data reduction and is a possible area of future development, particularly in the area of knowledge discovery from multimedia dataset.
4. *Data mining:* Data mining constitutes one or more of the following functions, namely, classification, regression, clustering, summarization, image retrieval, discovering association rules and functional dependencies, rule extraction, etc.
5. *Interpretation:* This includes interpreting the discovered patterns, as well as the possible (low-dimensional) visualization of the extracted patterns. Visualization is an important aid that increases understandability from the perspective of humans. One can evaluate the mined patterns automatically or semiautomatically to identify the truly interesting or useful patterns for the user.
6. *Using discovered knowledge:* It includes incorporating this knowledge into the performance system and taking actions based on the knowledge.

In other words, given huge volumes of heterogeneous data, the objective is to efficiently extract meaningful patterns that can be of interest and hence

useful to the user. The role of interestingness is to threshold or filter the large number of discovered patterns, and report only those which may be of some use. There are two approaches to designing a measure of interestingness of a pattern, namely, objective and subjective. The former uses the structure of the pattern and is generally quantitative. Often it fails to capture all the complexities of the pattern discovery process. The *subjective* approach, on the other hand, depends additionally on the *user* who examines the pattern. Two major reasons why a pattern is interesting from the subjective (user-oriented) point of view are as follows [21].

- *Unexpectedness*: When it is “surprising” to the user, and this potentially delivers new information to the user.
- *Actionability*: When the user can act on it to her/his advantage to fulfill the goal.

Though both these concepts are important, it has often been observed that actionability and unexpectedness are correlated. In literature, unexpectedness is often defined in terms of the dissimilarity of a discovered pattern from a predefined vocabulary provided by the user.

As an example, let us consider a database of student evaluations of different courses offered at some university. This can be defined as EVALUATE (TERM, YEAR, COURSE, SECTION, INSTRUCTOR, INSTRUCT_RATING, COURSE_RATING). We describe two patterns that are interesting in terms of actionability and unexpectedness respectively. The pattern that Professor X is consistently getting the overall INSTRUCT_RATING below overall COURSE_RATING can be of interest to the chairperson, because this shows that Professor X has room for improvement. If, on the other hand, in most of the course evaluations the overall INSTRUCT_RATING is higher than COURSE_RATING and it turns out that in most of Professor X’s rating the overall INSTRUCT_RATING is lower than COURSE_RATING, then such a pattern is unexpected and hence interesting.

Data mining is a step in the KDD process consisting of a particular enumeration of patterns over the data, subject to some computational limitations. The term *pattern* goes beyond its traditional sense to include models or structures in the data. *Historical* data are used to discover regularities and improve future decisions [22]. The data can consist of (say) a collection of time series descriptions that can be learned to predict later events in the series.

Data mining involves fitting models to or determining patterns from observed data. The fitted models play the role of inferred knowledge. Deciding whether the model reflects useful knowledge or not is a part of the overall KDD process for which subjective human judgment is usually required. Typically, a data mining algorithm constitutes some combination of the following three components.

- *The model*: The function of the model (e.g., classification, clustering) and its representational form (e.g., linear discriminants, decision trees).

A model contains parameters that are to be determined from data for the chosen function using the particular representational form or tool.

- *The preference criterion:* A basis for preference of one model or set of parameters over another, depending on the given data. The criterion is usually some form of goodness-of-fit function of the model to the data, perhaps tempered by a smoothing term to avoid over-fitting, or generating a model with too many degrees of freedom to be constrained by the given data.
- *The search algorithm:* The specification of an algorithm for finding particular models or patterns and parameters, given the data, model(s), and a preference criterion.

A particular data mining algorithm is usually an instantiation of the model-preference-search components. Some of the common model functions in current data mining practice include [13, 14]:

1. Classification: This model function classifies a data item into one of several predefined categorical classes.
2. Regression: The purpose of this model function is to map a data item to a real-valued prediction variable.
3. Clustering: This function maps a data item into one of several clusters, where clusters are natural groupings of data items based on similarity metrics or probability density models.
4. Rule generation: Here one mines or generates rules from the data. Association rule mining refers to discovering association relationships among different attributes. Dependency modeling corresponds to extracting significant dependencies among variables.
5. Summarization or condensation: This function provides a compact description for a subset of data. Data compression may play a significant role here, particularly for multimedia data, because of the advantage it offers to compactly represent the data with a reduced number of bits, thereby increasing the database storage bandwidth.
6. Sequence analysis: It models sequential patterns, like time-series analysis, gene sequences, etc. The goal is to model the states of the process generating the sequence, or to extract and report deviation and trends over time.

The rapid growth of interest in data mining [22] is due to the (i) advancement of the Internet technology and wide interest in multimedia applications in this domain, (ii) falling cost of large storage devices and increasing ease of collecting data over networks, (iii) sharing and distribution of data over the network, along with adding of new data in existing data repository, (iv)

development of robust and efficient machine learning algorithms to process this data, (v) advancement of computer architecture and falling cost of computational power, enabling use of computationally intensive methods for data analysis, (vi) inadequate scaling of conventional querying or analysis methods, prompting the need for new ways of interaction, (vii) strong competitive pressures from available commercial products, etc.

The notion of scalability relates to the efficient processing of large datasets, while generating from them the best possible models. The most commonly cited reason for scaling up is that increasing the size of the training set often increases the accuracy of learned classification models. In many cases, the degradation in accuracy when learning from smaller samples stems from overfitting, presence of noise, and existence of large number of features. Again, scaling up to very large datasets implies that fast learning algorithms must be developed. Finally, the goal of the learning (say, classification accuracy) must not be substantially sacrificed by a scaling algorithm. The three main approaches to scaling up include [23]

- designing a fast algorithm: improving computational complexity, optimizing the search and representation, finding approximate solutions to the computationally complex (NP complete or NP hard) problems, or taking advantage of the task's inherent parallelism;
- partitioning the data: dividing the data into subsets (based on instances or features), learning from one or more of the selected subsets, and possibly combining the results; and
- using a relational representation: addresses data that cannot feasibly be treated as a single flat file.

Some examples of mined or discovered patterns include

1. Classification:

- (a) People with age less than 25 and salary > 40K drive sports cars.
- (b) Set of images that contain a car as an object.

2. Association rules:

- (a) 80% of the images containing a car as an object also contain a blue sky.
- (b) 98% of people who purchase diapers also buy baby food.

3. Similar time sequences:

- (a) Stocks of companies *A* and *B* perform similarly.
- (b) Sale of furniture increases with the improvement of real estate business.

4. Outlier detection:

- (a) Residential customers for a telecom company, with businesses at home.
- (b) Digital radiographs of lungs, with suspicious spots.

1.3 DATA COMPRESSION

With growing demands of various applications, storage requirements of digital data are growing explosively. Some examples, demonstrating this, are given below.

- A high-quality audio signal requires approximately 1.5 megabits per second for digital representation and storage.
- A digitized 14×17 square-inch radiograph, scanned at $70 \mu\text{m}$, occupies nearly 45 megabytes of storage.
- A television-quality low-resolution color video of 30 frames per second, with each frame containing 640×480 pixels (24 bits per color pixel), needs more than 210 megabits per second of storage. As a result, a digitized one hour color movie would require approximately 95 gigabytes of storage.
- The storage requirement for the upcoming High-Definition-Television (HDTV)-quality video of resolution 1280×720 at 60 frames per second is many-fold. A digitized one-hour color movie of HDTV-quality video will require approximately 560 gigabytes of storage.
- A small document collection in electronic form in a digital library system may easily require to store several billion characters.
- The total amount of data spread over Internet sites is mind-boggling.

Although the cost of storage has decreased drastically over the past decade due to significant advancement in the microelectronics and storage technology, the requirement of data storage and data processing applications is growing explosively to outpace this achievement. Hence data compression continues to be a challenging area of research and development in both academia and industry, particularly in the context of large databases.

Interestingly enough, most of the datatypes for practical applications such as still image, video, voice, and text generally contain a significant amount of superfluous and redundant information in their canonical representation. Data redundancy may appear in different forms in the digital representation of different categories of datatypes. A few examples are as follows.

1. In English text files, common words (e.g., “*is*”, “*are*”, “*the*”) or similar patterns of character strings (e.g., ‘*ze*’, ‘*th*’, ‘*ing*’) are usually used repeatedly. It is also observed that the characters in an English text occur in a well-documented distribution, with letter “*e*” and “space” being the most popular.
2. In numeric data files, often we observe runs of similar numbers or predictable interdependency amongst the numbers.
3. The neighboring pixels in a typical image are highly correlated to each other, with the pixels in a smooth region of an image having similar values.
4. Two consecutive frames in a video are often mostly identical when motion in the scene is slow.
5. Some audio data beyond the human audible frequency range are useless for all practical purposes.

Data compression is the technique to reduce the redundancies in data representation in order to decrease data storage requirements and, hence, communication costs when transmitted through a communication network [24, 25]. Reducing the storage requirement is equivalent to increasing the capacity of the storage medium. If the compressed data are properly indexed, it may improve the performance of mining data in the compressed large database as well. This is particularly useful when interactivity is involved with a data mining system. Thus the development of efficient compression techniques, particularly suitable for data mining, will continue to be a design challenge for advanced database management systems and interactive multimedia applications.

Depending upon the application criteria, data compression techniques can be classified as *lossless* and *lossy*. In lossless methods we compress the data in such a way that the decompressed data can be an exact replica of the original data. Lossless compression techniques are applied to compress text, numeric, or character strings in a database – typically, medical data, etc. On the other hand, there are application areas where we can compromise with the accuracy of the decompressed data and can, therefore, afford to lose some information. For example, typical image, video, and audio compression techniques are *lossy*, since the approximation of the original data during reconstruction is good enough for human perception.

In our view, data compression is a field that has so far been neglected by the data mining community. The basic principle of data compression is to reduce the redundancies in data representation, in order to generate a shorter representation for the data to conserve data storage. In earlier discussions, we emphasized that *data reduction* is an important preprocessing task in data mining. Need for reduced representation of data is crucial for the success of very large multimedia database applications and the associated

economical usage of data storage. Multimedia databases are typically much larger than, say, business or financial data, simply because an attribute itself in a multimedia database could be a high-resolution digital image. Hence storage and subsequent access of thousands of high-resolution images, which are possibly interspersed with other datatypes as attributes, is a challenge. Data compression offers advantages in the storage management of such huge data. Although data compression has been recognized as a potential area for data reduction in literature [13], not much work has been reported so far on how the data compression techniques can be integrated in a data mining system.

Data compression can also play an important role in data condensation. An approach for dealing with the intractable problem of learning from huge databases is to select a small subset of data as representatives for learning. Large data can be viewed at varying degrees of detail in different regions of the feature space, thereby providing adequate importance depending on the underlying probability density [26]. However, these condensation techniques are useful only when the structure of data is well-organized. Multimedia data, being not so well-structured in its raw form, leads to a big bottleneck in the application of existing data mining principles. In order to avoid this problem, one approach could be to store some predetermined feature set of the multimedia data as an index at the header of the compressed file, and subsequently use this condensed information for the discovery of information or data mining.

We believe that integration of data compression principles and techniques in data mining systems will yield promising results, particularly in the age of multimedia information and their growing usage in the Internet. Soon there will arise the need to automatically discover or access information from such multimedia data domains, in place of well-organized business and financial data only. Keeping this goal in mind, we intended to devote significant discussions on data compression techniques and their principles in multimedia data domain involving text, numeric and non-numeric data, images, etc.

We have elaborated on the fundamentals of data compression and image compression principles and some popular algorithms in Chapter 3. Then we have described, in Chapter 9, how some data compression principles can improve the efficiency of information retrieval particularly suitable for multimedia data mining.

1.4 INFORMATION RETRIEVAL

Users approach large information spaces like the Web with different motives, namely, to (i) search for a specific piece of information or topic, (ii) gain familiarity with, or an overview of, some general topic or domain, and (iii) locate something that might be of interest, without a clear prior notion of what “interesting” should look like. The field of information retrieval devel-

ops methods that focus on the first situation, whereas the latter motives are mainly addressed in approaches dealing with exploration and visualization of the data.

Information retrieval [28] uses the Web (and digital libraries) to access multimedia information repositories consisting of mixed media data. The information retrieved can be text as well as image document, or a mixture of both. Hence it encompasses both text and image mining. Information retrieval automatically entails some amount of summarization or compression, along with retrieval based on content. Given a user query, the information system has to retrieve the documents which are related to that query. The potentially large size of the document collection implies that specialized indexing techniques must be used if efficient retrieval is to be achieved. This calls for proper indexing and searching, involving *pattern* or *string matching*.

With the explosive growth of the amount of information over the Web and the associated proliferation of the number of users around the world, the difficulty in assisting users in finding the best and most recent information has increased exponentially. The existing problems can be categorized as the absence of

- filtering: a user looking for some topic on the Internet receives too much information,
- ranking of retrieved documents: the system provides no qualitative distinction between the documents,
- support of relevance feedback: the user cannot report her/his subjective evaluation of the relevance of the document,
- personalization: there is a need of personal systems that serve the specific interests of the user and build user profile,
- adaptation: the system should notice when the user changes her/his interests.

Retrieval can be efficient in terms of both (a) a high recall from the Internet and (b) a fast response time at the expense of a poor precision. Recall is the percentage of relevant documents that are retrieved, while precision refers to the percentage of documents retrieved that are considered as relevant [29]. These are some of the factors that are considered when evaluating the relevance feedback provided by a user, which can again be explicit or implicit. An implicit feedback entails features such as the time spent in browsing a Web page, the number of mouse-clicks made therein, whether the page is printed or bookmarked, etc. Some of the recent generations of search engines involve Meta-search engines (like Harvester, MetaCrawler) and intelligent Software Agent technologies. The intelligent agent approach [30, 31] is recently gaining attention in the area of building an appropriate user interface for the Web.

Therefore, four main constituents can be identified in the process of information retrieval from the Internet. They are

1. Indexing: generation of document representation.
2. Querying: expression of user preferences through natural language or terms connected by logical operators.
3. Evaluation: performance of matching between user query and document representation.
4. User profile construction: storage of terms representing user preferences, especially to enhance the system retrieval during future accesses by the user.

1.5 TEXT MINING

Text is practically one of the most commonly used multimedia datatypes in day-to-day use. Text is the natural choice for formal exchange of information by common people through electronic mail, Internet chat, World Wide Web, digital libraries, electronic publications, and technical reports, to name a few. Moreover, huge volumes of text data and information exist in the so-called “gray literature” and they are not easily available to common users outside the normal book-selling channels. The gray literature includes technical reports, research reports, theses and dissertations, trade and business literature, conference and journal papers, government reports, and so on [32]. Gray literature is typically stored in text (or document) databases. The wealth of information embedded in the huge volumes of text (or document) databases distributed all over is enormous, and such databases are growing exponentially with the revolution of current Internet and information technology. The popular data mining algorithms have been developed to extract information mainly from well-structured classical databases, such as relational, transactional, processed warehouse data, etc. Multimedia data are not so structured and often less formal. Most of the textual data spread all over the world are not very formally structured either. The structure of textual data formation and the underlying syntax vary from one language to another language (both machine and human), one culture to another, and possibly user to user. Text mining can be classified as the special data mining techniques particularly suitable for knowledge and information discovery from textual data.

Automatic understanding of the content of textual data, and hence the extraction of knowledge from it, is a long-standing challenge in artificial intelligence. There were efforts to develop models and retrieval techniques for semistructured data from the database community. The information retrieval community developed techniques for indexing and searching unstructured text documents. However, these traditional techniques are not sufficient for knowledge discovery and mining of the ever-increasing volume of textual databases.

Although retrieval of text-based information was traditionally considered to be a branch of study in information retrieval only, text mining is currently

emerging as an area of interest of its own. This became very prominent with the development of search engines used in the *World Wide Web*, to search and retrieve information from the Internet. In order to develop efficient text mining techniques for search and access of textual information, it is important to take advantage of the principles behind classical string matching techniques for pattern search in text or string of characters, in addition to traditional data mining principles. We describe some of the classical string matching algorithms and their applications in Chapter 4.

In today's data processing environment, most of the text data is stored in compressed form. Hence access of text information in the compressed domain will become a challenge in the near future. There is practically no remarkable effort in this direction in the research community. In order to make progress in such efforts, we need to understand the principles behind the text compression methods and develop underlying text mining techniques exploiting these. Usually, classical text compression algorithms, such as the Lempel-Ziv family of algorithms, are used to compress text databases. We deal with some of these algorithms and their working principles in greater detail in Chapter 3.

Other established mathematical principles for data reduction have also been applied in text mining to improve the efficiency of these systems. One such technique is the application of *principal component analysis* based on the matrix theory of *singular value decomposition*. Use of *latent semantic analysis* based on the *principal component analysis* and some other text analysis schemes for text mining have been discussed in great detail in Section 9.2.

1.6 WEB MINING

Presently an enormous wealth of information is available on the Web. The objective is to mine interesting nuggets of information, like which airline has the cheapest flights in December, or search for an old friend, etc. Internet is definitely the largest multimedia data depository or library that ever existed. It is the most disorganized library as well. Hence mining the Web is a challenge.

The Web is a huge collection of documents that comprises (i) semistructured (HTML, XML) information, (ii) hyper-link information, and (iii) access and usage information and is (iv) dynamic; that is, new pages are constantly being generated. The Web has made cheaper the accessibility of a wider audience to various sources of information. The advances in all kinds of digital communication has provided greater access to networks. It has also created free access to a large publishing medium. These factors have allowed people to use the Web and modern digital libraries as a highly interactive medium. However, present-day search engines are plagued by several problems like the

- *abundance* problem, as 99% of the information is of no interest to 99% of the people,
- *limited coverage* of the Web, as Internet sources are hidden behind search interfaces,
- *limited query interface*, based on keyword-oriented search, and
- *limited customization* to individual users.

Web mining [27] refers to the use of data mining techniques to automatically retrieve, extract, and evaluate (generalize or analyze) information for knowledge discovery from Web documents and services. Considering the Web as a huge repository of distributed hypertext, the results from text mining have great influence in Web mining and information retrieval. Web data are typically unlabeled, distributed, heterogeneous, semistructured, time-varying, and high-dimensional. Hence some sort of human interface is needed to handle context-sensitive and imprecise queries and provide for summarization, deduction, personalization, and learning.

The major components of Web mining include

- information retrieval,
- information extraction,
- generalization, and
- analysis.

Information retrieval, as mentioned in Section 1.4, refers to the automatic retrieval of relevant documents, using document indexing and search engines. Information extraction helps identify document fragments that constitute the semantic core of the Web. Generalization relates to aspects from pattern recognition or machine learning, and it utilizes clustering and association rule mining. Analysis corresponds to the extraction, interpretation, validation, and visualization of the knowledge obtained from the Web.

Different aspects of Web mining have been discussed in Section 9.5.

1.7 IMAGE MINING

Image is another important class of multimedia datatypes. The World Wide Web is presently regarded as the largest global multimedia data repository, encompassing different types of images in addition to other multimedia datatypes. As a matter of fact, much of the information communicated in the real-world is in the form of images; accordingly, digital pictures play a pervasive role in the World Wide Web for visual communication. Image databases are typically

very large in size. We have witnessed an exponential growth in the generation and storage of digital images in different forms, because of the advent of electronic sensors (like CMOS or CCD) and image capture devices such as digital cameras, camcorders, scanners, etc.

There has been a lot of progress in the development of text-based search engines for the World Wide Web. However, search engines based on other multimedia datatypes do not exist. To make the data mining technology successful, it is very important to develop search engines in other multimedia datatypes, especially for image datatypes. Mining of data in the imagery domain is a challenge. Image mining [33] deals with the extraction of implicit knowledge, image data relationship, or other patterns not explicitly stored in the images. It is more than just an extension of data mining to the image domain. Image mining is an interdisciplinary endeavor that draws upon expertise in computer vision, pattern recognition, image processing, image retrieval, data mining, machine learning, database, artificial intelligence, and possibly compression.

Unlike low-level computer vision and image processing, the focus of image mining is in the extraction of patterns from a large collection of images. It, however, includes content-based retrieval as one of its functions. While current content-based image retrieval systems can handle queries about image contents based on one or more related image features such as color, shape, and other spatial information, the ultimate technology remains an important challenge. While data mining can involve absolute numeric values in relational databases, the images are better represented by relative values of pixels. Moreover, image mining inherently deals with spatial information and often involves multiple interpretations for the same visual pattern. Hence the mining algorithms here need to be subtly different than in traditional data mining.

A discovered image pattern also needs to be suitably represented to the user, often involving feature selection to improve visualization. The information representation framework for an image can be at different levels, namely, pixel, object, semantic concept, and pattern or knowledge levels. Conventional image mining techniques include object recognition, image retrieval, image indexing, image classification and clustering, and association rule mining. Intelligently classifying an image by its content is an important way to mine valuable information from a large image collection [34].

Since the storage and communication bandwidth required for image data is pervasive, there has been a great deal of activity in the international standard committees to develop standards for image compression. It is not practical to store the digital images in uncompressed or raw data form. Image compression standards aid in the seamless distribution and retrieval of compressed images from an image repository. Searching images and discovering knowledge directly from compressed image databases has not been explored enough. However, it is obvious that image mining in compressed domain will become a challenge in the near future, with the explosive growth of the image data

depository distributed all over in the World Wide Web. Hence it is crucial to understand the principles behind image compression and its standards, in order to make significant progress to achieve this goal.

We discuss the principles of multimedia data compression, including that for image datatypes, in Chapter 3. Different aspects of image mining are described in Section 9.3.

1.8 CLASSIFICATION

Classification is also described as supervised learning [35]. Let there be a database of tuples, each assigned a class label. The objective is to develop a model or profile for each class. An example of a profile with good credit is $25 \leq \text{age} \leq 40$ and $\text{income} > 40\text{K}$ or $\text{married} = \text{"yes"}$. Sample applications for classification include

- Signature identification in banking or sensitive document handling (match, no match).
- Digital fingerprint identification in security applications (match, no match).
- Credit card approval depending on customer background and financial credibility (good, bad).
- Bank location considering customer quality and business possibilities (good, fair, poor).
- Identification of tanks from a set of images (friendly, enemy).
- Treatment effectiveness of a drug in the presence of a set of disease symptoms (good, fair, poor).
- Detection of suspicious cells in a digital image of blood samples (yes, no).

The goal is to predict the class $C_i = f(x_1, \dots, x_n)$, where $x_1, \dots, x_n$ are the input attributes. The input to the classification algorithm is, typically, a dataset of training records with several attributes. There is one distinguished attribute called the dependent attribute. The remaining predictor attributes can be numerical or categorical in nature. A numerical attribute has continuous, quantitative values. A categorical attribute, on the other hand, takes up discrete, symbolic values that can also be class labels or categories. If the dependent attribute is categorical, the problem is called classification with this attribute being termed the class label. However, if the dependent attribute is numerical, the problem is termed regression. The goal of classification and regression is to build a concise model of the distribution of the dependent attribute in terms of the predictor attributes. The resulting model is used to

assign values to a database of testing records, where the values of the predictor attributes are known but the dependent attribute is to be determined. Classification methods can be categorized as follows.

1. Decision trees [36], which divide a decision space into piecewise constant regions. Typically, an information theoretic measure is used for assessing the discriminatory power of the attributes at each level of the tree.
2. Probabilistic or generative models, which calculate probabilities for hypotheses based on Bayes' theorem [35].
3. Nearest-neighbor classifiers, which compute minimum distance from instances or prototypes [35].
4. Regression, which can be linear or polynomial, of the form $ax_1 + bx_2 + c = C_i$ [37].
5. Neural networks [38], which partition by nonlinear boundaries. These incorporate learning, in a *data-rich* environment, such that all information is encoded in a distributed fashion among the connection weights.

Neural networks are introduced in Section 2.2.3, as a major soft computing tool. We have devoted the whole of Chapter 5 to the principles and techniques for classification.

1.9 CLUSTERING

A cluster is a collection of data objects which are similar to one another within the same cluster but dissimilar to the objects in other clusters. Cluster analysis refers to the grouping of a set of data objects into clusters. Clustering is also called unsupervised classification, where no predefined classes are assigned [35].

Some general applications of clustering include

- Pattern recognition.
- Spatial data analysis: creating thematic maps in geographic information systems (GIS) by clustering feature spaces, and detecting spatial clusters and explaining them in spatial data mining.
- Image processing: segmenting for object-background identification.
- Multimedia computing: finding the cluster of images containing flowers of similar color and shape from a multimedia database.
- Medical analysis: detecting abnormal growth from MRI.
- Bioinformatics: determining clusters of signatures from a gene database.

- Biometrics: creating clusters of facial images with similar fiduciary points.
- Economic science: undertaking market research.
- WWW: clustering Weblog data to discover groups of similar access patterns.

A good clustering method will produce high-quality clusters with high *intra*class similarity and low *inter*class similarity. The quality of a clustering result depends on both (a) the similarity measure used by the method and (b) its implementation. It is measured by the ability of the system to discover some or all of the hidden patterns.

Clustering approaches can be broadly categorized as

1. Partitional: Create an initial partition and then use an iterative control strategy to optimize an objective.
2. Hierarchical: Create a hierarchical decomposition (dendogram) of the set of data (or objects) using some termination criterion.
3. Density-based: Use connectivity and density functions.
4. Grid-based: Create multiple-level granular structure, by quantizing the feature space in terms of finite cells.

Clustering, when used for data mining, is required to be (i) scalable, (ii) able to deal with different types of attributes, (iii) able to discover clusters with arbitrary shape, (iv) having minimal requirements for domain knowledge to determine input parameters, (v) able to deal with noise and outliers, (vi) insensitive to order of input records, (vii) of high dimensionality, and (viii) interpretable and usable. Further details on clustering are provided in Chapter 6.

1.10 RULE MINING

Rule mining refers to the discovery of the relationship(s) between the attributes of a dataset, say, a set of *transactions*. Market basket data consist of a set of items bought together by customers, one such set of items being called a transaction. A lot of work has been done in recent years to find associations among items in large groups of transactions [39, 40].

A rule is normally expressed in the form $X \Rightarrow Y$, where X and Y are sets of attributes of the dataset. This implies that transactions which contain X also contain Y . A rule is normally expressed as IF *< some_conditions_satisfied >* THEN *< predict_values_for_some_other_attributes >*. So the association $X \Rightarrow Y$ is expressed as IF X THEN Y . A sample rule could be of the form

```

IF (salary  $\geq$  12000) AND (unpaid_loan = "no")
THEN (select_for_loan = "yes").

```

Rule mining can be categorized as

1. Association rule mining: An expression of the form $X \Rightarrow Y$, where X and Y are subsets of all attributes, and the implication holds with a confidence $\geq c$, where c is a user-defined threshold. This implies IF X THEN Y , with at least c confidence.
2. Classification rule mining: A supervised process uses a training dataset to generate the rules. The objective is to predict a predefined class or goal attribute, which can never appear in the antecedent part of a rule. The generated rules are used to predict the class attribute of an unknown test dataset.
3. Dependency rule modeling: This is also a supervised process, with the goal attribute being chosen from a predefined set of attributes. While non-goal attributes can occur only in the antecedent part of a rule, the goal attributes can appear in either its consequent or antecedent parts.

Let us consider an example from medical decision-making. Often data may be missing for various reasons; for example, some examinations can be risky for the patient or contraindications can exist, an urgent diagnostic decision may need to be made and some very informative but prolonged test results may have to be excluded from the feature set, or appropriate technical equipment may not be available. In such cases, the system can *query* the user for additional information only when it is particularly necessary to infer a decision. Again, one realizes that the final responsibility for any diagnostic decision always has to be accepted by the medical practitioner. So the physician may want to verify the justification behind the decision reached, based on personal expertise. This requires the system to be able to explain its mode of reasoning for any inferred decision or recommendation, preferably in classification rule form, to convince the user that its reasoning is correct.

Important association rule mining techniques have been considered in detail in Chapter 7. Generation of classification rules, in a modular framework, have been described in Chapter 8.

1.11 STRING MATCHING

String matching is a very important area of research for successful development of data mining systems, particularly for text databases and in mining of data through the Internet by a text-based search engine. In this section, we briefly introduce the string matching problem [24].

Let $P = a_1 a_2 \dots a_m$ and $T = b_1 b_2 \dots b_n$ denote finite strings (or sequences) of characters (or symbols) over a finite alphabet Σ , where m, n are positive

integers greater than 0. In its simplest form, the *pattern* or *string matching* problem consists of searching the text T to find the occurrence(s) of the pattern P in T ($m \leq n$).

Several variants of the basic problem can be considered. The pattern may consist of a finite set of sequences $P = \{P^1, P^2, \dots, P^k\}$, where each P^i is a pattern from the same alphabet and the problem is to search for occurrence(s) of any one of the members of the set in the text. The patterns may be *fully* or *partially specified*.

- Let \$ denote a “don’t care” or “wild card” character; then the pattern $A\$B$ denotes a set of patterns AAB, ABB, ACB , etc. – that is, any pattern that begins with A , ends with B , and has a single unspecified character in the middle. The character \$ is called a “fixed length don’t care” (*FLDC*) character and may appear at any place in the pattern.
- A special character ϕ is used to denote the infinite set of patterns $\phi = \{\$, \$\$, \$\$\$, \dots\}$ and is called a “variable length don’t care” (*VLDC*) character.

Patterns containing special characters \$ or ϕ are called partially specified; otherwise, they are termed fully specified.

The string matching problem has been extensively studied in the literature. Several linear time algorithms for the *exact* pattern matching problem (involving fully specified patterns) have been developed by researchers [41]–[43].

No linear time algorithm is yet known for the string matching problem with a partially specified pattern. The best known result for pattern matching using a pattern consisting of wild card characters is by Fischer and Patterson [44] with complexity $O(n \log^2 m \log \log m \log c)$, where c is the size of the alphabet. Several two-dimensional exact pattern matching algorithms have been proposed in Refs. [45]–[47].

There are other variation of the string matching when the pattern is not fully specified. For example, finding the occurrences of similar patterns with small differences in the text. Let us consider trying to find the occurrences of patterns similar to (say) “birth,” with maximum difference in two character positions in the text. Here the patterns “birth,” “broth,” “booth,” “worth,” “dirty,” etc., will be considered to be valid occurrence in the text. All these above variations of the string matching problem is usually known as *Approximate String Matching* in the literature.

The string (or pattern) matching problem becomes even more interesting when one attempts to directly match a pattern in a compressed text or database. String matching finds widespread applications in diverse areas such as text editing, text search, information retrieval, text mining, Web mining, Bioinformatics, etc. String matching is a very essential component in text analysis and retrieval in order to automatically extract the words, keywords, and set of terms in a document, and also in query processing when used in text mining.

We have devoted Chapter 4 to string matching, encompassing a detailed description of the classical algorithms along with a number of examples for each of them.

1.12 BIOINFORMATICS

A gene is a fundamental constituent of any living organism. Sequence of genes in a human body represent the signature(s) of the person. The genes are portions of the deoxyribonucleic acid, or DNA for short. J. D. Watson and F. H. Crick proposed a structure of DNA in 1953, consisting of two strands or chains. Each of these chains is composed of *phosphate* and *deoxyribose sugar* molecules joined together by covalent bonds. A *nitrogenous* base is attached to each sugar molecule. There are four bases: *adenine* [A], *cytosine* [C], *guanine* [G], and *thymine* [T]. From information theoretic perspective, the DNA can be considered as a string or sequence of symbols. Each symbol is one of the four above bases A, C, G, or T.

In the human body there are approximately 3 billion such base pairs. The whole stretch of the DNA is called the *genome* of an organism. Obviously, such a long stretch of DNA cannot be sequenced all at once. Mapping, search, and analysis of patterns in such long sequences can be combinatorially explosive and can be impractical to process even in today's powerful digital computers.

Typically, a DNA sequence may be 40,000–100,000 base pairs long. In practice, such a long stretch of DNA is first broken up into 400–2000 small fragments. Each such small fragment typically consists of approximately 1000 base pairs. These fragments are sequenced experimentally, and then reassembled together to reconstruct the original DNA sequence. Genes are encoded in these fragments of DNA. Understanding what parts of the genome encode which genes is a main area of study in computational molecular biology or Bioinformatics [7, 48]. The results of string matching algorithms and their derivatives have been applied in search, analysis and sequencing of DNA, and other developments in Bioinformatics.

Microarray experiments are done to produce gene expression patterns, that provide dynamic information about cell function. The huge volume of such data, and their high dimensions, make gene expression data to be suitable candidates for the application of data mining functions like clustering, visualization, and string matching. Visualization is used to transform these high-dimensional data to lower-dimensional, human understandable form. This aids subsequent useful analysis, leading to efficient knowledge discovery. Microarray technologies are utilized to evaluate the level of expression of thousands of genes, with applications in colon, breast, and blood cancer treatment [48].

Proteins are made up of polypeptide chains of amino acids, which consist of the DNA as the building block. General principles of protein structure, stability, and folding kinetics are being explored in Bioinformatics, using lat-

tice models. These models represent protein chains involving some parameters, and they allow complete explorations of conformational and sequence spaces. Interactions among spatially neighboring amino acids, during folding, are controlled by such factors as bond length, bond angle, electrostatic forces, hydrogen bonding, hydrophobicity, entropy, etc. [49]. The determination of an optimal conformation of a three-dimensional protein structure constitutes protein folding. This has wide-ranging applications in *pharmacogenomics*, and more specifically to drug design.

The different aspects of the applicability of data mining to Bioinformatics are described in detail in Chapter 10.

1.13 DATA WAREHOUSING

A data warehouse is a decision support database that is maintained separately from the organizations operational database. It supports information processing by providing a solid platform of consolidated, historical data for analysis. A data warehouse [13] is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of managements decision-making process. Data warehousing deals with the process of constructing and using data warehouses.

Database systems are of two types, namely, on-line transaction processing systems, like OLTP; and decision support systems, like warehouses, on-line analytical processing (OLAP), and mining. Historical data from OLTP systems form decision support systems, the goal being to learn from past experiences. While OLTP involves many short, update-intensive commands, a decision support system requires fewer but complex queries. OLTP is a major task of traditional relational database management systems. It involves day-to-day operations like purchasing, inventory, banking, manufacturing, payroll, registration, accounting, etc. OLAP, on the other hand, is a primary task of a data warehouse system. It concentrates on data analysis and decision making, based on the content of the data warehouse.

A data warehouse is subject-oriented, being organized around major subjects such as customer, product, and sales. It is constructed by integrating multiple, heterogeneous data sources, like relational databases, flat files, and on-line transaction records, in a uniform format. Data cleaning and data integration techniques are applied to ensure consistency in naming conventions, encoding structures, attribute measures, etc., among different data sources.

While an operational database is concerned with current value data, the data warehouse provides information from a historical perspective (e.g., past 5–10 years). Every key structure in the data warehouse contains an element of time, explicitly or implicitly, although the key of operational data may or may not contain the time element. Data warehouse constitutes a physically separate store of data, transformed from the operational environment. Operational update of data does not occur in the data warehouse environment.

It does not require transaction processing, recovery, and concurrency control mechanisms. It requires only two operations, namely, initial loading of data and its access.

Traditional heterogeneous databases build wrappers or mediators on top of the databases and adopt a query-driven approach. When a query is posed to a client site, a meta-dictionary is used to translate the query into a form appropriate for individual heterogeneous sites involved, and the results are integrated into a global answer set. This involves complex information filtering and a competition for resources. Data warehouses, on the other hand, are high-performance systems providing a multidimensional view for complex OLAP queries. Information from heterogeneous sources is integrated in advance, and it is stored in warehouses for direct query and analysis.

OLAP helps provide fast, interactive answers to large aggregate queries at multiple levels of abstraction. A *data cube* allows such multidimensional data to be effectively modeled and viewed in the n dimensions. Typical OLAP operations include

1. Roll up (drill-up): Summarize data by climbing up hierarchy or by dimension reduction.
2. Drill down (roll down): Reverse of roll-up from higher level summary to lower level summary or detailed data, or introducing new dimensions.
3. Slice and dice: Project and select.
4. Pivot (rotate): Reorient the cube, transform from 3D to a series of 2D planes, and provide better visualization.
5. Drill across: Involving more than one fact table.
6. Drill through: From the bottom level of the cube to its back-end relational tables (using structured query languages SQL).

1.14 APPLICATIONS AND CHALLENGES

Some of the important issues in data mining include the identification of applications for existing techniques, and developing new techniques for traditional as well as new application domains, like the Web, E-commerce, and Bioinformatics. Some of the existing practical uses of data mining exist in (i) tracking fraud, (ii) tracking game strategy, (iii) target marketing, (iv) holding on to good customers, and (v) weeding out bad customers, to name a few. There are many other areas we can envisage, where data mining can be applied. Some of these areas are as follows.

- Medicine: Determine disease outcome and effectiveness of treatments, by analyzing patient disease history to find some relationship between diseases.

- Molecular or pharmaceutical: Identify new drugs.
- Security: Face recognition, identification, biometrics, etc.
- Judiciary: Search and access of historical data on judgement of similar cases.
- Biometrics: Positive identification of a person from a large image, fingerprint or voice database.
- Multimedia retrieval: Search and identification of image, video, voice, and text from multimedia database, which may be compressed.
- Scientific data analysis: Identify new galaxies by searching for subclusters.
- Web site or Web store design, and promotion: Find affinity of visitors to Web pages, followed by subsequent layout modification.
- Marketing: Help marketers discover distinct groups in their customer bases, and then use this knowledge to develop targeted marketing programs.
- Land use: Identify areas of similar land use in an earth observation database.
- Insurance: Identify groups of motor insurance policy holders with a high average claim cost.
- City-planning: Identify groups of houses according to their house type, value, and geographical location.
- Geological studies: Infer that observed earthquake epicenters are likely to be clustered along continental faults.

The first generation of data mining algorithms has been demonstrated to be of significant value across a variety of real-world applications. But these work best for problems involving a large set of data collected into a single database, where the data are described by numeric or symbolic features. Here the data invariably do not contain text and image features interleaved with these features, and they are carefully and cleanly collected with a particular decision-making task in mind.

Development of new generation algorithms is expected to encompass more diverse sources and types of data that will support mixed-initiative data mining, where human experts collaborate with the computer to form hypotheses and test them. The main challenges to the data mining procedure, to be considered for future research, involve the following.

1. *Massive datasets and high dimensionality.* Huge datasets create combinatorially explosive search space for model induction, and they increase

the chances that a data mining algorithm will find spurious patterns that are not generally valid. Possible solutions include robust and efficient algorithms, sampling approximation methods, and parallel processing. Scaling up of existing techniques is needed – for example, in the cases of classification, clustering, and rule mining.

2. *User interaction and prior knowledge.* Data mining is inherently an interactive and iterative process. Users may interact at various stages, and domain knowledge may be used either in the form of a high-level specification of the model or at a more detailed level. Visualization of the extracted model is also desirable for better user interaction at different levels.
3. *Over-fitting and assessing the statistical significance.* Datasets used for mining are usually huge and available from distributed sources. As a result, often the presence of spurious data points leads to over-fitting of the models. Regularization and re-sampling methodologies need to be emphasized for model design.
4. *Understandability of patterns.* It is necessary to make the discoveries more understandable to humans. Possible solutions include rule structuring, natural language representation, and the visualization of data and knowledge.
5. *Nonstandard and incomplete data.* The data can be missing and/or noisy. These need to be handled appropriately.
6. *Mixed media data.* Learning from data that are represented by a combination of various media, like (say) numeric, symbolic, images, and text.
7. *Management of changing data and knowledge.* Rapidly changing data, in a database that is modified or deleted or augmented, may make previously discovered patterns invalid. Possible solutions include incremental methods for updating the patterns.
8. *Integration.* Data mining tools are often only a part of the entire decision-making system. It is desirable that they integrate smoothly, both with the database and the final decision-making procedure.
9. *Compression.* Storage of large multimedia databases is often required to be in compressed form. Hence the development of compression technology, particularly suitable for data mining, is required. It would be even more beneficial if data can be accessed in the compressed domain [24].
10. *Human Perceptual aspects for data mining.* Many multimedia data mining systems are intended to be used by humans. So it is a pragmatic

approach to design multimedia systems and underlying data mining techniques based on the needs and capabilities of the human perceptual system. The ultimate consumer of most perceptual information is the '*Human Perceptual System*'. Primarily, the *Human Perceptual System* consists of the '*Human Visual System*' and the '*Human Auditory System*'. How these systems work synergistically is still not completely understood and is a subject of ongoing research. We also need to focus some attention in this direction so that their underlying principles can be adopted while developing data mining techniques, in order to make these more amenable and natural to the human customer.

11. *Distributed database.* Interest in the development of data mining systems in a distributed environment will continue to grow. In today's networked society, data are not stored or archived in a single storage system unit. Problems arise while handling extremely large heterogeneous databases spread over multiple files, possibly in different disks or across the Web in different geographical locations. Often combining such data in a single very large file may be infeasible. Development of algorithms for mining data from distributed databases will open up newer areas of applications in the near future.

1.15 CONCLUSIONS AND DISCUSSION

Data mining is a good area of scientific study, holding ample promise for the research community. Recently a lot of progress has been reported for large databases, specifically involving association rules, classification, clustering, similar time sequences, similar text document retrieval, similar image retrieval, outlier discovery, etc. Many papers have been published in major conferences and leading journals. However, it still remains a promising and rich field with many challenging research issues.

In this chapter we have provided an introduction to knowledge discovery from databases and data mining. The major functions of data mining have been described from the perspectives of machine learning, pattern recognition, and artificial intelligence. Handling of multimedia data, their compression, matching, and their implications to text and image mining have been discussed. We have also stated principles of string matching, explaining how they can be applied in text retrieval and in Bioinformatics for DNA search type of operations. Different application domains and research challenges have also been highlighted.

Since the databases to be mined are often very large, parallel algorithms are desirable [50]. However, one has to explore a trade-off between computation, communication, memory usage, synchronization, and the use of problem-specific information, in order to select a suitable parallel algorithm for data mining. One can also partition the data appropriately and distribute

the subsets to multiple processors, learning concept descriptions in parallel and then combining them. This corresponds to loosely coupled collections of otherwise independent algorithms and is termed *distributed data mining* [51]. Traditional data mining algorithms require all data to be mined in a single, centralized data warehouse. A fundamental challenge is to develop distributed versions of data mining algorithms, so that data mining can be done while leaving some of the data in different places. In addition, appropriate protocols, languages, and network services are required for mining distributed data, handling the meta-data and the mappings required for mining the distributed data.

Spatial database systems involve spatial data – that is, point objects or spatially extended objects in a 2D/3D or some high-dimensional feature space. Knowledge discovery is becoming more and more important in these databases, as increasingly large amounts of data obtained from satellite images, X-ray crystallography, or other automatic equipment are being stored in the spatial framework. Image mining holds promise in handling such databases. Moreover, Bioinformatics offers applications in modeling or analyzing protein structures that are represented as spatial data.

There exist plenty of scope for the use of soft computing in data mining, because of the imprecise nature of data in many application domains. For example, neural nets can help in the learning, the fuzzy sets for natural language representation and imprecision handling, and the genetic algorithms for search and optimization. However, not much work has been reported in the use of soft computing tools in data mining. The relevance of soft computing lies in its ability to (i) handle subjectivity, imprecision, and uncertainty in queries, (ii) model document relevance as a *gradual* instead of a *crisp* property, (iii) provide deduction capability to the search engines, (iv) provide personalization and learning capability, and (v) deal with the dynamism, scale, and heterogeneity of Web documents.

We take this opportunity to compile in this book the existing literature on the various aspects of data mining, highlighting its application to multimedia information and Bioinformatics. Soft computing, an emergent technology, has also demonstrated ample promise in data mining. Chapter 2 focuses on an introduction to soft computing, its tools, and finally its role in the different functions of data mining. The fundamentals of multimedia data compression, particularly text and image compression, are dealt with in Chapter 3. Chapter 4 deals in-depth with various issues in string matching. Here we provide examples to show how patterns are matched in general text, as well as how they can be applied in DNA matching in Bioinformatics. The different tasks of data mining like classification, clustering and association rules are covered in Chapters 5, 6, and 7, respectively. The issue of rule generation and modular hybridization, in the soft computing framework, is described in Chapter 8. Multimedia data mining, including text mining, image mining, and Web mining, is dealt with in Chapter 9. Finally, certain aspects of Bioinformatics, as an application of data mining, are discussed in Chapter 10.

REFERENCES

1. U. Fayyad and R. Uthurusamy, "Data mining and knowledge discovery in databases," *Communications of the ACM*, vol. 39, pp. 24–27, 1996.
2. W. H. Inmon, "The data warehouse and data mining," *Communications of the ACM*, vol. 39, pp. 49–50, 1996.
3. T. Acharya and W. Metz, "Multimedia applications: Issues and challenges," in *Proceedings of the International Conference on Communications, Computers and Devices* (Indian Institute of Technology, Kharagpur, India), pp. 27–34, December 2000.
4. P. Piatetsky-Shapiro and W. J. Frawley, eds., *Knowledge Discovery in Databases*. Menlo Park, CA: AAAI/MIT Press, 1991.
5. President's Information Technology Advisory Committee's report, Washington, <http://www.ccic.gov/ac/interim/>, 1998.
6. M. Lesk, *Practical Digital Libraries: Books, Bytes, and Bucks*. San Francisco: Morgan Kaufmann, 1997.
7. S. L. Salzberg, D. B. Searls, and S. Kasif, eds., *Computational Methods in Molecular Biology*. Amsterdam: Elsevier Sciences B. V., 1998.
8. R. L. Blum, *Discovery and Representation of Causal Relationships from a Large Time-Oriented Clinical Database: The RX Project*, vol. 19 of *Lecture Notes in Medical Informatics*. New York: Springer-Verlag, 1982.
9. J. A. Major and D. R. Riedinger, "EFD—a hybrid knowledge statistical-based system for the detection of fraud," *International Journal of Intelligent Systems*, vol. 7, pp. 687–703, 1992.
10. R. Heider, "Troubleshooting CFM 56-3 engines for the Boeing 737—using CBR and data-mining," *Lecture Notes in Computer Science*, vol. 1168, pp. 512–523, 1996.
11. U. Fayyad, D. Haussler, and P. Stolorz, "Mining scientific data," *Communications of the ACM*, vol. 39, pp. 51–57, 1996.
12. O. Etzioni, "The World-Wide Web: Quagmire or goldmine?," *Communications of the ACM*, vol. 39, pp. 65–68, 1996.
13. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*. San Diego: Academic Press, 2001.
14. S. Mitra, S. K. Pal, and P. Mitra, "Data mining in soft computing framework: A survey," *IEEE Transactions on Neural Networks*, vol. 13, pp. 3–14, 2002.

15. D. Hand, H. Mannila, and P. Smyth, *Principles of Data Mining*. London: MIT Press, 2001.
16. M. Kantardzic, *Data Mining: Concepts, Models, Methods, and Algorithms*. Hoboken, NJ: Wiley Interscience, IEEE Press, 2002.
17. U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, and R. Uthrusamy, eds., *Advances in Knowledge Discovery and Data Mining*. Menlo Park, CA: AAAI/MIT Press, 1996.
18. "Special issue on knowledge discovery in data- and knowledge bases," *International Journal of Intelligent Systems*, vol. 7, no. 7, 1992.
19. K. J. Cios, W. Pedrycz, and R. Swiniarski, *Data Mining Methods for Knowledge Discovery*. Dordrecht: Kluwer, 1998.
20. U. Fayyad, G. P. Shapiro, and P. Smyth, "The KDD process for extracting useful knowledge from volumes of data," *Communications of the ACM*, vol. 39, pp. 27–34, 1996.
21. A. Silberschatz and A. Tuzhilin, "What makes patterns interesting in knowledge discovery systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, pp. 970–974, 1996.
22. T. M. Mitchell, "Machine learning and data mining," *Communications of the ACM*, vol. 42, pp. 30–36, 1999.
23. F. Provost and V. Kolluri, "A survey of methods for scaling up inductive algorithms," *Data Mining and Knowledge Discovery*, vol. 2, pp. 131–169, 1999.
24. T. Acharya, *VLSI Algorithms and Architectures for Data Compression*. Ph.D. thesis, Department of Computer Science, University of Central Florida, Orlando, FL, August 1994.
25. K. Sayood, *Introduction to Data Compression*. San Francisco: Morgan Kaufmann, 2000.
26. P. Mitra, C. A. Murthy, and S. K. Pal, "Density based multiscale data condensation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, pp. 734–747, 2002.
27. R. Kohavi, B. Masand, M. Spilipoulou, and J. Srivastava, "Web mining," *Data Mining and Knowledge Discovery*, vol. 6, pp. 5–8, 2002.
28. R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval*. Reading, MA: Addison-Wesley, 1999.
29. G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. New York: McGraw-Hill, 1983.

30. O. Etzioni and D. S. Weld, "Intelligent agents on the Internet: Facts, fiction, and forecast," *IEEE Expert*, vol. 10, pp. 44–49, August 1995.
31. P. Maes, "Agents that reduce work and information overload," *Communications of the ACM*, vol. 37, pp. 30–40, 1994.
32. I. H. Witten, A. Moffat, and T. C. Bell, *Managing Gigabytes: Compressing and Indexing Documents and Images*. San Francisco: Morgan Kaufmann, 1999.
33. J. Zhang, W. Hsu, and M. L. Lee, "Image mining: Issues, frameworks and techniques," in *Proceedings of the 2nd International Workshop on Multimedia Data Mining and the ACM SIGKDD Conference (MDM/KDD '01)* (San Francisco), pp. 13–20, August 2001.
34. A. Vailaya, A. T. Figueiredo, A. K. Jain, and H. J. Zhang, "Image classification for content-based indexing," *IEEE Transactions on Image Processing*, vol. 10, pp. 117–130, 2001.
35. J. T. Tou and R. C. Gonzalez, *Pattern Recognition Principles*. London: Addison-Wesley, 1974.
36. J. R. Quinlan, "Induction on decision trees," *Machine Learning*, vol. 1, pp. 81–106, 1986.
37. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Monterey, CA: Wadsworth and Brooks/Cole, 1984.
38. S. Haykin, *Neural Networks: A Comprehensive Foundation*. New York: Macmillan College Publishing Co., 1994.
39. R. Agrawal, T. Imielinski, and A. Swami, "Mining association rules between sets of items in large databases," in *Proceedings of 1993 ACM SIGMOD International Conference on Management of Data* (Washington, D.C.), pp. 207–216, May 1993.
40. R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proceedings of 20th International Conference on Very Large Databases*, pp. 478–499, September 1994.
41. D. Knuth, J. Morris, and V. Pratt, "Fast pattern matching in strings," *SIAM Journal of Computing*, vol. 6, pp. 323–350, 1977.
42. R. Boyer and J. A. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, pp. 762–772, 1977.
43. G. A. Stephen, *String Searching Algorithms*. Singapore: World Scientific, 2001.

44. M. J. Fischer and M. S. Paterson, "String matching and other products," in *Complexity of Computation, SIAM-AMS Proceedings* (R. M. Karp, ed.), vol. 7, pp. 113–125, 1974.
45. T. P. Baker, "A technique for extending rapid exact-match string matching to arrays of more than one dimension," *SIAM Journal of Computing*, vol. 7, pp. 533–541, 1978.
46. R. S. Bird, "Two dimensional pattern matching," *Information Processing Letters*, vol. 6, pp. 168–170, 1977.
47. A. Amir, G. M. Landau, and U. Vishkin, "Efficient pattern matching with scaling," *Journal of Algorithms*, vol. 13, pp. 2–32, 1992.
48. "Special Issue on Bioinformatics, Part I: Advances and Challenges," *Proceedings of the IEEE*, vol. 90, November 2002.
49. K. A. Dill, S. Bromberg, K. Yue, K. M. Fiebig, D. P. Yee, P. D. Thomas, and H. S. Chan, "Principles of protein folding—A perspective from simple exact models," *Protein Science*, vol. 4, pp. 561–602, 1995.
50. R. Agrawal and J. C. Shafer, "Parallel mining of association rules," *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, pp. 962–969, 1996.
51. H. Kargupta and P. Chan, eds., *Advances in Distributed and Parallel Knowledge Discovery*. Cambridge, MA: MIT Press, 2000.

This page intentionally left blank

2

Soft Computing

2.1 INTRODUCTION

Data mining is a form of knowledge discovery essential for solving problems in domains involving large volumes of data. The individual datasets may be gathered and studied collectively for purposes other than those for which they were originally created. New knowledge may also be obtained in the process, while eliminating the cost of additional data collection. Besides, data often exist in vast quantities over the Internet in an unstructured format. The application of data mining facilitates systematic analysis in such cases and helps the user in extracting relevant information. Sometimes different kinds of data can be interspersed for better semantic representation, and often data may be erroneous.

As an example, in medical data, numeric and textual information may be interspersed, different symbols can be used with the same meaning, redundancy often exists, and erroneous or misspelled medical terms are common. Hence a robust preprocessing system is required in order to extract any kind of knowledge from even medium-sized datasets.

Typically, real-life data must not only be cleaned of errors and redundancy, but must also be organized in a fashion that makes sense to the problem. There can exist imperfections in raw input data needed for knowledge acquisition, mainly due to uncertainty, vagueness, and incompleteness. While incompleteness arises due to missing or unknown data, uncertainty (or vagueness) can be caused by errors in physical measurements due to incorrect measuring devices or by a mixture of noisy and pure signals.

Soft computing is a consortium of methodologies that works synergistically and provides, in one form or another, flexible information processing capability for handling real-life ambiguous situations [1]. Its aim is to exploit the tolerance for imprecision, uncertainty, approximate reasoning, and partial truth in order to achieve tractability, robustness, and low-cost solutions. The guiding principle is to devise methods of computation that lead to an *acceptable* solution at *low* cost, by seeking for an approximate solution to an imprecisely or precisely formulated problem [2].

Recently, various soft computing methodologies have been applied to handle the different challenges posed by data mining [3]. The main constituents of soft computing, at this juncture, include fuzzy logic, neural networks, genetic algorithms, rough sets, and signal processing tools such as wavelets. Each of them contribute a distinct methodology for addressing problems in its domain. This is done in a cooperative, rather than a competitive, manner. The result is a more intelligent and robust system providing a human-interpretable, low-cost, approximate solution, as compared to traditional techniques.

This chapter provides an overview of the available literature on data mining, which is scarce, in the soft computing framework [3]. An introduction to soft computing and its constituent tools are provided in Section 2.2. Sections 2.3–2.8 explain the role of the different soft computing tools and their hybridizations, categorized on the basis of different data mining functions implemented. The utility and applicability of different soft computing methodologies is highlighted. It may be mentioned that there is no universally best data mining method; choosing particular soft computing tool(s) or some combination with traditional methods is entirely dependent on the particular application, and it requires human interaction to decide on the suitability of an approach.

Fuzzy sets provide a natural framework for the process in dealing with uncertainty or imprecise data. Generally, they are suitable for handling the issues related to understandability of patterns, incomplete and noisy data, and mixed media information and human interaction and can provide approximate solutions faster. Neural networks are nonparametric and robust and exhibit good learning and generalization capabilities in data-rich environments. Genetic algorithms (GAs) provide efficient search algorithms to optimally select a model, from mixed media data, based on some preference criterion or objective function. Rough sets are suitable for handling different types of uncertainty in data. Neural networks and rough sets are widely used for classification and rule generation. Application of wavelet-based signal processing techniques is new in the area of soft computing. Wavelet transformation of a signal results in decomposition of the original signal in different multiresolution subbands [4, 5]. This is useful in dealing with compression and retrieval of data, particularly images. Other approaches like case-based reasoning [6] and decision trees [7, 8] are also widely used to solve data mining problems.

Section 2.9 concludes the chapter. Some challenges to data mining and the possible application of soft computing methodologies are indicated.

2.2 WHAT IS SOFT COMPUTING?

Usually the primary considerations of traditional computing are precision, certainty, and rigor. We distinguish this as “hard” computing. In contrast, the principal notion in soft computing is that precision and certainty carry a cost; and that computation, reasoning, and decision-making should exploit (wherever possible) the tolerance for imprecision, uncertainty, approximate reasoning, and partial truth for obtaining low-cost solutions. This leads to the remarkable human ability of understanding distorted speech, deciphering sloppy handwriting, comprehending the nuances of natural language, summarizing text, recognizing and classifying images, driving a vehicle in dense traffic, and, more generally, making rational decisions in an environment of uncertainty and imprecision. The challenge, then, is to exploit the tolerance for imprecision by devising methods of computation that lead to *an acceptable solution at low cost*. This, in essence, is the guiding principle of soft computing [1].

There are ongoing efforts to integrate artificial neural networks (ANNs), fuzzy set theory, genetic algorithms (GAs), rough set theory and other methodologies in the *soft computing* paradigm. Hybridization [2, 9] exploiting the characteristics of these theories include *neuro-fuzzy*, *rough-fuzzy*, *neuro-genetic*, *fuzzy-genetic*, *neuro-rough*, *rough-neuro-fuzzy* approaches. However, among these, *neuro-fuzzy* computing is the most visible. Let us now begin our discussion by pointing out the relevance of soft computing.

2.2.1 Relevance

The traditional hard computing paradigm is seldom suitable for many real-life problems. Let us illustrate it with an example. Suppose that X is driving a car and X watches a “red light” (traffic signal). X has to stop. So X has to decide when to press the brake and how strongly. In a “precise framework,” the steps followed by X may be to find the distance of the car from the “light,” and then, depending on the current speed of the car, press the brake. To realize this, the car should be provided with a laser-gun-type arrangement so that the distance can be obtained. X should also know a set of rules of the form

“If the car is at a distance of d ft and moving at a speed of s ft/s, then press the brake with p poundal for t seconds right now.”

This is a precise rule governed by the laws of physics. Hence, if the brake is applied according to such rules, the car will stop where X wants it to.

Theoretically, such concepts are fine, but impractical because of the following reasons:

- The addition of a laser gun to a car increases its cost.
- The number of precise rules required will be too great to realize in a practical system.
- For the sake of argument, even if we assume that we know the rules to be followed, application of the brakes following the rule will be a very difficult task.

Precise solutions are not always feasible. In fact, we do not need a precise solution to such a problem. The exact position where the car stops is not important, but it should stop before the “red light” and should not hit any other car standing ahead of it. Hence an approximate idea about the distance of the car from the car or traffic signal ahead and the speed of the car should be enough. Under this situation X can control the car using rules of the form “If the car is moving *very fast* and the ‘red light’ is *close*, then press the brake *pretty hard*.” We can easily say that the action is purely guided by the intuition of an individual, the resultant decision being taken in *imprecise* terms.

Note that the rule has three vague clauses “very fast,” “close,” and “pretty hard.” These make the rule an imprecise one, and it will generate an approximate solution to the problem. The solution is less expensive and fast (real-time) also. This is one facet of what the soft computing paradigm for emulating the human-like decision making (also, a real-world computing system) attempts to achieve. Thus, to achieve higher machine IQ, the system should have the capability of modeling vagueness and making approximate decisions on that basis. Fuzzy sets are good for handling this aspect of soft computing. In fact, this distinguished characteristic of fuzzy sets led to the emergence of soft computing.

Let us now make the driving problem a bit more complex. Suppose that X is driving on a very crowded road and has to reach the destination D . From the present coordinate of X , there are a couple of alternative paths to reach D . Depending on the traffic conditions, X should try to pick up an optimal path. Note that the traffic conditions (traffic flows in either direction, number of traffic signals that will appear on a path, raining or clear, etc.) change with time, and hence what X thinks as the optimal path now may not remain optimal after some time. Consequently, X has to dynamically (adaptively) change the route. Human beings make approximate decisions for such problems on the basis of their experience (learning from previous driving experiences). If we want an intelligent system to achieve this capability, it should have the ability to learn from experience and examples. Artificial neural networks are adaptive systems and can deal with this aspect of the problem.

Any artificial neural network (ANN) that can be used for handling the problem just mentioned must be fed with relevant information. In other words, the ANN system has to be trained with adequate number of examples. The popular gradient descent (say, backpropagation)-type learning algorithms are usually very slow in learning and may get stuck at some local minimum. *Genetic algorithms* (GAs), in such situations, may be very effectively used for learning. If the number of free parameters of the network is large, GAs may also become slow, but for GA-based learning, the chance of getting stuck to a local minimum would be low. Consequently, we can expect a better generalization ability of the network.

In the remaining part of this section we present the basics of the different soft computing tools.

2.2.2 Fuzzy sets

We are continuously having to recognize people, objects, handwriting, voice, images, and other patterns, using distorted or unfamiliar, incomplete, occluded, fuzzy, and inconclusive data, where a pattern should be allowed to have membership or belongingness to more than one class. This is also very significant in (say) medical diagnosis, where a patient afflicted with a certain set of symptoms can be simultaneously suffering from more than one disease. Again, the symptoms need not necessarily be strictly numerical. It would be in *natural* terms, defined as linguistic and/or set variables such as *very high*, *more or less low*, *between 50°C and 55°C*. This is how the concept of *fuzziness* comes into the picture.

Let us explain the concept of membership with an example. You ask a friend to meet you at 10 a.m. tomorrow. It is highly likely that your friend will arrive *any time around* 10 a.m., say, from 9.55 a.m. to 10.05 a.m. This defines the concept of a membership function along the time axis, with a peak (membership of 1) at 10 a.m. sharp having a bandwidth of 10 min. As you move away either side from the peak, the membership approaches the value 0. The bandwidth, again, is problem- and context-dependent. Hence if the person is serious, the bandwidth would be less, whereas otherwise the bandwidth would usually be more. Thus we see that although 10 a.m. is a *crisp* concept with $\{0, 1\}$ *hard* characterizing function, in reality it becomes fuzzy with $[0, 1]$ graded membership function. One may note that the membership value reflects the degree of compatibility or similarity of an event with an imprecise concept representing a fuzzy set, whereas the probability of an event is related to the number of times it occurs (i.e., its frequency).

Fuzzy sets were introduced in 1965 by Zadeh [10] as a new way of representing vagueness in everyday life. This theory provides an approximate and yet effective means for describing the characteristics of a system that is too complex or ill-defined to admit precise mathematical analysis [11, 12]. The fuzzy approach is based on the premise that the key elements in human thinking are not just numbers but can be approximated to tables of fuzzy sets, or,

in other words, classes of objects in which the transition from membership to nonmembership is gradual rather than abrupt. Much of the logic behind human reasoning is not the traditional two-valued or even multivalued logic, but logic with fuzzy truths, fuzzy connectives, and fuzzy rules of inference.

Fuzzy set theory is reputed to handle, to a reasonable extent, uncertainties (arising from deficiencies of information) in various applications particularly in decision-making models under different kinds of risks, subjective judgment, vagueness, and ambiguity. The deficiencies may result from various reasons, namely, incomplete, imprecise, not fully reliable, vague, or contradictory information depending on the problem. Since this theory is a generalization of the classical set theory, it has greater flexibility to capture various aspects of incompleteness or imperfection in information about a situation.

The use of linguistic variables may be viewed as a form of data compression, which can be termed *granulation* [1]. The same effect can also be achieved by conventional quantization. However, in the case of quantization the values are intervals, whereas in the case of granulation the values are overlapping fuzzy sets. The advantages of granulation over quantization are that

- It is more general.
- It mimics the way in which humans interpret linguistic values.
- The transition from one linguistic value to a contiguous linguistic value is gradual rather than abrupt, resulting in continuity and robustness.

Again, the uncertainty in classification or clustering of patterns may arise from the overlapping nature of the various classes. This overlapping may result from fuzziness or randomness. In the conventional classification technique, it is usually assumed that a pattern belongs to only one class. This is not necessarily realistic physically, and certainly not mathematically. A pattern can and should be allowed to have degrees of membership in more than one class. It is therefore necessary to convey this information while classifying a pattern or clustering a dataset.

Let us now consider the problem of processing and recognizing a gray tone image pattern. In a conventional vision system, each operation in low level, middle level, and high level involves crisp decisions to make regions, features, primitives, relations, and interpretations crisp. Since the regions in an image are not always crisply defined, uncertainty can arise at every phase of recognition tasks. Therefore it becomes convenient and natural and may be appropriate to avoid committing ourselves to specific (hard) decision by allowing the segments or contours to be fuzzy subsets of the image; the subsets are characterized by the possibility (degree) of a pixel belonging to them.

A fuzzy set A in a space of points $R = \{r\}$ is a class of events with a continuum of grades of membership, and it is characterized by a membership function $\mu_A(r)$ that associates with each element in R a real number in the interval $[0, 1]$ with the value of $\mu_A(r)$ at r representing the grade of membership of r in A . Formally, a fuzzy set A with its finite number of supports

$r_1, r_2, \dots, r_n$ is defined as a collection of ordered pairs

$$\begin{aligned} A &= \{(\mu_A(r_i), r_i), i = 1, 2, \dots, n\} \\ &= \{(\frac{\mu_A(r_i)}{r_i}), i = 1, 2, \dots, n\}, \end{aligned}$$

where the support of A is an ordinary subset of R and is defined as

$$S(A) = \{r | r \in R \text{ and } \mu_A(r) > 0\}.$$

Here μ_i , the grade of membership of r_i in A , denotes the degree to which an event r_i may be a member of A or belong to A . Note that $\mu_i = 1$ indicates the strict containment of the event r_i in A . If, on the other hand, r_i does not belong to A , then $\mu_i = 0$.

If the support of a fuzzy set is only a single point $r_1 \in R$, then

$$A = \frac{\mu_1}{r_1}$$

is called a *fuzzy singleton*. Thus $A = (1/r_1)$, for $\mu_1 = 1$, would obviously denote a nonfuzzy singleton.

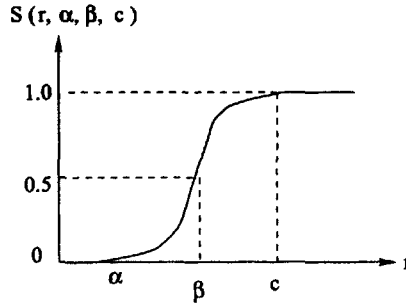
In terms of the constituent singletons the fuzzy set A with its finite number of supports $r_1, r_2, \dots, r_n$ can also be expressed in union form as

$$\begin{aligned} A &= \frac{\mu_1}{r_1} + \frac{\mu_2}{r_2} + \dots + \frac{\mu_n}{r_n} \\ &= \sum_i \frac{\mu_i}{r_i}, \quad i = 1, 2, \dots, n \\ &= \bigcup_i \frac{\mu_i}{r_i}, \quad i = 1, 2, \dots, n, \end{aligned} \tag{2.1}$$

where the $+$ sign denotes the union.

Fuzzy logic is based on the theory of fuzzy sets and, unlike classical logic, aims at modeling the imprecise (or inexact) modes of reasoning and thought processes (with linguistic variables) that play an essential role in the remarkable human ability to make rational decisions in an environment of uncertainty and imprecision. This ability depends, in turn, on our ability to infer an approximate answer to a question based on a store of knowledge that is inexact, incomplete, or not totally reliable. In fuzzy logic, everything, including truth, is a matter of degree [13]. Zadeh has developed a theory of approximate reasoning based on fuzzy set theory. By approximate reasoning we refer to a type of reasoning that is neither very exact nor very inexact. This theory aims at modeling the human reasoning and thinking process with linguistic variables [11] in order to handle both soft and hard data, as well as various types of uncertainty. Many aspects of the underlying concept have been incorporated in designing decision-making systems [14, 15].

Assignment of membership functions of a fuzzy subset is subjective in nature and reflects the context in which the problem is viewed. It cannot be assigned arbitrarily. In many cases, it is convenient to express the membership function of a fuzzy subset in terms of standard S and π functions. Note that fuzzy membership function and probability density function are conceptually different.

Fig. 2.1 Standard S function.

2.2.2.1 Membership functions It is frequently convenient to employ standardized functions with adjustable parameters (e.g., the S and π functions) which are defined in the following equations (see also Fig. 2.1):

$$\begin{aligned}
 S(r; \alpha, \beta, c) &= 0 && \text{for } r \leq \alpha \\
 &= 2\left(\frac{r-\alpha}{c-\alpha}\right)^2 && \text{for } \alpha \leq r \leq \beta \\
 &= 1 - 2\left(\frac{r-c}{c-\alpha}\right)^2 && \text{for } \beta \leq r \leq c \\
 &= 1 && \text{for } r \geq c.
 \end{aligned} \tag{2.2}$$

$$\begin{aligned}
 \pi(r; c, \lambda) &= S(r; c - \lambda, c - \frac{\lambda}{2}, c) && \text{for } r \leq c \\
 &= 1 - S(r; c, c + \frac{\lambda}{2}, c + \lambda) && \text{for } r \geq c.
 \end{aligned} \tag{2.3}$$

In $S(r; \alpha, \beta, c)$, the parameter β , $\beta = (\alpha + c)/2$, is the *crossover point*, that is, the value of r at which S takes the value 0.5. In $\pi(r; c, \lambda)$, λ is the *bandwidth*, that is, the distance between the crossover points of π , while c is the central point at which π is unity.

Let us consider the linguistic variable age (x). Here the linguistic values *young* and *old* play the role of primary fuzzy sets which have a specified meaning, for example,

$$\mu_{\text{young}} = 1 - S(20, 30, 40), \tag{2.4}$$

$$\mu_{\text{old}} = S(50, 60, 70), \tag{2.5}$$

where the S and π functions are defined by Eqs. (2.2) and (2.3), and μ_{young} and μ_{old} denote the membership functions of *young* and *old*, respectively.

In pattern recognition problems we often need to represent a class with fuzzy boundary in terms of a π function. A representation for such a π function, with range $[0, 1]$ and $r \in \mathbb{R}^n$, may be given as [2]

$$\pi(r; c, \lambda) = \begin{cases} 2 \left(1 - \frac{\|r - c\|}{\lambda}\right)^2, & \text{for } \frac{\lambda}{2} \leq \|r - c\| \leq \lambda \\ 1 - 2 \left(\frac{\|r - c\|}{\lambda}\right)^2, & \text{for } 0 \leq \|r - c\| \leq \frac{\lambda}{2} \\ 0, & \text{otherwise,} \end{cases} \tag{2.6}$$

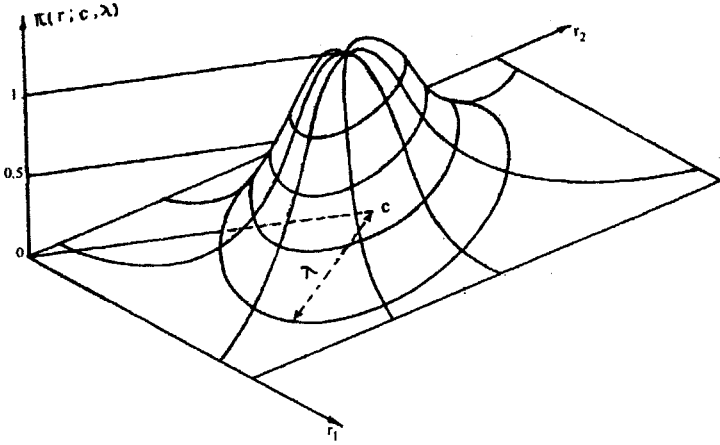


Fig. 2.2 π function when $r \in R^2$.

where $\lambda > 0$ is the radius of the π function with c as the central point and $\|\cdot\|$ denotes the Euclidean norm. This is shown in Fig. 2.2 for $r \in R^2$. Note that when the pattern r lies at the central point c of a class, then $\|r - c\| = 0$ and its membership value is maximum, that is, $\pi(c; c, \lambda) = 1$. The membership value of a point decreases as its distance from the central point c (i.e., $\|r - c\|$) increases. When $\|r - c\| = \lambda/2$, the membership value of r is 0.5, and this is called a *crossover point*.

2.2.2.2 Basic operations Basic operations related to fuzzy subsets A and B of R having membership values $\mu_A(r)$ and $\mu_B(r)$, $r \in R$ respectively, are summarized here [15].

- A is equal to B (i.e., $A = B$) $\Rightarrow \mu_A(r) = \mu_B(r)$, for all $r \in R$.
- A is a complement of B (i.e., $A = \overline{B}$) $\Rightarrow \mu_A(r) = \mu_{\overline{B}}(r) = 1 - \mu_B(r)$ for all $r \in R$.
- A is contained in B ($A \subseteq B$) $\Rightarrow \mu_A(r) \leq \mu_B(r)$ for all $r \in R$.
- The union of A and B ($A \cup B$) $\Rightarrow \mu_{A \cup B}(r) = \vee(\mu_A(r), \mu_B(r))$ for all $r \in R$, where $\vee$ denotes maximum.
- The intersection of A and B ($A \cap B$) $\Rightarrow \mu_{A \cap B}(r) = \wedge(\mu_A(r), \mu_B(r))$ for all $r \in R$, where $\wedge$ denotes minimum.

We also have the modifiers *not*, *very*, and *more or less*. These are explained, in terms of the linguistic value *young*, as follows:

$$\mu_{\text{not young}} = 1 - \mu_{\text{young}}, \quad (2.7)$$

$$\mu_{\text{very young}} = (\mu_{\text{young}})^2, \quad (2.8)$$

$$\mu_{\text{not very young}} = 1 - (\mu_{\text{young}})^2, \quad (2.9)$$

$$\mu_{\text{more or less young}} = (\mu_{\text{young}})^{0.5}. \quad (2.10)$$

2.2.3 Neural networks

There are millions of very simple processing elements or *neurons* in the brain, linked together in a massively parallel manner. This is believed to be responsible for the human intelligence and discriminating power. All information is stored in a distributed fashion among the connection weights. There is also a large amount of redundancy inherent among the connections, leading to a *graceful degradation* of performance in case of any damage. Artificial neural networks (ANNs) or connectionist models implement important aspects of a pattern recognition system like robustness, adaptivity, speed, and learning. An ANN learns through examples the discriminating characteristics among various pattern classes, by reducing the error and automatically discovering inherent relationships in a data-rich environment. No rules or programmed information sequences need to be specified beforehand. This procedure bears an analogy to how a baby learns to recognize objects, or perhaps learns to speak.

Artificial neural networks (ANNs) [16]–[21] are signal processing systems that try to emulate the behavior of biological nervous systems by providing a mathematical model of combination of numerous neurons connected in a network. These can be formally defined as *massively parallel interconnections of simple (usually adaptive) processing elements that interact with objects of the real world in a manner similar to biological systems*. ANNs attempt to replicate the *computational* power (low-level arithmetic processing ability) of biological neural networks and, thereby, hopefully endow machines with some of the (higher-level) *cognitive abilities* that biological organisms possess (due in part, perhaps, to their low-level computational prowess).

The origin of ANNs can be traced to the work of Hebb [22], where a local learning rule was proposed. This rule assumed that correlations between the states of two neurons determined the strength of the coupling between them. Subsequently, a synaptic connection that was very active grew in strength and vice versa.

The various models are designated by the network topology, node characteristics, and the status updating rules. Network topology refers to the structure of interconnections among the various nodes (neurons) in terms of layers and/or feedback or feedforward links. Node characteristics mainly specify the operations it can perform, such as summing the weighted inputs incident on it and then amplifying or applying some aggregation operators on it. The updating rules may be for weights and/or states of the processing elements (neurons). Normally, an objective function, representing the status

of the network, is defined such that its set of minima correspond to the set of stable states of the network.

Tasks that neural networks can perform include pattern classification, clustering or categorization, function approximation, prediction or forecasting, optimization, retrieval by content, and control. ANNs can be viewed as weighted directed graphs in which artificial neurons are nodes and directed edges (with weights) are connections between neuron outputs and neuron inputs. On the basis of the connection pattern (architecture), ANNs can be grouped into two categories:

- *Feedforward* networks, in which graphs have no loops – for example, single-layer perceptron, multilayer perceptron, radial basis function networks, Kohonen network
- *Recurrent* (or *feedback*) networks, in which loops occur because of feedback connections – for example, Hopfield network, adaptive resonance theory (ART) models

The computational neuron model proposed by McCulloch and Pitts [23] is a simple binary threshold unit.

$$\begin{aligned} \text{Thus } x_j(t+1) &= f(\sum_i w_{ij} x_i(t) - \theta_j), \\ \text{where } f(x) &= 1 \quad \text{if } x \geq 0 \\ &= 0 \quad \text{otherwise,} \end{aligned}$$

and x_j is the input of the j th neuron with threshold θ_j . If the synaptic weight $w_{ij} > 0$, then it is called an *excitatory* connection; if $w_{ij} < 0$, it is viewed as an *inhibitory* connection. A synchronous assembly of McCulloch–Pitts neurons is capable, in principle, of universal computation for suitably chosen weights [17]. Such an assembly can perform any computation that an ordinary digital computer can.

The adaptability of a neural network comes from its capability of learning from “environments.” Broadly, there are three paradigms of learning: supervised, unsupervised (or self-organized), and reinforcement. Sometimes, reinforcement is viewed as a special case of supervised learning. Under each category there are many algorithms. In supervised learning (learning with a teacher), adaptation is done on the basis of direct comparison of the network output with known correct or desired answer. Unsupervised learning does not learn any specific input–output relation. Here the network is tuned to the statistical regularities of the input data to form categories (or partitions) by optimizing, with respect to the free parameters of the network, some task-independent measure of quality of the representation. The reinforcement learning, on the other hand, attempts to learn the input–output mapping through trial and error with a view to maximizing a performance index called *reinforcement signal*. Here the system only knows whether the output is correct, but not what the correct output is.

ANNs are natural classifiers having resistance to noise, tolerance to distorted images or patterns (ability to generalize), superior ability to recognize partially occluded or degraded images or overlapping pattern classes or classes with highly nonlinear boundaries, and potential for parallel processing. They use nonparametric adaptive learning procedures, learn from examples, and discover important underlying regularities in the task domain.

For example, consider the case of supervised classification. Here a pattern is characterized by a number of features, each taking up different weights in characterizing the classes. A multilayer perceptron in which the input layer has neurons equal to the number of features and the output layer has neurons equal to the number of classes can be used to tackle this classification problem. Here the importance of different features will automatically be encoded in the connection links during training. The nonlinear decision boundaries are modeled, and class labels are assigned by taking collective decisions.

There has been widespread activity aimed at extracting the embedded knowledge in trained ANNs in the form of symbolic rules [2, 24, 25]. This serves to identify the attributes that, either individually or in a combination, are the most significant determinants of the decision or classification. Since all information is stored in a distributed manner among the neurons and their associated connectivity, any individual unit cannot essentially be associated with a single concept or feature of the problem domain.

Generally ANNs consider a fixed topology of neurons connected by links in a predefined manner. These connection weights are usually initialized by small random values. *Knowledge-based networks* [26, 27] constitute a special class of ANNs that consider crude domain knowledge to generate the initial network architecture, which is later refined in the presence of training data. The use of knowledge-based nets helps in reducing the searching space and time while the network traces the optimal solution. Typically, one extracts causal factors and functional dependencies from the data domain for initial encoding of the ANN [25, 28] and later generates refined rules from the trained network.

2.2.3.1 Single-layer perceptron The concept of *perceptron* [29, 30] was one of the most exciting developments during the early days of pattern recognition. The classical (single-layer) perceptron, given two classes of patterns, attempts to find a linear decision boundary separating the two classes.

A perceptron consists of a single neuron with adjustable weights, w_j , $j = 1, 2, \dots, n$, and threshold θ . Given an input vector $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, the net input to the neuron is

$$v = \sum_{j=1}^n w_j x_j - \theta. \quad (2.11)$$

The output y of the perceptron is +1 if $v > 0$ and is 0 otherwise. In a two-class classification problem, the perceptron assigns an input pattern to

one class if $y = 1$ and to the other class if $y = 0$. The linear equation $\sum_{j=1}^n w_j x_j - \theta = 0$ defines the decision boundary (a hyperplane in the n -dimensional input space) that halves the space. Rosenblatt [30] developed a learning procedure to determine the weights and threshold in a perceptron, given a set of training patterns. This algorithm is outlined as follows:

1. Initialize the weights and threshold to small random numbers.
2. Present a pattern vector $[x_1, x_2, \dots, x_n]^T$ and evaluate the output of the neuron.
3. Update the weights according to

$$w_j(t+1) = w_j(t) + \varepsilon(d - y)x_j, \quad (2.12)$$

where d is the desired output, t is the iteration number, and ε ($0.0 < \varepsilon < 1.0$) is the learning rate (step size).

Note that learning occurs only when the perceptron makes an error. This has an interesting explanation from the information theoretic perspective. Usually we expect new information when there is occurrence of an error, and hence it provides an opportunity for new learning.

Rosenblatt proved that when training patterns are drawn from two linearly separable classes, the perceptron learning procedure converges after a finite number of iterations. If the pattern space is not linearly separable, the perceptron fails [31]. A single-layer perceptron is inadequate for situations with multiple classes and nonlinear separating boundaries. Hence the invention of the multilayer perceptron network.

2.2.3.2 Multilayer perceptron (MLP) using backpropagation of error The multilayer perceptron (MLP) [18] consists of multiple layers of simple, two-state, sigmoid processing elements (nodes) or neurons that interact using weighted connections. After a lowermost input layer there are one or more intermediate *hidden* layers, followed by an output layer at the top. There exist no interconnections within a layer, while all neurons in a layer are fully connected to neurons in adjacent layers.

An external input vector is supplied to the network by clamping it at the nodes in the input layer. For conventional classification problems, during training, the appropriate output node is clamped to state 1 while the others are clamped to state 0. This is the desired output supplied by the *teacher*. The number of units in the output layer H corresponds to the number of output classes.

Consider the network given in Fig. 2.3. The total input x_j^{h+1} received by neuron j in layer $h+1$ is defined as

$$x_j^{h+1} = \sum_i y_i^h w_{ji}^h - \theta_j^{h+1}, \quad (2.13)$$

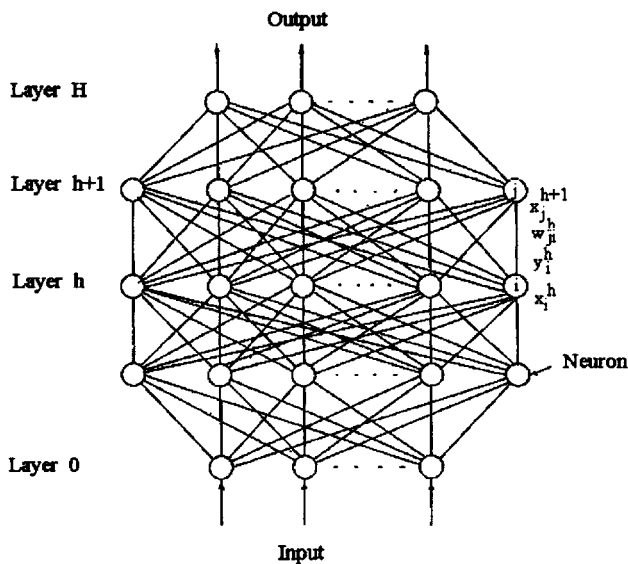


Fig. 2.3 MLP with three hidden layers.

where y_i^h is the state of the i th neuron in the preceding h th layer, w_{ji}^h is the weight of the connection from the i th neuron in layer h to the j th neuron in layer $h + 1$ and θ_j^{h+1} is the threshold of the j th neuron in layer $h + 1$. Threshold θ_j^{h+1} may be eliminated by giving the unit j in layer $h + 1$ an extra input line with a fixed activity level of 1 and a weight of $-\theta_j^{h+1}$.

The output of a neuron in any layer other than the input layer ($h > 0$) is a monotonic nonlinear function of its total input and is expressed as

$$y_j^h = \frac{1}{1 + e^{-x_j^h}}. \quad (2.14)$$

For nodes in the input layer

$$y_j^0 = x_j^0, \quad (2.15)$$

where x_j^0 is the j th component of the input vector clamped at the input layer. Learning consists of minimizing the error by updating the weights. It involves searching a very large parameter space and therefore is usually rather slow.

The least mean square (LMS) error in output vectors, for a given network weight vector $\mathbf{w}$, is defined as

$$E(\mathbf{w}) = \frac{1}{2} \sum_{j,p} (y_{j,p}^H(\mathbf{w}) - d_{j,p})^2, \quad (2.16)$$

where $y_{j,p}^H(\mathbf{w})$ is the state obtained for output node j in layer H for input-output pattern p and $d_{j,p}$ is its desired state specified by the teacher. One method for minimization of $E(\mathbf{w})$ is to apply the method of gradient descent by starting with any set of weights and repeatedly updating each weight by an amount

$$\Delta w_{ji}^h(t) = -\varepsilon \frac{\partial E}{\partial w_{ji}} + \alpha \Delta w_{ji}^h(t-1), \quad (2.17)$$

where the positive constant ε controls the descent, $0 \leq \alpha \leq 1$ is the damping coefficient or momentum, and t denotes the number of the iteration currently in progress. Generally ε and α are set at constant values, but there exist approaches that vary these parameters. Initially the connection weights w_{ji}^h between each pair of neurons i in layer h and j in layer $h+1$ are set to small random values lying in the range $[-0.5, 0.5]$.

From Eqs. (2.13)–(2.14) and (2.16), we have

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial y_j} \frac{dy_j}{dx_j} \frac{\partial x_j}{\partial w_{ji}} = \frac{\partial E}{\partial y_j} y_j^h (1 - y_j^h) y_i^{h-1}. \quad (2.18)$$

For the output layer ($h = H$), we substitute in Eq. (2.18)

$$\frac{\partial E}{\partial y_j} = y_j^H - d_j. \quad (2.19)$$

For the other layers, using Eq. (2.13), we substitute in Eq. (2.18)

$$\frac{\partial E}{\partial y_j} = \sum_k \frac{\partial E}{\partial y_k} \frac{dy_k}{dx_k} \frac{\partial x_k}{\partial y_j} = \sum_k \frac{\partial E}{\partial y_k} \frac{dy_k}{dx_k} w_{kj}^h, \quad (2.20)$$

where units j and k lie in layers h and $h+1$, respectively.

During training, each pattern of the training set is used in succession to clamp the input and output layers of the network. A sequence of forward and backward passes using Eqs. (2.13)–(2.20) constitute a *cycle*, and such a cycle through the entire training set is termed a *sweep*. After a number of sweeps through the training data, the error $E(\mathbf{w})$ in Eq. (2.16) may be minimized. At this stage the network is supposed to have discovered (learned) the relationship between the input and output vectors in the training samples.

In the testing phase the neural net is expected to be able to utilize the information encoded in its connection weights to assign the correct output labels for the test vectors that are now clamped only at the input layer. It should be noted that the optimal number of hidden layers and the number of units in each such layer are generally determined empirically, although growing, pruning, and other optimization techniques are also in vogue.

2.2.3.3 Kohonen network The essential constituents of Kohonen neural network model are as follows [19]:

- An array of neurons receiving coherent inputs, simultaneously, and computing a simple output function.

- A mechanism for comparing the neuronal outputs to select the neuron producing maximum output.
- A local interaction between the selected neuron and its neighbors.
- An adaptive mechanism that updates the interconnection weights.

The self-organizing feature map (SOFM) is an unsupervised learning network [19], which transforms p -dimensional input patterns to a q -dimensional (usually $q = 1$ or 2) discrete map in a topologically ordered fashion. Input points that are close in p -dimension are also mapped closely on the q -dimensional lattice. Each lattice cell is represented by a neuron that has a p -dimensional adaptable weight vector associated with it. With every input the match with each weight vector is computed. Then the best matching weight vector and some of its topological neighbors are adjusted to match the input points a little better. Initially, the process starts with a large neighborhood; with passage of time (iteration), the neighborhood size is reduced gradually. At a given time instant, within the neighborhood, the weight vector associated with each neuron is not updated equally. The strength of interaction between the winner and a neighboring node is inversely related to the distance (on the lattice) between them.

Consider the self-organizing network given in Fig. 2.4. Let M input signals be simultaneously incident on each of an $N \times N$ array of neurons. The output of the i th neuron is defined as

$$\eta_i(t) = \sigma \left[[\mathbf{m}_i(t)]^T \mathbf{x}(t) + \sum_{k \in S_i} w_{ki} \eta_k(t - \Delta t) \right], \quad (2.21)$$

where $\mathbf{x}$ is the M -dimensional input vector incident on it along the connection weight vector $\mathbf{m}_i$, k belongs to the subset S_i of neurons having interconnections with the i th neuron, w_{ki} denotes the fixed feedback coupling between the k th and i th neurons, $\sigma[\cdot]$ is a suitable sigmoidal output function, t denotes a discrete time index, and T stands for the transpose.

Initially the components of the $\mathbf{m}_i$ values are set to small random values lying in the range $[0, 0.5]$. If the best match between vectors $\mathbf{m}_i$ and $\mathbf{x}$ occurs at neuron c , then we have

$$\|\mathbf{x} - \mathbf{m}_c\| = \min_i \|\mathbf{x} - \mathbf{m}_i\|, \quad i = 1, 2, \dots, N^2, \quad (2.22)$$

where $\|\cdot\|$ indicates the Euclidean norm.

The weight updating is given as [19]

$$\mathbf{m}_i(t+1) = \begin{cases} \mathbf{m}_i(t) + \alpha(t) (\mathbf{x}(t) - \mathbf{m}_i(t)) & \text{for } i \in N_c \\ \mathbf{m}_i(t) & \text{otherwise,} \end{cases} \quad (2.23)$$

where $\alpha(t)$ is a positive constant that decays with time and N_c defines a topological neighborhood around the maximally responding neuron c , such that

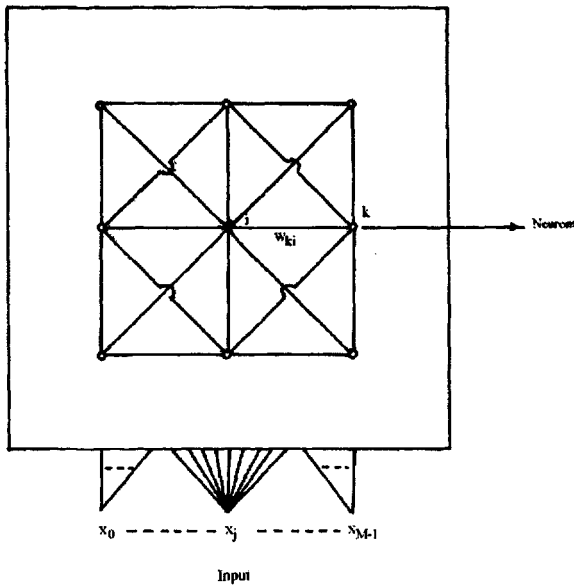


Fig. 2.4 Kohonen neural network.

it also decreases with time. Different parts of the network become selectively sensitized to different inputs in an ordered fashion so as to form a continuous map of the signal space. After a number of sweeps through the training data, with weight updating at each iteration obeying Eq. (2.23), the asymptotic values of m_i cause the output space to attain proper topological ordering. This is basically a variation of *unsupervised* learning.

2.2.3.4 Learning vector quantization (LVQ) Vector quantization can be seen as a mapping from an n -dimensional Euclidean space into a finite set of prototypes. Based on this principle, Kohonen proposed an unsupervised learning algorithm, which is a special case of SOFM and is known as LVQ [19]. In LVQ, only the weight vector associated with the winner node is updated with every data point by Eq. (2.23). The topological neighborhood is not updated here. Such a learning scheme, where all nodes compete to become the winner, is termed *competitive learning*. It is essentially a clustering network that does not care about preserving the topological order. Its main uses are for clustering, classification, and image data compression [32].

There exists a family of LVQs, termed LVQ1 and LVQ2 [19]. These algorithms are supervised learning schemes, essentially used as classifiers. The basic idea behind LVQ1 is as follows. If the winner prototype m_i has the same class label as that of the data point x , then bring m_i closer to x ; otherwise, move m_i away from x . Nonwinner nodes are not updated. LVQ2, a modified

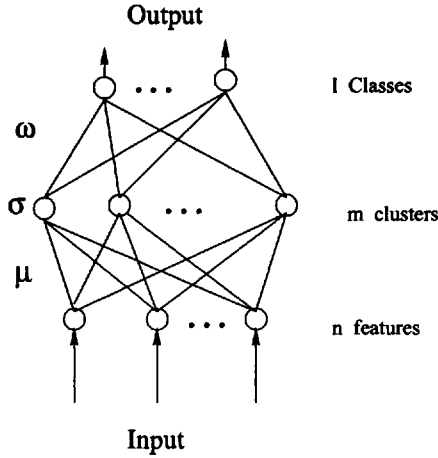


Fig. 2.5 Radial basis function network.

form of LVQ1, is designed to make the learning scheme comply better with Bayes' decision-making philosophy (described in Section 5.3). This algorithm considers the winner along with the runner-up (second winner).

2.2.3.5 Radial basis function network A radial basis function (RBF) network [33, 34] consists of two layers as shown in Fig. 2.5. The connection weight vectors of the input and output layers are denoted as μ and w , respectively. The basis (or kernel) functions in the hidden layer produce a localized response to the input stimulus. The output nodes form a weighted linear combination of the basis functions computed by the hidden nodes.

The input and output nodes correspond to the input features and output classes, while the hidden nodes represent the number of clusters (specified by the user) that partition the input space. Let $\mathbf{x} = (x_1, \dots, x_i, \dots, x_n) \in R^n$ and $\mathbf{y} = (y_1, \dots, y_i, \dots, y_l) \in R^l$ be the input and output, respectively, and let m be the number of hidden nodes.

The output u_j of the j th hidden node, using the *Gaussian kernel* function as a basis, is given by

$$u_j = \exp \left[-\frac{(\mathbf{x} - \mu_j)^T (\mathbf{x} - \mu_j)}{2\sigma_j^2} \right], \quad j = 1, 2, \dots, m, \quad (2.24)$$

where $\mathbf{x}$ is the input pattern, μ_j is its input weight vector (*i.e.*, the center of the Gaussian for node j), and σ_j^2 is the normalization parameter, such that $0 \leq u_j \leq 1$ (the closer the input is to the center of the Gaussian, the larger the response of the node).

The output y_j of the j th output node is

$$y_j = \mathbf{w}_j^T \mathbf{u}, \quad j = 1, 2, \dots, l, \quad (2.25)$$

where w_j is the weight vector for this node, and u is the vector of outputs from the hidden layer. The network performs a linear combination of the nonlinear basis functions of Eq. (2.24).

The problem is to minimize the error

$$E = \frac{1}{2} \sum_{p=1}^N \sum_{j=1}^l (y_{j,p} - d_{j,p})^2, \quad (2.26)$$

where $d_{j,p}$ and $y_{j,p}$ are desired and computed output at the j th node for the p th pattern, N is the size of the data set, and l is the number of output nodes.

Learning in the hidden layer, typically, uses the c -means clustering algorithm (described in Section 6.3.1.1). Let the cluster centers, so determined, be denoted as μ_j , $j = 1, \dots, m$. The normalization parameter σ_j represents a measure of the spread of data associated with each node.

Learning in the output layer is performed after the parameters of the basis functions have been determined. The weights are typically trained using the LMS algorithm given by

$$\Delta w_j = -\varepsilon(y_j - d_j)u, \quad (2.27)$$

where ε is the learning rate.

2.2.4 Neuro-fuzzy computing

The concept of ANNs was inspired by *biological neural networks*, which are inherently nonlinear, adaptive, highly parallel, robust, and fault tolerant. Fuzzy logic, on the other hand, is capable of modeling vagueness, handling uncertainty, and supporting human-type reasoning. One may therefore naturally think of judiciously integrating them by augmenting each other in order to build a more intelligent information system, in *neuro-fuzzy computing* paradigm [2, 35, 36], with recognition performance better than those obtained by the individual technologies. It incorporates both the generic and application-specific merits of ANNs and fuzzy logic into the hybridization.

Both fuzzy systems and ANNs are soft computing approaches to modeling expert behavior. The goal is to mimic the actions of an expert who solves complex problems. A learning process can be part of knowledge acquisition. In the absence of an expert, or sufficient time or data, one can resort to reinforcement learning instead of supervised learning. If one has knowledge expressed as linguistic rules, one can build a fuzzy system. On the other hand, if one has data, or can learn from a simulation or the real task, ANNs are more appropriate. The integration of neural and fuzzy systems leads to a symbiotic relationship, in which fuzzy systems provide a powerful framework for expert knowledge representation, while neural networks provide learning capabilities and suitability for computationally efficient hardware implementations.

It has been proved [37] that (i) any rule-based fuzzy system may be approximated by a neural net, and (ii) any neural net (feedforward, multilayered)

may be approximated by a rule-based fuzzy system. Jang and Sun [38] have shown that fuzzy systems are functionally equivalent to a class of radial basis function (RBF) networks, based on the similarity between the local receptive fields of the network and the membership functions of the fuzzy system.

Fuzzy systems can be broadly categorized into two families. The first includes linguistic models based on collections of IF-THEN rules, whose antecedents and consequents utilize fuzzy values. It uses fuzzy reasoning, and the system behavior can be described in *natural* terms. The *Mamdani* model [39] falls in this group. The knowledge is represented as

$$\begin{array}{ll} R^i : & \text{IF } x_1 \text{ is } A_1^i \text{ AND } x_2 \text{ is } A_2^i \dots \text{ AND } x_n \text{ is } A_m^i \\ & \text{THEN } y^i \text{ is } B^i, \end{array} \quad (2.28)$$

where $R^i (i = 1, 2, \dots, l)$ denotes the i th fuzzy rule, $x_j (j = 1, 2, \dots, n)$ is the input, y^i is the output of the fuzzy rule R^i , and $A_1^i, A_2^i, \dots, A_m^i, B^i (i = 1, 2, \dots, l)$ are fuzzy membership functions usually associated with linguistic terms.

The second category, based on *Sugeno*-type systems [40], uses a rule structure that has fuzzy antecedent and *functional* consequent parts. This can be viewed as the expansion of piecewise linear partition represented as

$$\begin{array}{ll} R^i : & \text{IF } x_1 \text{ is } A_1^i \text{ AND } x_2 \text{ is } A_2^i \dots \text{ AND } x_n \text{ is } A_m^i \\ & \text{THEN } y^i = a_0^i + a_1^i x_1 + \dots + a_n^i x_n. \end{array} \quad (2.29)$$

The approach approximates a nonlinear system with a combination of several linear systems, by decomposing the whole input space into several partial fuzzy spaces and representing each output space with a linear equation. Such models are capable of representing both qualitative and quantitative information and allow relatively easier application of powerful learning techniques for their identification from data. They are capable of approximating any continuous real-valued function on a compact set to any degree of accuracy [41].

There is always a trade-off between readability and precision. If one is interested in a more precise solution, then one is usually not so bothered about its linguistic interpretability. *Sugeno*-type systems are more suitable in such cases. Otherwise, the choice is for *Mamdani*-type systems.

Extraction of rules from neural nets enables humans to understand their prediction process in a better manner. This is because rules are a form of knowledge that human experts can easily verify, transmit, and expand. Representing rules in *natural* form aids in enhancing their comprehensibility for humans. This aspect is suitably handled using fuzzy set-based representations.

Neuro-fuzzy hybridization [2] is done broadly in two ways: a neural network equipped with the capability of handling fuzzy information [termed *fuzzy-neural network* (FNN)], and a fuzzy system augmented by neural networks to enhance some of its characteristics like flexibility, speed, and adaptability [termed *neural-fuzzy system* (NFS)].

In an FNN either the input signals and/or connection weights and/or the outputs are fuzzy subsets or a set of membership values to fuzzy sets (e.g., Refs. [42]–[44]). Usually, linguistic values (such as *low*, *medium*, and *high*) or fuzzy numbers or intervals are used to model these. Neural networks with fuzzy neurons are also termed FNNs because they are capable of processing fuzzy information.

A neural-fuzzy system (NFS), on the other hand, is designed to realize the process of fuzzy reasoning, where the connection weights of the network correspond to the parameters of fuzzy reasoning (e.g., Refs. [45] and [46]). Using the backpropagation-type learning algorithms, the NFS can identify fuzzy rules and learn membership functions of the fuzzy reasoning. Typically, the NFS architecture has distinct nodes for antecedent clauses, conjunction operators, and consequent clauses.

The state of the art for the different techniques of judiciously combining neuro-fuzzy concepts involves synthesis at various levels. In general, these methodologies can be broadly categorized as follows. Note that categories 1 and 3–5 relate to FNNs while category 2 refers to NFS.

1. Incorporating fuzziness into the neural net framework: fuzzifying the input data, assigning fuzzy labels to the training samples, possibly fuzzifying the learning procedure, and obtaining neural network outputs in terms of fuzzy sets [44, 47, 43].
2. Designing neural networks guided by fuzzy logic formalism: designing neural networks to implement fuzzy logic and fuzzy decision-making, and to realize membership functions representing fuzzy sets [48]–[46].
3. Changing the basic characteristics of the neurons: neurons are designed to perform various operations used in fuzzy set theory (like fuzzy union, intersection, aggregation) instead of the standard multiplication and addition operations [49, 50, 51].
4. Using measures of fuzziness as the error or instability of a network: the fuzziness or uncertainty measures of a fuzzy set are used to model the error or instability or energy function of the neural network-based system [52].
5. Making the individual neurons fuzzy: the input and output of the neurons are fuzzy sets and the activity of the networks, involving the fuzzy neurons, is also a fuzzy process [42].

2.2.5 Genetic algorithms

Genetic algorithms (GAs) [53, 54] are adaptive and robust computational search procedures, modeled on the mechanics of natural genetic systems. They act as a biological metaphor and try to emulate some of the processes observed

in natural evolution. While evolution operates on encodings of biological entities in the form of a collection of genes called a chromosome, GAs operate on string representation of possible solutions in terms of individuals or chromosomes containing the features. The feature value, the string structure and the string structure's decoded value in case of a GA correspond to the allele, genotype, and phenotype in natural evolution.

The components of a GA consist of

- Population of individuals
- Encoding or decoding mechanism of the individuals
- Objective function and an associated fitness evaluation criterion
- Selection procedure
- Genetic operators like recombination or crossover, mutation
- Probabilities to perform the genetic operations
- Replacement technique
- Termination conditions

Let us consider, as an example, the optimization of a function

$$y = f(x_1, x_2, \dots, x_p).$$

A binary vector is used as a chromosome to represent real values of the variables x_i , with the length of the vector depending on the required precision. A population is a set of individuals (chromosomes) representing the concatenated parameter set $x_1, x_2, \dots, x_p$, where each member refers to a coded *possible* solution. For example, a sample chromosome

$$0000|0100|\dots|1100$$

could correspond to $x_1 = 0000$, $x_2 = 0100$, and $x_p = 1100$. The chromosomes can be of fixed or variable size. Selection obeys the Darwinian survival of the fittest strategy, with the objective function playing the role of Nature (environment). Variation is introduced in the population through the genetic operations like recombination (crossover) and mutation. Normally the initial population is chosen randomly.

Encoding is used to convert parameter values into chromosomal representation. In case of continuous-valued parameters, a decimal-to-binary conversion is used. For example, using a 5-bit representation, 13 is encoded as 01101. In case of parameters having categorical values, a particular bit position in the chromosomal representation is set to 1 if it comes from a certain category. For example, the gender of a person can have values from {male, female}, such

that male/female is represented by the string 10/01. These strings (representing the parameters of a problem) are concatenated to form a chromosome.

Decoding is the reverse of encoding. For a continuous-valued parameter the binary representation is converted to a continuous value by the expression

$$\text{lower_bound} + \frac{\sum_{i=0}^{\text{bits_used}-1} \text{bit}_i * 2^i}{2^{\text{bits_used}} - 1} * (\text{upper_bound} - \text{lower_bound}).$$

Hence 01101 in five bits (*bits_used*) is decoded back to 13, using *lower_bound* = 0 and *upper_bound* = 31. In case of categorical parameters, the value is found by consulting the original mapping.

The fitness function provides a measure of a chromosome's performance. Selection gives more chance to better-fitted individuals, thereby mimicking the natural selection procedure. Some of the popular selection techniques include roulette wheel selection, stochastic universal sampling, linear normalization selection, and tournament selection. The roulette wheel selection procedure initially sums the fitness values (f_i s) of all the N chromosomes in the population, and it stores them in slots sized accordingly. Let this sum be given by *total_fitness*. The probability of selection p_i for the i th chromosome is expressed as

$$p_i = \frac{f_i}{\text{total_fitness}}, \quad (2.30)$$

while the cumulative probability q_i after inclusion of the i th chromosome is given by

$$q_i = \sum_{j=1}^i p_j. \quad (2.31)$$

Selection is made by spinning the roulette wheel N times, on each occasion generating a random number n_r in $[0, \text{total_fitness}]$. This returns the first chromosome whose fitness, when added to the fitness of the preceding population members, is greater than or equal to n_r . In rule form, we have

IF $n_r < q_1$ THEN select the first chromosome,
ELSE select the i th chromosome such that $q_{i-1} < n_r \leq q_i$.

For example, let there be five chromosomes with fitness values 40, 30, 18, 10, 2, having *total_fitness* = 100. These constitute slots sized 40%, 30%, 18%, 10%, and 2% of the area of the wheel. Each time one requires to select a chromosome, for applying crossover or mutation, a simple spin of the roulette wheel is made with n_r . Here, with $n_r = 45$, the algorithm selects the second chromosome, since $40 + 30 > 45$.

Recombination or crossover is modeled by choosing mating pairs from the selected chromosomes. Crossover probability p_c is used to determine whether a pair should be crossed over, and then the corresponding chromosome segments are interchanged. A random number n_{rc} is generated in the range $[0, 1]$. If

$n_{rc} < p_c$, the corresponding chromosome pair is selected for crossover. Again, crossover can be one point, two point, multipoint, or uniform. Let us consider, as an example, two parent chromosomes $xyxyxyxy$ and $abababab$ where x, y, a, b are binary. In one-point crossover at the 6th bit involving the parent chromosomes

$$\begin{array}{l} xyxyx|yxy \\ ababa|bab, \end{array}$$

one generates the children

$$\begin{array}{l} xyxyx|bab \\ ababa|yxy. \end{array}$$

Here the segment involving bits 6 to 8 is interchanged between the parents. In case of two-point crossover at the 3rd and 6th bits, involving parent chromosomes

$$\begin{array}{l} xy|xyx|yxy \\ ab|aba|bab, \end{array}$$

we obtain the children chromosomes

$$\begin{array}{l} xy|aba|yxy \\ ab|xyx|bab. \end{array}$$

Here the segment constituting bits 3 to 5 is swapped between the parents to generate the pair of offsprings.

Mutation is used to introduce diversity in the population. Mutation probability p_m determines whether a bit should be mutated, and then the corresponding location is flipped. For example, a mutation at the 3rd bit would transform the chromosome $00|1|000$ to $00|0|000$. Probabilities p_c and p_m can be fixed or variable, and they typically have values ranging between 0.6 to 0.9, and 0.001 to 0.01, respectively.

Let us consider a simple example related to minimizing the surface area A of a solid cylinder, given radius r and height h , to illustrate the working principle of GAs. Here the fitness function can be expressed as

$$A = 2\pi * r * h + 2\pi * r^2 = 2\pi * r(h + r).$$

We need to encode the parameters r and h in a chromosome. Using a 3-bit representation, we demonstrate encoding, crossover, and mutation. For $r_1 = 3$, $h_1 = 4$ and $r_2 = 4$, $h_2 = 3$, we generate parent chromosomes $011|100$ and $100|011$ with $A_1 = 132$, $A_2 = 176$, respectively. Let there be one-point crossover at bit 4, producing the children chromosomes $011|011$ and $100|100$. This is decoded as $r_{1c} = 3$, $h_{1c} = 3$ and $r_{2c} = 4$, $h_{2c} = 4$, with $A_{1c} = 16.16$ and $A_{2c} = 28.72$, respectively. Now, let there be mutation at bit 5 of the first child. This generates the chromosome $0110|01$, for $r_{1cm} = 3$ and $h_{1cm} = 1$,

with $A_{1cm} = 10.77$. This is the minimum value of fitness obtained thus far. Consecutive applications of the genetic operations of selection, crossover, and mutation, up to termination, enable the minimization (optimization) of the chosen fitness function.

The replacement techniques can be

1. Generational, where all the n individuals are replaced at a time by the n children created by reproduction. *Elitism* is often introduced to retain the best solution obtained so far.
2. Steady state, where $m < n$ members are replaced at a time by the m children reproduced.

The terminating criterion for the algorithm can be on the basis of

- execution for a fixed number of generations or iterations,
- a bound on the fitness value of the generated solution, or
- acquiring of a certain degree of homogeneity by the population.

GAs have been applied in diverse problems involving optimization, scheduling, graph coloring, genetic programming, pattern recognition, image processing, data mining, artificial immune systems, and financial prediction or bidding strategies.

2.2.6 Rough sets

The theory of *rough sets* [55] has recently emerged as another major mathematical tool for managing uncertainty that arises from granularity in the domain of discourse – that is, from the indiscernibility between objects in a set. The intention is to approximate a *rough* (imprecise) concept in the domain of discourse by a pair of *exact* concepts, called the lower and upper approximations. These exact concepts are determined by an *indiscernibility* relation on the domain, which, in turn, may be induced by a given set of *attributes* ascribed to the objects of the domain. The lower approximation is the set of objects definitely belonging to the vague concept, whereas the upper approximation is the set of objects possibly belonging to the same. These approximations are used to define the notions of *discernibility matrices*, *discernibility functions*, *reducts*, and *dependency factors*, all of which play a fundamental role in the reduction of knowledge. Figure 2.6 provides a schematic diagram of a rough set. Let us now present some requisite preliminaries of rough set theory.

An *information system* is a pair $S = \langle U, A \rangle$, where U is a nonempty finite set called the *universe* and A is a nonempty finite set of *attributes* $\{a\}$. An attribute a in A can be regarded as a function from the domain U to some value set V_a .

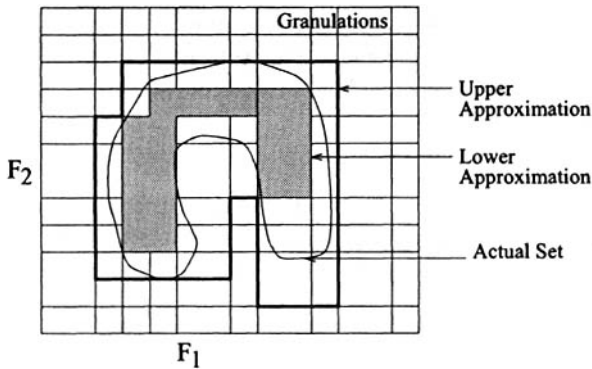


Fig. 2.6 Lower and upper approximations in a rough set.

With every subset of attributes $B \subseteq A$, one can easily associate an equivalence relation I_B on U :

$$I_B = \{(x, y) \in U : \text{for every } a \in B, a(x) = a(y)\}.$$

Then $I_B = \bigcap_{a \in B} I_a$.

If $X \subseteq U$, the sets $\{x \in U : [x]_B \subseteq X\}$ and $\{x \in U : [x]_B \cap X \neq \emptyset\}$, where $[x]_B$ denotes the equivalence class of the object $x \in U$ relative to I_B , are called the *B-lower* and *B-upper approximations* of X in S and denoted by $\underline{B}X$ and $\overline{B}X$, respectively.

$X (\subseteq U)$ is *B-exact* or *B-definable* in S if $\underline{B}X = \overline{B}X$. It may be observed that $\underline{B}X$ is the greatest *B-definable* set contained in X , and $\overline{B}X$ is the smallest *B-definable* set containing X .

Let us consider, for example, an *information system* $\langle U, \{a\} \rangle$ where the domain U consists of the students of a school, and there is a single attribute a – that of “belonging to a class.” Then U is partitioned by the classes of the school.

Now consider the situation when an infectious disease has spread in the school, and the authorities take the two following steps.

1. If at least one student of a class is infected, all the students of that class are vaccinated. Let $\overline{B}$ denote the union of such classes.
2. If every student of a class is infected, the class is temporarily suspended. Let $\underline{B}$ denote the union of such classes.

Then $\underline{B} \subseteq \overline{B}$. Given this information, let the following problem be posed:

- *Identify the collection of infected students.* Clearly, there cannot be a unique answer. But any set I that is given as an answer must contain $\underline{B}$ and at least one student from each class comprising $\overline{B}$. In other words, it must have $\underline{B}$ as its *lower approximation* and $\overline{B}$ as its *upper approximation*.
- I is then a *rough concept* or set in the information system $\langle U, \{a\} \rangle$. Further, it may be observed that any set I' given as another answer is *roughly*

equal to I , in the sense that both are represented (characterized) by $\overline{B}$ and $\underline{B}$.

The effectiveness of the theory of rough sets has been investigated in the domains of artificial intelligence and cognitive sciences, especially for representation of and reasoning with vague and/or imprecise knowledge, data classification and analysis, machine learning, and knowledge discovery [56]. Their role in data mining is elucidated in Section 2.6, with particular reference to rough clustering in Section 6.5.4.

2.2.7 Wavelets

Application of wavelets have had a growing impact in signal and image processing over the last two decades. But *wavelet* is by no means a new theory, and it existed in mathematics since 1909 when Haar discovered the Haar transform. Since then, mathematicians have been working on wavelets, and “wavelet analysis” used to be called “atomic decomposition” for a long time [57]. The *wave* in physics is defined as a disturbance propagated in media, typically as an oscillating function of time or space such as a sinusoid. The *wavelet* can be considered a snapshot of a wave oscillating within a short window of time or space. As a result, mathematically, the wavelet can be considered as a function which is both oscillating and localized.

Representation of a signal using sinusoids is very effective for *stationary signals*, which are statistically predictable and are time-invariant in nature. *Wavelet* representation is found to be very effective for *nonstationary* signals, which are not statistically predictable and time-varying in nature.

Variation of intensity to form edges is a very important visual characteristic of an image. From signal theoretic perspective, discontinuities of intensities occur at the edges in any image and hence it can be prominently visualized by the human eye. The time and frequency localization property of wavelets makes it attractive for analysis of images because of discontinuities at the edges.

Wavelets are functions generated from one single function called the *mother wavelet* by dilations (scalings) and translations (shifts) in time (frequency) domain. If the mother wavelet is denoted by $\psi(t)$, the other wavelets $\psi^{a,b}(t)$ for $a > 0$ and a real number b can be represented as

$$\psi^{a,b}(t) = \frac{1}{\sqrt{a}} \psi \left(\frac{t-b}{a} \right), \quad (2.32)$$

where a and b represent the parameters for dilations and translations in the time domain. The parameter a causes contraction in time domain when $a < 1$ and expansion when $a > 1$. In Fig. 2.7, we illustrate a mother wavelet and its contraction and dilation.

We discuss further details of wavelet transformation and its properties in Section 3.8.3, and we describe how it can be applied for efficient image compression. Its application to data clustering is provided in Section 6.5.3.

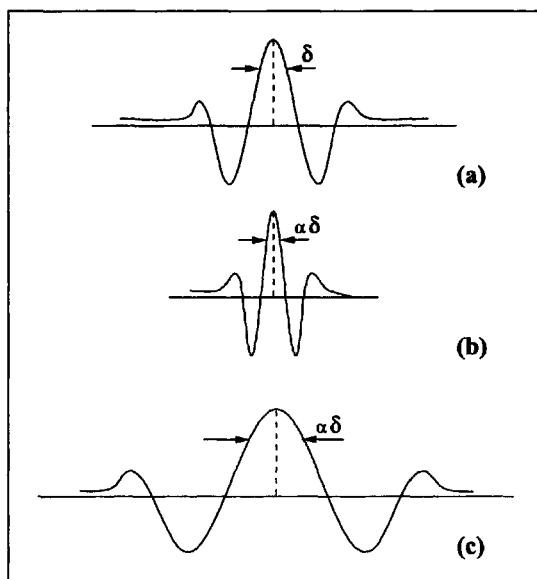


Fig. 2.7 (a) Mother wavelet $\psi(t)$. (b) $\psi(t/\alpha)$: $0 < \alpha < 1$. (c) $\psi(t/\alpha)$: $\alpha > 1$.

2.3 ROLE OF FUZZY SETS IN DATA MINING

Fuzzy sets constitute the earliest and most widely reported constituent of soft computing. As mentioned in Section 2.2.2, the modeling of imprecise and qualitative knowledge as well as the transmission and handling of uncertainty at various stages are possible through the use of fuzzy sets. In this section we provide a glimpse of the available literature pertaining to the use of fuzzy sets in data mining [3].

Knowledge discovery in databases is mainly concerned with identifying interesting patterns and describing them in a concise and meaningful manner [58]. Despite a growing versatility of knowledge discovery systems, there is an important component of human interaction that is inherent to any process of knowledge representation, manipulation, and processing. Fuzzy sets are naturally inclined towards coping with linguistic domain knowledge and producing more interpretable solutions.

The notion of *interestingness*, which encompasses several features such as validity, novelty, usefulness, and simplicity, can be quantified through fuzzy sets. Fuzzy dissimilarity of a discovered pattern with a user-defined vocabulary has been used as a measure of this interestingness [59]. As an extension to the above methodology, *unexpectedness* can also be defined in terms of a *belief system*, where if a belief b is based on previous evidence ξ , then $d(b|\xi)$ denotes the degree of belief b . In soft belief systems, a weight w_i is attached to each belief b_i . The degree of a belief may be measured with conditional

probability, Dempster–Shafer belief function, or frequency of the raw data. Here, the interestingness of a pattern X relative to a belief system B and evidence ξ may be formally defined as

$$I(X, B, \xi) = \sum_{b_i \in B} w_i |d(b_i|X, \xi) - d(b_i|\xi)|. \quad (2.33)$$

This definition of interestingness measures the amount by which the degrees of belief change as a result of a new pattern X .

There is a growing indisputable role of fuzzy set technology in the realm of data mining [60]. Various data browsers have been implemented using fuzzy set theory [61]. Analysis of real-world data in data mining often necessitates simultaneous dealing with different types of variables, namely, categorical, symbolic, and numerical data. Pedrycz [62] discusses some constructive and fuzzy set-driven computational vehicles of knowledge discovery and establishes the relationship between data mining and fuzzy modeling. The role of fuzzy sets is categorized below based on the different functions of data mining that are modeled.

2.3.1 Clustering

Data mining aims at sifting through large volumes of data in order to reveal useful information in the form of new relationships, patterns, or clusters, for decision-making by a user [63]. Fuzzy sets support a focused search, specified in linguistic terms, through data. They also help discover dependencies between the data in qualitative or semiquantitative format. In data mining, one is typically interested in a focused discovery of structure and an eventual quantification of functional dependencies existing therein. This helps prevent searching for meaningless or trivial patterns in a database. Researchers have developed fuzzy clustering algorithms for this purpose [64]. Russell and Lodwick [65] have explored fuzzy clustering methods for mining telecommunications customer and prospect databases to gain residential and business customer market share.

Pedrycz has designed fuzzy clustering algorithms [66] using (a) contextual information and (b) induced linguistic space, for better focusing of the search procedure in KDD. Krishnapuram et al. [67] have developed a robust fuzzy c -medoids algorithm for clustering Web data. Further details on these algorithms are provided in Section 6.5.1.

2.3.2 Granular computing

Achieving focus is important in data mining because there are too many attributes and values to be considered, which can result in combinatorial explosion. Most unsupervised data mining approaches try to achieve attribute focus by first recognizing the most interesting features. Mazlack [68] suggests

a converse approach of progressively reducing the dataset by partitioning and eliminating the least important attributes to reduce intra-item dissonance within the partitions. A *soft* focus is used to handle both crisp and imprecise data. It works by progressive reduction of cognitive dissonance, leading to an increase in useful information. The objective is to generate cohesive and comprehensible information *nuggets* by sifting out uninteresting attributes. A combined distance metric takes care of different types of attributes simultaneously, thus avoiding any taxonomic structure. Non-crisp values are handled by granularization followed by partitioning.

Granular computing [69] is useful in finding meaningful patterns in data by expressing and processing chunks of information (granules). These are regarded as essential entities in all cognitive pursuits geared toward establishing meaningful patterns in data. Soft granules can be defined in terms of membership functions. Increased granularity reduces attribute distinctiveness, resulting in loss of useful information, while finer grains lead to partitioning difficulty. The concept of granular computing allows one to concentrate all computational effort on some specific and problem-oriented subsets of a complete database. It also helps split an overall computing effort into several subtasks, leading to a *modularization* effect.

We deal with the classification aspect of granular computing in Section 5.4.6. Modularization in soft computing, for data mining, is described in Section 8.3.

2.3.3 Association rules

An important area of data mining research deals with the discovery of *association rules* [70], which describe interesting association relationship among different attributes. A boolean association involves binary attributes, a generalized association involves attributes that are hierarchically related, and a quantitative association involves attributes that can take on quantitative or categorical values. The use of fuzzy techniques has been considered to be one of the key components of data mining systems because of their affinity with human knowledge representation [71].

Wei and Chen [72] have mined generalized association rules with fuzzy taxonomic structures. A crisp taxonomy assumes that a child belongs to its ancestor with degree one. A fuzzy taxonomy is represented as a directed acyclic graph, each of whose edges represents a fuzzy *IS-A* relationship with degree μ ($0 \leq \mu \leq 1$). The partial belonging of an item in a taxonomy is taken into account while computing the degrees of support and confidence.

Au and Chan [73] utilize an *adjusted difference* between observed and expected frequency counts of attributes for discovering fuzzy association rules in relational databases. Instead of dividing quantitative attributes into fixed intervals, they employ linguistic terms to represent the revealed regularities and exceptions. Here no user-supplied thresholds are required, and quantitative values can be directly inferred from the rules. The linguistic representation leads to the discovery of *natural* and more understandable rules. The algo-

rithm allows one to discover both *positive* and *negative* rules, and can deal with fuzzy class boundaries as well as missing values in databases. The use of fuzzy techniques buries the boundaries of adjacent intervals of numeric quantities, resulting in resilience to noises such as inaccuracies in physical measurements of real life entities. The effectiveness of the algorithm was demonstrated on a transactional database of a PBX system and a database concerning industrial enterprises in mainland China.

We describe fuzzy association rules in greater detail in Section 7.10.

2.3.4 Functional dependencies

Fuzzy logic has been used for analyzing inference based on functional dependencies (FDs), between variables, in database relations. Fuzzy inference generalizes both imprecise (set-valued) and precise inference. Similarly, fuzzy relational databases generalize their classical and imprecise counterparts by supporting fuzzy information storage and retrieval [74]. FDs are interesting from knowledge discovery standpoint since they allow one to express, in a condensed form, some properties of the real world which are valid on a given database. These properties can then be used in various applications such as reverse engineering or query optimization. Bosc et al. [75] use a data mining algorithm to extract or discover extended FDs, represented by gradual rules composed of linguistic variables.

2.3.5 Data summarization

Summary discovery is one of the major components of knowledge discovery in databases. This provides the user with comprehensive information for grasping the essence from a large amount of information in a database. Fuzzy set theory is also used for data summarization [76]. Typically, fuzzy sets are used for an interactive top-down summary discovery process which utilizes fuzzy *IS-A* hierarchies as domain knowledge.

Linguistic summaries of large sets of data are derived as linguistically quantified propositions with a degree of validity [77]. This corresponds to the preference criterion involved in the mining task. The system consists of a summarizer (like *young*), a quantity in agreement (like *most*), and the truth or validity (say, 0.7).

It is found that often the most interesting linguistic summaries are non-trivial and human-consistent concepts, involving complicated combinations of attributes. In practice, this cannot be generated automatically and *human assistance or interaction* is required. Kacprzyk and Zadrozny [78] have developed *FQUERY* for an interactive linguistic summarization, using *natural* terms and *comprehensible* quantifiers. It supports various fuzzy elements in queries, including interval attributes with membership for matching in a fuzzy relation and importance coefficients. First the user has to formulate a

set of linguistic summaries of interest. The system then retrieves records from the database and calculates the validity of each summary. Finally, the most appropriate linguistic summary is selected. The scheme has also been used for fuzzy querying over the Internet, using browsers like Microsoft Explorer or Netscape Navigator. The definition of fuzzy values, fuzzy relations, and linguistic quantifiers is via Java applets.

Chiang et al. [79] have used fuzzy linguistic summary for mining time series data. The system provides human interaction, in the form of a graphic display tool, to help users premine a database and determine what knowledge could be discovered. The model is used to predict the on-line utilization ranks of different resources, including CPU and real storage.

2.3.6 Image mining

Recent increase in the size of *multimedia* information repositories, consisting of mixed media data, has made content-based image retrieval (CBIR) an active research area. Unlike traditional database techniques which retrieve images based on exact matching of keywords, CBIR systems represent the information content of an image by visual features such as color, texture, and shape, and they retrieve images based on similarity of features. Frigui [80] has developed an *interactive* and *iterative* image retrieval system that takes into account the *subjectivity* of human perception of visual content. The feature relevance weights are learned from the user's positive and negative feedback, and the Choquet integral is used as a dissimilarity measure. The smooth transition in the user's feedback is modeled by continuous fuzzy membership functions. Medasani and Krishnapuram [81] have designed a fuzzy approach to handle complex linguistic queries consisting of multiple attributes. Such queries are usually more *natural*, *user-friendly*, and *interpretable* for image retrieval. The degree to which an image satisfies an attribute is given by the membership value of the feature vector, corresponding to the image, in the membership function for the attribute. Fuzzy connectives are used to combine the degrees of satisfaction of multiple attributes in a complex query, to arrive at an overall degree of satisfaction while ranking images for retrieval.

Video as the format of computer-related material is becoming more and more common these days, and many Web pages involve small pieces of movies or video-clips or animation. Fuzzy time related queries are used in Ref. [82] to retrieve information inside a video. The queries are handled using Zadeh's principle of *computing with words*, which allows a human-friendly interface. The system is implemented on a Java Search Engine.

Querying for a target image and retrieving it from Web and image databases, based on image similarity, is presented in Ref. [83]. A fuzzy *c*-means algorithm is used to cluster intrinsic image characteristics extracted from subregions of the image. A measure of similarity between pairs of images is determined in terms of the rotation-invariant attributes like color, texture, and shape. Color is defined [84] in terms of the hue, saturation, and value representation of the

average color of the pixel in the region. Texture is represented in terms of the co-occurrence matrices in four directions involving Haralick's parameters [85]. For each database image, the system calculates its attribute matrix and does a whitening transformation before fuzzy clustering to store the representative centroids. When a target image is supplied, the system adopts a similar procedure in order to retrieve the most similar image (in terms of the stored centroid) from the database in response to a query.

In this section we have briefly summarized some research on soft computing-based image mining. The traditional image mining techniques, based on context-based image retrieval systems, have been covered in Section 9.3.

2.4 ROLE OF NEURAL NETWORKS IN DATA MINING

Neural networks were earlier thought to be unsuitable for data mining because of their inherent *black-box* nature. No information was available from them in symbolic form, suitable for verification or interpretation by humans. However, recent investigations have concentrated on extracting the embedded knowledge in trained networks in the form of symbolic rules [25]. Unlike fuzzy sets, the main contribution of neural nets towards data mining stems from rule extraction and clustering [3].

2.4.1 Rule extraction

In general, the primary input to a connectionist rule extraction algorithm is a representation of a trained (layered) neural network, in terms of its nodes, links, and sometimes the dataset. One or more hidden and output units are used to automatically derive the rules, which may later be combined and simplified to arrive at a more comprehensible rule set. These rules can also provide new insights into the application domain. The use of neural nets helps in (i) incorporating parallelism and (ii) tackling optimization problems in the data domain. The models are usually suitable in *data-rich* environments.

Typically, a network is first trained to achieve the required accuracy rate. Redundant connections of the network are then removed using a pruning algorithm. The link weights and activation values of the hidden units in the network are analyzed, and classification rules are generated [25, 86]. Further details on rule generation can be obtained in Section 8.2.

2.4.2 Rule evaluation

Here we provide some quantitative measures to evaluate the performance of the generated rules [87]. This relates to the *goodness of fit* chosen for the rules. Let the (i, j) th element of an $l \times l$ matrix, n_{ij} , indicate the number of objects (patterns) actually belonging to class i , but classified as class j .

- **Accuracy:** It is the correct classification percentage, provided by the rules on a test set defined as

$$\frac{n_{ic}}{n_i} * 100,$$

where n_i is equal to the number of objects in class i such that n_{ic} of these are correctly classified.

- **User's accuracy:** It gives a measure of the confidence that a classifier attributes to a region as belonging to a class. If n'_i objects are found to be classified into class i , then the user's accuracy (U) is defined as

$$U = \frac{n_{ic}}{n'_i} * 100.$$

In other words, it denotes the level of purity associated with a region.

- **Kappa:** The coefficient of agreement, kappa, measures the relationship of beyond chance agreement to expected disagreement. It uses all the cells in the confusion matrix, not just the diagonal elements. The kappa value for class i (K_i) is defined as

$$K_i = \frac{N.n_{ic} - n_i.n'_i}{N.n'_i - n_i.n'_i}, \quad (2.34)$$

where N indicates the total number of data samples. The estimate of kappa is the proportion of agreement, after chance agreement is removed from consideration. The numerator and denominator of overall kappa are obtained by summing the respective numerators and denominators of K_i separately over all classes.

- **Fidelity:** It is measured as the percentage of the test set for which network and the rulebase output agree [87].
- **Confusion:** This measure quantifies the goal that the "confusion should be restricted within minimum number of classes." Let $\hat{n}_{ij}$ be the mean of all n_{ij} for $i \neq j$. Then [87]

$$Conf = \frac{Card\{n_{ij} : n_{ij} \geq \hat{n}_{ij}, i \neq j\}}{l} \quad (2.35)$$

for an l class problem. The lower the value of confusion, the smaller the number of classes between which confusion occurs.

- **Coverage:** The percentage of examples from a test set for which *no rules are fired* is used as a measure of the uncovered region. A rulebase having a smaller uncovered region is superior.

- *Rulebase size*: This is measured in terms of the number of rules. The lower its value, the more compact the rulebase. This leads to better understandability.
- *Computational complexity*: This is measured in terms of the CPU time required.
- *Confidence*: The confidence of the rules is defined by a confidence factor *cf*. We use [87]

$$cf_j = \inf_{j: \text{all nodes in the path}} \frac{(\sum_i w_{ji} - \theta_j)}{\sum_i w_{ji}}, \quad (2.36)$$

where w_{ji} is the i th incoming link weight to node j and θ_j is its threshold.

2.4.3 Clustering and self-organization

One of the big challenges to data mining is the organization and retrieval of documents from archives. Kohonen et al. [88] have demonstrated the utility of a huge self-organizing map (SOM) with more than one million nodes to partition a little less than seven million patent abstracts, where the documents are represented by 500-dimensional feature vectors. Very large text collections have been automatically organized into *document maps* that are suitable for visualization and intuitive exploration of the information space. Vesanto et al. [89] employ a stepwise strategy by partitioning the data with a SOM, followed by its clustering. Alahakoon et al. [90] perform hierarchical clustering of SOMs, based on a spread factor which is independent of the dimensionality of the data. Further details of these algorithms are provided in Section 6.5.2.

Shalvi and De Claris [91] have designed a data mining technique, combining Kohonen's self-organizing neural network with data visualization, for clustering a set of pathological data containing information regarding the patients' drugs, topographies (body locations) and morphologies (physiological abnormalities). Koenig [92] has combined SOM and Sammon's nonlinear mapping for reducing the dimension of data representation for visualization purposes.

2.4.4 Regression

Neural networks have also been used for a variety of classification and regression tasks [93]. Time series prediction has been attempted by Lee and Liu [94]. They have employed a neural oscillatory elastic graph matching model, with hybrid radial basis functions, for tropical cyclone identification and tracking.

2.4.5 Information retrieval

The SOM has been used for information retrieval [95]. A map of text documents arranged using the SOM is organized in a meaningful manner, so that

items with similar content appear at nearby locations of the two-dimensional map display, such that the data is clustered. This results in an approximate model of the data distribution in the high-dimensional document space. A document map is automatically organized for browsing and visualization, and it is successfully utilized in speeding up document retrieval while maintaining high perceived quality. The objective of the search is to locate a small number N' of best documents in the order of goodness corresponding to a query. The strategy is outlined below.

- Indexing phase: Apply the SOM to partition a document collection of D documents into K subsets or clusters, representing each subset by its centroid.
- Search phase: For a given query,
 - Pre-select: select the best subsets, based on comparison with the centroids, and collect the documents in these subsets until K' documents ($K' \geq N'$) are obtained.
 - Refine: perform an exhaustive search among the K' prospective documents and return the N' best ones in the order of goodness.

A collection of 1460 document vectors was organized on a 10×15 SOM, using the WEBSOM principles, so that each map unit could contain an average of 10 documents.

2.5 **ROLE OF GENETIC ALGORITHMS IN DATA MINING**

GAs are adaptive, robust, efficient and global search methods, suitable in situations where the search space is large. They optimize a *fitness function*, corresponding to the preference criterion of data mining, to arrive at an optimal solution using certain genetic operators. Knowledge discovery systems have been developed using genetic programming concepts [96, 97]. The *MAS-SON* system [98], where intentional information is extracted for a given set of objects, is popular. The problem addressed is to find common characteristics of a set of objects in an object-oriented database. Genetic programming is used to automatically generate, evaluate, and select object-oriented queries. GAs are also used for several other purposes like fusion of multiple datatypes in multimedia databases, as well as automated program generation for mining multimedia data [99].

However, the literature in the domain of GA-based data mining is not as rich as that of fuzzy sets. We provide below a categorization of few such interesting systems based on the functions modeled [3].

2.5.1 Regression

Besides discovering human-interpretable patterns, data mining also encompasses prediction [58], where some variables or attributes in the database are used to determine unknown or future values of other variables of interest. The traditional weighted average or linear multiregression models for prediction require a basic assumption that there is no interaction among the attributes. GAs, on the other hand, are able to handle attribute interaction in a better manner. Xu et al. [100] have designed a multi-input single-output system using a nonlinear integral. An adaptive GA is used for learning the nonlinear multiregression from a set of training data.

Noda et al. [101] use GAs to discover *interesting* rules in a dependence modeling task, where different rules can predict different goal attributes. Generally, attributes with high information gain are good predictors of a class when considered individually. However attributes with low information gain could become more relevant when attribute interactions are taken into account. This phenomenon is associated with rule interestingness. The degree of interestingness of the consequent is computed based on the relative frequency of the value being predicted by it. In other words, the rarer the value of a goal attribute, the more interesting a rule it predicts. The authors attempt to discover (or mine) a few interesting rules (knowledge nuggets) instead of a large set of accurate (but not necessarily interesting) rules. The concept of interestingness has been represented using Eq. (2.33). It is also discussed in Section 7.4, with reference to association rule mining.

2.5.2 Association rules

Multiobjective GAs deal with finding the optimal solutions to problems having multiple objective functions or constraints. The solution to this is a *Pareto optimal* set of solutions, such that there exists no solution in the search space which dominates any member of this set. These are used for rule mining, which often involves a large search space with huge number of attributes and records. A global search is performed with multiple objectives, involving a combination of factors like predictive accuracy, comprehensibility, and interestingness.

Rules are typically encoded in two ways, namely, the Michigan approach (each individual encoding a single rule) and the Pittsburgh approach (each individual encoding a set of rules). The antecedent and consequent parts are encoded separately. To avoid generation of invalid chromosomes during crossover, some alignment is necessary in the position of the different attribute values. While a categorical attribute is represented by its value, a continuous-valued attribute is encoded by its binary representation. Selection and mutation operators are the same as in standard GAs. Although conventional crossover may be used, one can also resort to generalized or specialized crossover operators that involve logical OR/AND operations over the appropriate segment of the parent chromosomes.

Lopes et al. [102] evolve association rules of IF C THEN P type, which provide a high degree of accuracy and coverage. While the *accuracy* of a rule measures its degree of confidence, its *coverage* is interpreted as the comprehensive inclusion of all the records that satisfy the rule. Hence

$$\text{Accuracy} = \frac{|C \cap P|}{|C \cap P| + |C \cap \bar{P}|} \quad (2.37)$$

and

$$\text{Coverage} = \frac{|C \cap P|}{|C \cap P| + |\bar{C} \cap P|} \quad (2.38)$$

are defined. Note that other quantitative measures for rule evaluation have been discussed in Section 2.4.2, with reference to neural networks.

2.6 **ROLE OF ROUGH SETS IN DATA MINING**

The theory of rough sets [55] has proved to be useful in a variety of KDD processes. It offers mathematical tools to discover hidden patterns in data; therefore its importance, as far as data mining is concerned, can in no way be overlooked [3]. A fundamental principle of a rough set-based learning system is to discover redundancies and dependencies between the given features of a problem to be classified. It approximates a given concept from below and from above, using *lower* and *upper approximations*.

A rough set learning algorithm can be used to obtain a set of rules in IF-THEN form from a *decision table*. Every decision rule has two conditional probabilities associated with it, namely, certainty and coverage factors. These are closely related to the fundamental concepts of lower and upper approximations [103]. The rough set method provides an effective tool for extracting knowledge from databases. Here one first creates a knowledge base, classifying objects and attributes within the created decision tables. Then a knowledge discovery process is initiated to remove some undesirable attributes. Finally the data dependency is analyzed, in the reduced database, to find the minimal subset of attributes called *reduct*.

Rough set applications to data mining generally proceed along the following directions.

1. *Decision rule induction from attribute value table* [104]–[106]. Most of these methods are based on generation of discernibility matrices and reducts.
2. *Data filtration by template generation* [107]. This mainly involves extracting elementary blocks from data based on equivalence relation. Genetic algorithms are also sometimes used in this stage for searching, so that the methodologies can be used for large datasets.

Besides these, reduction of memory and computational requirements for rule generation, and working on dynamic databases [106] are also considered.

Some of the rough set-based systems developed for data mining include (i) the KDD-R system based on the VPRS (Variable Precision Rough Set) model [108] and (ii) the rule induction system based on LERS (Learning from Examples based on Rough Set Theory) [109]. LERS has been extended in Ref. [110] to handle *missing* attributes using the closest fit.

Document clustering has been recognized as a means for improving the efficiency and effectiveness of information retrieval and text mining. A non-hierarchical document clustering algorithm [111], based on a tolerance rough set model, has been applied to large document databases characterized by a few index terms or keywords. Unlike hierarchical algorithms, requiring time and space complexities of $O(N^3)$ and $O(N^2)$ respectively (with N being the total number of terms in a textual database), this approach requires complexities of $O(N \log N)$ and $O(N)$. The concept of upper approximation in rough sets makes it possible to exploit the semantic relationship between a few index terms in a large text document.

2.7 ROLE OF WAVELETS IN DATA MINING

Role of wavelets in different aspects of data mining is gaining significant importance. Today it has become a very powerful signal processing tool in different application areas such as image processing, compression, image indexing and retrieval, digital libraries, image clustering, and databases [112]–[116].

Spatial data mining aims to handle the huge amounts of spatial data obtained from satellite images, medical equipments, Geographic Information Systems (GIS), image database exploration, etc. The objective is to automate the process of understanding spatial data by concise representation and reorganization, to accommodate data semantics. Clustering is often required at hierarchical levels of coarseness, grouping the spatial objects at different levels of accuracy. This gives rise to the concept of multiresolution representation of an image. Wavelets [5] are found to be very useful in appropriately modeling such situations because of the nonstationary property of the image signals formed around the edges and correlation amongst the image pixels.

Wavelet transform is a signal processing technique that decomposes a signal or image into different frequency subbands at number of levels and multiple resolutions. In every level of decomposition, the high-frequency subband captures the discontinuities in the signals – for example, the edge information in an image. The low-frequency subband is nothing but a subsampled version of the original image, with similar statistical and spatial properties as the original signal. As a result, the low-frequency subband can be further decomposed into higher levels of resolution, and it helps in representing spatial objects in different coarser levels of accuracy in multiresolution subbands. This property

led to the application of wavelet transforms in edge detection, object isolation, object detection, medical image fusion, and others [117].

One has to apply the wavelet transform on the feature space to find dense regions or clusters. Wavelet transform is not a single uniquely defined mathematical function like the Discrete Cosine Transform. There are many wavelet basis functions available in the literature. The wavelet transform is usually represented as a pair of Finite Impulse Response (FIR) filters, namely, the high-pass filter and the low-pass filter. A one-dimensional signal s can be filtered by convolving the filter coefficients c_k of such a filter with the signal values

$$\hat{s}_i = \sum_{k=1}^M c_k s_{i+k} - \frac{M}{2}, \quad (2.39)$$

as an example, where M is the number of coefficients in the filter and $\hat{s}$ is the result of the convolution. The *Cohen–Daubechies–Feauveau* (2,2) biorthogonal wavelet is one of the most commonly used wavelet transform in data clustering applications. It is a hat-shaped filter that emphasizes regions where points cluster, while suppressing the weaker information along their boundary. This makes it easier to find the connected components in the transformed space.

Wavelets have been used for efficiently clustering large datasets [116]. This is discussed in Section 6.5.3. We provide further details on wavelet transformation and its application to image compression in Section 3.8.3.

2.8 ROLE OF HYBRIDIZATIONS IN DATA MINING

Let us first consider neuro-fuzzy hybridization in the context of data mining [3]. The rule generation aspect of neural networks is utilized to extract more *natural* rules from fuzzy neural networks [118], incorporating the better interpretability and understandability of fuzzy sets. The fuzzy MLP [119] and fuzzy Kohonen network [120] have been used for linguistic rule generation and inferencing. Here the input, besides being in quantitative, linguistic, or set forms, or a combination of these, can also be incomplete. Output decision is provided in terms of class membership values. The models are capable of

- Inferencing based on complete and/or partial information
- Querying the user for unknown input variables that are key to reaching a decision
- Producing justification for inferences in the form of IF-THEN rules.

The connection weights and node activation values of the trained network are used in the process. A *certainty factor* determines the confidence in an output decision. Figure 2.8 gives an overall view of the various stages involved in the process of inferencing and rule generation.

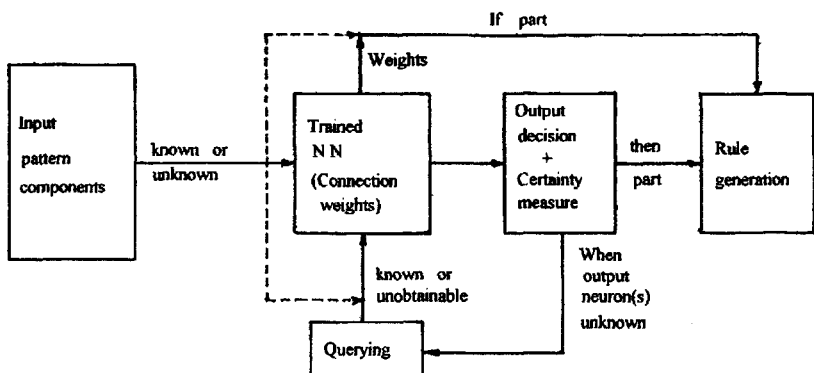


Fig. 2.8 Block diagram of inferencing and rule generation phases.

Zhang et al. [121] have designed a granular neural network to deal with numerical-linguistic data fusion and granular knowledge discovery in databases. The network is capable of learning internal granular relations between input and output and predicting new relations. Low-level granular data can be compressed to generate high-level granular knowledge in the form of rules.

A neuro-fuzzy knowledge-based network by Mitra et al. [122] is capable of generating both *positive* and *negative* rules in linguistic form to justify any decision reached. In the absence of positive information regarding the belonging of a pattern to class C_k , the complementary information is used for generating negative rules. The network topology is automatically determined, in terms of the *a priori* class information and distribution of pattern points in the feature space, followed by refinement using growing and/or pruning of links and nodes.

Banerjee et al. [28] have used a *rough-neuro-fuzzy* integration to design a knowledge-based system, where the theory of rough sets is utilized for extracting domain knowledge. The extracted crude domain knowledge is encoded among the connection weights. Rules are generated from a decision table by computing relative reducts. The network topology is automatically determined and the dependency factors of these rules are encoded as the initial connection weights. The hidden nodes model the conjuncts in the antecedent part of a rule, while the output nodes model the disjuncts.

A promising direction in mining a huge dataset is to (a) partition it, (b) develop classifiers for each module, and (c) combine the results. A modular approach has been pursued [87, 123, 124] to combine the knowledge-based rough-fuzzy MLP subnetworks or modules generated for each class, using GAs. An l -class classification problem is split into l two-class problems. Figure 2.9 depicts the knowledge flow for the entire process for $l = 2$. Dependency rules, shown on the top left corner of the figure, are extracted directly from real-valued attribute table consisting of fuzzy membership values by adap-

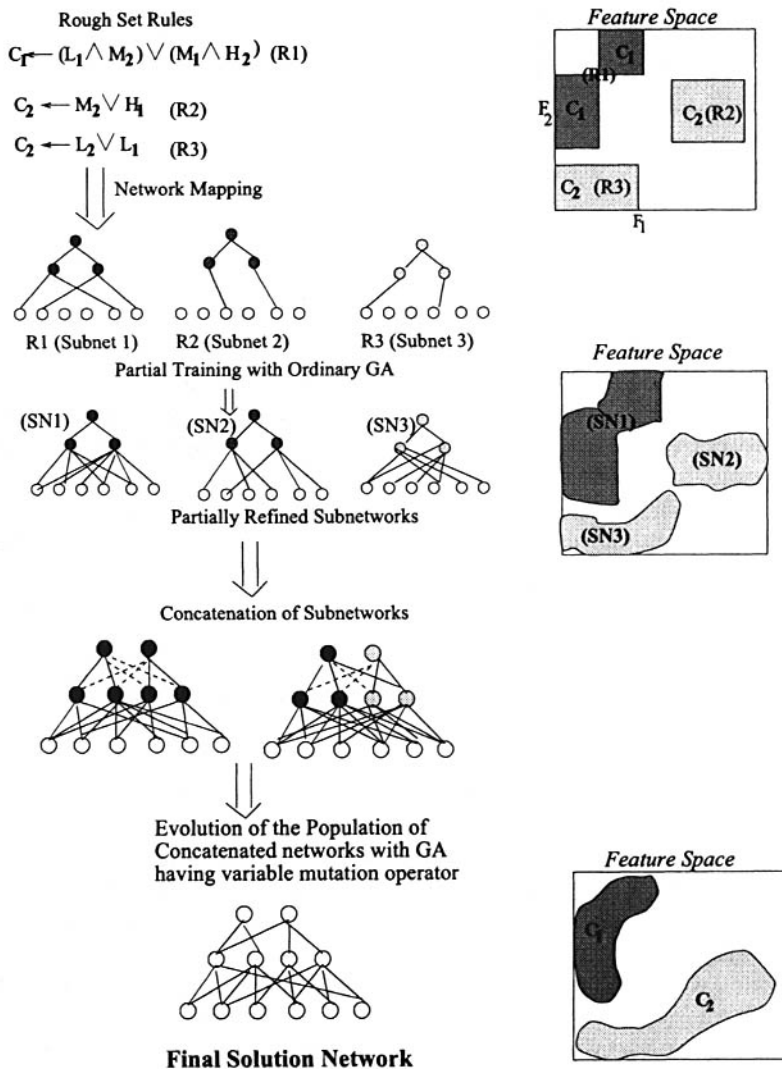


Fig. 2.9 Knowledge flow in a modular rough-neuro-fuzzy-genetic system.

tively applying a threshold. The nature of the decision boundaries, at each stage, are depicted on the right side of the figure. The topology of the subnetworks are initially mapped using the dependency rules. Evolution using GA leads to partial refinement. The final network (represented as a concatenation of the subnetworks) is evolved using a GA with restricted mutation operator, in a novel *rough-neuro-fuzzy-genetic* framework. The *divide-and-conquer* strategy, followed by evolutionary optimization, is found to enhance the performance of the network. The method is described in further detail in Section 8.3.

George and Srikanth [125] have used a *fuzzy-genetic* integration, where GAs are applied to determine the most appropriate data summary. Kiem and Phuc [126] have developed a *rough-neuro-genetic* hybridization for discovering conceptual clusters from a large database.

2.9 CONCLUSIONS AND DISCUSSION

Current research in data mining mainly focuses on the discovery algorithm and visualization techniques. There is a growing awareness that, in practice, it is easy to discover a huge number of patterns in a database where most of these patterns are actually obvious, redundant, and useless or uninteresting to the user. To prevent the user from being overwhelmed by a large number of uninteresting patterns, techniques are needed to identify only the useful or interesting patterns and present them to the user.

Soft computing methodologies, involving fuzzy sets, neural networks, genetic algorithms, rough sets, wavelets, and their hybridizations, have recently been used to solve data mining problems. They strive to provide approximate solutions at low cost, thereby speeding up the process. A categorization has been provided based on the different soft computing tools and their hybridizations used, the mining function implemented, and the preference criterion selected by the model.

Fuzzy sets, which constitute the oldest component of soft computing, are suitable for handling the issues related to understandability of patterns, incomplete or noisy data, mixed media information, and human interaction and can provide approximate solutions faster. They have been mainly used in clustering, discovering association rules and functional dependencies, summarization, time series analysis, Web applications, and image retrieval.

Neural networks are suitable in data-rich environments and are typically used for extracting embedded knowledge in the form of rules, quantitative evaluation of these rules, clustering or self-organization, classification, regression, Web mining, and information retrieval. They have an advantage over other types of machine learning algorithms for scaling [127].

Genetic algorithms provide efficient search algorithms to select a model, from mixed media data, based on some preference criterion or objective function. They have been employed in regression, discovering association rules

and Web mining. Rough sets are suitable for handling different types of uncertainty in data, and they have been mainly utilized for extracting knowledge in the form of rules.

Hybridizations typically enjoy the generic and application-specific merits of the individual soft computing tools that they integrate. Data mining functions modeled by such systems include rule extraction, data summarization, clustering, incorporation of domain knowledge, and partitioning. It is to be noted that the notion of partitioning (i.e., the modular approach) provides an effective direction for scaling up algorithms and speeding up convergence.

An efficient integration of soft computing tools, according to Zadeh's Computational Theory of Perceptions [128], is needed. Feature evaluation and dimensionality reduction help improve prediction accuracy. Some recent work in this direction are available in Refs. [129]–[131]. Other issues, requiring attention, include (a) the choice of metrics and evaluation techniques to handle dynamic changes in data and (b) a quantitative evaluation of system performance.

Recently, several commercial data mining tools have been developed based on soft computing methodologies. These include

- Data Mining Suite, using fuzzy logic;
- Braincell, Cognos 4Thought and IBM Intelligent Miner for Data, using neural networks; and
- Nuggets, using GAs.

REFERENCES

1. L. A. Zadeh, "Fuzzy logic, neural networks, and soft computing," *Communications of the ACM*, vol. 37, pp. 77–84, 1994.
2. S. K. Pal and S. Mitra, *Neuro-fuzzy Pattern Recognition: Methods in Soft Computing*. New York: John Wiley & Sons, 1999.
3. S. Mitra, S. K. Pal, and P. Mitra, "Data mining in soft computing framework: A survey," *IEEE Transactions on Neural Networks*, vol. 13, pp. 3–14, 2002.
4. I. Daubechies, "The wavelet transform, time-frequency localization and signal analysis," *IEEE Transactions on Information Theory*, vol. 36, pp. 961–1005, 1990.
5. I. Daubechies, *Ten Lectures on Wavelets*. CBMS, Philadelphia: Society for Industrial and Applied Mathematics, 1992.

6. R. Heider, "Troubleshooting CFM 56-3 engines for the Boeing 737 - using CBR and data-mining," *Lecture Notes in Computer Science*, vol. 1168, pp. 512-523, 1996.
7. J. Furnkranz, J. Petrak, and R. Trappl, "Knowledge discovery in international conflict databases," *Applied Artificial Intelligence*, vol. 11, pp. 91-118, 1997.
8. J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.
9. S. K. Pal and A. Skowron, eds., *Rough Fuzzy Hybridization: A New Trend in Decision Making*. Singapore: Springer-Verlag, 1999.
10. L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, pp. 338-353, 1965.
11. L. A. Zadeh, "The concept of a linguistic variable and its application to approximate reasoning: Part 1, 2, and 3," *Information Sciences*, vol. 8, 8, 9, pp. 199-249, 301-357, 43-80, 1975.
12. L. A. Zadeh, "Fuzzy sets as a basis for a theory of possibility," *Fuzzy Sets and Systems*, vol. 1, pp. 3-28, 1978.
13. L. A. Zadeh, "The role of fuzzy logic in the management of uncertainty in expert systems," *Fuzzy Sets and Systems*, vol. 11, pp. 199-227, 1983.
14. H. J. Zimmermann, *Fuzzy Sets, Decision Making and Expert Systems*. Boston, MA: Kluwer Academic Publishers, 1987.
15. G. J. Klir and B. Yuan, *Fuzzy Sets and Fuzzy Logic: Theory and Applications*. Englewood Cliffs, NJ: Prentice-Hall, 1995.
16. S. Haykin, *Neural Networks: A Comprehensive Foundation*. New York: Macmillan College Publishing Co., 1994.
17. J. Hertz, A. Krogh, and R. G. Palmer, *Introduction to the Theory of Neural Computation*. Reading, MA: Addison-Wesley, 1994.
18. D. E. Rumelhart and J. L. McClelland, eds., *Parallel Distributed Processing: Explorations in the Microstructures of Cognition*, vol. 1. Cambridge, MA: MIT Press, 1986.
19. T. Kohonen, *Self-Organization and Associative Memory*. Berlin: Springer-Verlag, 1989.
20. J. M. Zurada, *Introduction to Artificial Neural Systems*. New York: West Publishing Company, 1992.
21. D. R. Hush and B. G. Horne, "Progress in supervised neural networks," *IEEE Signal Processing Magazine*, pp. 8-39, January 1993.

22. D. O. Hebb, *The Organization of Behaviour*. New York: John Wiley & Sons, 1949.
23. W. S. McCulloch and W. Pitts, "A logical calculus of the idea immanent in nervous activity," *Bulletin of Mathematical Biophysics*, vol. 5, pp. 115–133, 1943.
24. G. G. Towell and J. W. Shavlik, "Extracting refined rules from knowledge-based neural networks," *Machine Learning*, vol. 13, pp. 71–101, 1993.
25. A. B. Tickle, R. Andrews, M. Golea, and J. Diederich, "The truth will come to light: Directions and challenges in extracting the knowledge embedded within trained artificial neural networks," *IEEE Transactions on Neural Networks*, vol. 9, pp. 1057–1068, 1998.
26. L. M. Fu, "Knowledge-based connectionism for revising domain theories," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, pp. 173–182, 1993.
27. G. G. Towell and J. W. Shavlik, "Knowledge-based artificial neural networks," *Artificial Intelligence*, vol. 70, pp. 119–165, 1994.
28. M. Banerjee, S. Mitra, and S. K. Pal, "Rough fuzzy MLP: Knowledge encoding and classification," *IEEE Transactions on Neural Networks*, vol. 9, pp. 1203–1216, 1998.
29. F. Rosenblatt, "The perceptron: A probabilistic model for information storage and organization in the brain," *Psychological Review*, vol. 65, pp. 386–408, 1958.
30. F. Rosenblatt, *Principles of Neurodynamics, Perceptrons and the Theory of Brain Mechanisms*. Washington, D.C.: Spartan Books, 1961.
31. M. Minsky and S. Papert, *Perceptrons: An Introduction to Computational Geometry*. Cambridge, MA: MIT Press, 1969.
32. K. L. Oehler and R. M. Gray, "Combining image compression and classification using vector quantization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 17, pp. 461–473, 1995.
33. J. Moody and C. J. Darken, "Fast learning in networks of locally-tuned processing units," *Neural Computation*, vol. 1, pp. 281–294, 1989.
34. D. R. Hush, B. Horne, and J. M. Salas, "Error surfaces for multilayer perceptrons," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 22, pp. 1152–1161, 1993.
35. C. T. Lin and C. S. George Lee, *Neural Fuzzy Systems—A Neuro-Fuzzy Synergism to Intelligent Systems*. Englewood Cliffs, NJ: Prentice-Hall, 1996.

36. W. Pedrycz, *Computational Intelligence: An Introduction*. Boca Raton, NY: CRC Press, 1998.
37. Y. Hayashi and J. J. Buckley, "Approximations between fuzzy expert systems and neural networks," *International Journal of Approximate Reasoning*, vol. 10, pp. 63-73, 1994.
38. J. S. R. Jang and C. T. Sun, "Functional equivalence between radial basis function networks and fuzzy inference systems," *IEEE Transactions on Neural Networks*, vol. 4, pp. 156-159, 1993.
39. E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man Machine Studies*, vol. 7, pp. 1-13, 1975.
40. T. Takagi and M. Sugeno, "Fuzzy identification of systems and its application to modeling and control," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 15, pp. 116-132, 1985.
41. J. J. Buckley and T. Feuring, *Fuzzy and Neural: Interactions and Applications*. Studies in Fuzziness and Soft Computing, Heidelberg: Physica-Verlag, 1999.
42. S. C. Lee and E. T. Lee, "Fuzzy neural networks," *Mathematical Biosciences*, vol. 23, pp. 151-177, 1975.
43. S. K. Pal and S. Mitra, "Multi-layer perceptron, fuzzy sets and classification," *IEEE Transactions on Neural Networks*, vol. 3, pp. 683-697, 1992.
44. J. K. Keller and D. J. Hunt, "Incorporating fuzzy membership functions into the perceptron algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 7, pp. 693-699, 1985.
45. J. M. Keller, R. R. Yager, and H. Tahani, "Neural network implementation of fuzzy logic," *Fuzzy Sets and Systems*, vol. 45, pp. 1-12, 1992.
46. J. R. Jang, "ANFIS: Adaptive-network-based fuzzy inference system," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, pp. 665-685, 1993.
47. S. Mitra, "Fuzzy MLP based expert system for medical diagnosis," *Fuzzy Sets and Systems*, vol. 65, pp. 285-296, 1994.
48. D. Nauck, F. Klawonn, and R. Kruse, *Foundations of Neuro-Fuzzy Systems*. Chichester, England: John Wiley & Sons, 1997.
49. J. M. Keller, R. Krishnapuram, and F. C. -H. Rhee, "Evidence aggregation networks for fuzzy logic inference," *IEEE Transactions on Neural Networks*, vol. 3, pp. 761-769, 1992.

50. S. Mitra and S. K. Pal, "Logical operation based fuzzy MLP for classification and rule generation," *Neural Networks*, vol. 7, pp. 353–373, 1994.
51. G. A. Carpenter, S. Grossberg, N. Markuzon, J. H. Reynolds, and D. B. Rosen, "Fuzzy ARTMAP: A neural network architecture for incremental supervised learning of analog multidimensional maps," *IEEE Transactions on Neural Networks*, vol. 3, pp. 698–713, 1992.
52. A. Ghosh, N. R. Pal, and S. K. Pal, "Self-organization for object extraction using multilayer neural network and fuzziness measures," *IEEE Transactions on Fuzzy Systems*, vol. 1, pp. 54–68, 1993.
53. D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Reading, MA: Addison-Wesley, 1989.
54. Z. Michalewicz, *Genetic Algorithms + Data Structures = Evolutionary Programs*. Berlin: Springer-Verlag, 1994.
55. Z. Pawlak, *Rough Sets, Theoretical Aspects of Reasoning about Data*. Dordrecht: Kluwer Academic, 1991.
56. R. Slowiński, ed., *Intelligent Decision Support, Handbook of Applications and Advances of the Rough Sets Theory*. Dordrecht: Kluwer Academic, 1992.
57. Y. Meyer, *Wavelets: Algorithms and Applications*. Philadelphia: SIAM - Society for Industrial and Applied Mathematics, 1993.
58. U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, and R. Uthurusamy, eds., *Advances in Knowledge Discovery and Data Mining*. Menlo Park, CA: AAAI/MIT Press, 1996.
59. B. Liu, W. Hsu, L. F. Mun, and H. Y. Lee, "Finding interesting patterns using user expectation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 11, pp. 817–832, 1999.
60. R. R. Yager, "Database discovery using fuzzy sets," *International Journal of Intelligent Systems*, vol. 11, pp. 691–712, 1996.
61. J. F. Baldwin, "Knowledge from data using fuzzy methods," *Pattern Recognition Letters*, vol. 17, pp. 593–600, 1996.
62. W. Pedrycz, "Fuzzy set technology in knowledge discovery," *Fuzzy Sets and Systems*, vol. 98, pp. 279–290, 1998.
63. P. Piatetsky-Shapiro and W. J. Frawley, eds., *Knowledge Discovery in Databases*. Menlo Park, CA: AAAI/MIT Press, 1991.
64. I. B. Turksen, "Fuzzy data mining and expert system development," in *Proceedings of IEEE International Conference on Systems, Man, and Cybernetics* (San Diego, CA), pp. 2057–2061, October 1998.

65. S. Russell and W. Lodwick, "Fuzzy clustering in data mining for telco database marketing campaigns," in *Proceedings of NAFIPS 99* (New York), pp. 720–726, June 1999.
66. W. Pedrycz, "Conditional fuzzy c-means," *Pattern Recognition Letters*, vol. 17, pp. 625–632, 1996.
67. R. Krishnapuram, A. Joshi, O. Nasraoui, and L. Yi, "Low complexity fuzzy relational clustering algorithms for web mining," *IEEE Transactions on Fuzzy Systems*, vol. 9, pp. 595–607, 2001.
68. L. J. Mazlack, "Softly focusing on data," in *Proceedings of NAFIPS 99*, (New York), pp. 700–704, June 1999.
69. L. A. Zadeh, "Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic," *Fuzzy Sets and Systems*, vol. 19, pp. 111–127, 1997.
70. R. Agrawal, T. Imielinski, and A. Swami, "Mining association rules between sets of items in large databases," in *Proceedings of 1993 ACM SIGMOD International Conference on Management of Data* (Washington, D.C.), pp. 207–216, May 1993.
71. A. Maeda, H. Ashida, Y. Taniguchi, and Y. Takahashi, "Data mining system using fuzzy rule induction," in *Proceedings of IEEE International Conference on Fuzzy Systems FUZZ IEEE 95* (Yokohama, Japan), pp. 45–46, March 1995.
72. Q. Wei and G. Chen, "Mining generalized association rules with fuzzy taxonomic structures," in *Proceedings of NAFIPS 99* (New York), pp. 477–481, June 1999.
73. W. H. Au and K. C. C. Chan, "An effective algorithm for discovering fuzzy rules in relational databases," in *Proceedings of IEEE International Conference on Fuzzy Systems FUZZ IEEE 98* (Alaska), pp. 1314–1319, May 1998.
74. J. Hale and S. Shenoi, "Analyzing FD inference in relational databases," *Data and Knowledge Engineering*, vol. 18, pp. 167–183, 1996.
75. P. Bosc, O. Pivert, and L. Ughetto, "Database mining for the discovery of extended functional dependencies," in *Proceedings of NAFIPS 99* (New York), pp. 580–584, June 1999.
76. D. H. Lee and M. H. Kim, "Database summarization using fuzzy ISA hierarchies," *IEEE Transactions on Systems Man and Cybernetics. Part B-Cybernetics*, vol. 27, pp. 68–78, 1997.

77. R. R. Yager, "On linguistic summaries of data," in *Knowledge Discovery in Databases* (W. Frawley and G. Piatetsky-Shapiro, eds.), pp. 347–363, Menlo Park, CA: AAAI/MIT Press, 1991.
78. J. Kacprzyk and S. Zadrozny, "Data mining via linguistic summaries of data: an interactive approach," in *Proceedings of IIZUKA 98* (Fukuoka, Japan), pp. 668–671, October 1998.
79. D. A. Chiang, L. R. Chow, and Y. F. Wang, "Mining time series data by a fuzzy linguistic summary system," *Fuzzy Sets and Systems*, vol. 112, pp. 419–432, 2000.
80. H. Frigui, "Adaptive image retrieval using the fuzzy integral," in *Proceedings of NAFIPS 99* (New York), pp. 575–579, June 1999.
81. S. Medasani and R. Krishnapuram, "A fuzzy approach to complex linguistic query based image retrieval," in *Proceedings of NAFIPS 99* (New York), pp. 590–594, June 1999.
82. M. Detyniecki, C. Seyrat, and R. Yager, "Interacting with web video objects," in *Proceedings of the 18th International Conference of the North American Fuzzy Information Processing Society (NAFIPS'99)*, pp. 914–917, 1999.
83. A. Filho, G. L. A. Mota, M. M. B. R. Vellasco, and M. A. C. Pacheco, "Query by image similarity using a fuzzy logic approach," in *Proceedings of Fourth International Conference on Computational Intelligence and Multimedia Applications (ICCIMA 2001)*, pp. 389–394, 2001.
84. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Reading, MA: Addison-Wesley, 1992.
85. R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 3, pp. 610–621, 1973.
86. H. J. Lu, R. Setiono, and H. Liu, "Effective data mining using neural networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, pp. 957–961, 1996.
87. S. K. Pal, S. Mitra, and P. Mitra, "Rough Fuzzy MLP: Modular evolution, rule generation and evaluation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, pp. 14–25, 2003.
88. T. Kohonen, S. Kaski, K. Lagus, J. Salojärvi, J. Honkela, V. Paatero, and A. Saarela, "Self organization of a massive document collection," *IEEE Transactions on Neural Networks*, vol. 11, pp. 574–585, 2000.
89. J. Vesanto and E. Alhoniemi, "Clustering of the self-organizing map," *IEEE Transactions on Neural Networks*, vol. 11, pp. 586–600, 2000.

90. D. Alahakoon, S. K. Halgamuge, and B. Srinivasan, "Dynamic self organizing maps with controlled growth for knowledge discovery," *IEEE Transactions on Neural Networks*, vol. 11, pp. 601–614, 2000.
91. D. Shalvi and N. De Claris, "Unsupervised neural network approach to medical data mining techniques," in *Proceedings of IEEE International Joint Conference on Neural Networks* (Alaska), pp. 171–176, May 1998.
92. A. Koenig, "Interactive visualization and analysis of hierarchical neural projections for data mining," *IEEE Transactions on Neural Networks*, vol. 11, pp. 615–624, 2000.
93. V. Ciesielski and G. Palstra, "Using a hybrid neural/expert system for database mining in market survey data," in *Proceedings Second International Conference on Knowledge Discovery and Data Mining (KDD-96)* (Portland, OR), p. 38, AAAI Press, August 2–4, 1996.
94. R. S. T. Lee and J. N. K. Liu, "Tropical cyclone identification and tracking system using integrated neural oscillatory elastic graph matching and hybrid RBF network track mining techniques," *IEEE Transactions on Neural Networks*, vol. 11, pp. 680–689, 2000.
95. K. Lagus, "Text retrieval using self-organized document maps," *Neural Processing Letters*, vol. 15, pp. 21–29, 2002.
96. I. W. Flockhart and N. J. Radcliffe, "A genetic algorithm-based approach to data mining," in *The Second International Conference on Knowledge Discovery and Data Mining (KDD-96)* (Portland, OR), p. 299, AAAI Press, August 2–4 1996.
97. M. L. Raymer, W. F. Punch, E. D. Goodman, and L. A. Kuhn, "Genetic programming for improved data mining: An application to the biochemistry of protein interactions," in *Genetic Programming 1996: Proceedings of the First Annual Conference* (Stanford University, CA), pp. 375–380, MIT Press, 28–31 July 1996.
98. T. Ryu and C. F. Eick, "MASSON: discovering commonalties in collection of objects using genetic programming," in *Genetic Programming 1996: Proceedings of First Annual Conference* (Stanford University, CA), pp. 200–208, MIT Press, July 28–31 1996.
99. A. Teller and M. Veloso, "Program evolution for data mining," *The International Journal of Expert Systems*, vol. 8, pp. 216–236, 1995.
100. K. Xu, Z. Wang, and K. S. Leung, "Using a new type of nonlinear integral for multi-regression: An application of evolutionary algorithms in data mining," in *Proceedings of IEEE International Conference on Systems, Man, and Cybernetics* (San Diego, CA), pp. 2326–2331, October 1998.

101. E. Noda, A. A. Freitas, and H. S. Lopes, "Discovering interesting prediction rules with a genetic algorithm," in *Proceedings of IEEE Congress on Evolutionary Computation CEC 99* (Washington, D.C.), pp. 1322–1329, July 1999.
102. C. Lopes, M. Pacheco, M. Vellasco, and E. Passos, "Rule-evolver: An evolutionary approach for data mining," in *Proceedings of RSFDGrC '99* (Yamaguchi, Japan), pp. 458–462, November 1999.
103. J. F. Peters and A. Skowron, "A rough set approach to knowledge discovery," *International Journal of Intelligent Systems*, vol. 17, pp. 109–112, 2002.
104. T. Mollestad and A. Skowron, "A rough set framework for data mining of propositional default rules," *Lecture Notes in Computer Science*, vol. 1079, pp. 448–457, 1996.
105. A. Skowron, "Extracting laws from decision tables—a rough set approach," *Computational Intelligence*, vol. 11, pp. 371–388, 1995.
106. N. Shan and W. Ziarko, "Data-based acquisition and incremental modification of classification rules," *Computational Intelligence*, vol. 11, pp. 357–370, 1995.
107. L. Polkowski and A. Skowron, *Rough Sets in Knowledge Discovery 1 and 2*. Heidelberg: Physica-Verlag, 1998.
108. W. Ziarko and N. Shan, "KDD-R: A comprehensive system for knowledge discovery in databases using rough sets," in *Proceedings of Third International Workshop on Rough Sets and Soft Computing RSSC '94*, pp. 164–173, 1994.
109. J. W. Grzymala-Busse, "LERS—A knowledge discovery system," in *Rough Sets in Knowledge Discovery 2, Applications, Case Studies and Software Systems* (L. Polkowski and A. Skowron, eds.), pp. 562–565, Heidelberg: Physica-Verlag, 1998.
110. J. W. Grzymala-Busse, W. J. Grzymala-Busse, and L. K. Goodwin, "A closest fit approach to missing attribute values in preterm birth data," in *Proceedings of RSFDGrC '99* (Yamaguchi, Japan), pp. 405–413, November 1999.
111. T. B. Ho and N. B. Nguyen, "Nonhierarchical document clustering based on a tolerance rough set model," *International Journal of Intelligent Systems*, vol. 17, pp. 199–212, 2002.
112. S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, pp. 674–693, 1989.

113. M. Antonini, M. Barlaud, P. Mathieu, and I. Daubechies, "Image coding using wavelet transform," *IEEE Transactions on Image Processing*, vol. 1, pp. 205–220, 1992.
114. F. Wang and Q. J. Zhang, "Incorporating functional knowledge into neural networks," in *Proceedings of IEEE International Conference on Neural Networks* (Houston, TX), pp. 266–269, 1997.
115. J. Z. Wang, G. Wiederhold, O. Firchein, and S. Wei, "Content-based image indexing and searching using Daubechies' wavelets," *International Journal of Digital Libraries*, vol. 1, pp. 311–328, 1998.
116. G. Sheikholeslami, S. Chatterjee, and A. Zhang, "WaveCluster: A multi-resolution clustering approach for very large spatial databases," in *Proceedings of 1998 International Conference on Very Large Data Bases (VLDB '98)* (New York), pp. 428–439, August 1998.
117. R. M. Rao and A. S. Bopardikar, *Wavelet Transforms: Introduction to Theory and Applications*. Massachusetts: Addison Wesley, 1998.
118. S. Mitra and Y. Hayashi, "Neuro-fuzzy rule generation: Survey in soft computing framework," *IEEE Transactions on Neural Networks*, vol. 11, pp. 748–768, 2000.
119. S. Mitra and S. K. Pal, "Fuzzy multi-layer perceptron, inferencing and rule generation," *IEEE Transactions on Neural Networks*, vol. 6, pp. 51–63, 1995.
120. S. Mitra and S. K. Pal, "Fuzzy self organization, inferencing and rule generation," *IEEE Transactions on Systems, Man and Cybernetics, Part A: Systems and Humans*, vol. 26, pp. 608–620, 1996.
121. Y. Q. Zhang, M. D. Fraser, R. A. Gagliano, and A. Kandel, "Granular neural networks for numerical-linguistic data fusion and knowledge discovery," *IEEE Transactions on Neural Networks*, vol. 11, pp. 658–667, 2000.
122. S. Mitra, R. K. De, and S. K. Pal, "Knowledge-based fuzzy MLP for classification and rule generation," *IEEE Transactions on Neural Networks*, vol. 8, pp. 1338–1350, 1997.
123. S. Mitra, P. Mitra, and S. K. Pal, "Evolutionary modular design of rough knowledge-based network using fuzzy attributes," *Neurocomputing*, vol. 36, pp. 45–66, 2001.
124. P. Mitra, S. Mitra, and S. K. Pal, "Staging of cervical cancer with soft computing," *IEEE Transactions on Biomedical Engineering*, vol. 47, pp. 934–940, 2000.

125. R. George and R. Srikanth, "Data summarization using genetic algorithms and fuzzy logic," in *Genetic Algorithms and Soft Computing* (F. Herrera and J. L. Verdegay, eds.), pp. 599–611, Heidelberg: Physica-Verlag, 1996.
126. H. Kiem and D. Phuc, "Using rough genetic and Kohonen's neural network for conceptual cluster discovery in data mining," in *Proceedings of RSFDGrC '99* (Yamaguchi, Japan), pp. 448–452, November 1999.
127. Y. Bengio, J. M. Buhmann, M. Embrechts, and J. M. Zurada, "Introduction to the special issue on neural networks for data mining and knowledge discovery," *IEEE Transactions on Neural Networks*, vol. 11, pp. 545–549, 2000.
128. L. A. Zadeh, "A new direction in AI: Towards a computational theory of perceptions," *AI Magazine*, vol. 22, pp. 73–84, 2001.
129. S. Bengio and Y. Bengio, "Taking on the curse of dimensionality in joint distribution using neural networks," *IEEE Transactions on Neural Networks*, vol. 11, pp. 550–557, 2000.
130. R. Kewley, M. Embrecht, and C. Breneman, "Data strip mining for the virtual design of pharmaceuticals with neural networks," *IEEE Transactions on Neural Networks*, vol. 11, pp. 668–679, 2000.
131. C. K. Shin, S. J. Yu, U. T. Yun, and H. K. Kim, "A hybrid approach of neural network and memory based learning to data mining," *IEEE Transactions on Neural Networks*, vol. 11, pp. 637–646, 2000.

3

Multimedia Data Compression

3.1 INTRODUCTION

Multimedia data mining is a growing area of interest, and its advancement will have impact on how we store, access, and process different datatypes for different application areas in the near future. Data mining usually deals with large datasets and involves the access of relevant information from them. Hence it becomes necessary to apply data compression in large datasets, in order to reduce storage requirements for practical data processing applications, particularly in the area of multimedia applications. The development of efficient compression techniques will continue to be a design challenge and an area of interest to researchers.

Although the basic premises of data compression offer promises to potentially improve the efficiency of data mining techniques, not much attention has been focused in this direction by researchers. In our view, data compression has been neglected by the data mining community. However, limited efforts have been made to reduce high-dimensional data to lower dimensions for its compact representation and better visualization. Classical data mining techniques deal with mining information from databases represented in the canonical form. Access of data in the compressed domain and development of data compression techniques particularly suitable for data mining, whereby it would be possible to efficiently index the compressed data for fast search and access from large databases, remains a challenge. This will immensely benefit Web mining as well since huge volumes of data are distributed worldwide, all over the Web, in compressed form.

In order to address the challenging problem of mining data in the compressed domain, it becomes essential to understand the principles behind current data compression techniques. Researchers can build on this knowledge to propose new data mining techniques in the compressed domain, thereby influencing future multimedia applications through mining.

The main advantage of compression is that it reduces the data storage requirements. It also offers an attractive approach to reduce the communication cost in transmitting high volumes of data over long-haul links via higher effective utilization of the available bandwidth in the data links. This significantly aids in reducing the cost of communication, due to the data rate reduction. Thereby, data compression also increases the quality of multimedia presentation through limited bandwidth communication channels. As a result, the audience can experience rich quality signals for audio-visual data representation.

For example, because of the application of sophisticated compression technologies applied in multimedia data, we can receive toll quality audio at the other side of the globe through the good old telecommunication channels at a much better price as compared to that of a decade ago. Because of the significant progress in image compression techniques, a single 6-MHz broadcast television channel can carry HDTV signals to provide better-quality audio and video at higher rates and enhanced resolution, without additional bandwidth requirements. Due to the reduced data rate offered by the compression techniques, the computer network and Internet usage is becoming more and more image and graphics friendly, rather than being just data and text centric phenomena. In short, high-performance compression has created new opportunities of creative applications such as digital library, digital archival, video teleconferencing, telemedicine, digital entertainment, to name a few. Researchers need to pay significant attention to develop techniques for mining of multimedia data in the compressed domains, in order to further excel in the usage of data mining technologies in multimedia applications.

The organization of this chapter is as follows. Section 3.2 introduces the basic concepts from information theory. Sections 3.3–3.5 provide details on the classification of compression algorithms, a data compression model, and the different measures of compression performance. Section 3.6 presents some source coding algorithms. This is followed by a treatise on Principal Component Analysis in Section 3.7. Principles of still image compression are described in Section 3.8. The JPEG image compression standard, JPEG lossless coding algorithm, and baseline JPEG compression are explained in Sections 3.9–3.11. Text compression is elaborated upon in Section 3.12. Finally, Section 3.13 concludes the chapter.

3.2 INFORMATION THEORY CONCEPTS

The *Mathematical Theory of Communication* [1]–[4], also called the *Information Theory*, was pioneered by Claude E. Shannon in 1948. It is considered to be the theoretical foundation of data compression research.

Representation of data is a combination of *information* and *redundancy* [1]. Information is the portion of data that must be preserved permanently in its original form in order to correctly interpret the meaning or purpose of the data. Redundancy, on the other hand, is that portion of data that can be removed when it is not needed or can be reinserted to interpret the data when needed. Data *compression* is essentially a redundancy reduction technique. The redundancy in data representation is reduced in such a way that it can be subsequently reinserted to recover the original data, through a process called *decompression* of this data. In literature, sometimes data compression is referred to as *coding*, while decompression is termed as *decoding*.

Usually development of a data compression scheme can be broadly divided into two phases, namely, *modeling* and *coding*. In the modeling phase, information about redundancy that exists in the data is extracted and described in a model. This enables us to determine how the actual data differs from the model, and it allows us to encode the difference in the *coding* phase. Obviously, a data compression algorithm becomes more effective if the model is closer to the characteristics of the data generating process which we often call the *source*. The model can be obtained by empirical observation of the statistics of the data generated by the source. In an empirical sense, any information-generating process can be described as a source that emits a sequence of symbols chosen from a finite set of all possible symbols generated by the source. This finite set of symbols is often called an *alphabet*. For example, we can think of this text as being generated by a source with an alphabet containing all the ASCII characters.

3.2.1 Discrete memoryless model and entropy

If the symbols produced by the information source are statistically independent to each other, the source is called a *discrete memoryless source*. This is described by its source alphabet $A = \{a_1, a_2, \dots, a_N\}$ and the associated probabilities of occurrence $P = \{p(a_1), p(a_2), \dots, p(a_N)\}$ of the symbols $a_1, a_2, \dots, a_N$ in A .

The definition of *discrete memoryless source* model provides us a very powerful concept of quantification of *average information content per symbol* of the source and *entropy* of the data. The concept of “entropy” was first used by physicists as a thermodynamic parameter to measure the degree of “disorder” or “chaos” in a thermodynamic or molecular system. In statistical sense, we can view this as a measure of the degree of “surprise” or “uncertainty.” In an intuitive sense, it is reasonable to assume that the appearance of a less prob-

able event (symbol) gives us more surprise and hence we expect that it might carry more information. On the contrary, the more probable event (symbol) will carry less information because it was expected more. Note an analogy to the concept of surprising or interesting rules explained in Section 7.4.

With the above intuitive explanation, we can comprehend Shannon's definition of the relation between the source symbol probabilities and corresponding codes. The amount of *information* content, $I(a_i)$, in a source symbol a_i , in terms of its associated probability of occurrence $p(a_i)$ is

$$I(a_i) = \log_2 \frac{1}{p(a_i)} = -\log_2 p(a_i).$$

The base 2 in the logarithm indicates that the information is expressed in binary units or bits. In terms of binary representation of the codes, a symbol a_i that is expected to occur with probability $p(a_i)$ is best represented in approximately $-\log_2 p(a_i)$ bits. As a result, a symbol with higher probability of occurrence in a message is coded using a fewer number of bits.

If we average the amount of information content over all the possible symbols of the discrete memoryless source, we can find the average amount of information content per source symbol from the discrete memoryless source. This is expressed as

$$E = \sum_{i=1}^N p(a_i) I(a_i) = - \sum_{i=1}^N p(a_i) \log_2 p(a_i), \quad (3.1)$$

and is popularly known as "*entropy*" in information theory. Hence entropy is the expected length of a binary code over all possible symbols in a discrete memoryless source. Note the analogy of this definition with Eq. (5.1), expressing entropy in the context of pattern classification.

The concept of entropy is very powerful. In "stationary" systems, where the probabilities of occurrence of the source symbols are fixed, it provides a bound for the compression that can be achieved. This is a very convenient measure of the performance of a coding system.

3.2.2 Noiseless Source Coding Theorem

The *Noiseless Source Coding Theorem* by Shannon [1] establishes the minimum average code word length per source symbol that can be achieved, which in turn provides an upper bound on the achievable compression losslessly. The *Noiseless Source Coding Theorem* is also known as *Shannon's first theorem*. This is one of the major source coding results in information theory [1]–[3].

If the data generated from a discrete memoryless source A is considered to be grouped together in blocks of n symbols, to form an n -extended source, then the new source A^n has N^n possible symbols $\{a_i\}$, with probability $P(a_i) = P(a_{i_1})P(a_{i_2}) \cdots P(a_{i_n})$, $i = 1, 2, \dots, N^n$. By deriving the entropy of the new

n -extended source, it can be proven that

$$E(A^n) = nE(A),$$

where $E(A)$ is the entropy of the original source A . Let us now consider encoding the blocks of n source symbols, at a time, into binary codewords. For any $\epsilon > 0$, it is possible to construct a codeword for the block in such a way that the average number of bits per original source symbol, $\bar{L}$, satisfies

$$E(A) \leq \bar{L} < E(A) + \epsilon. \quad (3.2)$$

The left-hand inequality must be satisfied for any uniquely decodable code for the block of n source symbols.

The *Noiseless Source Coding Theorem* states that any infinitely long sequence of source symbols emanating from a discrete memoryless source can be losslessly encoded with a code whose average number of bits per source symbol is arbitrarily close to, but not less than, the source entropy E in bits. Hence this theorem provides us the intuitive (statistical) yardstick to measure the information emerging from a source.

3.2.2.1 Example 1: Let us consider a *discrete memoryless source* with alphabet $A_1 = \{\alpha, \beta, \gamma, \delta\}$ having associated probabilities $p(\alpha) = 0.65$, $p(\beta) = 0.20$, $p(\gamma) = 0.10$, and $p(\delta) = 0.05$, respectively. The entropy of this source is $E = -(0.65 \log_2 0.65 + 0.20 \log_2 0.20 + 0.10 \log_2 0.10 + 0.05 \log_2 0.05)$, which is approximately 1.42 bits/symbol. As a result, a 2000-symbols-long datum can be represented using approximately 2820 bits.

Knowing something about the structure of the data sequence often helps in reducing the entropy estimation of the source. Let us consider that the numeric data sequence generated by a source of alphabet $A_2 = \{0, 1, 2, 3\}$ is $D = 0\ 1\ 1\ 2\ 3\ 3\ 3\ 3\ 3\ 3\ 3\ 3\ 2\ 2\ 2\ 3\ 3\ 3\ 3$, as an example. The probability of appearance of the symbols in alphabet A_2 are $p(0) = 0.05$, $p(1) = 0.10$, $p(2) = 0.20$, and $p(3) = 0.65$, respectively, as in alphabet A_1 . Hence the estimated entropy of the sequence D is $E = 1.42$ bits per symbol. If we assume that correlation exists between two consecutive samples in this data sequence, we can reduce this correlation by simply subtracting a sample by its previous sample to generate the residual values $r_i = s_i - s_{i-1}$ for each sample s_i . Based on this assumption of the model, the sequence of residuals of the original data sequence is $\bar{D} = 0\ 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0\ 0\ 0\ 0\ -1\ 0\ 0\ 1\ 0\ 0\ 0$, consisting of three symbols in a modified alphabet $\bar{A}_2 = \{-1, 1, 0\}$. The probability of occurrence of the symbols in the new alphabet $\bar{A}$ are $p(-1) = 0.05$, $p(1) = 0.2$, and $p(0) = 0.75$, respectively, as computed by the number of occurrences in the residual sequence. The estimated entropy of the transformed sequence is $\bar{E} = -(0.05 \log_2 0.05 + 0.2 \log_2 0.2 + 0.75 \log_2 0.75) = 0.992$, that is, 0.992 bits/symbol. Hence the data sequence can be represented with fewer number of bits, resulting in compression.

3.3 CLASSIFICATION OF COMPRESSION ALGORITHMS

In an abstract sense, we can describe *data compression* as a method that takes an input data D and generates a shorter representation of the data $c(D)$ with fewer number of bits compared to that of D . The reverse process is called *decompression*, and it takes the compressed data $c(D)$ and generates or reconstructs the data D' as shown in Fig. 3.1. Sometimes the *compression* (coding) and *decompression* (decoding) systems together are called a "CODEC," as marked by the broken box in Fig. 3.1. The reconstructed data D' could be identical to the original data D or it could be an approximation of the original data D , depending upon the reconstruction requirements. If the reconstructed data D' is an exact replica of the original data D , we call the algorithm applied to compress D and decompress $c(D)$ to be *lossless*. Otherwise, we call the algorithms to be *lossy*. Hence as far as the reversibility of the original data is concerned, the data compression algorithms can be broadly classified into two categories, namely, "*lossless*" and "*lossy*."

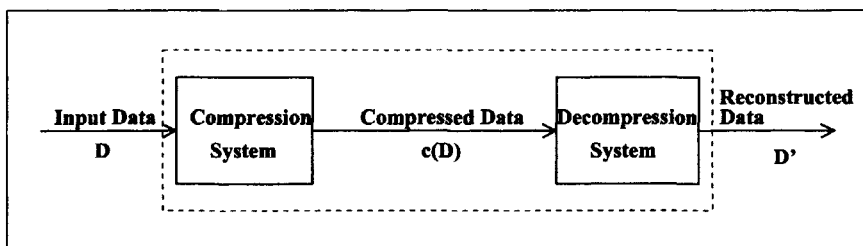


Fig. 3.1 Block diagram of CODEC.

Usually we need to apply lossless data compression techniques on text data or scientific data. For example, we cannot afford to compress the electronic copy of this textbook using a lossy compression technique. It is expected that we should be able to reconstruct the same text after the decompression process. A small error in the reconstructed text can have a completely different meaning. We do not expect the sentence "You should *not* delete this file" in a text to change to "You should *now* delete this file" as a result of some error introduced by a lossy compression or decompression algorithm. Similarly, if we compress a huge ASCII file containing a program written in "C" language, for example, we expect to get back the same "C" code after decompression because of obvious reasons.

The lossy compression techniques are usually applicable to data where high fidelity of reconstructed data is not required, for perception, by the human perceptual system. Examples of such types of data are image, video, graphics, speech, audio, etc. Some image compression applications may, however, require the compression scheme to be lossless; that is, each pixel of the decompressed image should be exactly identical to the original one. Medical

imaging is an example of such an application, where compressing the digital radiographs with a lossy scheme could be a disaster if it has to compromise with the diagnostic accuracy. Similar observations are true for astronomical images of galaxies and stars.

3.4 A DATA COMPRESSION MODEL

A model of a typical data compression system can be described using the block diagram shown in Fig. 3.2. A data compression system mainly constitutes three major steps, namely, (i) removal or reduction in data redundancy, (ii) reduction in entropy, and (iii) entropy encoding.

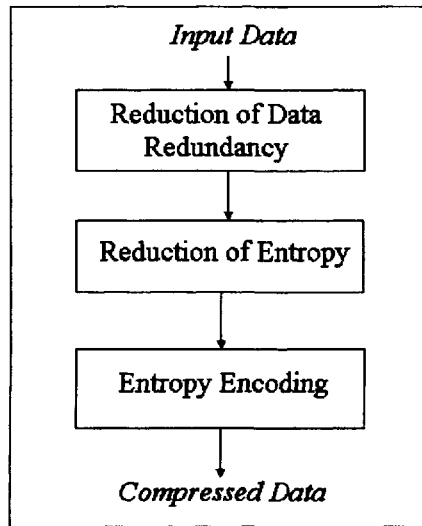


Fig. 3.2 A data compression model.

The redundancy in data may appear in different forms. For example, the neighboring pixels in a typical image are very much spatially correlated to each other. By correlation we mean that the pixel values are very similar in the non-edge smooth regions of the image [5, 6]. This correlation of the neighboring pixels is termed as spatial correlation. In case of moving pictures, the consecutive frames could be almost similar, with or without minor displacement, if the motion is slow. The composition of words or sentences in a natural text follows some context model, based on the grammar being used. Similarly, the records in a typical numeric database may have some sort of

relationship amongst the atomic entities which comprise each record in the database. There are rhythms and pauses in regular intervals in any natural audio or speech data. All these redundancies in data representation can be reduced in order to achieve potential compression.

Removal or reduction in data redundancy is typically achieved by transforming the original data from one form of representation to another, in order to decorrelate the spatial information redundancies present in the data. The popular techniques used for spatial redundancy reduction are prediction of data samples using some model, transformation of the original data from spatial to frequency domain using methods such as Discrete Cosine Transform (DCT), decomposition of the original dataset into different subbands as in Discrete Wavelet Transformation (DWT), etc. In principle, this spatial redundancy reduction potentially yields more compact representation of the information in the original dataset, in terms of fewer transformed coefficients or equivalent, and hence makes it amenable to represent the data with fewer number of bits in order to achieve compression.

The next major step in a lossy data compression system is “quantization.” Quantization techniques are applied on the decorrelated or transformed data, in order to further reduce the number of symbols or coefficients, by masking nonsignificant parts and preserving only the meaningful information in the data. This leads to reduction in entropy of the data, and hence makes it further amenable to compression by allocating less number of bits for transmission or storage. The reduction in entropy is achieved by dropping nonsignificant information in the transformed data and preserving fewer significant symbols only. For example, in case of an image transformed in frequency domain, the high-frequency transformed coefficients can be dropped because the human vision system is not sensitive to these. By preserving a smaller number of transformed coefficients in the meaningful low-frequency range, we can maintain the fidelity of the reconstructed image. The nature and amount of quantization dictate the quality of the reconstructed data. The quantized coefficients are then losslessly encoded, using some entropy encoding scheme to compactly represent the quantized data for storage or transmission. Since the entropy of the quantized data is less than that of the original one, it can be represented by a smaller number of bits compared to the original data set and hence we achieve compression.

The decompression system is just an inverse process to reconstruct the data.

3.5 MEASURES OF COMPRESSION PERFORMANCE

As in any other system, the metrics of performance of a data compression algorithm is an important criteria for its selection. The performance measures of data compression algorithms can be looked at from different perspectives depending upon the application requirements, namely, amount of compression

achieved, objective and subjective quality of the reconstructed data, relative complexity of the algorithm, speed of execution, etc. We explain some of these below.

3.5.1 Compression ratio and bits per sample

The most popular performance measure of a data compression algorithm is the '*compression ratio*'. It is defined as the ratio of the number of bits in the original data to the number of bits in the compressed data. Consider a gray scale image of size 256×256 . If each pixel is represented by a single byte, the image needs 65536 bytes of storage. If the compressed version of the image can be stored in 4096 bytes, the compression ratio achieved by the compression algorithm will be 16:1.

A variation of the *compression ratio* is '*bits per sample*'. This metric indicates the average number of bits to represent a single sample of the data – for example, *bits per pixel* for image coding. In case of an image, each pixel represents a sample. On the other hand, in case of a text file, each sample corresponds to a character in the text. If 65536 pixels of an image are compressed to 4096 bytes, we can say that the compression algorithm achieved 0.5 bits per pixel on the average. Hence the *bits per sample* can be measured by the ratio of the *number of bits* of a single uncompressed sample to the *compression ratio*.

3.5.2 Quality metric

The quality or fidelity metric is important for lossy compression algorithms used in video, image, voice, etc., because here the reconstructed data differs from the original one and the human perceptual system is the ultimate judge of the reconstructed quality. For example, if there is no perceivable difference between the reconstructed data and the original version, then the compression algorithm can be claimed to have achieved a very high quality or fidelity. The difference of the reconstructed data from the original one is called the *distortion*, and a lower distortion implies a higher quality of the reconstructed data. A quality measure can either be very subjective based on human perception, or be objectively defined using mathematical or statistical evaluation.

3.5.2.1 Subjective quality metric: There is no universally accepted measure for the subjective quality metrics. Often the subjective quality metric is defined as the *mean observer score* (MOS). Sometimes it is also called the *mean opinion score*. There are different statistical ways to compute MOS. In one of the simplest methods, a statistically significant number of observers are randomly chosen to evaluate the visual quality of the reconstructed images. All the images are compressed and decompressed by the same algorithm. Each observer assigns a numeric score to each reconstructed image based on

his or her perception of quality of the image, say, within a range 1–5 to describe the quality of the image, with 5 and 1 being the best and worst quality, respectively. The average of the scores assigned by all the observers is the MOS, and it is considered as a viable subjective metric if all the observers are unbiased and evaluate the images under the same viewing or experimental conditions. There are different variations of this approach to calculate MOS, namely, absolute comparison, paired comparison, blind evaluation, etc.

The techniques of measurement of MOS could well be different for different perceptual data. For example, the methodology to evaluate the subjective quality of a still image could be entirely different from that for video or voice data.

3.5.2.2 Objective quality metric: There is no universally accepted measure for the objective quality of data compression algorithms either. The most widely used objective quality metrics are root-mean-squared error (*RMSE*), signal-to-noise ratio (*SNR*), and peak signal-to-noise ratio (*PSNR*). If I is an $M \times N$ image and $\bar{I}$ is the corresponding reconstructed image after compression and decompression, *RMSE* is calculated as

$$RMSE = \sqrt{\frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N [I(i, j) - \bar{I}(i, j)]^2}, \quad (3.3)$$

where i, j refer to the pixel position in the image. The *SNR* in decibel unit (dB) is expressed as $SNR =$

$$20 \log_{10} \left(\frac{\sqrt{\frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N I^2(i, j)}}{RMSE} \right) = 10 \log_{10} \left(\frac{\sum_{i=1}^M \sum_{j=1}^N I^2(i, j)}{\sum_{i=1}^M \sum_{j=1}^N [I(i, j) - \bar{I}(i, j)]^2} \right). \quad (3.4)$$

In case of an 8-bit image, the corresponding *PSNR* in dB is computed as

$$PSNR = 20 \log_{10} \left(\frac{255}{RMSE} \right), \quad (3.5)$$

where 255 is the maximum possible pixel value in 8 bits.

It should be noted that a lower *RMSE* (or equivalently, higher *SNR* or *PSNR*) does not necessarily always indicate a higher subjective quality. In fact these objective error metrics do not always correlate well with the subjective quality metrics. There are many cases where the *PSNR* of a reconstructed image can be reasonably high, but the subjective quality is really bad when visualized by human eyes. Hence the choice of the objective or subjective metrics, to evaluate a compression and decompression algorithm, often depends upon the application criteria.

Similar objective quality metrics are used for audio and speech signals as well.

3.5.3 Coding complexity

When the computational requirement to implement the CODEC in a particular computing platform is an important criterion, we often consider the coding complexity and computation time of a compression algorithm to be a performance measure. Implementation of a compression algorithm using special purpose digital signal processor (DSP) architectures is common in communication systems. In portable systems, the coding complexity is an important criterion from the perspective of low-power hardware implementation. The computational requirement is usually measured in terms of the number of arithmetic operations and the memory requirement. Usually, the number of arithmetic operations is described by MOPS (millions of operations per second). But in compression literature, the term MIPS (millions of instructions per second) is often used to measure the compression performance of a specific computing engine's architecture.

3.6 SOURCE CODING ALGORITHMS

From the information theoretic perspective, *source coding* can mean both lossless and lossy compression. However, researchers often use it to indicate lossless coding only. In the signal processing community, source coding is used to mean source model based coding. In this section, we describe some basic source coding algorithms such as Run-length coding and Huffman coding in greater detail.

3.6.1 Run-length coding

Run-length coding is a simple approach to source coding when there exists a long run of the same data, in a consecutive manner, in a dataset. As an example, the data $d = '6\ 6\ 6\ 6\ 6\ 6\ 0\ 9\ 0\ 5\ 5\ 5\ 5\ 5\ 2\ 2\ 2\ 2\ 2\ 1\ 3\ 4\ 4\ 4\ 4\ 4\ \dots'$ contains long runs of 6's, 5's 2's 4's, etc. Rather than coding each sample in the run individually, the data can be represented compactly by simply indicating the value of the sample and the length of its run when it appears. For example, if a portion of an image is represented by "5 5 5 5 5 5 19 19 19 19 19 19 19 19 19 19 19 0 0 0 0 0 0 0 8 23 23 23 23 23 23," this can be run-length encoded as (5 7) (19 12) (0 8) (8 1) (23 6). For ease of understanding, we have shown a pair in each parenthesis. Here the first value represents the pixel, while the second indicates the length of its run.

In some cases, the appearance of runs of symbols may not be very apparent. But the data can possibly be preprocessed in order to aid run-length coding. Consider the data $d = '26\ 29\ 32\ 35\ 38\ 41\ 44\ 50\ 56\ 62\ 68\ 78\ 88\ 98\ 108\ 118\ 116\ 114\ 112\ 110\ 108\ 106\ 104\ 102\ 100\ 98\ 96'$. We can simply preprocess this data by taking the sample difference $e(i) = d(i) - d(i - 1)$, to produce the processed data $\bar{e} = '26\ 3\ 3\ 3\ 3\ 3\ 3\ 6\ 6\ 6\ 6\ 10\ 10\ 10\ 10\ 10\ -2\ -2\ -2\ -2\ -2$

$-2 \ -2 \ -2 \ -2 \ -2 \ -2'$. This preprocessed data can now easily be run-length encoded as (26 1) (3 6) (6 4) (10 5) $(-2 \ 11)$. A variation of this technique is applied in the baseline JPEG standard for still picture compression [7]. The same technique can be applied to numeric databases as well.

On the other hand, binary (black and white) images, such as facsimile, usually consist of runs of 0's or 1's. As an example, if a segment of a binary image is represented as $d =$

"0000000001111111111000000000000001110000000000001001111111111," it can be compactly represented as $c(d) = (9, 11, 15, 3, 13, 1, 2, 10)$ by simply listing the lengths of alternate runs of 0's and 1's. While the original binary data d requires 65 bits for storage, its compact representation $c(d)$ requires 32 bits only under the assumption that each length of run is being represented by 4 bits. The early facsimile compression standard (CCITT Group 3, CCITT Group 4) algorithms have been developed based on this principle [8].

3.6.2 Huffman coding

In 1952, D. A. Huffman [9] invented a coding technique to produce the shortest possible average code length, given the source symbol set and the associated probability of occurrence of the symbols. The Huffman coding technique is based on the following two observations regarding optimum prefix codes.

- The more frequently occurring symbols can be allocated with shorter codewords than the less frequently occurring symbols.
- The two least frequently occurring symbols will have codewords of the same length, and they differ only in the least significant bit.

The average of the length of these codes is closed to the entropy of the source. Let us assume that there are m source symbols $\{s_1, s_2, \dots, s_m\}$ with associated probabilities of occurrence $\{p_1, p_2, \dots, p_m\}$. Using these probability values, we can generate a set of Huffman codes of the source symbols. The Huffman codes can be mapped into a binary tree, popularly known as the Huffman tree. We describe below the algorithm to generate the Huffman codes of the source symbols.

1. Produce a set $N = \{N_1, N_2, \dots, N_m\}$ of m nodes as leaves of a binary tree. Assign a node N_i with the source symbol s_i , $i = 1, 2, \dots, m$, and label the node with the associated probability p_i .

(Example: As shown in Fig. 3.3, we start with eight nodes $N_0, N_1, N_2, N_3, N_4, N_5, N_6, N_7$ corresponding to the eight source symbols a, b, c, d, e, f, g, h , respectively. Probability of occurrence of each symbol is indicated in the associated parentheses.)

2. Find the two nodes with the two lowest probability symbols from the current node set, and produce a new node as a parent of these two nodes.

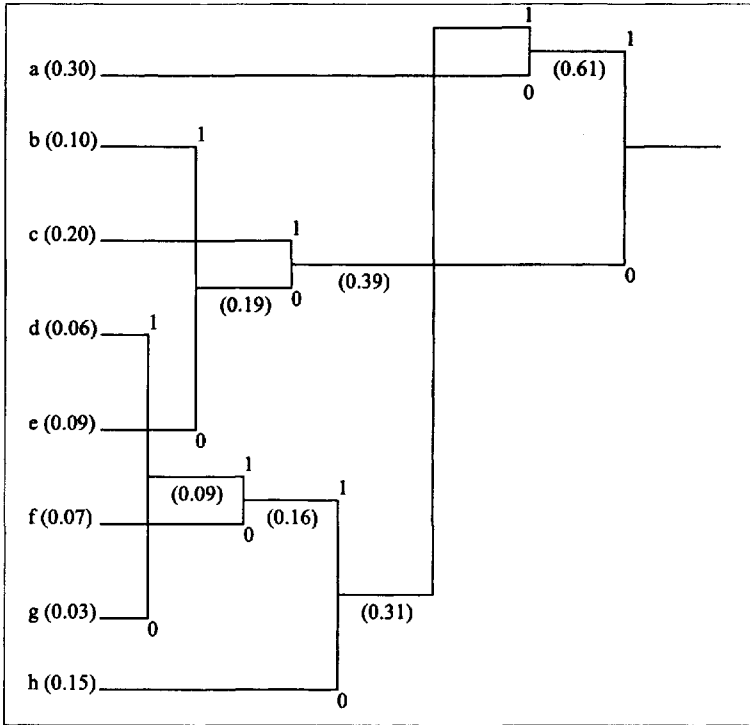


Fig. 3.3 Huffman tree construction for Example 2.

(**Example:** From Fig. 3.3 we find that the two lowest probability symbols g and d are associated with nodes N_6 and N_3 , respectively. The new node N_8 becomes the parent of N_3 and N_6 .)

3. Label the probability of this new parent node as the sum of the probabilities of its two child nodes.

(**Example:** The new node N_8 is now labeled by probability 0.09, which is the sum of the probabilities 0.06 and 0.03 of the symbols d and g associated with the nodes N_3 and N_6 , respectively.)

4. Label the branch of one child node of the new parent node as 1 and branch of the other child node as 0.

(**Example:** The branch N_3 to N_8 is labeled by 1 and the branch N_6 to N_8 is labeled by 0.)

5. Update the node set by replacing the two child nodes with smallest probabilities by the newly generated parent node. If the number of nodes remaining in the node set is greater than 1, go to Step 2.

(**Example:** The new node set now contains the nodes N_0 , N_1 , N_2 , N_4 , N_5 , N_7 , N_8 and the associated probabilities are 0.30, 0.10, 0.20, 0.09,

0.07, 0.15, 0.09, respectively. Since there are more than one node in the node set, Steps 2 to 5 are repeated and the nodes N_9 , N_{10} , N_{11} , N_{12} , N_{13} , N_{14} are generated in the next six iterations, until the node set consists of N_{14} only.)

6. Traverse the generated binary tree from the root node to each leaf node N_i , $i = 1, 2, \dots, m$, to produce the codeword of the corresponding symbol s_i , which is a concatenation of the binary labels (0 or 1) of the branches from the root to the leaf node.

(**Example:** The Huffman code of symbol h is "110," formed by concatenating the binary labels of the branches N_{14} to N_{13} , N_{13} to N_{11} and N_{11} to N_7 .)

Table 3.1 Huffman code table

Symbol	Probability	Huffman code
a	0.30	10
b	0.10	001
c	0.20	01
d	0.06	11111
e	0.09	000
f	0.07	1110
g	0.03	11110
h	0.15	110

3.6.2.1 Example 2: Assume an alphabet $S = \{a, b, c, d, e, f, g, h\}$ with probabilities $p(a) = 0.30$, $p(b) = 0.10$, $p(c) = 0.20$, $p(d) = 0.06$, $p(e) = 0.09$, $p(f) = 0.07$, $p(g) = 0.03$ and $p(h) = 0.15$, respectively. The Huffman tree for this source is depicted in Fig. 3.3, while the Huffman code is shown in Table 3.1.

Let us consider a string M of 200 symbols generated from the above source, where the numbers of occurrences of a , b , c , d , e , f , g and h in M are 60, 20, 40, 12, 18, 14, 6, and 30, respectively. Size of the encoded message M using the Huffman codes in Table 3.1 will be 550 bits. Here it requires 2.75 bits per symbol on the average. On the other hand, the length of the encoded message M will be 600 bits if it is encoded by a fixed length code of length 3 for each of the symbols. This simple example demonstrates how we can achieve compression using variable-length coding or source coding techniques.

3.7 PRINCIPAL COMPONENT ANALYSIS FOR DATA COMPRESSION

Principal Component Analysis has been a popular technique for data compression. It forms the basis of the Karhunen–Loeve (KL) transform for compact representation of data [5, 6, 10, 11]. The KL transform and the theory behind *principal component analysis* are of fundamental importance in signal and image processing. The principle has also found its place in data mining for reduction of large-dimensional datasets. It has been successfully applied to text analysis and retrieval for text mining as well [12]. The principal component analysis has been developed based on the matrix theory for *Singular Value Decomposition (SVD)*.

According to singular value decomposition (SVD) theory, for any arbitrary $M \times N$ matrix F of rank L there exists an $M \times M$ unitary matrix U and an $N \times N$ unitary matrix V so that

$$U^T F V = \Lambda^{\frac{1}{2}}, \quad (3.6)$$

where

$$\Lambda^{\frac{1}{2}} = \begin{bmatrix} \lambda^{\frac{1}{2}}(1) & & & & & \\ & \lambda^{\frac{1}{2}}(2) & & & & \\ & & \ddots & & & \\ & & & \lambda^{\frac{1}{2}}(L) & & \\ & & & & 0 & \\ & & & & & \ddots \\ & & & & & & 0 \end{bmatrix}$$

is an $M \times N$ diagonal matrix and the first L diagonal elements $\lambda^{\frac{1}{2}}(i)$, for $i = 1, 2, \dots, L$, are called the *singular* values of input matrix F . Since U and V are unitary matrices, we have

$$U U^T = I_M,$$

$$V V^T = I_N,$$

where I_M and I_N are the identity matrices of dimension M and N , respectively. As a result, the input matrix F can be decomposed as

$$F = U \Lambda^{\frac{1}{2}} V^T. \quad (3.7)$$

The columns of U are chosen as the eigenvectors u_m of the symmetric matrix FF^T so that

$$U^T(FF^T)U = \begin{bmatrix} \lambda(1) & & & & & \\ & \lambda(2) & & & & \\ & & \ddots & & & \\ & & & \lambda(L) & & \\ & & & & 0 & \\ & & & & & \ddots \\ & & & & & & 0 \end{bmatrix}, \quad (3.8)$$

where $\lambda(i)$, $i = 1, 2, \dots, L$, are the nonzero eigenvalues of FF^T . Similarly, the columns of matrix V are eigenvectors v_n of the symmetric matrix F^TF as defined by

$$V^T(F^TF)V = \begin{bmatrix} \lambda(1) & & & & & \\ & \lambda(2) & & & & \\ & & \ddots & & & \\ & & & \lambda(L) & & \\ & & & & 0 & \\ & & & & & \ddots \\ & & & & & & 0 \end{bmatrix}, \quad (3.9)$$

where $\lambda(i)$, $i = 1, 2, \dots, L$ are the corresponding nonzero eigenvalues of F^TF . The input matrix can be represented in series form by these eigenvalues and eigenvectors as

$$F = \sum_{i=1}^L \lambda^{\frac{1}{2}}(i) u_i v_i^T. \quad (3.10)$$

If the eigenvalues $\lambda(i)$, for $i = 1, 2, \dots, L$ are sorted in decreasing order and only first K from the sorted list are significant ($K < L$), then we can approximate the input matrix F by a smaller-dimensional matrix $\bar{F}$ using these first K eigenvalues and corresponding eigenvectors only.

The eigenvector corresponding to the highest eigenvalue of F^TF is called the *first principal component*. Likewise, the *second principal component* is the eigenvector corresponding to the next highest eigenvalue of F^TF , and so on. Hence the k th *principal component* is the eigenvector corresponding to the k th largest eigenvalue of F^TF .

The principal component analysis-based data reduction technique has been very popular in data mining, particularly to reduce the high-order dimensionality of data to lower orders. This is also the foundation of Karhunen-Loeve Transform used in many multimedia processing applications [5, 6, 10, 11]. *Latent Semantic Analysis* (LSA) technique has been developed based on this theory of *SVD*, using a compact representation of the data in terms of a few

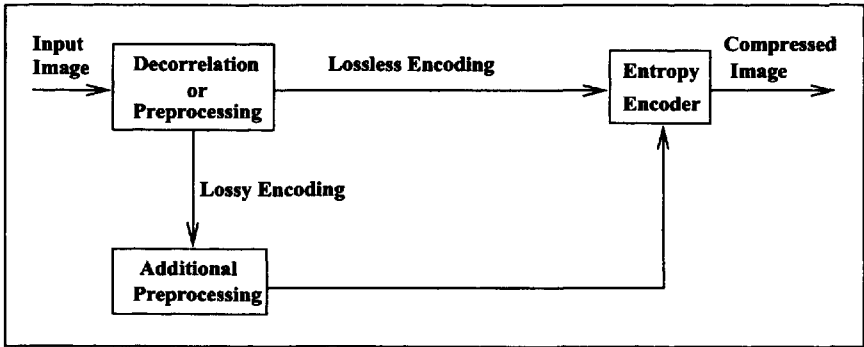


Fig. 3.4 A general image compression framework.

principal components only. LSA has been effectively applied in text analysis suitable for text mining [12], and has been described in detail in Section 9.2.5.

3.8 PRINCIPLES OF STILL IMAGE COMPRESSION

The statistical analysis of a typical image indicates that there is a strong correlation amongst neighboring pixels. This causes redundancy of information in the image. In general, still image compression techniques rely on two fundamental redundancy reduction principles, namely, *Spatial* and *Statistical*. Spatial redundancy is the similarity of neighboring pixels in an image and is reduced by decorrelating the pixels. The statistical redundancy reduction is referred to as *entropy encoding*, accomplished by a source coding algorithm.

The general model of a still image compression framework is shown as a block diagram in Fig. 3.4. The *decorrelation* or *preprocessing* block is the step for reducing the spatial redundancy of the image pixels. In lossless coding mode, this decorrelated image is directly processed by the entropy encoder. On the other hand, for lossy compression, the decorrelated image is further preprocessed as shown in Fig. 3.4 in order to mask irrelevant information depending upon the application criteria. This process is popularly called *Quantization*. The quantized pixels are then entropy-encoded using a source coding algorithm to compactly represent the image. We now discuss different image coding principles in the following sections.

3.8.1 Predictive coding

Since the adjacent pixels in a typical image are highly correlated, it is possible to extract a great deal of information about a pixel from its neighboring pixel values. In predictive coding, a pixel value is predicted by a set of previously encoded neighboring pixels. For an ideal prediction model, the predicted value

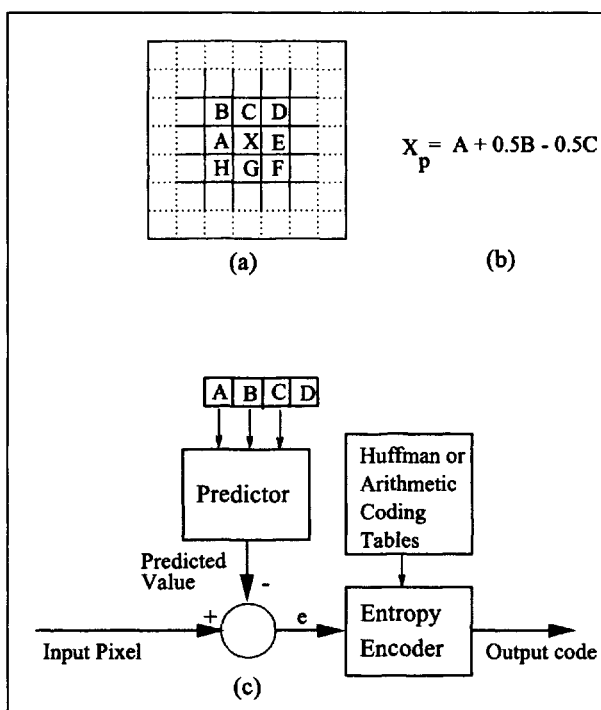


Fig. 3.5 DPCM showing (a) neighbors of pixel X to be encoded, (b) prediction function, and (c) block diagram.

of the pixel is similar to the actual value. But in reality, using an effective prediction model, we can predict a pixel value which is very close to its actual value. A practical approach to the prediction model is to take a linear combination of the previously encoded neighboring pixels. The reason for taking the previously encoded pixels is that the same pixels will be available to the decoder, when it decodes the pixels in the same order that they were encoded. The difference between the actual pixel value and the predicted value is called *differential* or *prediction error*. The error value e is then entropy-encoded using a variable-length encoding technique to generate the compressed image. This method is popularly known as *Differential Pulse Code Modulation* (DPCM). The block diagram is provided in Fig. 3.5(c).

In Fig. 3.5(a), let pixels A, B, C, D, E, F, G , and H be the immediate neighbors of pixel X to be encoded. If we follow the raster scan order convention to access the image from left-to-right and top-to-bottom, the previously encoded pixels available to the predictor will be A, B, C, D . We assume that the prediction function is $X_p = A + 0.5B - 0.5C$, using A, B , and C only, as shown in Fig. 3.5(b). The prediction error $e = X - X_p$ has less entropy as compared to X , because of the reduction in spatial redundancy. As

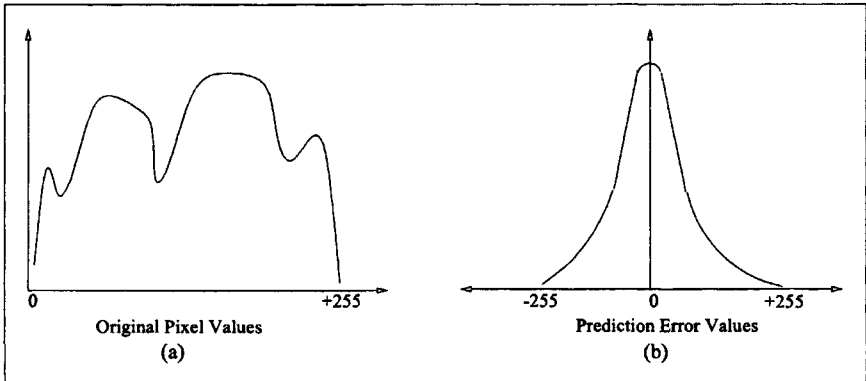


Fig. 3.6 Histogram of (a) a typical image and (b) its prediction error.

a result, the error value e can be encoded by less number of bits to achieve compression. Effectiveness of the error image for entropy encoding, as compared to direct encoding of the image, can be explained by the corresponding histograms shown in Fig. 3.6.

Statistical distribution of pixels in a typical image is uniform distribution in nature, as shown in Fig. 3.6(a). Because of uniform distribution, average number of bits per pixels in the range $[0, 255]$ will be eight. In Fig. 3.6(b), we provide the statistical distribution of the prediction error values of the same image after applying the DPCM technique. The prediction error values belong to the range $[-255, +255]$. It is clear from Fig. 3.6(b) that the statistical distribution of the prediction errors of a typical image is Laplacian in nature, and most of the prediction errors are skewed around zero. As a result, we can apply a statistics-based entropy encoding technique to allocate smaller binary codes to the prediction error values close to zero and larger codes to bigger error values. Hence the average number of bits per pixel error will be less than eight, thereby resulting in compression.

3.8.2 Transform coding

In predictive coding, the coding process takes place pixel by pixel. *Transform coding* is an effective way of coding a group of spatially correlated pixels [10]. This technique takes advantage of the fact that the energy of most natural images is mainly concentrated in the low-frequency regions.

A suitable transformation technique produces fewer number of correlated transformed coefficients as compared to the original image, and a significant amount of image information is concentrated in these fewer correlated transformed coefficients. As a result, we can discard or mask the insignificant transformed coefficients, mainly consisting of the high-frequency components, using a suitable quantization technique without affecting the desired recon-

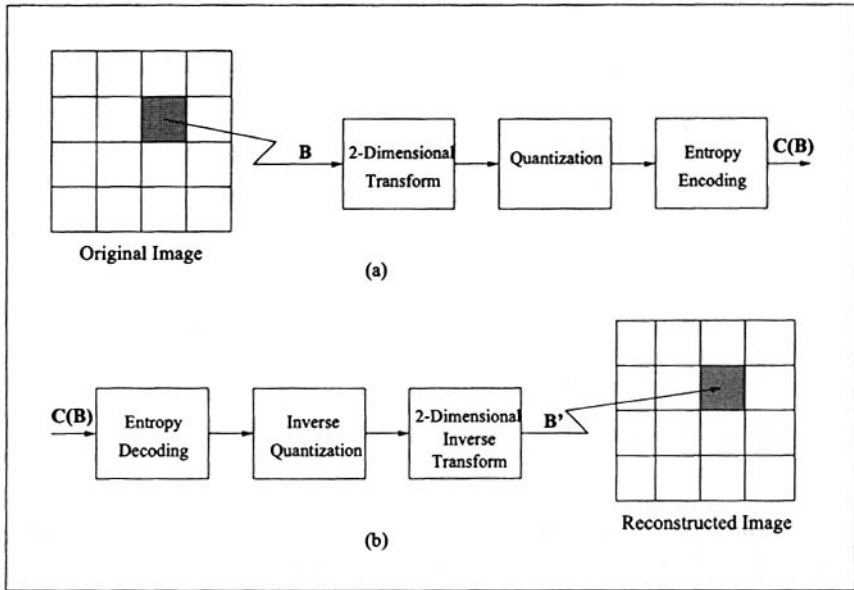


Fig. 3.7 Transform coding-based (a) compression and (b) decompression.

structed image quality. This is possible because the human visual system has perceptual masking effects, so that the high-frequency components are not as sensitive to reconstruction errors as compared to their low-frequency counterparts. If the quantization process is not too coarse, then the reconstructed image can be perceptually similar to the original one.

The general framework for transform coding-based image compression systems is depicted in Fig. 3.7(a). Usually the input image is first divided into a number of smaller rectangular blocks B . Each of these blocks are then independently transformed, using the chosen linear transformation technique. The transformed coefficients are quantized and entropy encoded into bit-stream $c(B)$ in order to achieve compression. During the decompression, as shown in Fig. 3.7(b), the compressed bit stream $c(B)$ is first entropy-decoded to generate the quantized coefficients. This is followed by inverse quantization in order to generate an approximation of the transformed coefficients. The inverse transformation is applied on these coefficients to reconstruct the image block B' . The composition of the reconstructed image blocks forms the reconstructed image, as shown in Fig. 3.7(b).

In transform coding, the selection of the transformation technique is a major decision. The main motivation behind transformation from the spatial domain to another domain (usually frequency domain) is to represent the data in a more compact form in the transformed domain. The optimum transformation also minimizes the mean squared error of the reconstructed image. The *Karhunen-Loeve Transform* (KLT) [5, 10] has been proven to

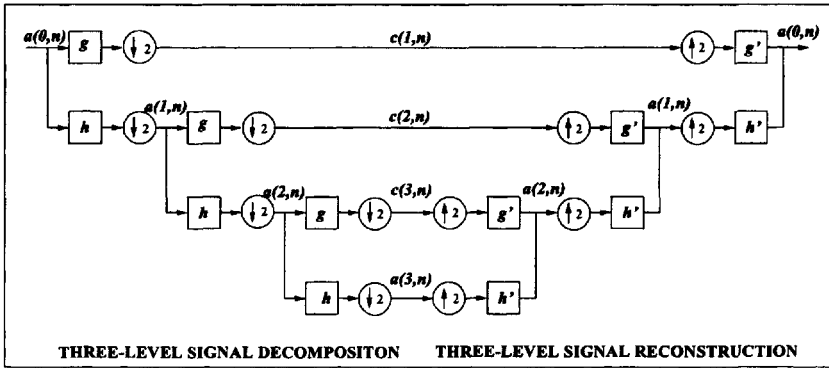


Fig. 3.8 Three-level multi-resolution wavelet decomposition and reconstruction of a signal.

be optimal in terms of the compaction efficiency, by representing the image using few principal components containing a significant portion of the image information. KLT packs the most energy of an image block in the least number of transformed coefficients. It completely decorrelates the pixels, and hence minimizes the spatial redundancy in the image block.

Although KLT is optimum, it is not efficient for practical implementations. The basis functions of KLT are input data-dependent, because they are formed by the eigenvectors of the autocorrelation matrix of the input signal. There is no fast algorithm for practical implementation of KLT, because of its dependency on the input source signal. As a result, we have to choose a suboptimal transform, so that the basis functions are not signal-dependent and a fast algorithm can exist in order to have a practical implementation. A number of suboptimal transforms like the Discrete Fourier Transform (DFT), Discrete Cosine Transform (DCT), Discrete Sine Transform (DST), and Discrete Hadamard Transform (DHT), to name a few, have been used in digital image compression [6, 10]. Of these the DCT is the most popular block-based transform, because its performance is very close to that of KLT and a number of fast algorithms exist for DCT [13]. DCT is the basis for most of the image and video compression algorithms, especially the still image compression standard JPEG in lossy mode and the video compression standards MPEG-1, MPEG-2, MPEG-4, H.263, etc. [14].

3.8.3 Wavelet coding

Representation of a signal using Fourier series in terms of the sinusoids is well known in the signal processing community, provably for more than a century, and is known to be effective for *stationary* as well as *nonstationary* signals. The non-stationary signals are not statistically predictable, especially in the regions of discontinuities. In a typical image, discontinuities occur at the

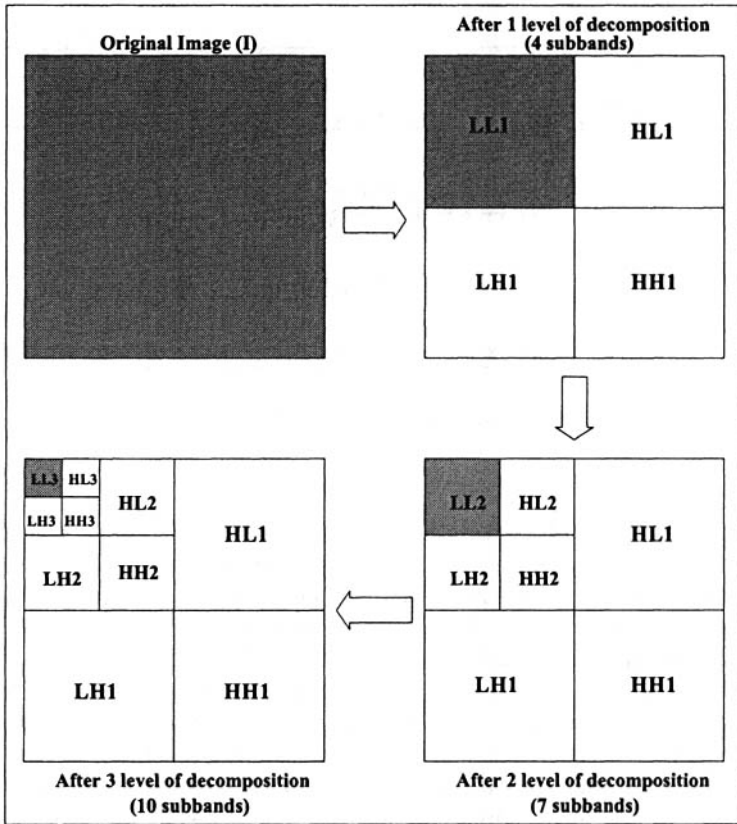


Fig. 3.9 Three level multiresolution wavelet decomposition of an image.

edges. The basics of wavelet representation of signals have been discussed in Section 2.2.7, using Eq. (2.32) and Fig. 2.7. In this section, we introduce how the wavelet transformation is applied in image coding. Wavelet coding is a transform coding technique that is not limited to the block-based implementation only. Usually the wavelet transform is performed on the whole image. Wavelet transform decomposes the input signal into low-frequency and high-frequency subbands.

In 1989, Mallat [15] proposed the multiresolution approach for wavelet decomposition of signals using a pyramidal FIR filter structure of QMF filter pairs, which practically mapped the wavelet decomposition into the subband coding paradigm. Once we express the wavelet decomposition in terms of FIR filters, the same general principles of subband coding of digital images applies. In multiresolution analysis, it can be proven that the decomposition of signals using discrete wavelet transform can be expressed in terms of FIR

filters [16] as

$$c_{m,n}(f) = \sum_k a_{m-1,k}(f) g_{2n-k} \quad (3.11)$$

and

$$a_{m,n}(f) = \sum_k a_{m-1,k}(f) h_{2n-k}, \quad (3.12)$$

where g and h are the high-pass and low-pass discrete FIR filters whose coefficients are related as $g_i = (-1)^i h_{-i+1}$. If the input signal $f(t)$ is given in discrete sampled form, say $a_{0,n}(f)$, to indicate original discrete samples at resolution 0, then the above equations describe a multiresolution subband decomposition of samples into $a_{m,n}(f)$ and $c_{m,n}(f)$ at level m using the subband decomposed samples $c_{m-1,n}(f)$ at level $m-1$, with a low-pass FIR filter h and high-pass FIR filter g . The output subbands in every level are formed by retaining every other filtered output sample, which results in decimation of the output by a factor of 2. These filters are called the analysis filters. Using the corresponding synthesis filters h' and g' , it can be shown that the signal $a_{0,n}$ can be exactly reconstructed in m levels using the formula

$$a_{m-1,i}(f) = \sum_n a_{m,n}(f) h'_{2n-i} + \sum_n c_{m,n}(f) g'_{2n-i}. \quad (3.13)$$

Figure 3.8 depicts the multiresolution decomposition approach using the analysis filters g and h , and reconstruction of the same using synthesis filters g' and h' .

Image compression techniques using Discrete Wavelet Transform (DWT) have received wide attention in recent years [17, 18]. Using a separable two-dimensional filtering function, two-dimensional DWT can be computed by applying one-dimensional DWT row-wise and column-wise independently. In Fig. 3.9, we show an example of hierarchical wavelet decomposition of an image into ten subbands after three levels of decomposition. After the first level of decomposition, the original image is decomposed into four subbands $LL1$, $HL1$, $LH1$, and $HH1$. The $LL1$ subband is the low-frequency subband which can be considered as a 2:1 subsampled (horizontally and vertically) version of the original image I , and its statistical characteristic is similar to the original image as shown by the shaded regions in Fig. 3.9. Here $HL1$, $LH1$, and $HH1$ are called the high-frequency subbands, where $HL1$ and $LH1$ correspond to the horizontal and vertical high frequencies, respectively. $HH1$ constitutes the high frequencies that are not in either horizontal or vertical orientations. Each of these spatially oriented (horizontal, vertical, or diagonal) subbands mostly contain information of local discontinuities in the image, and the bulk of the energy in each of the high-frequency subbands are concentrated in the vicinity of areas which correspond to edge activity in the original image.

Since the low-frequency subband $LL1$ has similar spatial and statistical characteristics as the original image, it can be further decomposed into four

subbands *LL2*, *HL2*, *LH2*, and *HH2*. Continuing the same method for decomposition in *LL2*, the original image is decomposed into 10 subbands *LL3*, *HL3*, *LH3*, *HH3*, *HL2*, *LH2*, *HH2*, *HL1*, *LH1*, and *HH1* after three levels of pyramidal multiresolution subband decomposition, as shown in Fig. 3.9. The same procedure can continue to further decompose *LL3* into higher levels.

Using the right wavelet filters and choosing an effective quantization strategy for each subband can yield good compression performance. Each decomposed subband may be encoded separately using a suitable coding scheme. We can allocate different bit-rates to different subbands. Because of the hierarchical nature of the subbands in wavelet decomposition, a smaller number of bits need to be allocated to the high-frequency subbands in a lower level as compared to the high-frequency subbands in upper levels. This helps maintain good fidelity of the reconstructed image and thereby achieves good compression. Experimental results show that we can even allocate zero bits to the *HH1* subband and still maintain good reconstructed quality in most of the natural images.

3.9 IMAGE COMPRESSION STANDARD: JPEG

JPEG is the acronym for *Joint Photographic Experts Group*. It is the first international image compression standard for continuous-tone still images, including both gray scale and color images [7]. The goal of this standard is to support a variety of applications for compression of continuous-tone still images (i) of different sizes, (ii) in any color space, (iii) in order to achieve compression performance at or near the state of the art, (iv) with user-adjustable compression ratios, and (v) with very good to excellent reconstructed quality. Another goal of this standard is that it should have manageable computational complexity for widespread practical implementation. JPEG defines the following four modes of operation [7].

1. *Sequential Lossless Mode*: Compress the image in a single scan, and the decoded image is an exact replica of the original image.
2. *Sequential DCT-Based Mode*: Compress the image in a single scan using DCT-based lossy compression technique. As a result, the decoded image is not an exact replica but an approximation of the original image.
3. *Progressive DCT-Based Mode*: Compress the image in multiple scans and also decompress the image in multiple scans, with each successive scan producing a better quality image.
4. *Hierarchical Mode*: Compress the image at multiple resolutions for display on different devices.

The three DCT-based modes (2, 3, and 4) in JPEG provide lossy compression, because the precision limitation to digitally compute DCT (and

its inverse) and the quantization process introduce distortion in the reconstructed image. For sequential lossless mode of compression, predictive coding (DPCM) is used instead of the DCT-based transformation and also there is no quantization involved. The simplest form of sequential DCT-based JPEG is called the **baseline JPEG** algorithm. We shall describe the *JPEG lossless* algorithm and the *baseline JPEG* algorithm in greater detail in the following two sections.

3.10 THE JPEG LOSSLESS CODING ALGORITHM

Lossless JPEG compression is based on the principles of predictive coding. In this scheme, the value of a pixel X is first predicted by using one or more of the previously encoded adjacent pixels A , B , and C as shown in Fig. 3.10(a). It then encodes the difference between the original pixel and its predicted value, usually called the *prediction error* or *prediction residual*, by either Huffman coding or binary arithmetic coding (QM-coder) [7].

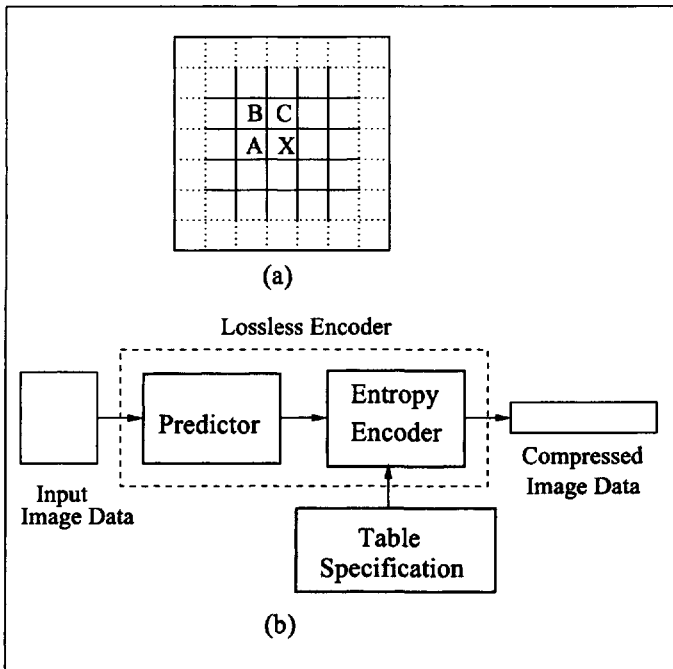


Fig. 3.10 JPEG lossless model with (a) 3-pixel prediction neighborhood and (b) encoder diagram.

There are eight possible options for prediction as shown in Table 3.2. Options 1 to 3 are one-dimensional predictors, while options 4 to 7 deal with two

dimensions. Depending upon the nature of the image, one predictor may yield better compression result as compared to another. However, experimental results on various kinds of images show that on the average their performance are relatively close to each other [19]. The chosen option for prediction is indicated in the header of the compressed file, so that both the encoder and decoder use the same function for prediction.

Table 3.2 Prediction functions in lossless JPEG

Option	Prediction function	Type of prediction
0	No prediction	Differential Coding
1	$X_p = A$	1-D Horizontal Prediction
2	$X_p = B$	1-D Vertical Prediction
3	$X_p = C$	1-D Diagonal Prediction
4	$X_p = A + B - C$	2-D Prediction
5	$X_p = A + \frac{1}{2}(B - C)$	2-D Prediction
6	$X_p = B + \frac{1}{2}(A - C)$	2-D Prediction
7	$X_p = \frac{1}{2}(A + B)$	2-D Prediction

Table 3.3 Categories of prediction error values

Category	Prediction error value
0	0
1	-1, +1
2	-3, -2, +2, +3
3	-7, ..., -4, +4, ..., +7
4	-15, ..., -8, +8, ..., +15
5	-31, ..., -16, +16, ..., +31
6	-63, ..., -32, +32, ..., +63
7	-127, ..., -64, +64, ..., +127
8	-255, ..., -128, +128, ..., +255
9	-511, ..., -256, +256, ..., +511
10	-1023, ..., -512, +512, ..., +1023
11	-2047, ..., -1024, +1024, ..., +2047
12	-4095, ..., -2048, +2048, ..., +4095
13	-8191, ..., -4096, +4096, ..., +8191
14	-16383, ..., -8192, +8192, ..., +16383
15	-32767, ..., -16384, +16384, ..., +32767
16	+32768

In the lossless mode, the standard allows precision P of the input source image to be 2 bits to 16 bits wide. Since there is no previously encoded pixel

known to the encoder when it encodes the very first pixel in the very first row of the image, it is handled differently. For a given input precision P and a point transform parameter P_t , the predicted value for the first pixel in the first line is 2^{P-P_t-1} . By default, we can assume $P_t = 0$. For details of the point transform parameter, the reader is advised to consult the JPEG standard [7]. For all other pixels (except the first one) in the first line, we use option 1 for prediction function. Except for the first line, option 2 is used to predict the very first pixel in all other lines. For all other pixels, we select one of the eight options for prediction function from Table 3.2. Once a predictor is selected, it is used for all other pixels in the block.

In lossless JPEG standard, the prediction error values are computed *modulo* 2^{16} to take into consideration the full precision allowed in this mode. These error values are first represented as a pair of symbols (*CATEGORY*, *MAGNITUDE*). The first symbol *CATEGORY* represents the category of the error value. The second symbol *MAGNITUDE* represents the Variable-Length Integer (VLI) for the prediction error value. *CATEGORY* represents the number of bits to encode *MAGNITUDE* in terms of VLI. All the possible prediction error values, *modulo* 2^{16} , and their corresponding categories are shown in Table 3.3. Only the *CATEGORY* in the symbol pair for each prediction error value is Huffman coded.

The codeword for the symbol pair (*CATEGORY*, *MAGNITUDE*) is formed in two steps. First it assigns the Huffman code of the *CATEGORY*. This Huffman code is then appended with additional *CATEGORY* number of bits to represent the *MAGNITUDE* in VLI. If the prediction error value is positive, the *MAGNITUDE* is directly binary represented by a VLI using *CATEGORY* number of bits and hence it starts with bit 1. If the error value is negative, the VLI is 1's complement of its absolute value and hence it starts with bit 0.

For example, the prediction error value "25" is represented by the pair (5, 25) because the number 25 belongs to category 5 in Table 3.3 and hence 25 is represented by a 5-bit VLI. If the Huffman code for category 5 is "011," then the binary codeword for the error value 25 will be "01111001." The first three bits correspond to the Huffman code "011" for category 5 and next 5 bits "11001" is the VLI for 25. Similarly, the prediction error value -25 will be represented as "01100110." Here the last 5 bits '00110' is the 1's complement of "11001" to represent -25 and, since -25 belongs to the same category 5, the first three bits of the codeword correspond to Huffman code of category 5.

Use of the *CATEGORY* of the error values greatly simplifies the Huffman coder. Without this categorization, we would require to use a Huffman table with 2^{16} entries for all the 2^{16} possible symbols of prediction error values. Detailed information for implementation of the JPEG lossless coding can be found in Annex H of the JPEG standard [7].

3.11 BASELINE JPEG COMPRESSION

Among the four modes of the JPEG family, the baseline JPEG compression algorithm is most widely used. It is defined for compression of continuous-tone images with one to four components. The number of components for gray scale images is one, whereas a color image can have up to four color components. The baseline JPEG allows only 8-bit samples within each component of the source image. An example of a four-component color image is a CMYK (cyan, magenta, yellow and black) image which is used in many applications such as printing, scanning etc.

A color image for display has three color components RGB (red, green and blue). In a typical color image, the spatial intercomponent correlation between the red, green, and blue color components is significant. In order to achieve good compression performance, the correlation between color components is first reduced by converting the RGB image into a decorrelating color space. In baseline JPEG, a three-color RGB image is first transformed into a Luminance-Chrominance (L-C) color space such as YCbCr, YUV, CIELAB, etc. The advantage of converting an image into Luminance-Chrominance color space is that the luminance and chrominance components are very much decorrelated between each other. Moreover, the chrominance channels contain many redundant information and can easily be subsampled without sacrificing any visual quality of the reconstructed image.

3.11.1 Color space conversion

In this section, we consider color space conversion only from RGB to YCbCr and vice versa. There are several ways to convert from RGB to YCbCr color space. Here we adopt the CCIR (International Radio Consultative Committee) Recommendation 601-1. This is the typical method for color conversion used in baseline JPEG compression. According to CCIR 601-1 Recommendation, the transformation from RGB to YCbCr is done based on the mathematical expression

$$\begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} = \begin{pmatrix} 0.29900 & 0.58700 & 0.11400 \\ -0.16874 & -0.33126 & 0.50000 \\ 0.50000 & -0.41869 & -0.08131 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix}.$$

Color space conversion from RGB to YCbCr using the above transformation may result in negative numbers for Cb and Cr, while Y is always positive. In order to represent Cb and Cr as unsigned 8-bit integers, they are level-shifted by adding 128 to each sample followed by rounding and saturating the value

in the range $[0, 255]$. Hence the above transformation can be expressed as

$$\begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} = \begin{pmatrix} 0.29900 & 0.58700 & 0.11400 \\ -0.16874 & -0.33126 & 0.50000 \\ 0.50000 & -0.41869 & -0.08131 \end{pmatrix} \begin{pmatrix} R \\ G \\ B \end{pmatrix} + \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix}, \quad (3.14)$$

in order to produce 8-bit unsigned integers for each of the components in the YCbCr domain. Accordingly, the inverse transformation from YCbCr to RGB is done as

$$\begin{pmatrix} R \\ G \\ B \end{pmatrix} = \begin{pmatrix} 1.0 & 0.0 & 1.40210 \\ 1.0 & -0.34414 & -0.71414 \\ 1.0 & 1.77180 & 0.0 \end{pmatrix} \begin{pmatrix} Y \\ Cb \\ Cr \end{pmatrix} - \begin{pmatrix} 0 \\ 128 \\ 128 \end{pmatrix}. \quad (3.15)$$

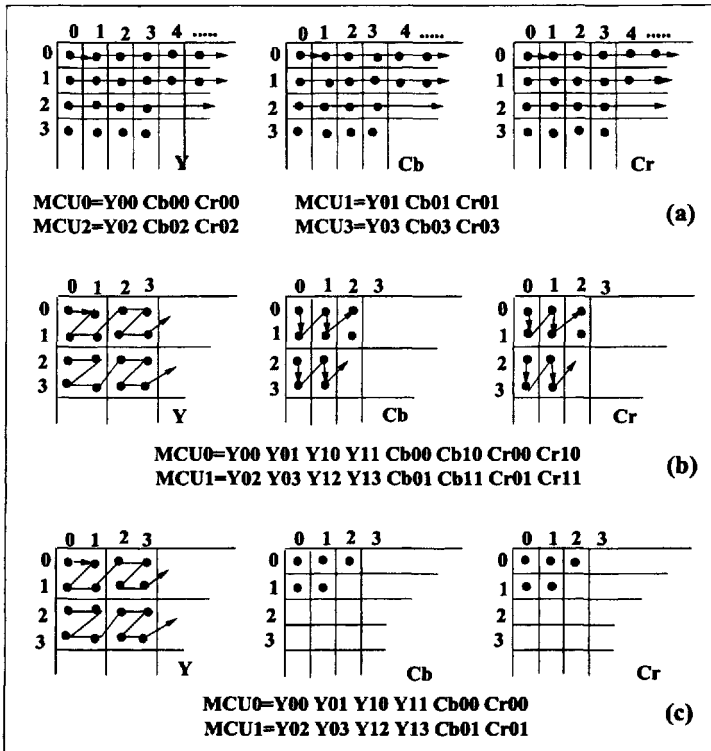


Fig. 3.11 Definition of MCUs for (a) YCbCr 4:4:4, (b) YCbCr 4:2:2, (c) YCbCr 4:2:0.

After the color space conversion, most of the spatial information of the image is contained in the luminance component (Y). The chrominance components (Cb and Cr) contain mostly redundant color information, and we lose little information by subsampling these components both horizontally and/or

vertically. We can subsample the chrominance components by simply throwing away every other sample in each row and/or each column, if desired. If we subsample the redundant chrominance components both horizontally and vertically, the amount of data required to represent the color image is reduced to half because each chrominance component now has only half resolution both in horizontal and vertical directions. This color format is called 4:2:0 color subsampling format.

Baseline JPEG also supports 4:2:2 and 4:4:4 color formats. Each chrominance component in the 4:2:2 color format has the same vertical resolution as the luminance component, but the horizontal resolution is halved by dropping alternate samples in each row. In the 4:4:4 format, both the chrominance components Cb and Cr have identical vertical and horizontal resolution as the luminance component. Hence no subsampling is done here. The subsampling operation to generate the 4:2:0 or 4:2:2 color format is the first lossy step.

3.11.2 Source image data arrangement

In the previous section we have seen that the dimension of each of the color components Y, Cb, and Cr could be different, depending upon the color subsampling format. Each color component is divided into 8×8 nonoverlapping blocks. Selecting one or more such data blocks from each of the color components, we can form what is called a *minimum coded unit* (MCU) in JPEG. The standard defines the arrangement of the data blocks in either interleaved or noninterleaved scanning order of the color components. In a noninterleaved scan, the data blocks in each color component are stored and processed separately in raster scan order, left-to-right and top-to-bottom. In interleaved order, data blocks from all the color components appear in each MCU. Definition of the MCUs for 4:4:4, 4:2:2, and 4:2:0 formats of YCbCr images in interleaved scan is shown in Fig. 3.11.

Each dot in Fig. 3.11 represents an 8×8 data block. In 4:4:4 format interleaved scan, each MCU consists of a data block from each of the Y, Cb, and Cr component as shown in Fig. 3.11(a). The order of processing these blocks is in the scan order from left-to-right and top-to-bottom. For example, the first MCU consists of the first data block Y00 from the Y component followed sequentially by the first data blocks Cb00 from the Cb component and Cr00 from the Cr component as shown in Fig. 3.11(a). The next MCU consists of Y01, Cb01, and Cr01, respectively. After all the MCUs consisting of the 8×8 data blocks from the first row (as depicted in Fig. 3.11(a)) are encoded, the second row of 8×8 blocks are scanned in a similar fashion. This procedure is continued until the last 8×8 block in the raster scan is encoded.

In 4:2:2 format, each MCU consists of a 2×2 unit of four data blocks from the Y component followed by a 2×1 unit of two data blocks from each of the Cb and Cr components. The corresponding order of processing is shown in Fig. 3.11(b). In 4:2:0 format, each MCU consists of 2×2 units of four data blocks from the Y component followed by one from each of the Cb and

Cr components, and the corresponding order of processing is illustrated in Fig. 3.11(c).

3.11.3 The baseline compression algorithm

The baseline JPEG algorithm follows the principles of block-based transform coding. Block diagram of the baseline JPEG algorithm for a gray scale image with a single component is shown in Fig. 3.12. For a color image, the same algorithm is applied to each 8×8 data block based on the source image data arrangement described in Section 3.11.2.

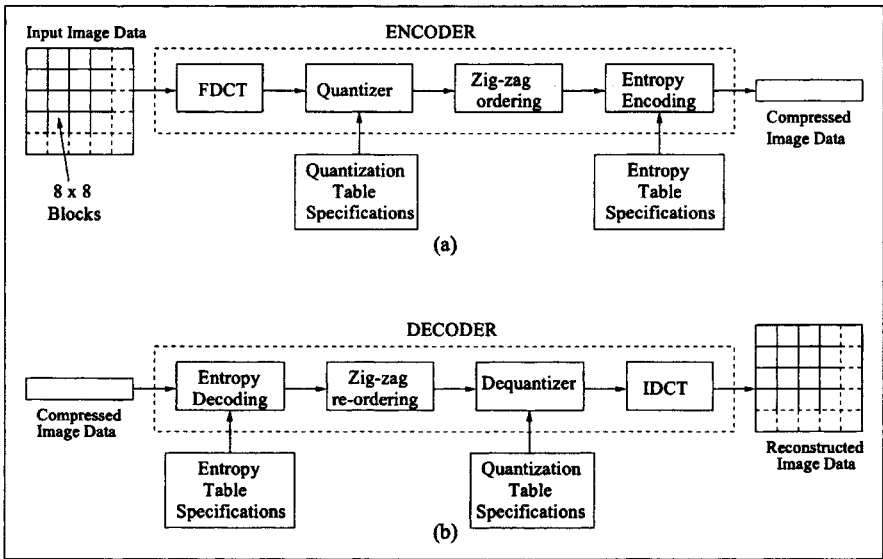


Fig. 3.12 JPEG baseline (a) compression and (b) decompression.

The image component is first divided into nonoverlapping 8×8 blocks in the raster scan order left-to-right and top-to-bottom as depicted in Fig. 3.12(a). Each block is then encoded separately by the *Encoder*, shown by the broken box in Fig. 3.12(a). The first step is to level shift each pixel in the block to convert into a signed integer, by subtracting 128 from each pixel. Each level shifted pixel in 8×8 block is then transformed into the frequency domain via forward DCT (FDCT). The FDCT of 8×8 block of pixels $f(x, y)$ for $(x, y = 0, 1, \dots, 7)$ is defined as

$$F(u, v) = \frac{1}{4} C(u) C(v) \sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cos \left[\frac{\pi(2x+1)u}{16} \right] \cos \left[\frac{\pi(2y+1)v}{16} \right], \quad (3.16)$$

for $u = 0, 1, \dots, 7$ and $v = 0, 1, \dots, 7$, where

$$C(k) = \begin{cases} \frac{1}{\sqrt{2}} & \text{for } k = 0 \\ 1 & \text{otherwise.} \end{cases}$$

The transformed 8×8 block now consists of 64 DCT coefficients. The first coefficient $F(0,0)$ is the DC component of the block, while the other 63 coefficients are the AC components $AC_{u,v} = F(u,v)$ of the block as shown in Fig. 3.13. The DC component $F(0,0)$ is essentially the sum of 64 pixels in the input 8×8 pixel block multiplied by the scaling factor $\frac{1}{4}C(u)C(v) = \frac{1}{8}$, as in Eq. (3.16).

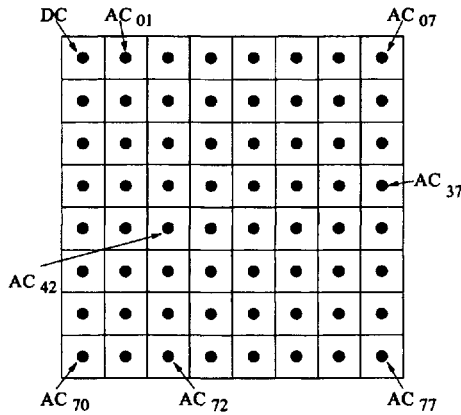


Fig. 3.13 DC and AC components of the transformed block.

The next step in the compression process is to quantize the transformed coefficients. This step is primarily responsible for losing information, and hence introduces distortion in the reconstructed image. That is the reason why baseline JPEG is a lossy compression. Each of the 64 DCT coefficients are uniformly quantized. The 64 quantization step-size parameters for uniform quantization of the 64 DCT coefficients form an 8×8 *Quantization Matrix*. Each element in the *Quantization Matrix* is an integer between 1 and 255. Each DCT coefficient $F(u,v)$ is divided by the corresponding quantizer step-size parameter $Q(u,v)$ in the *Quantization Matrix* and is rounded to the nearest integer as

$$F_q(u,v) = \text{Round} \left(\frac{F(u,v)}{Q(u,v)} \right). \quad (3.17)$$

The JPEG standard does not define any fixed *Quantization Matrix*, and it is the prerogative of the user to select the matrix. There are two quantization matrices provided in Annex K of the JPEG standard for reference, but not as a requirement. These two quantization matrices are shown in Tables 3.4 and 3.5, respectively.

Table 3.4 Luminance Quantization Matrix

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Table 3.5 Chrominance Quantization Matrix

17	18	24	47	99	99	99	99
18	21	26	66	99	99	99	99
24	26	56	99	99	99	99	99
47	66	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99
99	99	99	99	99	99	99	99

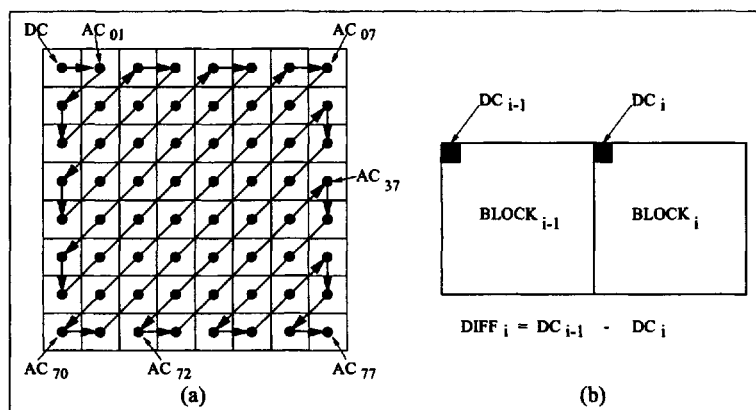


Fig. 3.14 Encoding of quantized DCT coefficients, with (a) zig-zag ordering of AC coefficients, and (b) differential coding of DC.

Table 3.4 is the *Luminance Quantization Matrix* for quantizing the transformed coefficients of the luminance component of an image. Similarly, Table 3.5 is the *Chrominance Quantization Matrix* for quantizing the transformed coefficients of the chrominance components of the image. These two quantization tables have been designed based on the psychovisual experiments by Lohscheller [20] to determine the visibility thresholds for 2-D basis functions. These tables may not be suitable for all kinds of images, but they provide reasonably good result for most natural images with 8-bit precision for luminance and chrominance samples. If the elements in these tables are divided by 2, we get perceptually lossless compression, whereby the reconstructed image is indistinguishable from the original one by human eyes. When the quantization tables are designed based on the perceptual masking properties of the human eye, many of the small DCT coefficients (high-frequency samples) are zeroed out to aid significant compression. This is done by using larger quantization step-size parameters for higher-frequency AC components, as depicted in Tables 3.4 and 3.5.

Quality of the reconstructed image and the achieved compression can be controlled by a user, by selecting a quality factor Q_JPEG to tune the elements in the quantization tables, as proposed by the *Independent JPEG Group* and implemented in their software [21]. The value of Q_JPEG may vary from 1 to 100. The quantization matrices in Tables 3.4 and 3.5 have been set for $Q_JPEG = 50$. For other Q_JPEG values, each element in both the tables are simply scaled by a factor α defined as [21]

$$\alpha = \begin{cases} \frac{50}{Q_JPEG} & \text{if } 1 \leq Q_JPEG \leq 50, \\ 2 - \frac{Q_JPEG}{50} & \text{if } 50 \leq Q_JPEG \leq 100, \end{cases} \quad (3.18)$$

subject to the condition that the minimum value of the scaled quantization matrix elements $\alpha Q(u, v)$ is 1. For the best reconstructed quality, Q_JPEG is set to 100.

After quantization of the DCT coefficients, the quantized DC coefficient is encoded by differential encoding. The DC coefficient DC_i of the current block is subtracted from the DC coefficient DC_{i-1} of the previous block and the difference

$$DIFF_i = DC_{i-1} - DC_i \quad (3.19)$$

is encoded as shown in Fig. 3.14(b). This is done to exploit the spatial correlation between the DC values of the adjacent blocks.

Encoding of the AC coefficients is not straightforward. Instead of encoding each AC coefficient in the block, only the significant (nonzero) coefficients are encoded in an efficient manner such that the runs of zeros preceding a nonzero value is embedded into the encoding. Usually there are few significant low-frequency AC coefficients in the whole 8×8 block, and most of the higher-frequency coefficients are quantized to zeros. In order to exploit this property, the AC coefficients are ordered in a particular irregular order sequence as shown in Fig. 3.14(a). This irregular ordering of the AC coefficients is called the *zig-zag ordering*. It is done to keep the low-frequency coefficients together, and it forms long runs of zeros corresponding to the higher-frequency quantized coefficients. This zig-zag sequence is then broken into runs of zeros ending in a nonzero value.

Before we explain the entropy encoding procedure, let us demonstrate the results of level shifting, DCT, quantization, and zig-zag ordering with an example 8×8 block extracted from a natural image.

110	110	118	118	121	126	131	131
108	111	125	122	120	125	134	135
106	119	129	127	125	127	138	144
110	126	130	133	133	131	141	148
115	116	119	120	122	125	137	139
115	106	99	110	107	116	130	127
110	91	82	101	99	104	120	118
103	76	70	95	92	91	107	106

Example: One 8×8 data block

-18	-18	-10	-10	-7	-2	3	3
-20	-17	-3	-6	-8	-3	6	7
-22	-9	1	-1	-3	-1	10	16
-18	-2	2	5	5	3	13	20
-13	-12	-9	-8	-6	-3	9	11
-13	-22	-29	-18	-21	-12	2	-1
-18	-37	-46	-27	29	-24	-8	-10
-25	-52	-58	-33	-36	-37	-21	-22

Level shifted 8×8 data block

-89.00	-63.47	18.21	-6.85	7.50	13.45	-7.00	0.13
74.14	-2.90	-19.93	-21.04	-17.88	-10.81	8.29	5.26
-63.65	3.10	5.08	14.82	10.12	9.33	1.31	-0.62
3.73	2.85	6.67	8.99	-3.38	1.54	1.04	-0.62
2.50	0.57	-4.46	0.52	3.00	-2.89	-0.32	1.33
7.52	-1.80	-0.63	-0.10	0.41	-3.21	-2.74	-2.07
-3.40	0.43	0.81	0.28	-0.40	-0.19	-0.58	-1.09
-2.26	-0.88	1.73	0.23	-0.21	-0.12	1.23	1.61

DCT coefficients of the above 8×8 block

-6	-6	2	0	0	0	0	0
6	0	-1	-1	-1	0	0	0
-5	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Results of DCT coefficients quantized by *Luminance Quantization Matrix*

The entropy encoding procedure for the differentially encoded DC coefficient is identical to the entropy encoding of the prediction error values, as explained in Section 3.10 for lossless JPEG. For 8-bit images in baseline JPEG, the DCT coefficients fall in the range $[-1023, +1023]$. Since the DC coefficient is differentially encoded, the differential value of DC falls in the range $[-2047, +2047]$. Assuming that the DC coefficient of the previous block is -4

as an example, we get the differential DC value of the present block to be -2 . Using Table 3.3, we find that this belongs to category 2 and hence -2 is described as (2, "01"). If the Huffman code of category 2 is "011," then -2 is coded as "01101," where the last two bits "01" represent the variable-length integer (VLI) code of -2 . There are two Huffman tables (Tables K.3 and K.4) for encoding the DC coefficients in Annex K of the baseline JPEG standard for reference. But the user can choose any table and add them as part of the header of the compressed file [7]. Table K.3 is supplied for coding the Luminance DC differences as a reference. Table K.4 is supplied for Chrominance DC differences.

After zig-zag ordering of the AC coefficients in the example, the resulting sequence becomes

[illegible]

This sequence of AC coefficients can be mapped into an *intermediate sequence* of combination of two symbols *symbol₁* and *symbol₂*. Here *symbol₁* is represented by a pair (*RUNLENGTH*, *CATEGORY*), where *RUNLENGTH* is the number of consecutive zeros preceding the nonzero AC coefficient being encoded and *CATEGORY* is the number of bits to represent the VLI code of this nonzero AC coefficient. Again *symbol₂* is a single piece of information designated (*AMPLITUDE*), which is encoded by the VLI code of the nonzero AC coefficient. Accordingly, the zig-zag sequence in the example can be compactly represented as

$(0, 3)(-6), (0, 3)(6), (0, 3)(-5), (1, 2)(2), (1, 1)(-1), (5, 1)(-1), (2, 1)(-1), (0, 1)(1), (0, 0).$

The first significant (nonzero) AC coefficient in the zig-zag sequence is found to be -6 . It is represented as $(0, 3)(-6)$ because it precedes with no run of zeros (i.e., $RUNLENGTH = 0$) and the $AMPLITUDE = -6$ belongs to $CATEGORY = 3$. Similarly, the following two nonzero coefficients 6 and -5 are represented as $(0, 3)(6)$ and $(0, 3)(-5)$, respectively. The next significant coefficient 2 is represented by $(1, 2)(2)$ because it precedes a zero coefficient (i.e., $RUNLENGTH = 1$) and $AMPLITUDE = 2$ belongs to $CATEGORY = 2$. Again, the next significant symbol is represented as $(1, 1)(-1)$. The following significant coefficient -1 is represented as $(5, 1)(-1)$ because it precedes five zeros (i.e., $RUNLENGTH = 5$) and $AMPLITUDE = -1$ belongs to $CATEGORY = 1$. Following the same procedure, the next two nonzero coefficients -1 and 1 are represented by $(2, 1)(-1)$ and $(0, 1)(1)$, respectively. There are no other nonzero coefficients in the remaining of the zig-zag sequence. A special symbol $(0, 0)$ is used to indicate that the remaining elements in the zig-zag block are all zeros. Each $(RUNLENGTH, CATEGORY)$ pair is encoded using a Huffman code, while the corresponding $AMPLITUDE$ is encoded by the VLI code.

There are two special symbols in encoding the zig-zag sequence of AC coefficients, namely, (0, 0) and (15, 0). The first special symbol (0, 0) is referred as EOB (end-of-block), to indicate that the remaining elements in the zig-zag block are zeros. The other special symbol (15, 0) is also referred as ZRL (Zero-Run-Length) and is used to indicate a run of 16 zeros. Maximum length of a run of zeros allowed in baseline JPEG is 16. If there are more than 16 zeros, then the run is broken into a number of runs of zeros of length 16. For example, consider 57 zeros before a nonzero coefficient, say -29. This will be represented by (15, 0) (15, 0) (15, 0), (9, 5)(-29). Here the first three (15, 0) pairs represent 48 zeros and (9, 5)(-29) represents 9 zeros followed by the coefficient -29 which belongs to category 5.

The baseline JPEG allows a maximum of four Huffman tables, - that is, two for encoding AC coefficients and two for encoding DC coefficients. In luminance-chrominance image data, usually two Huffman tables (one for AC and one for DC) each are used for encoding the luminance and chrominance data. The Huffman tables used during the compression process are stored as header information in the compressed image file, in order to uniquely decode the coefficients during the decompression process. There are two Huffman tables (Tables K.5 and K.6) for encoding the AC coefficients, and two others (Tables K.3 and K.4) for encoding the DC coefficients in Annex K of the baseline JPEG standard for reference. The users can choose any table of their choice and store it as part of the header of the compressed file [7]. Tables K.3 and K.5 are recommended for luminance DC differences and AC coefficients. Tables K.4 and K.6 are recommended for corresponding chrominance channels.

Let us now allocate the variable-length codes in the last example. The codewords for (0, 0), (0, 1), (0, 3), (1, 1), (1, 2), (2, 1) and (5, 1), from Table K.5, are 1010, 00, 100, 1100, 11011, 11100, and 1111010, respectively. VLI codes for the nonzero AC coefficients 1, -1, 2, -5, 6, and -6 are 1, 0, 10, 010, 110, and 001, respectively. Codeword for the differential DC value is 01101. The compressed bit-stream for the 8×8 block is shown below, and it requires only 52 bits as opposed to the 512 bits required by the original 8×8 block of 8-bit pixels. We have

'01101 100001 100110 100010 1101110 11000 11110100 111000 001 1010',

where the first five bits "01101" represent the DC coefficient and the other 47 bits represent the AC coefficients. Therefore, we achieve approximately 10:1 compression using the baseline JPEG to compress the block.

3.11.4 Decompression process in baseline JPEG

Decompression is the inverse process to decode the compressed bit-stream, in order to properly reconstruct the image. Block diagram of the baseline decompression algorithm is provided in Fig. 3.12(b). During the decompression

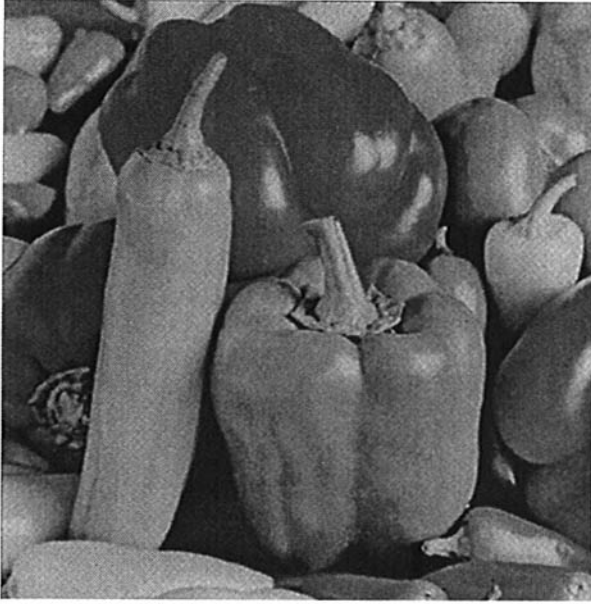


Fig. 3.15 Original pepper image.

process, the system first parses the header of the compressed file in order to retrieve all the relevant information, namely, image type, number of components, format, quantization matrices, and the Huffman tables that were used to compress the original image, etc.

After parsing the header information, the decompression algorithm is applied on the compressed bit-stream as shown in Fig. 3.12(b). The *entropy decoding* step in Fig. 3.12(b) decodes the bit-stream of the compressed data using the Huffman tables that were used during the compression process. The purpose of this step is to regenerate the zig-zag ordered sequence of the quantized DCT coefficients. This zig-zag sequence is then reordered by the *zig-zag reordering* step to create the 8×8 block of quantized DCT coefficients. Each DCT coefficient in the quantized block is inverse-quantized as

$$F'(u, v) = F_q(u, v) * Q(u, v), \quad (3.20)$$

where $Q(u, v)$ is the quantization step-size parameter from the same quantization table that was used during the compression process. After inverse-quantization, the DCT coefficients $F'(u, v)$ are inverse transformed to spatial domain data via inverse DCT (IDCT). The IDCT of an 8×8 block $F'(u, v)$,

for $u, v = 0, 1, \dots, 7$, is defined as

$$f(x, y) = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 C(u)C(v)F'(u, v) \cos \left[\frac{\pi(2x+1)u}{16} \right] \cos \left[\frac{\pi(2y+1)v}{16} \right] \quad (3.21)$$

for $x = 0, 1, \dots, 7$ and $y = 0, 1, \dots, 7$.

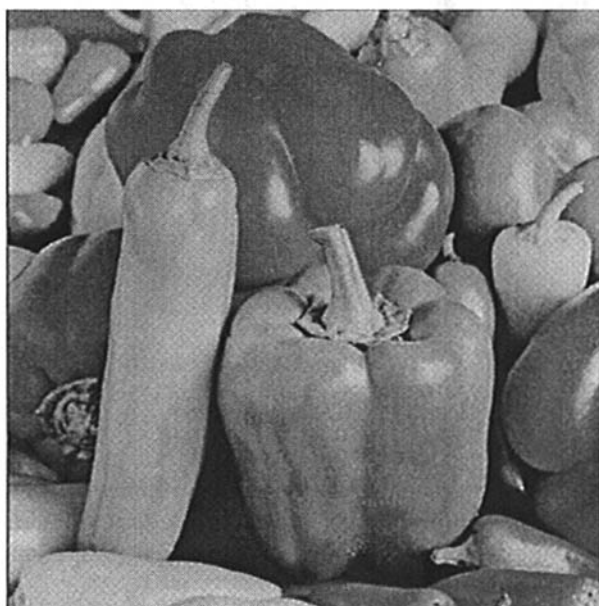


Fig. 3.16 Pepper image compressed with quality factor 75.

After decompression of all the MCUs from the compressed bit-stream, the image components are reconstructed. For a gray scale image, there is only one component and no color transformation is required. For color image, the reconstructed Y, Cb, and Cr components are inverse-transformed to the RGB color space. We show in color the famous *Pepper* image in Fig. 3.15. When compressed, using the baseline JPEG algorithm with quality factor $Q_JPEG = 75$, the reconstructed image is found to be perceptually almost identical to the original image. This is demonstrated in Fig. 3.16. When we compress the same image with a quality factor $Q_JPEG = 10$, we can see prominent artifacts in the image as shown in Fig. 3.17. Such artifacts, caused by lossy JPEG compression/decompression, are called *blocking* artifacts. This happens because of the discontinuities created at the 8×8 block boundaries, since these blocks are compressed and decompressed independently.

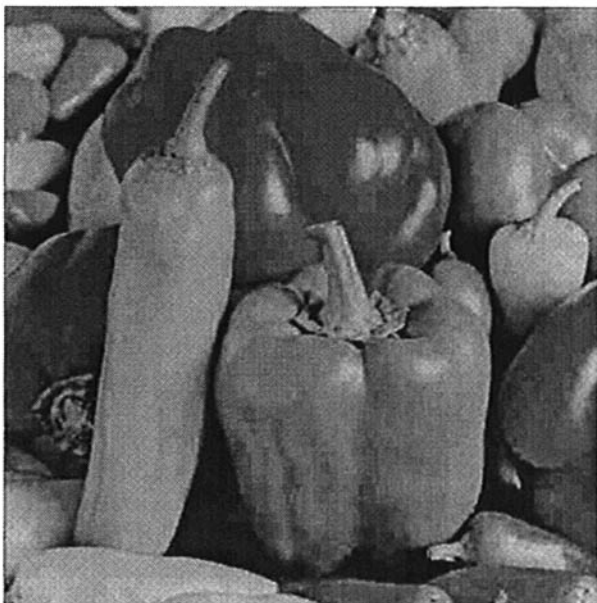


Fig. 3.17 Pepper image compressed with quality factor 10.

3.11.5 JPEG2000: Next generation still picture coding standard

Goal of the JPEG2000 [22, 23] standardization activity is to advance imaging applications in the new era of Internet and wireless communication. This new standard is expected to address the areas where the current JPEG standard fails to produce the best quality or performance requirements.

The current JPEG standard for still image compression is very much suitable for compressing images at 0.5 bits per pixel or higher. However, the reconstructed image quality significantly degrades at lower bit rates. Reconstructed quality of a JPEG compressed image is unacceptable below 0.25 bits per pixel. JPEG results in poor compression performance when applied to bi-level images for compound documents, such as facsimile, scan, and text-type imagery. Current JPEG standard has 44 different modes of operation, with many of these modes being very application-specific and not widely used in practice. As a result, the interchangeability between the different modes of JPEG applications is a difficult proposition.

JPEG2000 is targeted for more compression efficiency in terms of compression ratio and image quality, especially at very low bit-rates (below 0.25 bits per pixel). JPEG2000 will have a single common decompression architecture to encompass different modes and types of applications, so that it is suitable for greater interchange between applications encompassing different features. The same decompression architecture will be suitable both for bi-level and

continuous tone images, because the JPEG2000 system is capable of compressing and decompressing images with various dynamic ranges between 1 to 16 bits for each color component. This standard is expected to handle new paradigms of distributed imaging applications, especially Internet imaging. JPEG2000 will provide an open systems approach to imaging applications. The interesting feature of this new standard is that the same algorithm can be used for both lossless and lossy compression of still images.

Unlike DCT in current JPEG, the JPEG2000 coding algorithm is being defined based on the principles of Discrete Wavelet Transform (DWT), which offers multiresolution and efficient progressive encoding/decoding system. Progressive decoding can be achieved in terms of resolution, while being progressive in terms of visual quality from the same encoded bit-stream and selectable during the decoding time. This capability is particularly suitable for client-server applications such as World Wide Web and many other distributed networking environments. This is also suitable for retrieval, archival, print, and color facsimile-type applications.

There are two types of wavelet filters to accomplish DWT in the JPEG2000 standard. One type of wavelet filters generates noninteger values for the transformed coefficients, which are mainly used for lossy image compression and cannot be used for lossless image compression. The other type of wavelet filters generates integer-transformed coefficients, which are used mainly for lossless compression. However, they can be used in lossy mode also, when the transformed coefficients are quantized after wavelet transformation. In current JPEG, the steps in lossless mode of compression are entirely different from the lossy mode. In JPEG2000 a common algorithm is used in both lossy and lossless modes, based on the selection of corresponding wavelet filters. JPEG2000-compressed images are very much suitable for transmission through a noisy environment like wireless channels, because of the error resilience features embedded into the bit stream.

This technology will enable regions of particular interest in an image to be encoded with greater fidelity (*Region of Interest* coding) compared to other areas in the image, and it will also enable random access to the compressed data for manipulation of the images in the compressed domain. The random access of the code-stream will allow operations such as rotation, translation, scaling, filtering, etc., without decompressing the whole image. The file format of JPEG2000 is expected to handle the features of metadata such as *Watermarking* of images, *Intellectual Property Rights* as per the WIPO (World Intellectual Property Organization) compliant, *Content Registration*, *JPEG Registration Authority*, etc. The plan of this standard is to provide appropriate interfaces with MPEG-4, in order to insert and extract still pictures to/from moving video and maintain corresponding Intellectual Property information. JPEG2000 is a good candidate for usage in multimedia data mining, because of the metadata information that can be handled using this standard.

3.12 TEXT COMPRESSION

Text compression is inherently lossless in nature. Like image and video compression, there is no standard committee devoted to define text compression standards under the auspices of an international standard body. However, industry adopted text compression schemes, mainly based on the principles of text processing and string matching. We shall describe some of them here.

Dictionary-based coding techniques are particularly suitable for compressing text datatypes. Typically, redundancy in text appears in the form of common words, which repeat quite often, in addition to the statistical redundancy of individual characters. In order to achieve significantly enhanced compression performance, it is desirable to exploit both these aspects of redundancy. We can handle the redundancy of frequently appearing common words by constructing a *dictionary* and replacing each common word in the text file by an *index* to the dictionary. This approach is popularly known as the *dictionary-based coding* scheme. The dictionary could be *static* or *dynamic*.

In *static dictionary coding*, the dictionary is fixed during both compression and decompression. The simplest example of this scheme is to express (or encode) the words "Sunday," "Monday," ..., "Saturday" by the indices 1, 2, ..., 7. A *dynamic dictionary coding*, however, builds a *dictionary* dynamically using the message itself that is being encoded or decoded. The basic idea behind most of the dynamic dictionary-based robust lossless text compression schemes is to first *parse* the text (which can be considered as a string of characters) into a sequence of *substrings* and then generate compressed codes of these substrings. Jacob Ziv and Abraham Lempel described efficient dynamic dictionary encoders, popularly known as LZ77 [24] and LZ78 [25], by replacing a group of characters of the text (*phrases*) with a pointer to where they have occurred earlier in the portion of the text that has already been encoded. Many variations of these algorithms have been developed after that. They are collectively called the *Ziv-Lempel* or *Lempel-Ziv* (LZ) family, namely, LZSS [26], LZW [27], LZC [28], LZWAJ [29], etc. For example, LZSS, a variation of LZ77, is the basis of the text compression engine in popularly used compression utilities like *zip*, *gzip*, *pkzip*, *winzip*. The LZW algorithm, a variant of LZ78 scheme, is the core of the *Unix compress* utility.

There exist other categories of text compression algorithms. Some of the popular ones include variants of a technique called *Prediction by Partial Matching* (PPM) [30, 31]. PPM is a statistical compression scheme based on context modeling of the symbols. The already encoded portion of the text is used as context to determine the probability of the symbol being encoded. It relies on the *Arithmetic coding scheme* [32] to achieve good compression performance. Although there are a number of variations of the PPM algorithm for text compression, such as PPMA, PPMB, PPMC, etc., none of them are supported by any underlying theory. However, the PPM algorithms are relatively slow as compared to the LZ family of algorithms.

A comparatively recent development in context based text compression is the *block-sorting* scheme based on the Burrows and Wheeler Transform (BWT) [33]. This is analogous to transformation-based image compression schemes. The text is first divided into a number of blocks. Each block is then transformed into a form more amenable to compression. The transformation is done by permuting the characters in each block, so that the characters occurring in a similar context get aligned near each other. The permuted block is then compressed using a suitable coder, which exploits the locality of context of the symbols in the permuted block. During decompression, the decoder first decodes the permuted block which is then inverse-transformed (BWT) in order to reconstruct the original text.

3.12.1 The LZ77 algorithm

LZ77 is the first form of Ziv–Lempel coding proposed by Ziv and Lempel in 1977 [24]. In this approach a fixed-size buffer, containing the previously encoded character sequence that precedes the current coding position, can be considered as a dictionary. The encoder matches the input sequence through a sliding window, as illustrated in Fig. 3.18. The window is divided into two parts, namely, (i) a *search window* that consists of the already encoded character sequence, and (ii) a *lookahead buffer* that contains the character sequence to be encoded as shown in Fig. 3.18.

In order to encode a sequence in the lookahead buffer, the *search window* is scanned to find the longest match in it with a prefix of the lookahead buffer. The match can overlap with the lookahead buffer, but obviously cannot be the lookahead buffer itself. Once the longest match is found, it is coded as a triple $\langle \text{offset}, \text{length}, C(\text{char}) \rangle$, where *offset* is the distance of the first character of the longest match in the search window from the lookahead buffer, *length* is the length of the match, and $C(\text{char})$ is the binary codeword of the first mismatching symbol *char* that follows the match in the lookahead buffer. The window is shifted left by *length* + 1 symbols to begin the next search.

3.12.1.1 Example 3–LZ77 coding: Let the character sequence to be encoded be given as $\dots \text{baabacbaacbcdcbdcac} \dots$. We assume that the size of the *search window* is 8 and that of the *lookahead buffer* is 6. Let us assume that the substring *baabacba* in the search window has already been encoded and the substring *acbcd* in the lookahead buffer is to be encoded, as shown in Fig. 3.18(a). After scanning the search window, the longest match is found to be the substring ‘*acb*’ of length 3 at a distance 4 from the lookahead buffer. The character following the prefix ‘*acb*’ in the lookahead buffer is ‘*c*’. Hence the triple to output is $\langle 4, 3, C(c) \rangle$, where $C(c)$ is the codeword for the character *c*. Since the match length is 3, we shift the window left by 4 characters.

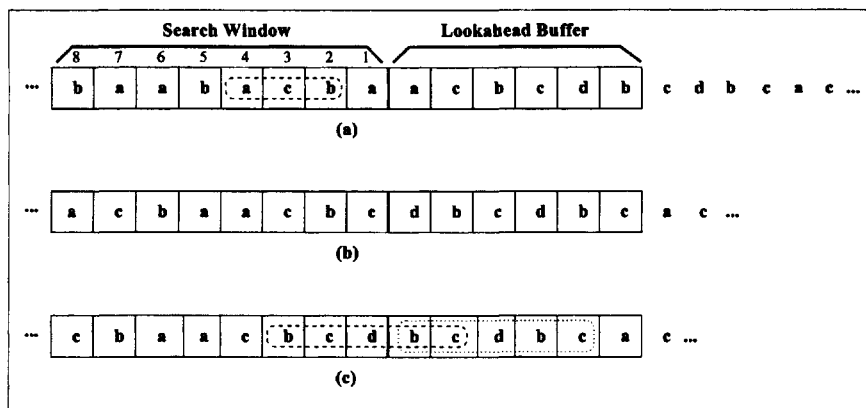


Fig. 3.18 LZ77 coding: An example with sliding window.

Now the first character in the lookahead buffer is 'd' as illustrated in Fig. 3.18(b), and there is no match for 'd' in the search window. Hence we output the triple $\langle 0, 0, C(d) \rangle$ and shift the sliding window by one.

The longest match in the sliding window is the substring 'bcdbc' as depicted in Fig. 3.18(c). It is to be noted that the matching substring starts in character position 3 in the search window, and it overlaps with the first two characters bcdbc in the lookahead buffer. Hence we output the triple $\langle 3, 5, C(a) \rangle$ and shift the sliding window left by 6 characters to continue.

There are many variations of LZ77 coding, mainly to further improve the performance or implementation efficiency of the scheme. Popular compression softwares like Zip and PKZip use a variation of the LZ77 coding scheme, called LZSS coding [26].

3.12.2 The LZ78 algorithm

LZ78 is the other key algorithm in the LZ family proposed by Ziv and Lempel in 1978 [25]. Instead of using the previously encoded sequence of symbols (or string) in the sliding window as the implicit dictionary, the LZ78 algorithm explicitly builds a dictionary of string patterns dynamically in both the encoder and decoder. The encoder searches this dictionary to find the longest match with the prefix of the input string and encodes it as a pair $\langle i, C(S) \rangle$, where i is the index of the matched substring in the dictionary and $C(S)$ is the codeword of the first symbol S following the matched portion of the input. A new entry is then added to the dictionary, corresponding to the matched substring concatenated by the symbol S . The codeword $C(S)$ is usually a Huffman-type variable-length code of the source symbol S .

In order to achieve further compression, the index i in the output pair can be encoded using some Huffman-type variable-length binary encoding

by exploiting the statistics of the indices. But for the sake of simplicity of explanation, we avoid detailed discussion here.

3.12.2.1 Example 4—LZ78 encoding: Let us consider the sequence of symbols *bacababbaabbabbaaacbbc*. Initially the dictionary is empty. Since the first input symbol *b* has no match in the dictionary, the encoder outputs the pair $\langle 0, C(b) \rangle$ and inserts the first entry *b* into the dictionary with index 1 as shown in Table 3.6.

Table 3.6 Dictionary after Step 1

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>

Similarly, the next input symbol *a* has no match in the dictionary. Hence the encoder outputs the pair $\langle 0, C(a) \rangle$ and inserts new entry *a* at index 2 in the dictionary as indicated in Table 3.7.

Table 3.7 Dictionary after Step 2

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>
$\langle 0, C(a) \rangle$	2	<i>a</i>

Because the next input symbol *c* has no match in the dictionary, the encoder outputs the pair $\langle 0, C(c) \rangle$ and inserts the new entry *c* at index 3 as shown in Table 3.8.

Table 3.8 Dictionary after Step 3

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>
$\langle 0, C(a) \rangle$	2	<i>a</i>
$\langle 0, C(c) \rangle$	3	<i>c</i>

Now the input symbol *a* matches with entry 2 in the dictionary, but *ab* fails to generate a match. So the encoder outputs the pair $\langle 2, C(b) \rangle$ and inserts new entry *ab* at index 4 in the dictionary, as indicated in Table 3.9.

Table 3.9 Dictionary after Step 4

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>
$\langle 0, C(a) \rangle$	2	<i>a</i>
$\langle 0, C(c) \rangle$	3	<i>c</i>
$\langle 2, C(b) \rangle$	4	<i>ab</i>

The next two symbols *ab* match with entry 4 in the dictionary, but *abb* does not have any match. So the encoder outputs the pair $\langle 4, C(b) \rangle$ and inserts a new entry *abb* at index 5 in the dictionary, as shown in Table 3.10.

Table 3.10 Dictionary after Step 5

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>
$\langle 0, C(a) \rangle$	2	<i>a</i>
$\langle 0, C(c) \rangle$	3	<i>c</i>
$\langle 2, C(b) \rangle$	4	<i>ab</i>
$\langle 4, C(b) \rangle$	5	<i>abb</i>

Table 3.11 Final LZ78 dictionary

Encoder output	Index	Entry
$\langle 0, C(b) \rangle$	1	<i>b</i>
$\langle 0, C(a) \rangle$	2	<i>a</i>
$\langle 0, C(c) \rangle$	3	<i>c</i>
$\langle 2, C(b) \rangle$	4	<i>ab</i>
$\langle 4, C(b) \rangle$	5	<i>abb</i>
$\langle 2, C(a) \rangle$	6	<i>aa</i>
$\langle 1, C(b) \rangle$	7	<i>bb</i>
$\langle 5, C(a) \rangle$	8	<i>abba</i>
$\langle 6, C(c) \rangle$	9	<i>aac</i>
$\langle 7, C(c) \rangle$	10	<i>bbc</i>

Continuing the above procedure the encoder generates the output pairs $\langle 2, C(a) \rangle$, $\langle 1, C(b) \rangle$, $\langle 5, C(a) \rangle$, $\langle 6, C(c) \rangle$ and $\langle 7, C(c) \rangle$, and builds the dictionary accordingly. The final dictionary is depicted in Table 3.11. The encoded output of the sequence is $\langle 0, C(b) \rangle$, $\langle 0, C(a) \rangle$,

$\langle 0, C(c) \rangle$, $\langle 2, C(b) \rangle$, $\langle 4, C(b) \rangle$, $\langle 2, C(a) \rangle$, $\langle 1, C(b) \rangle$, $\langle 5, C(a) \rangle$,
 $\langle 6, C(c) \rangle$, $\langle 7, C(c) \rangle$.

3.12.2.2 Example 5—LZ78 decoding: We now decode the encoded data to explain how the LZ78 decoding process works. The decoder also dynamically builds a dictionary, which is the same as that built by the encoder. Initially the dictionary contains nothing. Since the first input pair to the decoder is $\langle 0, C(b) \rangle$, it first decodes the symbol b from the codeword $C(b)$. As the decoded index is 0, it outputs the symbol b and inserts the first entry $\langle 1, b \rangle$ in the dictionary as shown in Table 3.6.

The next input pair to the decoder is $\langle 0, C(a) \rangle$. As a result, the decoder outputs the symbol a and inserts the next entry $\langle 2, a \rangle$ in the dictionary as indicated in Table 3.7. The following input pair being $\langle 0, C(c) \rangle$, the decoder outputs the symbol c and inserts the next entry $\langle 3, c \rangle$ in the dictionary as shown in Table 3.8.

The next input pair is $\langle 2, C(b) \rangle$, which indicates that the new output is the pattern for entry 2 in the dictionary concatenated by the decoded symbol b . Since entry 2 represents a , the output will be ab . A new pattern ab is now inserted in index 4 of the dictionary.

The following input pair is $\langle 4, C(b) \rangle$. As a result, the decoder outputs the string abb and inserts it in the dictionary in entry 5. Analogously, the decoder reads the next pair $\langle 2, C(a) \rangle$ and generates the output aa , inserting it in the dictionary in entry 6. Continuing in a similar fashion, the subsequent decoder outputs are bb , $abba$, aac and bbc ; and these are inserted in the dictionary at indices 7, 8, 9, and 10, respectively. The final dictionary is identical to the one generated in Table 3.11. The final decoder output is *bacababbaabbabbaaacbbc*, and it exactly matches with the original input sequence.

Generally the LZ78 algorithm is easier for implementation and less memory-consuming, as compared to the LZ77. This is because of the simpler data structure used in LZ78 to output sequence of pairs only, as opposed to the triples in LZ77. There exist a number of variations of the LZ78 algorithm, the most popular being the algorithm by Welch [27] known as the LZW algorithm. We describe this algorithm in the following section.

3.12.3 The LZW algorithm

The inclusion of the explicit codeword $C(S)$ of the symbol S along with the index i , in the output $\langle i, C(S) \rangle$ of the LZ78 encoding algorithm, is often found to be very wasteful. The inefficiency is overcome in the LZW algorithm, by omitting $C(S)$ and transmitting the index i only. This is accomplished by initializing the dictionary with a list of single symbol patterns, to include all the symbols of the source alphabet. In each step, the index of the longest match from the input in the dictionary is output and a new pattern is inserted in the dictionary. This new pattern is formed by concatenating the longest match with the next character in the input stream. As a result, the last

symbol (or character) of this new pattern is encoded as the first character of the next one.

3.12.3.1 Example 6—LZW encoding: The LZW encoding algorithm is explained below with an example to encode the string *babacbabababcb*.

Table 3.12 LZW dictionary for encoding

Index	Pattern	Derived as
1	<i>a</i>	initial
2	<i>b</i>	
3	<i>c</i>	
4	<i>ba</i>	2 + <i>a</i>
5	<i>ab</i>	1 + <i>b</i>
6	<i>bac</i>	4 + <i>c</i>
7	<i>cb</i>	3 + <i>b</i>
8	<i>bab</i>	4 + <i>b</i>
9	<i>baba</i>	8 + <i>a</i>
10	<i>abc</i>	5 + <i>c</i>

The dictionary generation is shown in Table 3.12. Initially, the dictionary consists of single symbol (or character) patterns *a*, *b*, and *c* from the input alphabet {*a*, *b*, *c*}. The index of the patterns in the dictionary are 1, 2, and 3, respectively.

After receiving the first character *b*, the encoder finds the match at index 2. But the pattern *ba*, with the first two characters, does not have a match in the current dictionary. Hence the encoder outputs index 2 to encode the first character *b*, and inserts the new pattern *ba* to index 4 in the dictionary.

The second input character *a* has a match in the dictionary with index 1, but *ab* formed by the second and third characters does not have a match. As a result, the encoder outputs index 1 to encode *a* and inserts the new pattern *ab* in the dictionary at index 5.

Now the next two characters *ba* match with the pattern at index 4 in the dictionary, but *bac* does not. Hence the encoder outputs index 4 to encode *ba*, and it inserts the new pattern *bac* into the dictionary at index 6.

The following character *c* now matches with index 3, but *cb* does not. Hence the encoder outputs index 3 to encode *c*, and it inserts *cb* in the dictionary to index 7.

The subsequent two characters *ba* have a match at index 4, but *bab* does not. Hence the encoder outputs the index 4 to encode *ba*, and it inserts the new pattern *bab* in the dictionary to index 8.

The next three characters *bab* have a match in the dictionary to index 8, but *baba* does not. Hence the encoder now outputs the index 8 to encode *bab*, and it inserts the new pattern *baba* in the dictionary at index 9.

The following two characters *ab* now match with the pattern at index 5 in the dictionary, but *abc* does not. Hence the encoder outputs index 5 to encode *ab*, and it inserts the new pattern *abc* in the dictionary to index 10.

The subsequent two characters *cb* have a match at index 7 in the dictionary. Hence the encoder outputs the index 7 to encode *cb*, and it stops. As a result, the output of the LZW encoder is 2 1 4 3 4 8 5 7.

It should be noted that statistical probabilities of appearance of the pointers from the LZW encoder can be further exploited by using Huffman-coding type variable length entropy encoding schemes. This may result in further reduction of the output data, and hence it can enhance the text compression performance.

3.12.3.2 Example 7—LZW decoding: Here we take the same encoder output from Example 6, and decode it using the LZW algorithm. The input to the decoder is 2, 1, 4, 3, 4, 8, 5, 7.

Like the encoder, the decoder starts with the initial dictionary having three entries for *a*, *b*, *c* and indices 1, 2, 3. After visiting the first index 2, the decoder outputs the corresponding pattern *b* from the dictionary.

The next output is *a*, corresponding to the second input index 1. At this point, the decoder inserts a new pattern *ba* in the dictionary to index 4. This new pattern *ba* is formed by concatenating the first character *a* of the current output pattern *a* at the end of the last output pattern *b*.

The next input index is 4, which corresponds to the pattern *ba* in the dictionary. Hence the decoder outputs *ba*, and it inserts the new pattern *ab* in the dictionary to index 5. The new pattern *ab* is again formed by concatenating the first character *b* of the current output pattern *ba* at the end of the last output pattern *a*.

The next input index is 3, which corresponds to *c* in the current dictionary. The decoder hence outputs *c* and inserts a new pattern *bac* in the dictionary to index 6. This pattern *bac* has been formed by concatenating *c* at the end of the previous output or matching pattern *ba*.

The next output of the decoder is *ba* because of the input index 4. The decoder now inserts the new pattern *cb* in the dictionary to index 7. This pattern is again formed by concatenating the first character *b* of the current output *ba* at the end of the previous output *c*. At this point, the dictionary has only 7 entries as shown in Table 3.13. So far the decoding process was straightforward.

The next input to the decoder is index 8. But the dictionary does not have any pattern at index 8. This tricky situation arises during decoding, if a pattern has been encoded using the pattern immediately preceding it during the encoding process. As a result, the last character of the pattern is the same as the first character. Hence the decoder creates the output by concatenating the first character of the previous output with the previous output itself. Since the previous output was *ba*, the decoder outputs *bab* in the current decoding

Table 3.13 LZW dictionary for decoding

Index	Pattern	Derived as
1	<i>a</i>	initial
2	<i>b</i>	
3	<i>c</i>	
4	<i>ba</i>	2 + <i>a</i>
5	<i>ab</i>	1 + <i>b</i>
6	<i>bac</i>	4 + <i>c</i>
7	<i>cb</i>	3 + <i>b</i>

step and inserts this new pattern in the dictionary to index 8. The following input index 5 corresponds to the pattern *ab*, and hence the decoder outputs *ab* and inserts the new pattern *baba* in the dictionary at index 9. This pattern *baba* is formed by concatenating the first character *a* of the current output *ab* at the end of the previous output *bab*.

The next input index 7 corresponds to the pattern *cb*. The decoder outputs *cb* and obviously inserts the new pattern *abc* in the dictionary and stops. At this point the final dictionary is exactly identical to the final dictionary that was formed during the encoding process as shown in Table 3.12 in the previous example.

3.12.4 Other applications of Lempel–Ziv coding

LZ coding techniques are not necessarily applicable to text compression only. Variants of the LZ coding techniques have been found to be effective to compress many other datatypes. They can be effectively used to compress general-purpose data effectively, for archival and storage. LZ coding techniques can be applied to compress databases (both numeric and text), graphical charts, geographical maps, and many other special kinds of images. The LZ-based coding schemes have also been adopted in many international coding standards.

LZW-based coding has been found to be effective to losslessly compress different kinds of images. The widely used image file format ‘GIF’ (Graphical Interchange Format) is an implementation of the LZW algorithm. This is very similar to the popular *compress* utility in UNIX. GIF is very effective in compressing computer-generated graphical images and pseudo-color or color-mapped images. TIFF (Tag Image File Format) is another industry standard based on LZ coding. This is useful for compressing *dithered binary images*, which simulate gray scale images through a variation of the density of black dots. The CCITT (previously ITU-T) *Recommendation V.42 bis* is a compression standard of data over a telephone network. The compression mode

of this standard uses the LZW algorithm to compress data to be transmitted through the modem.

3.13 CONCLUSIONS AND DISCUSSION

In this chapter we have introduced the fundamental principles behind multimedia data compression. Data compression has great potential in the near future to improve the efficiency of data mining systems, by exploiting the benefits of compact and shorter representation of data. This is particularly important because data mining techniques typically deal with large databases, and data storage management is a big issue for managing such large databases. However, the data mining community has hitherto failed to take advantage of the knowledge in the area of data compression and develop special data mining techniques based on the principles behind data compression. Nevertheless, there have been limited efforts at usage of data compression to reduce the high dimensionality of multimedia datasets, with applications for mining multimedia information in a limited manner. Multimedia data mining is covered in detail in Chapter 9.

We have discussed various issues of multimedia data compression, along with some theoretical foundations. We presented some basic source coding algorithms, often used in data compression, in order to introduce this area of development to the readers. We have described the principles behind the popular algorithms for image and text type multimedia data. We avoided discussion on compression of other datatypes such as video, audio, and speech because it is beyond the scope of this book. The advantages of data compression are manifold and will enable more multimedia applications at reduced costs, thereby aiding its usage by a larger population, with newer applications, in the near future.

REFERENCES

1. C. E. Shannon and W. Weaver, *The Mathematical Theory of Communication*. Urbana, IL: University of Illinois Press, 1949.
2. C. E. Shannon, "Certain results in coding theory for noisy channels," *Information Control*, vol. 1, pp. 6–25, 1957.
3. C. E. Shannon, "Coding theorems for a discrete source with a fidelity criterion," Technical Report, IRE National Convention Record, 1959.
4. B. McMillan, "The basic theorems of information theory," *Annals of Mathematics and Statistics*, vol. 24, pp. 196–219, 1953.

5. A. N. Netravali and B. Haskell, *Digital Pictures*. New York: Plenum Press, 1988.
6. A. K. Jain, *Fundamentals of Image Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1989.
7. W. B. Pennebaker and J. L. Mitchell, *JPEG: Still Image Data Compression Standard*. New York: Chapman & Hall, 1993.
8. R. Hunter and A. H. Robinson, "International digital facsimile standard," *Proceedings of IEEE*, vol. 68, pp. 854–867, 1980.
9. D. A. Huffman, "A method for the construction of minimum redundancy codes," *Proceedings of the IRE*, vol. 40, pp. 1098–1101, 1952.
10. R. J. Clarke, *Transform Coding of Images*. New York: Academic Press, 1985.
11. I. T. Jolliffe, *Principal Component Analysis*. New York: Springer-Verlag, 1986.
12. D. Hand, H. Mannila and P. Smyth, *Principles of Data Mining*. Cambridge, MA: The MIT Press, 2001.
13. K. R. Rao and P. Yip, *Discrete Cosine Transform - Algorithms, Advantages, Applications*. San Diego, CA: Academic Press, 1990.
14. M. Ghanbari, *Video Coding: An Introduction to Standard Codecs*, vol. 42 of *Telecommunications Series*. London, United Kingdom: IEEE, 1999.
15. S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, pp. 674–693, 1989.
16. I. Daubechies, *Ten Lectures on Wavelets*. CBMS, Philadelphia: Society for Industrial and Applied Mathematics, 1992.
17. M. Antonini, M. Barlaud, P. Mathieu, and I. Daubechies, "Image coding using wavelet transform," *IEEE Transactions on Image Processing*, vol. 1, pp. 205–220, 1992.
18. J. M. Shapiro, "Embedded image coding using zerotrees of wavelet coefficients," *IEEE Transactions on Signal Processing*, vol. 41, pp. 3445–3462, 1993.
19. T. Acharya and A. Mukherjee, "High-speed parallel VLSI architectures for image decorrelation," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 9, pp. 343–365, 1995.
20. H. Lohscheller, "A subjectively adapted image communication system," *IEEE Transactions on Communications*, vol. 32, pp. 1316–1322, 1984.

21. "C source code of JPEG encoder research 6b," Sixth public release of the Independent JPEG group's free JPEG software, The Independent JPEG Group, ftp://ftp.uu.net/graphics/jpeg/jpegsrc_v6b.tar.gz, March 1998.
22. "Information technology - JPEG2000 Image Coding System," Final Committee Draft Version 1.0 ISO/IEC JTC 1/SC 29/WG 1 N1646R, March 2000.
23. D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Boston, USA: Kluwer Academic Publishers, 2002.
24. J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Transactions on Information Theory*, vol. 23, pp. 337-343, 1977.
25. J. Ziv and A. Lempel, "Compression of individual sequences via variable-rate coding," *IEEE Transactions on Information Theory*, vol. 24, pp. 530-536, 1978.
26. J. A. Storer and T. G. Syzanski, "Data compression via textual substitution," *Journal of the ACM*, vol. 29, pp. 928-951, 1982.
27. T. Welch, "A technique for high-performance data compression," *IEEE Computer*, vol. 17, pp. 8-19, 1984.
28. T. C. Bell, J. G. Cleary, and I. H. Witten, *Text Compression*. Englewood Cliffs, NJ: Prentice-Hall, 1990.
29. T. Acharya and J. F. JáJá, "An on-line variable-length binary encoding of text," *Information Sciences*, vol. 94, pp. 1-22, 1996.
30. J. G. Cleary and I. H. Witten, "Data compression using adaptive coding and partial string matching," *IEEE Transactions on Communications*, vol. 32, pp. 396-402, 1984.
31. A. Moffat, "Implementing the PPM data compression scheme," *IEEE Transactions on Communications*, vol. 38, pp. 1917-1921, 1990.
32. I. H. Witten, R. M. Neal, and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, pp. 520-540, 1987.
33. M. Burrows and D. J. Wheeler, "A block-sorting lossless data compression algorithm," Technical Report 124, Digital Equipment Corporation, Palo Alto, CA, May 1994.

String Matching

4.1 INTRODUCTION

Text probably got much more attention compared to other media datatypes, in research and development for information retrieval and data mining, because of the wealth of work done in the area of searching patterns in text files during the last three decades. This resulted in the growth of text processing softwares, text information retrieval systems, digital libraries, etc. The single most important reason for this growth is the development of numerous classical algorithms and their efficient implementations in string matching [1, 2]. The results in string matching algorithms influenced the development of text-based search engines, and these are being widely used in the *World Wide Web*.

Text mining is becoming a very practical and important area of development. Given the practical importance of classification and search of patterns in large collection of text data in the Internet, newswire, electronic media, digital library, large textual databases, and their ability to generate knowledge from these vast resources, the development in the area of text mining continues to increase. Development of string matching algorithms also influenced the areas of computational biology, and molecular biology, along with the success of the Human Genome Project. String matching algorithms have been used in DNA search, DNA sequencing, and many other problems in Bioinformatics as well. In our judgment, understanding of the principles in string matching is important for further development in data mining and its applications in multimedia as well as Bioinformatics.

The remaining part of this section introduces the preliminaries of string matching. Classical linear order string matching algorithms are described in Section 4.2. The use of string matching in Bioinformatics is highlighted in Section 4.3. Issues in approximate string matching are dealt with in Section 4.4. Compressed string matching is considered in Section 4.5. Finally, Section 4.6 concludes the chapter.

4.1.1 Some definitions and preliminaries

Before we proceed to detailed algorithmic description, it is essential to prepare the readers with some basic definitions.

Alphabet: Symbols, or characters, are considered to be the basic elemental building blocks in string matching. An *alphabet* is a specific set of symbols. It is usually a finite set. For instance, $\Sigma = \{a, b, c, d, e\}$ is an alphabet containing symbols a, b, c, d , and e .

String: A *string* is a sequence of instances of *symbols*, or characters, over a finite alphabet Σ . For instance, both 'baacbcba' and 'adaeabedeed' are strings over the alphabet $\Sigma = \{a, b, c, d, e\}$.

The *length* of a string s , say, is the number of instances of the symbols or characters in the string. The string s may be expressed as $s = s_1s_2 \cdots s_m$, where each s_i is an instance of a symbol, or character, from the alphabet and m is the length of the string s . Often length of the string is represented as $|s|$. An *empty* string, ϵ , is a special string with length 0. The concatenation of two strings $x = x_1x_2 \cdots x_p$ and $y = y_1y_2 \cdots y_q$, denoted by xy , is equal to the string $x_1x_2 \cdots x_py_1y_2 \cdots y_q$. The length of xy is $p + q$, where p and q are the lengths of strings x and y , respectively. As an example, the concatenation of two string 'straight' and 'forward' is 'straightforward'. The concatenation of the empty string ϵ with any string is that string itself.

Substring: A string $x = x_1x_2 \cdots x_k$ is a *substring* of another string $y = y_1y_2y_3 \cdots y_n$, if and only if there exists an i , $0 < i \leq n$, so that $y_{i+j-1} = x_j$ for $j = 1, 2, \dots, k$.

As an example, the string 'bad' is a substring of a string 'dabadaba'. Hence the substring of a string can be formed by deleting zero or more characters from the beginning and/or end of the string. The empty string is a special substring of any string, and hence it is the shortest length substring of any string. The longest substring of a string is the string itself.

Suffix: The *suffix* of a string s is a substring formed by deleting zero or more characters from the beginning of s . Hence y is a suffix of string s , if there exists a substring x such that $s = xy$. In other words, suffix of $s = s_1s_2 \cdots s_n$ is any substring $s_{n-j}s_{n-j+1} \cdots s_n$ where $0 \leq j \leq n$. The empty string ϵ is the shortest suffix and the string itself is the longest suffix of any string. Any

suffix other than the empty string and the string itself is called a *proper suffix*. As an example, 'oil' is the proper suffix of a string 'turmoil'. This is pictorially depicted in Fig. 4.1(a).

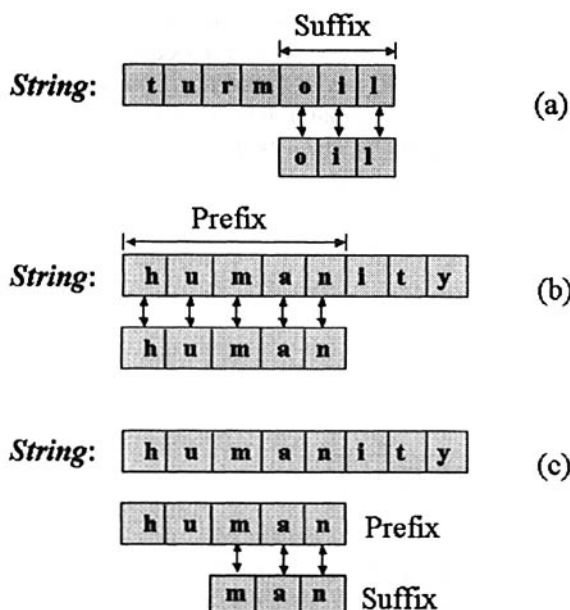


Fig. 4.1 Examples of (a) suffix, (b) prefix, and (c) suffix of prefix of a string.

Prefix: The *prefix* of a string s is a substring formed by deleting zero or more characters from the end of s .

Hence x is a prefix of s if there exist a substring y so that $s = xy$. In other words, prefix of $s = s_1s_2 \cdots s_n$ is any substring $s_1s_2 \cdots s_k$, where $0 \leq k \leq n$. The empty string ϵ is the shortest prefix and the string itself is the longest prefix of any string. Any prefix other than the empty string and the string itself is called a *proper prefix*. As an example, 'human' is the proper prefix of the string "humanity." This is pictorially depicted in Fig. 4.1(b).

It is interesting to note that the substring 'man' is a suffix of a prefix of the string "humanity," as pictorially depicted in Fig. 4.1(c).

Factor: A string y is a factor of a string s if s can be represented as $s = xyz$, where x and y are the prefix and suffix of s . The substring 'man' is a factor of the string 'humanity'. In other words, a factor of a string is a *suffix* of a *prefix* or *prefix* of a *suffix* of a string.

4.1.2 String matching problem

String matching essentially is the technique of finding the occurrence of a particular string, called a pattern, in another string called the text. The *String matching problem* can be formulated as follows.

Let us assume that a pattern $p = p_1p_2 \cdots p_m$ of length m and a text $t = t_1t_2 \cdots t_n$ of length n are two strings formed over the same finite alphabet Σ such that $m < n$. We say that the pattern p occurs in text t at the beginning of text location k if $1 \leq k \leq n - m$ and $t_{k+i-1} = p_i$ for $1 \leq i \leq m$. The *string matching problem* is the problem of finding all the text locations where the given pattern p occurs in the given text t . The string matching problem has been depicted pictorially in Fig. 4.2.

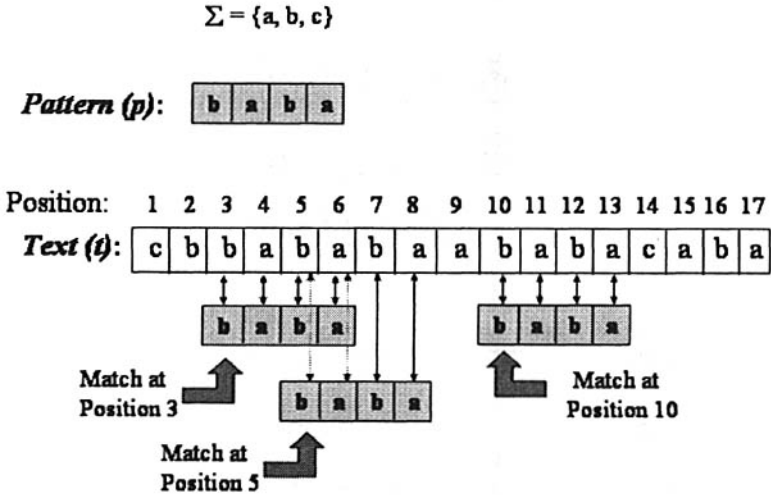


Fig. 4.2 The string matching problem.

Example 1: Let us assume a text string $t = 'cbbababababacaba'$ and a pattern $p = 'baba'$ over the finite alphabet $\Sigma = \{a, b, c\}$ as shown in Fig. 4.2. The pattern 'baba' occurs in text locations 3, 5, and 10, respectively. Note that locations 3 and 5 overlap each other. A valid string matching algorithm should be able to recognize such overlapping occurrences as well.

There are different interesting variations of the string matching problem. In many applications, the search pattern may not be a simple sequence of characters or symbols. It could be fully or partially specified. If all the symbols or characters p_i for $1 \leq i \leq m$ in the pattern $p = p_1 p_2 \cdots p_m$ are definitely known, the pattern is called a fully specified pattern. If one or more symbol p_j for any $1 \leq j \leq m$ in the pattern is not specifically known, the pattern can be called a partially specified pattern. An unspecified symbol in the pattern can be denoted by a "don't care" or a "wild card" character.

As an example, let us assume an alphabet $\Sigma = \{A, B, C, D\}$. A pattern $P = 'BCCA'$ is fully specified. However a pattern $P = BC * A$ is partially specified because the third symbol $*$ in P is not known. If $*$ can be any of the symbols from the alphabet Σ , the occurrence of pattern P could be the occurrence of any one of the possible patterns $'BCAA'$, $'BCBA'$, $'BCCA'$, and $'BCDA'$. This unspecified symbol $*$ in P is called the *fixed length don't care (FLDC)* character. However if the pattern P consists of a don't care character ϕ which can be any substring $\{\$, \$\$, \$\$, \dots\}$ of any arbitrary length, then the don't care character ϕ is called the *variable length don't care (VLDC)* character. As an example, let us assume that the partially specified pattern is $P = 'BC\phi A'$. Since ϕ can contain any pattern from the infinite set of substrings $\{\$, \$\$, \$\$, \dots\}$, the possible occurrences of the pattern in the text will be any matched substring beginning with the prefix $'BA'$ and ending with the suffix A – for example, $'BCA'$, $'BCAAAA'$, $'BCABCBDBCDDBA'$, etc. Partially specified pattern matching is useful in searching for text information when the pattern is partially known.

The pattern may also consist of a finite set of sequences instead of just a single string. Here the pattern matching problem can be extended to search for occurrence(s) of any one of the members of the set in the text, while reading the text once only. The patterns of interest may contain wild cards as explained above. They may also contain regular expressions. Use of regular expressions in patterns can be very powerful, because a set of search patterns can be expressed by using a regular expression in the form of a simple string as well as concatenations, unions, and repetitions of other subexpressions. Obviously, the algorithms to solve such problems are very complex, and they still remain a challenge in computer science.

The matching criteria can also vary by permitting slight limited difference between a pattern and its occurrences in the text. This type of pattern matching is popularly known as *approximate matching* and is particularly useful in information retrieval, text processing, and molecular and computational biology. Approximate matching is a powerful tool in automatic detection of spelling errors in texts, distance measures in DNA analysis, DNA sequence and matching, etc. The progress in simple and approximate string matching may have significant influence in text mining and Bioinformatics as well.

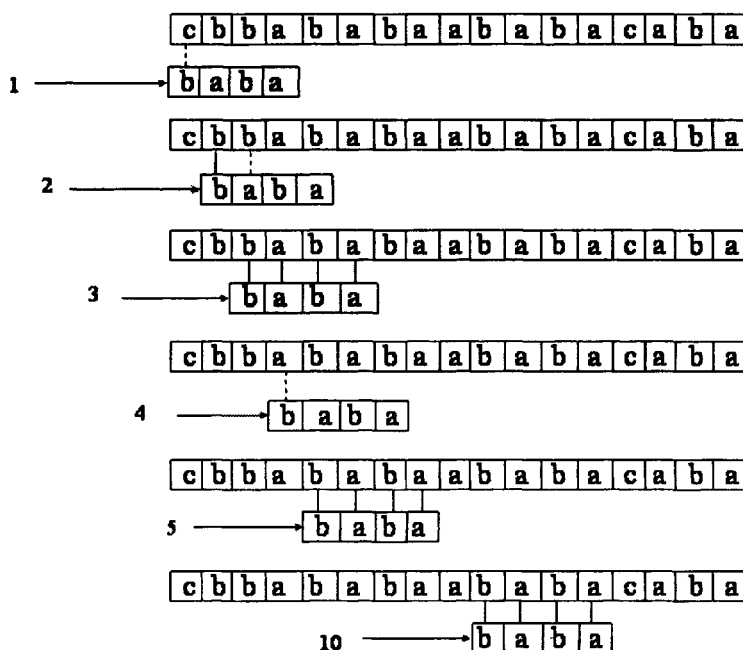


Fig. 4.3 Example of brute force approach to string matching.

4.1.3 Brute force string matching

In a brute force approach, the string matching algorithm compares a pattern character by character in each and every location of the text. Starting at the beginning of the text string, we compare the characters of the pattern one after another with the corresponding characters in the text, until a mismatch is found or the complete pattern is exhausted. If the pattern is exhausted, we claim to have found a match at the beginning of the text. If a mismatch of character is detected before the pattern is exhausted, then the pattern does not occur at the beginning of the text. We start the matching all over again at the next character in the text, and continue the same procedure.

Example 2: We illustrate how the brute force pattern matching algorithm finds occurrences of a pattern 'baba' in a text 'cbbabababababacaba' in Fig. 4.3. The algorithm starts by comparing the first character "b" of the pattern with the first character 'c' of the text string. The broken line connecting them shows that these two characters don't match. Hence the pattern gets shifted to the second character location in the text, in order to start the pattern comparison all over again from the second character location in the string. In the second step, the first character "b" of the pattern is compared

with the second character 'b' of the text. Since they are the same, this is shown by connecting them with a solid line. Now the second character of the pattern "a" is compared with the next character 'b' in the text, and they are shown to be connected by a broken line as they mismatch in this location. Hence the pattern matching starts all over again at text character location 3 now. We can see that all the four consecutive characters 'b', 'a', 'b', and 'a' in pattern p are now matched with the consecutive four character in the text starting in location 3, and they are shown to be connected by solid lines. Hereby we have found the first occurrence of the pattern in the text at location 3. The pattern is now shifted to restart the matching process from text character location 4. The first character "b" does not match with the character 'a' in location 4, and hence the pattern is shifted to location 5. Here we find that the pattern 'b a b a' matches with the consecutive four characters in the text starting in location 5 and hence determines an occurrence of the pattern in location 5 of the text. Continuing in the same manner, the other occurrences of the pattern is obtained at text character location 10, as shown in Fig. 4.3. We now formally describe the brute force algorithm for pattern matching.

BRUTE-FORCE-STRING-MATCHING (p, t)

1. Compute pattern length, $m \leftarrow |p|$;
2. Compute text length, $n \leftarrow |t|$;
3. Initialize text pointer, $s \leftarrow 1$;
4. Initialize pattern pointer, $i \leftarrow 1$;
5. if $p_i = t_{s+i}$ (i.e., i th character in pattern matches with $(s+i)$ th character in text) then increment pattern pointer, $i \leftarrow i + 1$,
else go to step 7;
6. if $(i \leq m)$ then go to step 5;
7. if $i > m$ (i.e., search is successful) then print "Pattern occurs at text position" s ;
8. Increment text pointer, $s \leftarrow s + 1$ for next search;
9. if $s \leq n - m + 1$ (i.e., the text is not exhausted) then go to step 4 to repeat above.

The above brute force approach requires the input text string to be buffered, because the text needs to be backtracked whenever there is an unsuccessful match with a symbol in the pattern. The computational complexity of the algorithm is $O(m * n)$ in the worst case. However, there are efficient algorithms for string matching, which take only a linear order of computational

complexity $O(m + n)$ in the worst case. Moreover, there is no need of buffering, because these algorithms do not backtrack in the event of occurrence of a mismatch in a pattern symbol.

4.2 LINEAR-ORDER STRING MATCHING ALGORITHMS

String matching algorithms with linear-order computational complexity are very useful in many practical text-based applications such as edit, search and retrieval of text, and development of search engine, and therein lies its possible influence in text data mining. In this section, we first discuss a practical approach of developing a linear-order string matching algorithm with finite automata, in order to avoid the problem of buffering due to backtracking in the text itself. This definition of finite automaton and its property of not backtracking when there is a mismatch has influenced the development of the Knuth–Morris–Pratt algorithm [1] in 1977 as the first linear-order algorithm with computational complexity $O(m + n)$. This was followed by flurries of activity in the computer science community to develop efficient linear-order algorithms for string matching, along with exploration of simple implementation of them. In the remaining part of this section, we describe some of the classical linear order algorithms that established the foundation of research and development in string matching. These include the Boyer–Moore [2], Boyer–Moore–Horspool [3], and Karp–Rabin [4, 5] algorithms.

4.2.1 String matching with finite automata

Finite automata has been used as a tool in string matching. For every pattern p , we can always build a *finite automaton*, which we call a *string-matching automaton* for the corresponding pattern. The string-matching automaton is built from the pattern as a preprocessing step before matching. The text is then scanned through the automaton to find occurrences of the pattern in the text. A finite automaton M can be considered as a 5-tuple $(Q, q_0, S, \Sigma, \delta)$, where

- Q is a finite set of *states* of the automaton,
- $q_0 \in Q$ is a special state called the *start state*,
- $S \subseteq Q$ is a distinguished set of states called the *stop states*,
- Σ is the finite input *alphabet*, and
- δ is a function from $Q \times \Sigma$ into Q , called the *transition function* of the automaton M .

To understand string matching with finite automata, it is not necessary for readers to have a complete understanding of the automata theory. The

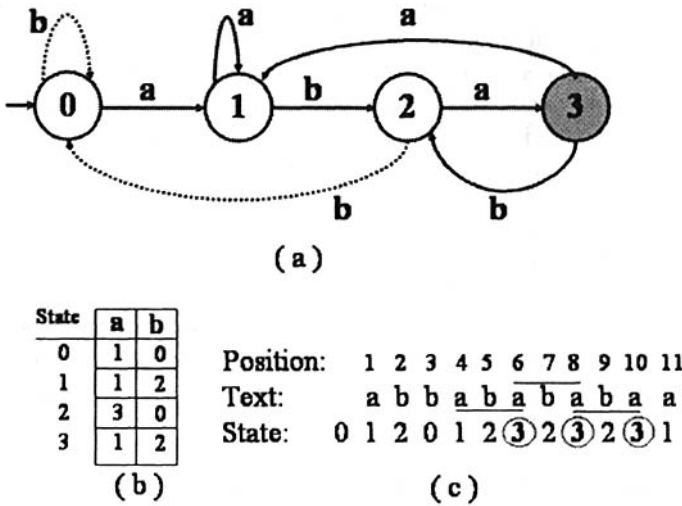


Fig. 4.4 Finite automata: (a) state diagram, (b) state transition table, and (c) pattern matching example.

automaton M always begins at the start state q_0 and reads the characters of the string sequentially one at a time. If the automaton M is in a state q_i and reads a character or symbol $\sigma \in \Sigma$, it makes a transition from state q_i to another state, say q_j , and we denote the transition as $q_j = \delta(q_i, \sigma)$. If the state $q_j \in S$, we say that the automaton M has accepted the string scanned so far. The finite automaton can be represented by a state-transition diagram as shown in Fig. 4.4(a). We explain the finite state diagram for a pattern 'a b a' with three types of nodes as follows:

- A *start* node. This represents the start state q_0 of M . In Fig. 4.4(a), state 0 is the start state.
- A *stop* node. In simple string matching there is only one stop node and the machine transits to this state when a valid occurrence of the pattern appears in the string. In Fig. 4.4(a), state 3 represents the stop state. This is specially indicated by the shaded node in Fig. 4.4(a).
- Finite number of *internal* or *read* nodes. These are the nodes representing the states of the machine other than the start or stop nodes. The machine reads only one character or symbol of the string in each of these nodes.

Example 3: We illustrate an example of string matching with finite automaton in Fig. 4.4. In this example, we assume that the pattern is $p = "aba"$ and the text to be searched is $t = 'abbabababaa'$ over the alphabet $\Sigma = \{a, b\}$. State diagram of the finite automaton is shown in Fig. 4.4(a) with four states $Q = \{0, 1, 2, 3\}$ of which $q_0 = 0$ is the *start* state and $S = 3$ is the *stop* state. The state transition function δ is explained through the tabular representation in Fig. 4.4(b). The text and the corresponding state of the automaton, after it scans each character of the text, is shown in Fig. 4.4(c). As indicated in Fig. 4.4(c), there are three occurrences of the pattern $p = "aba"$ in the text $t = 'abbabababaa'$ because the automaton goes to the stop state at character positions 6, 8, and 10 in the text. Since the pattern length is 3, the matched locations of the pattern in the text correspond to character positions 4, 6 and 8 respectively. The matched patterns in the string are marked by two underlines and one overline in the text of Fig. 4.4(c).

4.2.1.1 Computational complexity: Once the state diagram (or the state transition table) of the finite automaton of a pattern is constructed, we can scan the text to search for the pattern by comparing each text character only once, not requiring any backtrack when there is a mismatch. Hence we can find all the occurrences of the pattern in the text of length n in $O(n)$ time. This is a major improvement as compared to the naive brute force approach to pattern matching of Section 4.1.3. However, there is an overhead for preprocessing the pattern in terms of time and space complexity in order to (a) construct the state diagram or the state transition table for the pattern and (b) store the table in the memory for pattern matching. The state transition table contains m entries for a pattern of length m and for each of the symbol in the alphabet Σ . As a result, the preprocessing requires $O(m * |\Sigma|)$ time to construct the state transition table. Hence total computational complexity for string matching using the finite automaton becomes $O(n + m * |\Sigma|)$. However, m is usually much smaller compared to n . Therefore for small alphabet Σ the computational complexity, on the average, becomes linear in order.

4.2.2 Knuth–Morris–Pratt algorithm

The linear-order $O(m + n)$ algorithm proposed by Knuth, Morris, and Pratt [1] is the oldest and one of the most popular classical algorithms for string matching. The fundamental idea behind this algorithm is to avoid backtracking on the text when a mismatch occurs, by exploiting the knowledge of the matched substring in the text prior to the mismatch. During the search process, all the characters in the text are read forward sequentially one after another. Unlike constructing the state transition table by preprocessing the pattern in $O(m|\Sigma|)$ time, as in the finite automata based technique, the Knuth–Morris–Pratt algorithm first creates an auxiliary table *Next* with m entries in $O(m)$ time by analyzing the pattern p . This *Next* table is then used to shift the text forward by *Next*(i) characters in the event of a mismatch at

the i th character in the pattern in $O(n)$ time, where $Next(i)$ is the i th entry in this table. Hence the overall complexity of the algorithm is $O(m + n)$. We have explained this through a diagram in Fig. 4.5.

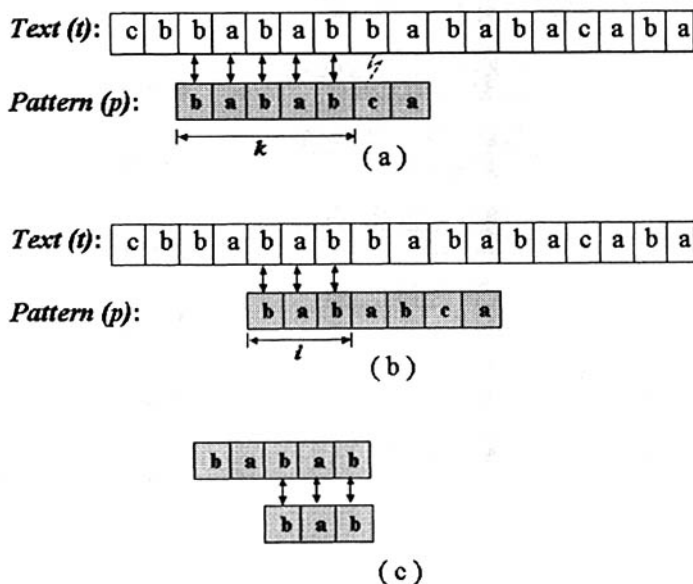


Fig. 4.5 Generation of the $Next$ table. (a) First k characters of p matches with the text, (b) p is shifted by $k - i$ positions right, because (c) first i characters of p is also suffix of the first k matched characters.

As described in Fig. 4.5(a), if there is a mismatch at the j th symbol of the text with the $(k + 1)$ th symbol (p_{k+1}) of the pattern $p = p_1p_2 \cdots p_m$, then the prefix $p_1p_2 \cdots p_k$ is the same as the substring $t_{j-k}t_{j-k+1} \cdots t_{j-1}$ in the text, which is same as the suffix of the text matched so far. Hence we can decide how much the text needs to be shifted forward in the event of a mismatch, by observing the already-matched portion of the pattern only. This shift is dependent on the structure of the pattern and the position of mismatch in the pattern. Therefore the $Next$ table is independent of the text, and it can be generated by analyzing the pattern itself before scanning the text. In order to determine the k th entry in the $Next$ table, we just need to find the longest overlap of a proper prefix of pattern p with a suffix of the already-matched portion of the pattern p as shown in Fig. 4.5(c). Precisely,

$$Next(k) = \max\{i : i < k \text{ and } p_1p_2 \cdots p_i = p_{k-i+1}p_{k-i+2} \cdots p_k\}. \quad (4.1)$$

To reiterate, the value of $Next(k)$ is the maximum $i < k$, such that the prefix $p_1p_2 \cdots p_i$ of the pattern $p = p_1p_2 \cdots p_m$ is a suffix $p_1p_2 \cdots p_{k-1}$, that is, $p_1p_2 \cdots p_i = p_{k-i+1} \cdots p_{k-1}$. The value $Next(k)$ is assigned to be 1 if such

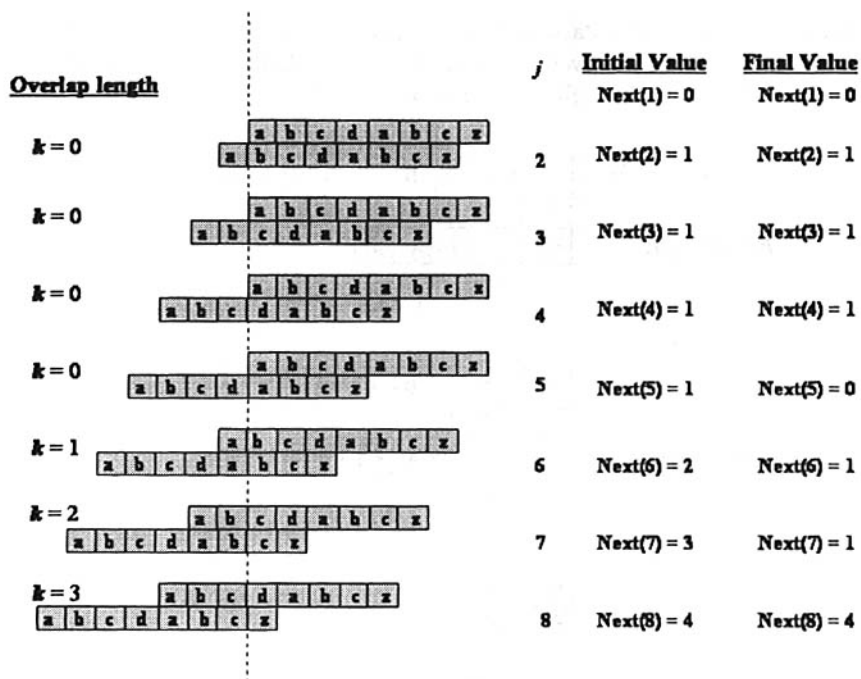


Fig. 4.6 Generation of the *Next* table for pattern $p = "abcdabcz."$

a prefix does not exist, indicating a prefix that is an empty string ϵ only. To compute $Next(k)$, the pattern $p_1p_2 \cdots p_{k-1}$ is overlapped with itself by sliding one copy of itself over another, character by character from left to right, until all the overlapping characters match or there is none left to slide. The overlapping substring then defines the desired prefix, and $Next(k)$ is the length of the overlapping prefix plus 1. We demonstrate this with an example in Fig. 4.6 to compute the *Next* table for the pattern $p = "abcdabcz."$

The *Next* value for the first character of the pattern is always 0 because there is no mismatching substring prior to the first character. In Fig. 4.6, we show the pattern overlapped with itself. The left-hand side of the vertical line shows the substring $p_1p_2 \cdots p_{j-1}$ as well as the overlapping of the longest prefix of this substring with its suffix, for the computation of $Next(j)$. For $j = 2, 3, 4$, and 5 , the overlap length is 0 because only the empty string ϵ can be the longest prefix, which is also a suffix of all the substrings 'a', 'ab', 'abc', and 'abcd', respectively, as shown in Fig. 4.6. Hence the initial values of $Next(2)$, $Next(3)$, $Next(4)$ and $Next(5)$ are 1's. For $j = 6$, 'a' is the longest prefix that is also a suffix of the substring 'abcd a'. Since the overlap length is 1, the value of $Next(6)$ becomes 2. For $j = 7$, the longest prefix of the substring 'abcd ab' is 'ab', that is also a suffix of 'abcd ab'. Now the value

of $Next(7)$ is 3, because the overlap length is 2. Similarly 'abc' is the longest prefix and also a suffix of the substring 'abcdabc' for $j = 8$, and accordingly the value of $Next(8)$ is 4.

These $Next(j)$ values can be considered during the string matching in the event of a mismatch of the j th character of a pattern with the i th character of the text, and accordingly the pattern is shifted so that the next character in the text after mismatching t_{i+1} can now be aligned with $p_{Next(j)+1}$ in the pattern to continue the forward matching without backtracking.

We may further improve the $Next$ table by considering the actual symbol causing a mismatch. Let us consider the above example with the initial $Next$ table as shown in Fig. 4.6. If there is a mismatch between a text character t_i and pattern symbol p_7 (i.e., the second 'c' in the pattern), then the next comparison is done between the same t_i and p_3 ($= p_{Next(7)}$). However, the comparison will fail again because p_3 is also the same character 'c' and will shift the comparison to p_1 as $Next(3) = 1$. Hence we can further improve the $Next$ table by taking the actual symbol causing the mismatch into consideration. The modified $Next(k)$ (final value) is expressed as

$$Next(k) = \max\{i : i < k \text{ and } p_1 p_2 \cdots p_i = p_{k-i+1} p_{k-i+2} \cdots p_k \text{ and } x_i \neq x_k\}. \quad (4.2)$$

Using this definition, the final values of the new $Next$ table are computed as shown in Fig. 4.6. For example, the initial value of $Next(5)$ is 1. However, $p_5 = p_1 = 'a'$, and hence we replace the initial value of $Next(5)$ by the value of $Next(1)$ that is 0. Similarly, initial value of $Next(6)$ is 2. Since $p_6 = p_2 = 'b'$ and $p_6 \neq p_1$ ($= p_{Next(2)}$), the final value of $Next(6)$ is 1. Iterating this procedure for all the entries in the $Next$ table, we generate the final values of $Next$ as depicted in Fig. 4.6. The formal algorithm to generate the $Next$ table is shown below.

GENERATE-NEXT-TABLE (p)

1. Initialize pattern pointer, $j \leftarrow 1$;
2. Initialize overlap length of the patterns, $k \leftarrow 0$;
3. Initialize $Next$ table, $Next(1) \leftarrow 0$ (special value for mismatch at p_1);
4. **while** ($k > 0$ and $p_j \neq p_k$) **do** $k \leftarrow Next(k)$;
5. Increment pattern pointer, $j \leftarrow j + 1$;
6. Increment overlap length, $k \leftarrow k + 1$;
7. **if** ($p_j = p_k$) **then** $Next(j) \leftarrow Next(k)$ **else** $Next(j) \leftarrow k$;
8. **if** ($j < m$) **then** go to step 4;
9. Stop.

The string matching algorithm using the above *Next* table is described below.

KNUTH-MORRIS-PRATT-STRING-MATCHING (p, t, Next)

1. Initialize the pattern index, $j \leftarrow 1$;
2. Initialize the text index, $k \leftarrow 1$;
3. Set length of the pattern, $m \leftarrow |p|$;
4. Set length of the text, $n \leftarrow |t|$;
5. **while** $j > 0$ and $p_j \neq t_i$ (i.e., there is a mismatch) **do**
 shift pattern pointer ($j \leftarrow \text{Next}(j)$);
6. Advance text pointer, $i \leftarrow i + 1$;
7. Advance pattern pointer, $j \leftarrow j + 1$;
8. **if** $j \geq m$ (i.e., match is successful) **then**
 print "pattern occurs at text index" $i - m$
 else shift pattern pointer, $j \leftarrow \text{Next}(j)$.
9. **if** $i \leq n$ and $j \leq m$ (i.e., matching is not complete) **then**
 go to step 5 to continue pattern matching
 else Stop.

Example 4: An example of pattern matching with the Knuth-Morris-Pratt algorithm is shown in Fig. 4.7. We consider finding the occurrences of a pattern $p = \text{"babab"}$ in a text $t = \text{'ababababababababababab'}$. First we compute the next table $[0, 1, 0, 1, 3]$ for the pattern $p = \text{"babab"}$ as shown in Fig. 4.7(a). Matching details are depicted in Fig. 4.7(b). The indices i and j represent, respectively, the character positions in text and the pattern being matched. The symbol " y " for $t_i = p_j$ represents a match of the text character t_i with the corresponding pattern character p_j . The symbol " Y " indicates occurrence of the pattern p ending at the text position i shown by a circle. The symbol " N " represents mismatch of t_i and corresponding p_j . Whenever the result of comparison $t_i = p_j$ is either Y or N , j is replaced by $\text{Next}(j)$. For $i = 1$ and $j = 1$ in the figure, we witness the first mismatch. This is indicated by the symbol " N " and hence the value of $j = 1$ is replaced by $j = \text{Next}(1) = 0$. Both i and j are incremented and the characters p_j and t_i are compared, until there is a mismatch or occurrence of the end of the pattern in the text. We witness the end of occurrence of the pattern at $i = 6$ and hence the text location 2 is marked by a down arrow ($\downarrow$) to indicate the beginning of the first occurrence of the pattern in the text. Similarly, the pattern occurs in the text beginning at indices 4, 6 and ending at indices 8 and 10, respectively. At $i = 12$, the text character t_{12} doesn't match with the corresponding pattern

character p_5 , and hence $j = 5$ is replaced by $Next(5) = 3$. Again t_{12} doesn't match with p_3 and hence j is replaced by $Next(3) = 0$ again. Both i and j are incremented and matching continues forward. The next pattern occurrence is obtained after we find the character match at $i = 17$, and hence the match occurs beginning at $i = 13$.

4.2.2.1 Computational complexity: The computational complexity of Knuth–Morris–Pratt algorithm is $O(n)$ in both the worst and average cases for the pattern matching phase. By analyzing the matching algorithm, it can be shown that the assignment $j \leftarrow Next(j)$ in step 5 never exceeds the total execution of the increment operation $i \leftarrow i + 1$ in step 6. The pattern is therefore shifted to the right for a total of at most n times, and hence the computation complexity of the matching phase is $O(n)$. Similarly, we can show that the processing time for initialization of the $Next$ table is of the same order $O(m)$. As a result, the worst case overall computational complexity of the algorithm is $O(m + n)$. The worst case computation time happens when a Fibonacci string pattern is matched in a text.

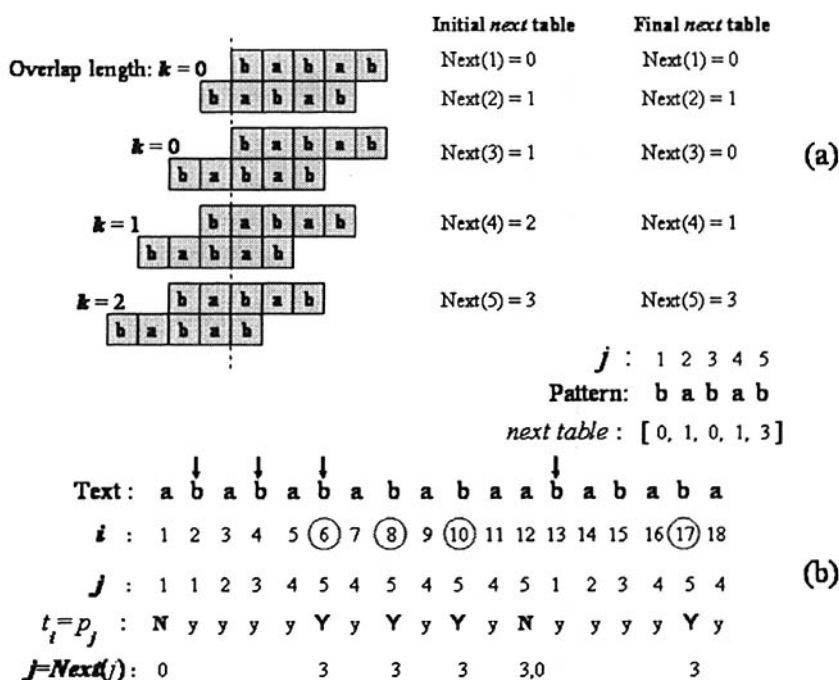


Fig. 4.7 Example of Knuth–Morris–Pratt pattern matching algorithm with pattern $p = "babab"$ and text $t = 'abababababababababab'$.

4.2.3 Boyer–Moore algorithm

Boyer and Moore proposed their algorithm for string matching [2] around the same time that Knuth, Morris, and Pratt came out with theirs, in 1977. Both the algorithms became historically famous in the research and development of string matching, mainly because of their application to text processing. Although the computational complexity of both the algorithms is on the average linear, but Boyer–Moore algorithm is likely to be more efficient than the Knuth–Morris–Pratt algorithm for a relatively longer pattern p and reasonably large alphabet Σ .

The key insight of the Boyer–Moore algorithm is that some of the characters in the text can be skipped entirely without comparing them with the pattern, because it can be shown that they can never contribute to an occurrence of the pattern in the text. In Boyer–Moore algorithm, although the text is scanned left to right, comparisons of the pattern and the text are done backwards right to left along the search window while reading the longest suffix of the search window that is also a suffix of the pattern.

The first comparison is made between the last pattern character p_m and the text character t_m , where m is the length of the pattern p . If p_m mismatches with t_m and the character t_m does not at all appear in pattern p , then it is a wastage in comparing the first $m - 1$ characters of the pattern with the first $m - 1$ characters of the text since the pattern cannot occur in any of the first m positions of the text. As a result, the pattern can be shifted safely m places to the right so that the next comparison happens between p_m and t_{2m} . Consider searching for a pattern, say “ $ababz$ ”, in a text which does not contain the character ‘ z ’ in any of its positions. The total number of comparisons in the text will then be only $\frac{n}{5}$ instead of n . This is a significant performance improvement as compared to prefix comparison-based string matching, such as Knuth–Morris–Pratt or the finite automaton-based algorithms.

In general, if p_m does not match with t_i and t_i does not appear in the pattern $p = p_1p_2 \cdots p_m$, then we simply ignore comparing all the previous $m - 1$ text characters and shift the pattern m places to the right of t_i in the text. This is illustrated with an example in Fig. 4.8(a) for a pattern “ $bcbab$ ” of length five, aligned with the text beginning at index 12. Here $p_5 = 'b'$ does not match with $t_{16} = 'd'$ and ‘ d ’ does not appear in any position of the pattern ‘ $bcbab$ ’. Hence the pattern is shifted right by five places and aligned with the text beginning at index 17, as shown in Fig. 4.8(b), such that further comparison resumes from this location.

On the other hand, if $p_m \neq t_i$ and t_i does appear in the pattern such that the rightmost appearance of t_i in pattern is p_{m-j} , then the pattern can safely be shifted by j places to the right of t_i in the text in order to align p_{m-j} with t_i . Thereafter, comparison of p_m starts again with t_{i+j} . As an example, $p_m = p_5 = 'b'$ does not match with $t_i = t_{21} = 'c'$ as shown in Fig. 4.8(b). However, ‘ c ’ appears in the pattern and its rightmost appearance is $p_{m-j} = p_2 = 'c'$. Hence the pattern ‘ $bcbab$ ’ is shifted right by $j = 3$ places in order to align

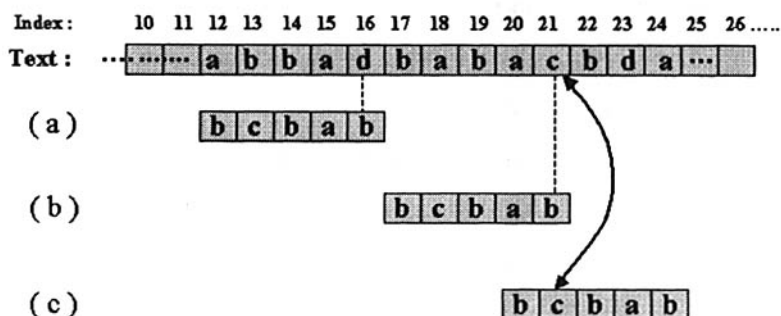


Fig. 4.8 Example of skipping character comparisons in Boyer-Moore algorithm for pattern matching. (a) Current pattern position, (b) pattern completely shifted right because 'd' does not appear in the pattern, (c) pattern is shifted by three positions to align with character 'c'.

$p_2 = 'c'$ with $t_{21} = 'c'$, as indicated by the curved arrow in Fig. 4.8(c), and further comparison of p_5 resumes with text character $t_{i+j} = t_{24}$.

If a match is found between p_m and t_i , then the preceding characters in the text from t_i are compared sequentially right to left with the corresponding positions in the pattern until there is a mismatch or the pattern is completely matched. If the pattern gets completely matched, this implies that the pattern occurs at location i . Hence the pattern is shifted by one place to the right, and the matching procedure resumes.

The number of positions to slide forward, upon mismatch, depends on the character t_i being matched with the rightmost character p_m of the pattern. These numbers can be stored in an array or table, say *skip* with $|\Sigma|$ entries in the table, where Σ is the alphabet over the text and the pattern. The entry for a symbol $\sigma \in \Sigma$ in the *skip* table is $skip(\sigma) = m - j$ when p_j is the rightmost occurrence of σ in pattern p , and $skip(\sigma) = m$ if σ does not appear in the pattern at all. Hence we can compute the *skip* table using the following algorithm.

GENERATE-SKIP-TABLE(Σ, p)

1. Set pattern length, $m \leftarrow |p|$;
2. Initialize *skip* table, $skip(\sigma) = m$ for all symbols $\sigma \in \Sigma$;
3. Initialize pattern index, $j \leftarrow 1$;
4. for j th character p_j in the pattern, set $skip(p_j) \leftarrow m - j$;
5. Increment pattern index, $j \leftarrow j + 1$;
6. if $j \leq m$ (i.e., the pattern is not complete) then go to step 4;
7. Stop.

The nature of shift of the pattern has been explained with an example in Fig. 4.9 to find the occurrences of the pattern string 'match' in the text string 'one of them matches and others mismatch from'. The procedure requires only 19 character comparisons, as opposed to 44 or more comparisons by Knuth-Morris-Pratt or the finite automaton-based string matching algorithms.

When a match is found between p_m and t_i , subsequent comparisons are made with preceding characters in the text from t_i sequentially right-to-left with the corresponding positions in the pattern. If a mismatch is found at p_j (i.e., $p_j \neq t_{i-m+j}$), then the suffix $u = p_{j+1}p_{j+2} \cdots p_m$ of length $m - j$ of the pattern is said to match with the text substring $u = t_{i-m+j+1} \cdots t_i$. If the rightmost occurrence of the mismatching character t_{i-m+j} in the pattern is p_{m-k} , then the pattern is then shifted by k positions right from the mismatching position in the text to align p_{m-k} with t_{i-m+j} and the matching procedure resumes further. However, the shift will be only one position right if $j < m - k$, in order to avoid negative shift to align p_{m-k} with t_{i-m+j} .

It is also possible that a greater shift is obtained, as compared to the above case, when a mismatch occurs after a partial match of a substring. The idea is to find a suffix $u = p_{j+1}p_{j+2} \cdots p_m$, occurring in another pattern, as a factor of p . Then the pattern can be shifted safely forward to the right, so that $u = t_{i-m+j+1} \cdots t_i$ in the text matches with the next occurrence of u in the pattern. If no such factor exists in the pattern, we cannot safely move the whole pattern right to the mismatching character. In this case, the algorithm computes the longest prefix v of p that is also a proper suffix of u . The pattern is then shifted by $m - |v|$ positions to align with v in the text. The possible shift can be precomputed from the pattern itself and stored in an array or 'shift' table.

During the search stage, the shift for mismatch at pattern location p_j and mismatching text character t_i is chosen as $\max\{skip(t_i), shift(j)\}$.

The Boyer-Moore search algorithm has worst-case computational complexity of the order $O(m * n)$. However, it is sublinear on the average case. Many

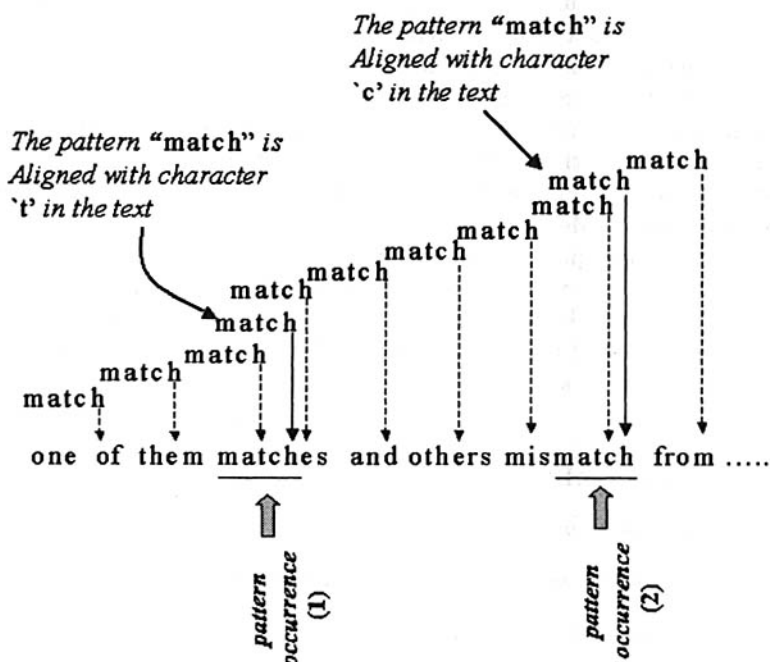


Fig. 4.9 Example of Boyer-Moore pattern matching.

variations of Boyer-Moore idea have been proposed to define worst-case algorithms of linear order [2]. Although it theoretically provides high-performance, the Boyer-Moore (as well as Knuth-Morris-Pratt) algorithm requires complicated preprocessing of the pattern before beginning the actual search of occurrences of the pattern in the string. Hence the Boyer-Moore algorithm, in spite of its promise of sublinear performance on the average, has not been used in many applications in its original form.

Horspool was the first to propose a very simplified version of the Boyer-Moore algorithm [3], by dispensing the processing and use of the *shift* array all together. It uses a variation of the original *skip* array only, and it ensures linear-order computational complexity on the average as well. This is popularly known as the *Boyer-Moore-Horspool* algorithm for string matching.

4.2.4 Boyer-Moore-Horspool algorithm

In the Boyer-Moore-Horspool algorithm [3], we compare the text character t_i with the last character p_m of the pattern. If they match, then we compare the

preceding characters of the text with corresponding characters in the pattern sequentially right to left, until we detect either an occurrence of the pattern or a mismatch on a text character. Irrespective of the match, we slide the pattern according to the next occurrence of the character t_i in the pattern. The number of positions to be moved is determined by the value of $skip(t_i)$.

Computation of the skip table in the Boyer–Moore–Horspool algorithm has a subtle difference with the original skip table definition proposed in the Boyer–Moore algorithm. If we carefully observe the *skip* table generation procedure in the Boyer–Moore algorithm, we find that the value of $skip(p_m)$ is always 0. In the Horspool version, $skip(p_m) = m$ if p_m is unique within the pattern (i.e., the character p_m does not appear in any other location in the pattern); otherwise $skip(p_m) = m - k$, where p_{m-k} is the penultimate (rightmost) appearance of the character p_m in the pattern. The preprocessing algorithm for computation of skip table in the Boyer–Moore–Horspool algorithm is as follows:

GENERATE-SKIP-TABLE(Σ, p)

1. Set pattern length, $m \leftarrow |p|$;
2. Initialize *skip* table, $skip(\sigma) = m$ for all symbols $\sigma \in \Sigma$;
3. Initialize pattern index, $j \leftarrow 1$;
4. for j th character p_j in the pattern, set $skip(p_j) \leftarrow m - j$;
5. Increment pattern index, $j \leftarrow j + 1$;
6. if $j \leq m - 1$ then go to step 4;
7. Stop.

Note that the preprocessing algorithm for computation of skip table in the Boyer–Moore–Horspool algorithm differs from the computation of the skip table in the Boyer–Moore algorithm only in step 6. Here we now have “if ($j < m$) then go to step 4.” Since step 4 is iterated here for $j = 1$ to $m - 1$, the value of $skip(p_m)$ is never zero. This value will be m if p_m is unique, while it becomes $m - j$ if p_j is the penultimate appearance of p_m in p . We show in Fig. 4.10 two examples of skip tables, generated for patterns “*abcde*” and “*aecde*”, using the Boyer–Moore (BM) and Boyer–Moore–Horspool (BMH) algorithms.

The Boyer–Moore–Horspool pattern matching algorithm is formally presented below.

BOYER–MOORE–HORSPOOL-ALGORITHM(t, p, Σ)

1. Initialize pattern length, $m \leftarrow |p|$;
2. Initialize the text length, $n \leftarrow |t|$;

3. Compute *skip* table GENERATE-SKIP-TABLE(Σ, p);
4. Initialize text pointer, $i \leftarrow 0$;
5. Initialize pattern pointer, $j \leftarrow m$;
6. **while** $j > 0$ and $t_{i+j} = p_j$ (i.e., string character matches with pattern character) **do** move pattern pointer to left, $j \leftarrow j - 1$;
7. **if** $j = 0$ (i.e., match is successful) **then**
print "*pattern occurs at text index*" $i + 1$;
8. Shift the text pointer, $i \leftarrow i + \text{skip}(t_{i+m})$;
9. **if** $i \leq n - m$ (i.e., text not yet fully traversed) **then**
go to step 5 to continue matching process.
10. stop.

Alphabet (Σ) = {a, b, c, d, e, f, g}

Pattern 1 = a b c d e

Skip Table 1:

$\sigma \in \Sigma$	BM	BMH
a	4	4
b	3	3
c	2	2
d	1	1
e	0	5
f	5	5
g	5	5

$m = 5$

Pattern 2 = a e c d e

Skip Table 2:

$\sigma \in \Sigma$	BM	BMH
a	4	4
b	5	5
c	2	2
d	1	1
e	0	3
f	5	5
g	5	5

Fig. 4.10 Example of skip tables.

Example 5: Here we demonstrate the effectiveness of the Boyer-Moore-Horspool algorithm in matching patterns from simple English text. An example is shown in Fig. 4.11. The alphabet considered in this example is $\Sigma = \{a, c, d, e, f, h, i, m, n, o, r, s, t, ' '\}$. The symbol ' ' represents the blank character. Since the pattern "match" has only five characters {a, c, h, m, t}, the rest of the characters in Σ are considered to belong to the *don't care*

category as denoted by \$ in Fig. 4.11. The broken arrows ($\downarrow$) represent the position where the text character (t_i) mismatches with the corresponding pattern character, and hence the pattern needs to be shifted right by $skip(t_i)$. The value of $skip(t_i)$ is shown as the label of each broken arrow, that is aligned with each mismatching character t_i and the last character of the pattern (i.e., $p_5 = 'h'$).

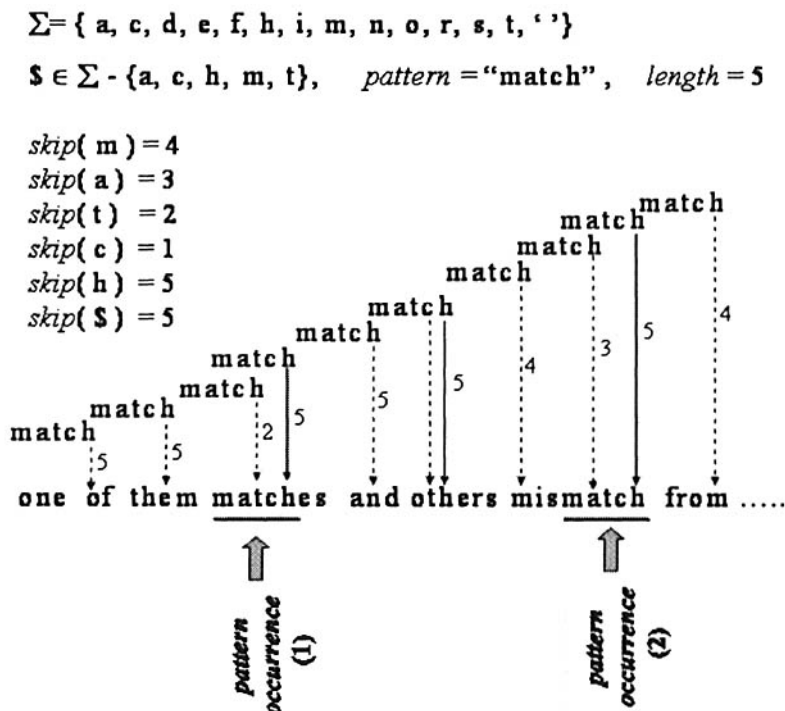


Fig. 4.11 Example of Boyer-Moore-Horspool string matching.

For example, the matching process begins by comparing $t_5 = 'o'$ with $p_5 = 'h'$. Since they mismatch, the pattern is shifted by 5 positions right because $skip(o) = 5$. Now the text character $t_{10} = 'e'$ is compared with $p_5 = 'h'$. Since they mismatch, the pattern is again shifted by 5 positions right because $skip(e) = 5$. Subsequently, $t_{15} = 't'$ mismatches with 'h' and the pattern is shifted by 2 because $skip(t) = 2$. Now the text character under consideration is $t_{17} = 'h'$, which matches with the rightmost character of the pattern. Hence all the preceding characters are compared, and we find a complete match of the pattern. Hence occurrence of the pattern beginning at text position 13 is reported. The pattern is now shifted right by $skip(h) = 5$ positions.

Continuing in this manner, we find the second occurrence of the pattern at location 35 as shown in Fig. 4.11.

It should be noted that the total number of character comparisons in this example is 19, which is the same as the number of comparisons required by the original Boyer–Moore algorithm shown in Fig. 4.9. Moreover, Boyer–Moore–Horspool algorithm is not only simpler for implementation, it also requires less preprocessing overhead and often provides better average computation performance.

4.2.5 Karp–Rabin algorithm

In Karp–Rabin algorithm [4, 5], instead of directly comparing the pattern characters with the text characters, the text is first pre-processed to map into a sequence of integers. Here each character position in the text is mapped into an integer, and this sequence of numbers is then compared with a fixed integer representing the pattern. In general, if there are d symbols in the alphabet Σ , then each symbol or character can be considered as a digit in the radix- d notation for number representation. Hence we can map the pattern $p = "p_1p_2 \cdots p_m"$ into a radix- d integer number I^p whose decimal equivalent is

$$I^p = p_1 * d^{m-1} + p_2 * d^{m-2} + \cdots + p_{m-1} * d^1 + p_m * d^0. \quad (4.3)$$

Similarly we generate a radix- d integer number I_i^t for each character location t_i in text $t = 't_1t_2t_3 \cdots t_n'$, whose decimal equivalent is

$$I - i = t_i * d^{m-1} + t_{i+1} * d^{m-2} + \cdots + t_{i+m-2} * d^1 + t_{i+m-1} * t^0. \quad (4.4)$$

Example 6: For the purpose of simple explanation, let us assume that the alphabet consists of the decimal digits $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. With this decimal notation, we can assume that a string of m consecutive symbols or characters is equivalent to an m -digits decimal number. The character string '52837031' can be considered to be the decimal number 52,837,031. We provide a simple pattern matching example, using Karp–Rabin algorithm, in Fig. 4.12.

Here we consider a pattern $p = "343"$ of length 3 and a text '1034343201' of length 10 over an alphabet $\Sigma = \{0, 1, 2, 3, 4\}$. Hence each symbol in the alphabet is a digit in the 5-radix number system notation. The decimal equivalent of the integer map of the pattern $p = "343"$ is the integer

$$I^p = 3 * 5^2 + 4 * 5^1 + 3 * 5^0 = 98.$$

The corresponding integer map, with 3-symbols length substring in first 8 character positions in the text, is $I^t = \{28, 19, 98, 119, 98, 117, 85, 51\}$ as shown in Fig. 4.12. We do not consider the integer map in the last two symbol positions because the pattern length is 3 and there will never be a match beginning either at position 9 or 10. As shown by dotted arrows, the first integer 28 is

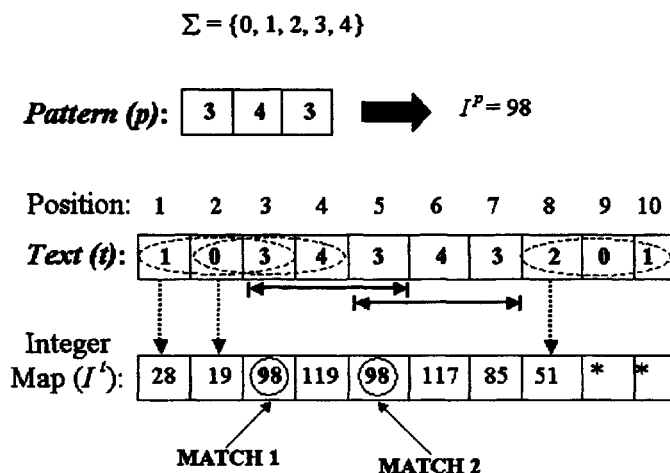


Fig. 4.12 Example of Karp-Rabin string matching.

obtained by taking the decimal equivalent of the first three consecutive symbols starting at position 1 (i.e., $1 * 5^2 + 0 * 5^1 + 3 * 5^0 = 28$). Analogously, the second integer 19 is obtained from the three consecutive symbols starting at the second position, and continuing in a similar manner the last integer 51 in position 8 is obtained by decimal number representation with the three consecutive symbols starting at position 8. Upon scanning the integer map and comparing each decimal with the integer number 98 (representing the pattern), we find two matches at positions 3 and 5, respectively. These, indeed, are the valid matches when compared with the original text string. We can compute I^P in time $O(m)$, because we can express it as

$$I^P = ((\dots(p_1 * d + p_2) * d + p_3) * d + \dots + p_{m-1}) * d + p_m. \quad (4.5)$$

This can be recursively computed in m steps, with 1 multiplication and 1 addition in each step, as follows.

1. Initialize integer map and pattern index, $I^P \leftarrow 0, i = 1$;
2. Update integer map and increment pattern index, $I^P \leftarrow I^P * d + p_i, i \leftarrow i + 1$;
3. if $i \leq m$ (i.e., pattern not completed) then go to step 2 to continue computation;

4. Return integer I^p .

Here the first integer map I_1^t for the first text character (t_1) can be computed as

$$I_1^t = ((\cdots (t_1 * d + t_2) * d + t_3) * d + \cdots + t_{m-1}) * d + t_m$$

in $O(m)$ time. The remaining integer map, for the rest of the characters $t_2, t_3, \dots, t_{n-m+1}$ in the text, can be computed in $O(n - m)$ time. This is because I_{i+1}^t can be computed incrementally from the value of I_i^t using

$$I_{i+1}^t = (I_i^t - d^{m-1} * t_i) * d + t_{i+m}. \quad (4.6)$$

It is obvious that the multiplication factor d^{m-1} can be precomputed in $O(m)$ in the worst case.

As an example, $I_5^t = 98$ represents the integer map of the substring '343' beginning at position 5 in the text as shown in Fig. 4.12. To compute the next integer map I_6^t , we need to drop the high-order digit $t_5 = 3$ from the substring and add the low-order digit $t_8 = 2$ as

$$I_6^t = (I_5^t - 5^2 * t_5) * 5 + t_8 = (98 - 25 * 3) * 5 + 2 = 117.$$

After the above preprocessing, I^t is scanned from left to right and compared with I^p to find all occurrences of pattern p in text t . Hence the Karp-Rabin algorithm can be computed in $O(m + n)$ time. However, there is a practical problem with the above simplified approach for string matching, because of the limited precision of digital computers for both processing and storage of the numbers I^p and I_i^t . If the alphabet $d = |\Sigma|$ and m are large, each necessary arithmetic computation for I^p and I_i^t with limited precision digital computer cannot be performed in constant time. This problem has been solved by adopting a hash function to compute the integers to represent the signature of a substring, in order to represent them within the permitted precision. The hash function permits generation of a signature so that I_{i+1}^t can be derived easily from I_i^t .

The hash function is carefully chosen by adopting the *modulo-q* operation, selecting q in such a manner that $d * q$ fits within the precision of a single computer word, where d is the size of the alphabet Σ . Adjusting the recurrence Eq. (4.6) to work with *modulo-q*, we get

$$I_{i+1}^t = (I_i^t - k * t_i) * d + I_{i+m}^t \pmod{q}, \quad (4.7)$$

where $k \equiv d^{m-1} \pmod{q}$. The distributive property of the *mod* function, namely,

$$(x + y) \pmod{z} = (x \pmod{z} + y \pmod{z}) \pmod{z}, \quad (4.8)$$

allows only the remainders to be stored after each stage of computation and helps to keep the results small enough to fit within the allowed precision of

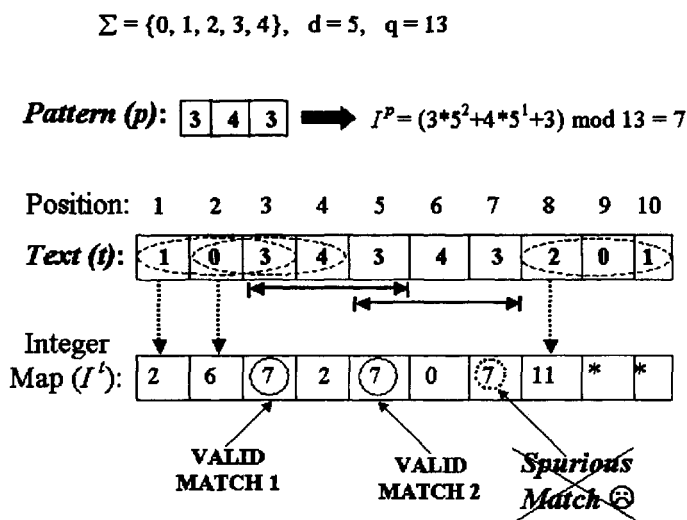


Fig. 4.13 Example of Karp-Rabin string matching algorithm with modulo operation.

a computer word. In Fig. 4.13, we demonstrate an example of the above modulo-based matching operation.

The problem with the above approach is that the *modulo* operation is a many-to-one function, and hence not unique. As a result, it can generate *spurious* matches in addition to desired valid matches in the string as shown in Fig. 4.13. For a large value of q , however, the appearance of spurious matches will be significantly smaller. Therefore when a potential match is detected, the substring is directly compared with the original pattern to check the validity of the match. The algorithm is formally explained below.

KARP-RABIN-STRING-MATCHING ALGORITHM(p, t)

1. Set pattern length, $m \leftarrow |p|$;
2. Initialize integer map for pattern, $I^p \leftarrow 0$;
3. Initialize integer map for text, $I_1^t \leftarrow 0$;
4. Set $k \leftarrow d^{m-1} \pmod{q}$;
5. **for** $j \leftarrow 1$ to m **do**
 $I^p \leftarrow (I^p \cdot d + p_j) \pmod{q}, I_1^t \leftarrow (I_1^t \cdot d + t_j) \pmod{q}$;

6. Initialize text pointer, $i \leftarrow 1$;
7. if $I_i^t \neq I^p$ (i.e., mismatch in current text position) then
go to step 9 to continue at the next text position;
8. if $t_i t_{i+1} \cdots t_{i+m-1} = p_1 p_2 \cdots p_m$ (i.e., match at text position i) then
print "Valid match at location" i ;
9. if $i \leq n - m$ (i.e., text not completely scanned) then
 $I_{i+1}^t \leftarrow (I_i^t - k * t_i) * d + I_{i+m}^t \pmod{q}$ (i.e., compute hash function at position $i + 1$);
10. Increment text pointer, $i \leftarrow i + 1$;
11. if $i \leq n - m + 1$ (i.e., text not complete) then
go to step 7 to continue;
12. Stop.

Karp and Rabin proposed an algorithm [5] in 1987 to reduce the probability of occurrence of spurious matches, by randomly selecting a prime q on the occurrence of a spurious match, reinitializing the integer map after the spurious match location, and thereafter continuing with the search.

4.3 STRING MATCHING IN BIOINFORMATICS

From information theoretic perspective, the DNA can be considered as a string or sequence of symbols, where each symbol is one of the four bases *adenine* [A], *cytosine* [C], *guanine* [G] and *thymine* [T]. Hence the alphabet in DNA string search can be assumed to be $\Sigma = \{A, C, T, G\}$. Let us consider a DNA fragment 'AGATACGATATATACGATATAGA', in which we would like to search for a string 'ATATA'. Here we show the application of Knuth–Morris–Pratt algorithm and Boyer–Moore–Horspool algorithm in matching the DNA substring 'ATATA' in the DNA fragment 'AGATACGATATATACGATATAGA'.

Example 7: We apply the Knuth–Morris–Pratt algorithm to match the DNA substring 'ATATA' in the DNA fragment 'AGATACGATATATACGATATAGA'. The *Next* table for pattern "ATATA" is the same as the *Next* table for pattern "babab" of Example 4 as derived in Fig. 4.7. The matching process to find the occurrences of DNA string 'ATATA' in DNA fragment 'AGATACGATATATACGATATAGA' is depicted in Fig. 4.14.

Example 8: Here we consider the Boyer–Moore–Horspool algorithm for the same DNA search, to detect occurrences of the DNA string 'ATATA' in

$j : 1 \ 2 \ 3 \ 4 \ 5$
Pattern: A T A T A
Next table: [0, 1, 0, 1, 3]

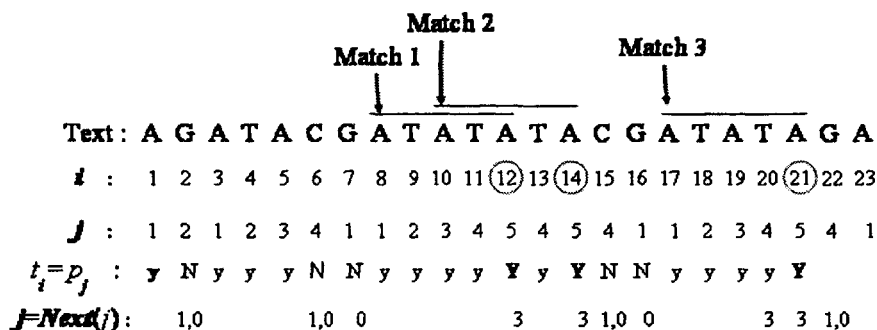


Fig. 4.14 Example of searching the DNA string 'ATATA' in the DNA fragment 'AGATACGATATATACGATATAGA' with Knuth-Morris-Pratt algorithm.

the DNA fragment 'AGATACGATATATACGATATAGA'. The matching process is illustrated in Fig. 4.15. The character $t_5 = 'A'$ is compared with the last character of the pattern $p_5 = 'A'$, and they are found to be the same. The comparison continues right to left until we detect a mismatch at $t_2 = 'G'$ with $p_2 = 'T'$. The pattern is shifted by 2 because $\text{skip}(A) = 2$. Continuing this process, we find three occurrences of the pattern "ATATA" in the DNA fragment 'AGATACGATATATACGATATAGA' as shown in Fig. 4.15. Total number of character comparisons required in this example is 23.

The principles and results of string matching have been used to solve many problems in Bioinformatics. In the following section we describe the concepts and principles behind approximate string matching algorithms and their solutions. The concept of approximate string matching is also a very powerful tool in DNA sequencing, alignment, homologue search, and many other similar problems in Bioinformatics. We described these problems and their solutions in greater detail in Chapter 10.

$$\Sigma = \{A, C, G, T\}$$

pattern = "ATATA", length = 5

skip(A) = 2

skip(C) = 5

skip(G) = 5

skip(T) = 1

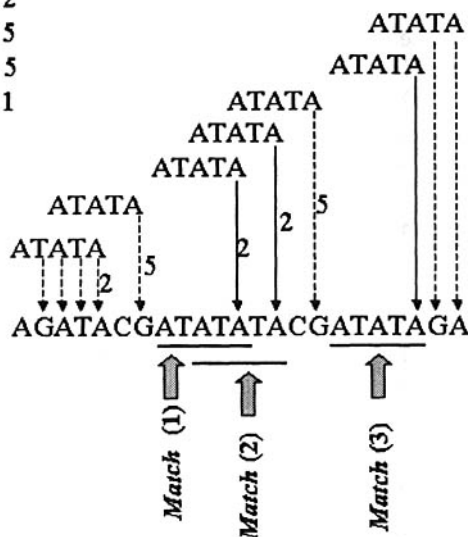


Fig. 4.15 Example of Boyer-Moore-Horspool string matching using DNA.

4.4 APPROXIMATE STRING MATCHING

In the preceding sections we considered exact matching of patterns in text. However, the string matching problem becomes challenging when the pattern is not an exact one. This may be partially specified, as discussed in Section 1.11. A generalization of the string matching problem is *Approximate String Matching*.

The approximate string matching problem deals with finding the occurrences of substrings in a text $t = t_1 t_2 \dots t_n$ which are similar to a given pattern $p = p_1 p_2 \dots p_m$. By the word similar, we mean to allow for a limited number $k > 0$ of *differences* between the pattern and its occurrence in the text. Before proceeding further, let us provide some basic definitions of difference, also termed *distance*.

4.4.1 Basic definitions

There are many definitions of “*difference*”, such as *Hamming distance*, *Levenshtein distance*, *Edit distance*, etc. There are other complex measures of ‘*difference*’, mainly in computational biology, but most of the popular and useful algorithms have been developed based on the *Levenshtein distance*.

4.4.1.1 Hamming distance: This is always measured between two strings of equal length. Hamming distance between two strings is equal to the number of symbol positions at which they differ. For example Hamming distance between strings $D = \text{'SUNDAY'}$ and $D' = \text{'MONDAY'}$ is 2 because they differ in the first two character positions only.

4.4.1.2 Levenshtein distance: A string $X = x_1x_2 \cdots x_p$ can be transformed to another string $Y = y_1y_2 \cdots y_q$ by applying one or more of the following three ‘edit operations’ in each character of the string, namely, *insertion*, *deletion*, and *substitution*. The *Levenshtein distance* $d(X, Y)$ between the strings X and Y is the minimum number of edit operations required to transform the string X into Y , or vice versa. For example, the *Levenshtein distance* between two strings $D = \text{'SATURDAY'}$ and $D' = \text{'SUNDAY'}$ is $d(D, D') = 3$, because we can delete characters ‘A’ and ‘T’ and substitute ‘R’ in ‘SATURDAY’ by ‘N’ to convert ‘SATURDAY’ to ‘SUNDAY’.

4.4.1.3 Edit distance: If the string $X = x_1x_2 \cdots x_p$ can be transformed to $Y = y_1y_2 \cdots y_q$ by applying one or more *insertion* and *deletion* operations only, then the *edit distance* between X and Y is the minimum number of *insertion* and/or *deletion* operations required to transform the string X into Y , or vice versa. For example, the *edit distance* between $D = \text{'SATURDAY'}$ and $D' = \text{'SUNDAY'}$ is $d(D, D') = 4$ because we can delete characters ‘A’, ‘T’, and ‘R’ in ‘SATURDAY’ and then insert ‘N’ to convert ‘SATURDAY’ to ‘SUNDAY’. Although the substitution operation is not directly applied to measure the *edit distance*, it can be accomplished by applying a *deletion* operation followed by an *insertion* operation in a character position.

4.4.1.4 k -Approximate string matching problem: With the above definition of *difference* or *distance function(s)*, we can formally define the approximate string matching problem as follows.

Given a pattern $p = p_1p_2 \cdots p_m$ of length m characters, text $t = t_1t_2 \cdots t_n$ of length n , where $0 < m \leq n$, a positive integer k , and a *distance function* d , find all the substrings y of text t such that

$$d(p, y) \leq k. \quad (4.9)$$

When the distance function d represents the Hamming distance, the problem is called an approximate string matching with *k-mismatches*. When d

represents the edit distance, it is called an approximate string matching with *k-differences* or *k-errors*.

4.4.2 Wagner–Fischer algorithm for computation of string distance

The popular Wagner–Fischer algorithm [6] for computation of distance between two strings is based on the dynamic-programming approach [7]. Accordingly, distances between the prefixes of the strings are successively computed from the previous values until the final result.

Let us assume that we are interested in computing the distance between two strings $p = p_1p_2 \cdots p_m$ and $t = t_1t_2 \cdots t_n$. Also, let us assume that $d_{i,j}$ represents the distance between the prefixes $p(i) = p_1p_2 \cdots p_i$ and $t(j) = t_1t_2 \cdots t_j$ of strings p and t , where i is the length of prefix $p(i)$ and j is length of prefix of $t(j)$, respectively. Hence

$$d_{i,j} = d(p(i), t(j))$$

and $d_{m,n}$ is the distance between the two strings $p = p_1p_2 \cdots p_m$ and $t = t_1t_2 \cdots t_n$. Let us assume that $w(p_i, t_j)$ represents the cost of symbol substitution from p_i to t_j if $p_i \neq t_j$, $w(p_i, \epsilon)$ is the cost of deleting symbol p_i , and $w(\epsilon, t_j)$ is the cost of inserting symbol t_j in a string. During computation of the string distance, the values of $d_{i,j}$ are recorded in a two-dimensional array $d[m+1, n+1]$. The value of $d_{i,j}$ is computed using the recurrence formula

$$d_{i,j} = \min\{d_{i-1,j-1} + w(p_i, t_j), d_{i-1,j} + w(p_i, \epsilon), d_{i,j-1} + w(\epsilon, t_j)\}. \quad (4.10)$$

The boundary conditions for this recurrence relation are as follows.

$$\begin{aligned} d_{0,0} &= 0, \\ d_{i,0} &= \sum_{k=1}^i w(p_k, \epsilon) \quad \text{for } 1 \leq i \leq m, \\ d_{0,j} &= \sum_{k=1}^j w(\epsilon, t_k) \quad \text{for } 1 \leq j \leq n. \end{aligned} \quad (4.11)$$

For Levenshtein distance, we assume unit values for each *deletion*, *insertion*, and *substitution* operations. The corresponding cost values are expressed as

$$\begin{aligned} w(p_i, \epsilon) &= 1, \\ w(\epsilon, t_j) &= 1, \\ w(p_i, t_j) &= \begin{cases} 1 & \text{if } p_i \neq t_j \\ 0 & \text{if } p_i = t_j \end{cases}. \end{aligned} \quad (4.12)$$

Hence, for Levenshtein distance, the boundary computation are done as follows.

$$d_{0,0} = 0,$$

$$\begin{aligned}
d_{i,0} &= i \quad \text{for } 1 \leq i \leq m, \\
d_{0,j} &= j \quad \text{for } 1 \leq j \leq n.
\end{aligned}
\tag{4.13}$$

Using the array d , it is possible to determine a minimal-cost trace and hence a least-cost editing sequence from string p to string t . The algorithm can be expressed as follows.

LEAST-COST-TRACE-COMPUTATION($d[m+1, n+1]$, p , t)

1. Initialize $i \leftarrow m$, for string p of length m ;
2. Initialize $j \leftarrow n$, for string t of length n ;
3. **if** ($d_{i,j} = d_{i-1,j} + w(p_i, \epsilon)$) **then**
 compute $i \leftarrow i - 1$ and go to step 7 to continue;
4. **if** ($d_{i,j} = d_{i,j-1} + w(\epsilon, t_j)$) **then**
 compute $j \leftarrow j - 1$ and go to step 7 to continue;
5. print (i, j) for symbol substitution from p_i to t_j ;
6. compute $i \leftarrow i - 1, j \leftarrow j - 1$;
7. **if** $i > 0$ and $j > 0$ (i.e., scanning not complete) **then**
 go to step 3 to continue cost computation;
8. Stop.

4.4.2.1 Example 9: Let us consider two strings $p = \text{'SUNDAY'}$ and $t = \text{'SATURDAY'}$. The length of these two strings are 6 and 8, respectively. Hence we compute the values in the array $d[7, 9]$ based on the recurrence formula for $d_{i,j}$ for $i = 0$ to 6 and $j = 0$ to 8. Entries in the two-dimensional array $d[7, 9]$ are shown in Fig. 4.16(a).

Here the value of $d_{m,n} = d_{6,8} = 3$. Hence the string distance (Levenshtein distance) between the strings 'SATURDAY' and 'SUNDAY' is 3. The minimal-cost trace and hence the least-cost editing sequence can be generated from the completed distance array. Accordingly, by applying LEAST-COST-TRACE-COMPUTATION for $d[7, 9]$, the following trace T is generated by listing the output of the algorithm in reverse order.

$$T = \{(1, 1), (2, 4), (4, 6), (5, 7), (6, 8)\}.$$

The transformation of the string 'SUNDAY' to 'SATURDAY' is depicted in Fig. 4.16(b). The characters in p (i.e., 'SUNDAY' in this example) untouched by edges are substituted (i.e., 'N' is substituted by 'R'), and this is indicated by a broken arrow. The characters in t (i.e., 'SATURDAY' in this example) untouched are inserted (i.e., 'A' and 'T').

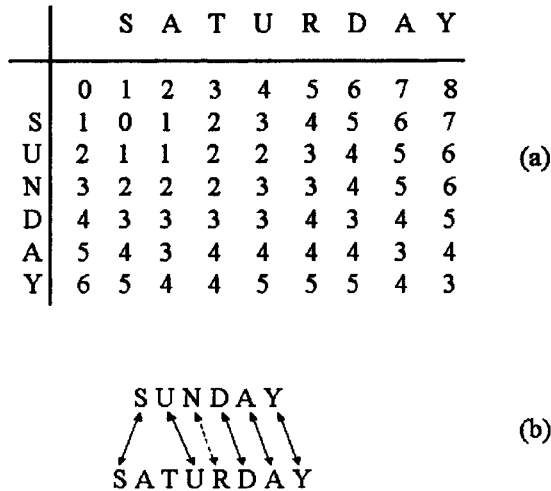


Fig. 4.16 Wagner–Fischer algorithm showing (a) matrix computation for distance function and (b) transformation, of the string ‘SUNDAY’ to ‘SATURDAY’.

4.4.2.2 Longest common subsequence problem: A *longest common subsequence* of two strings is a subsequence, common to both strings, having the maximal length. Given two strings p and t , with $|p| = m$ and $|t| = n$, where $0 < m \leq n$, the *longest common subsequence problem* is to find the longest common subsequence $lcs(p, t)$ of two strings p and t as well as its length $|lcs(p, t)|$.

After computation of the complete *distance array* and finding the least-cost trace T , it is fairly straightforward to find the $lcs(p, t)$. The constituents of the $lcs(p, t)$ are p_i , or equivalently t_j , such that $(i, j) \in T$ and $p_i = t_j$. So in above example, the longest common subsequence for strings $p = \text{‘SUNDAY’}$ and $t = \text{‘SATURDAY’}$ is

$$lcs(p, t) = p_1 p_2 p_4 p_5 p_6 = t_1 t_4 t_6 t_7 t_8 = \text{‘SUDAY’}.$$

The longest common subsequence has been used in many application areas, such as detection and correction of spelling error. It has also been used in Bioinformatics for molecular sequence matching, both for exact match and for common substrings up to k -mismatches [8].

		A	C	E	D	E	F	A	B	C	E	E
	0	1	2	3	4	5	6	7	8	9	10	11
A	0	0	0	0	0	0	0	0	0	0	0	0
B	1	0	1	1	1	1	1	0	1	1	1	1
C	2	1	1	2	2	2	2	1	0	1	2	2
D	3	2	1	2	3	3	3	2	1	0	1	2
E	4	3	2	2	2	3	4	3	2	1	1	2
E	5	4	3	2	3	2	3	4	3	2	1	1

Fig. 4.17 Example of text search with k -mismatches for $k = 2$.

4.4.3 Text search with k -differences

The dynamic programming approach for computation of *Levenshtein distance* between two strings provides the foundation of text searching with k -differences or k -errors. Here we search for occurrences of the pattern p in text t , with a maximum difference of k characters between the pattern and a text substring in text t . The only difference of this approach with Levenshtein distance computation is that we must allow for a substring occurrence to begin at any text position. This is achieved by adjusting the boundary condition in Eq. (4.13) to $d_{0,j} = 0$ for all $j = 0$ to n , because the minimum distance between the empty string ϵ and any substring of t is 0. The computation of the recurrence relation to generate all other $d_{i,j}$ are identical to that of the Levenshtein distance by Eq. (4.13). Upon completion of the computation of all the entries in the array $d[m+1, n+1]$, any value not exceeding k in the last row m indicates a position in the text where a substring having at most k -differences with the pattern ends.

4.4.3.1 Example 10: Let us consider two strings, namely, a pattern $p = "ABCDE"$ and a text $t = 'ACEDEFABCEE'$. The complete distance array corresponding to these two strings is shown in Fig. 4.17. For $k = 2$,

we find from row 5 of the array that there are five occurrences of the pattern “ABCDE” with up to 2-differences in the text, ending at text positions 3, 5, 9, 10, and 11. The corresponding matching text substrings are ‘ACE’, ‘ACEDE’, ‘ABC’, ‘ABCE’, and ‘ABCEE’, respectively.

4.5 COMPRESSED PATTERN MATCHING

One important operation that is fundamental to many database applications is to be able to search a large database to find the occurrence of pattern(s). In order to exploit the benefits of data compression, to conserve internal processor storage and computation resources, it is naturally desirable to perform pattern matching directly on compressed data without inherently decompressing it. We call this the *compressed pattern matching* problem.

Extra overhead caused by data compression is a major bottleneck in its use in many applications, where data need to be retrieved and manipulated often based on the pattern search operation. One way to correct this problem is to develop string or pattern matching techniques to operate directly on compressed data. Although effective data compression techniques have been around for almost half a century, little work has been addressed in this direction.

Let $c(t)$ denote a compressed text string corresponding to a text t . The *compressed pattern matching problem* is to find the occurrence(s) of a pattern p in t by searching directly into the compressed text $c(t)$.

The compressed pattern matching problem becomes even more challenging when the pattern is not fully specified, because of appearance of don't care character(s) in it. The problem of searching a compressed text using Huffman coding type tree-based coding or run-length encoding seems superficially straightforward. The idea is to apply any well-known string search algorithm [1, 3, 5, 9] on $c(t)$ with respect to the compressed pattern $c(p)$. A close examination of the algorithm reveals that such an approach is not very practical in many applications. If we use Huffman encoding of Section 3.6.2, an *implicit decompression* process has to be performed to determine the *character boundary* (that is the starting bit of each encoded symbol). We have demonstrated this with an example in Fig. 4.18.

Acharya [10] designed hardware algorithms to handle this problem, by generating a signal to indicate the boundary of a character in the compressed text where the linear pattern matching takes place. If a potential match is found in the compressed text, it is further checked to determine whether a character boundary signal is detected as well. If a character boundary is detected in conjunction with the occurrence of the compressed pattern, then the occurrence of the pattern is considered to be a valid match as depicted in the block diagram of Fig. 4.18. Acharya [10] also proposed how to handle the compressed pattern matching problem, in these hardware algorithms, even when the patterns are partially specified with either *fixed-length don't care characters* (FLDC)

Character	Huffman Code
a	0
b	10
c	110
d	111

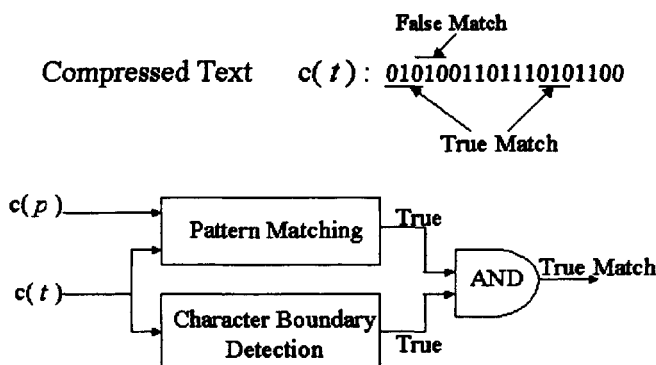
Text (t): a b b a c d a b c aPattern (p): a bCompressed Pattern $c(p)$: 010

Fig. 4.18 Example of compressed pattern matching using Huffman code.

or *variable-length don't care characters* (VLDC). The same working principle can be applied to any tree-based code such as Shannon-Fano code [11], Elias Code [12], etc.

The compressed pattern matching problem with Lempel-Ziv codes (of Section 3.12) is very difficult, due to the fact that for Lempel-Ziv codes a substring may have multiple encoding in the compressed file [10]. Some attention has recently been paid to the search of patterns in text compressed with variations of Lempel-Ziv coding. The first algorithm for pattern search in LZ78 coded files was proposed in [13]. A randomized algorithm to determine whether a pattern is present in an LZ77 compressed text was presented in Ref. [14]. However, these algorithms are very complex. There is tremendous need for further research and development in this area of compressed pattern matching, for practical applications and their usage in data mining in the near future. Acharya proposed a general formulation for compressed pattern matching in Lempel-Ziv codes in two steps. The first step is to preprocess the given pattern to be searched using the "Codebook" generated during Lempel-Ziv compression of the text. The compressed matching is done in the second step, using a graph generated in the pattern preprocessing step [10].

Although the computational complexity of these methods are polynomial in nature, still it is very high for any practical implementation. Hence de-

velopment of computationally efficient algorithms for pattern matching in compressed domain remains a challenge. Development of these algorithms especially suitable for data mining applications will be even more interesting to explore and offer promises for performance of future data mining systems.

4.6 CONCLUSIONS AND DISCUSSION

In this chapter, we presented fundamentals of string matching algorithms and their applications in different domains. We have presented several classical and pioneering algorithms in this area of study. We have also introduced the concept of the compressed pattern matching problem, in order to explore how pattern matching can be employed directly in compressed databases without involving explicit decompression. In addition to the description of the algorithms, we have presented a number of examples in each case to demonstrate clearly how they work.

The area of compressed pattern matching being still nascent, we wanted to motivate readers to pay attention in this direction in order to make significant progress in text mining and data mining applications in general. The results of string matching have paved the way for many text processing applications, including search, edit, and indexing of text databases. The search engines for the World Wide Web have been designed based on the results from string matching research. Text based search and retrieval systems are being used in text mining and Web mining applications as well. These issues are covered in further detail in Chapter 9.

The principles of string matching have been widely used in Bioinformatics for DNA sequence search, sequencing, alignment, etc. The concepts behind approximate string matching have been particularly useful for homology search and related problems in large genome databases. All these are described in greater detail in Section 10.3.4 of Chapter 10.

REFERENCES

1. D. Knuth, J. Morris, and V. Pratt, "Fast pattern matching in strings," *SIAM Journal of Computing*, vol. 6, pp. 323–350, 1977.
2. G. A. Stephen, *String Searching Algorithms*. Singapore: World Scientific, 2001.
3. R. N. Horspool, "Practical fast searching in strings," *Software-Practice and Experience*, vol. 10, pp. 501–506, 1980.

4. R. M. Karp and M. O. Rabin, "Efficient randomized pattern matching algorithms," Tech. Rep. TR-31-81, Harvard University Center for Research in Computing, 1981.
5. R. M. Karp and M. O. Rabin, "Efficient randomized pattern matching algorithms," *IBM Journal of Research and Development*, vol. 31, pp. 249–60, 1987.
6. R. A. Wagner and M. J. Fischer, "The string-to-string correction problem," *Journal of the ACM*, vol. 21, pp. 168–73, 1974.
7. R. Bellman and S. Dreyfus, *Applied Dynamic Programming*. Princeton, NJ: Princeton University Press, 1962.
8. R. Arratia, L. Gordon, and M. Waterman, "An extreme value theory for sequence matching," *The Annals of Statistics*, vol. 14, pp. 971–993, 1986.
9. R. Boyer and J. A. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, pp. 762–772, 1977.
10. T. Acharya, *VLSI Algorithms and Architectures for Data Compression*. PhD thesis, Department of Computer Science, University of Central Florida, Orlando, FL, August 1994.
11. R. M. Fano, *Transmission of Information*. Cambridge, MA: MIT Press, 1949.
12. P. Elias, "Universal codeword sets and representations of the integers," *IEEE Transactions on Information Theory*, vol. 21, pp. 194–203, 1975.
13. A. Aho, G. Benson, and M. Farach, "Let sleeping files lie: Pattern matching in z-compressed files," *Journal of Computer and Systems Sciences*, vol. 52, no. 2, 1996.
14. M. Farach and M. Thorup, "String matching in Lempel–Ziv compressed strings," *Algorithmica*, vol. 20, no. 4, 1998.

5

Classification in Data Mining

5.1 INTRODUCTION

As the quantity and variety of data available increases, there arises a commensurate need for robust, efficient, and versatile data exploration techniques that can be supervised or unsupervised. Classification, as explained in Section 1.8, is a method of categorizing or assigning class labels to a pattern set under the supervision of a *teacher*. Decision boundaries are generated to discriminate between patterns belonging to different classes. The patterns are initially partitioned into training and test sets, and the classifier is trained on the former. The test set is used to evaluate the generalization capability of the classifier. Examples of classification from diverse domains include (i) medical patients based on the disease, (ii) a set of images containing a red rose, from an image database, (iii) a set of documents describing “data mining”, from a document database, (iv) equipment malfunction based on cause, and (v) loan applicants based on their likelihood of payment. For example, in the latter case the problem is to predict a new applicant’s loan eligibility given old data about the customers (like age, salary, profession, location) and their payment patterns.

A decision tree classifier is one of the most widely used supervised learning methods used for data exploration. It is easy to interpret and can be re-represented as *If-then-else* rules. It approximates a function by piecewise constant regions and does not require any prior knowledge of the data distribution. This classifier works well on noisy data. A decision tree aids in data exploration in the following manner [1].

- It reduces a volume of data by transformation into a more compact form, that preserves the essential characteristics and provides an accurate summary.
- It discovers whether the data contains well-separated classes of objects, such that the classes can be interpreted meaningfully in the context of a substantive theory.
- It maps data in the form of a tree so that prediction values can be generated by backtracking from the leaves to its root. This may be used to predict the outcome for a new data or query.

The concept of decision trees was popularized by Quinlan with ID3 [2], which stands for *Interactive Dichotomizer 3*. Systems based on this approach use an information theoretic measure, like entropy, for assessing the discriminatory power of each attribute. The most important feature of decision trees is their capability to break down a complex decision-making process into a collection of simpler decisions, thereby providing an easily interpretable solution [3]. ID3 is a popular and efficient method of making decisions for classification of *symbolic* data and is generally not suitable in cases where numerical values are to be operated upon. Since most real-life problems deal with nonsymbolic (numeric, continuous) data, they must be discretized prior to attribute selection. Classification and Regression Trees (CART) [4] and C4.5/C5.0 [5], however, do not require such prior discretization. Here the thresholds are dynamically computed depending on the conditions along a path, and they often result in the multiple use of a particular attribute with different thresholds. This can, however, lead to an increased accuracy at the cost of reduced comprehensibility.

The major decision tree algorithms are grouped as (i) classifiers from the machine learning community: ID3, C4.5, CART; and (ii) classifiers for large databases [6]: SLIQ, SPRINT, SONAR, RainForest. Generally, a pruning phase is followed by a building phase. During the building phase the algorithm recursively splits nodes, using the best splitting attribute for that node. It is found that smaller, imperfect decision trees generally achieve better accuracy. Hence leaf nodes are recursively pruned to prevent over-fitting.

As discussed in Section 2.2.3, the advantages of artificial neural networks (ANNs) for classification include the learning of complicated, or highly nonlinear, class boundaries, fast application, and handling of a large number of features. Like decision trees, they are also nonparametric. The major disadvantages of ANNs encompass a slow training time, harder interpretation, and a difficult implementation in terms of the optimal number of nodes. Some of the popular ANN models, used for classification, include multilayer perceptron and radial basis function networks [7].

Both decision trees and ANNs are the most commonly used tools for pattern classification. Note that the decision tree approach is *monothetic*. It considers the utility of individual attributes one at a time and may miss the case

when multiple attributes are weakly predictive separately but become strongly predictive in combination. However, neural approaches are *polythetic*. Here multiple attributes can be considered simultaneously.

Probabilistic learning [8] is used to calculate explicit probabilities for the hypotheses and is among the most practical approaches to certain types of learning problems. When this is incremental, each training example can be used to incrementally increase or decrease the probability that a hypothesis is correct. Prior knowledge can also be combined with the observed data. One can use probabilistic prediction to infer multiple hypotheses, weighted by their probabilities. Even when Bayesian methods are computationally intractable, they can provide a standard of optimal decision making against which other methods can be measured.

Instance-based learners work on the basis of minimum distance from instances or prototypes [8]. Some typical models include the k -nearest neighbor classifier, radial basis function networks, and case-based reasoning. Nearest-neighbor classifiers typically define the proximity between instances, find the neighbors of a new instance, and then assign to it the label for the majority class of its neighbors. Case-based reasoning [9] is generally used when the attributes are more complicated than simple real-valued.

Support vector machines (SVMs) are a general class of learning architectures, inspired by the statistical learning theory, that performs *structural risk minimization* on a nested set structure of separating hyperplanes [10]. Given a training data, the SVM learning algorithm generates the optimal separating hyperplane in terms of generalization error. SVMs have been found to be very useful in handling data mining problems.

Section 5.2 of this chapter deals with different decision tree classifier models. Issues related to overfitting, pruning, and rule extraction are discussed in this context. Fusion of decision trees and ANNs is also presented. This is followed by Bayesian classifiers, instance-based learners, and support vector machines in Sections 5.3–5.5, respectively.

A problem with ID3 is that it cannot provide any information about the intersection region where the pattern classes are overlapping. This can be handled using fuzzy decision trees. Section 5.6 describes a method of designing fuzzy ID3 and extracting linguistic rules from this for encoding a fuzzy MLP [11, 12]. This generates a fuzzy knowledge-based network. Note that knowledge-based networks provide for initial embedding of prior knowledge about the domain. This is a desirable feature for data mining. Use of fuzzy sets enables uncertainty handling in this framework. Fuzzy decision trees provide a way of encoding a fuzzy knowledge-based network. Details on methodologies involving other soft computing tools are provided in Section 8.2.3.

The fuzzy ID3 formulates a scheme for automatic linguistic discretization of continuous attributes, based on quantiles. A novel concept of measuring the goodness of a decision tree, in terms of its compactness (size) and efficient performance, is provided. Linguistic rules are evaluated using quantitative indices. The knowledge encoding of the network incorporates the frequency

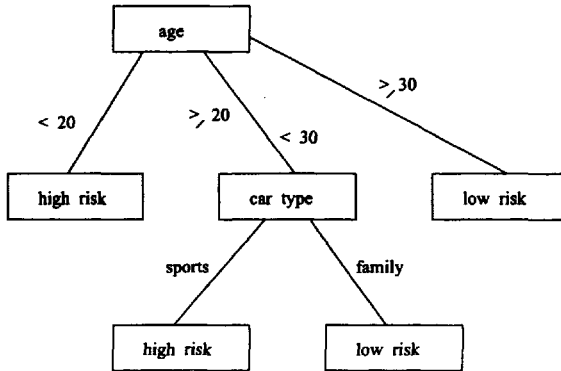


Fig. 5.1 A decision tree.

of samples and depth of the attributes in the fuzzy decision tree. Fuzziness measures, in terms of class memberships, are used at the node level of the tree to take care of overlapping classes. The effectiveness of the network, in terms of recognition scores, structure of decision tree, performance of rules, and network size, is demonstrated on real life data. The chapter is concluded in Section 5.7.

5.2 DECISION TREE CLASSIFIERS

A decision tree classifier splits a dataset on the basis of discrete decisions, using certain thresholds on the attribute values. Figure 5.1 depicts a typical decision tree demonstrating the risk factor associated with rash driving. There is a root node on top of the tree structure, indicating the feature (or attribute) that is split first for highest discrimination. The internal nodes of the tree represent simple decision rules on one or more attributes, while the leaf nodes are the predicted class labels. Tree traversal along the left branch in the figure indicates that persons with *age* < 20 involve the *high risk* category, while the right branch depicts that people with *age* > 30 are associated with *low risk* irrespective of the *car type*. The third (middle) branch, on the other hand, is traversed for $30 > \text{age} \geq 20$ and leads to a second split on attribute *car type* before arriving at a final decision.

An object X is, therefore, classified by passing it through the tree starting at the root node. The test at each internal node along the path is applied to the attributes of X , to determine the next branch along which X should go down. The label at the leaf node at which X ends up is output as its classification. An object is *misclassified* by a tree if the classification output by the tree is not the same as the object's correct class label. The proportion of objects correctly classified by a decision tree is known as its *accuracy*, whereas

the proportion of misclassified objects is the *error*. There exist several well-known tree learning algorithms in literature [1, 6]. Some of these are ID3 [2], its successor C4.5 [5], CART [4], SLIQ [13], SPRINT [14], SONAR [15], and RainForest [16].

Greedy top-down construction is the most commonly used method for tree growing. A hierarchical model is constructed top-down, starting from the entire data, partitioning it into subsets, and recursing the partitioning procedure using a splitting rule. When more than one tree can describe a dataset perfectly, one needs metrics to quantify the *goodness* of trees.

The process of tree building starts with an empty tree and the entire training set, and it broadly proceeds as follows until no more splits are possible.

1. If all the training examples at the current node t belong to category C_i , create a leaf node with the class (category) C_i .
2. Otherwise, score each one of the set of possible splits S , using a *goodness measure*.
3. Choose the best split S^* as the test at the current node.
4. Create as many child nodes as there are distinct outcomes of S^* , and partition the training data using S^* into the child nodes.
5. A child node t is said to be *pure* if all the training samples at t belong to the same class. Repeat the previous steps on all impure child nodes.

Tree construction can proceed by maximizing global *mutual information* of the whole tree, or by locally optimizing *information gain*. Sometimes distance measures like the *Gini index* of diversity [Eq. (5.2)] is also used. Both locally optimizing information gain as well as distance-based splitting criteria are found to produce small, shallow and accurate trees. In order to split at a node, the algorithm proceeds as follows:

- Intuitively pick an attribute that best separates instances of the different classes.
- Quantify the intuitive factor for measuring the separability: Define an impurity $I(S)$ of an arbitrary set S consisting of l classes. It can be
 - Information entropy, measured as

$$Entropy(S) = - \sum_{i=1}^l p_i \log p_i, \quad (5.1)$$

where p_i is the relative frequency of class i in S (*a priori* probability). This has a value of zero when all the patterns belong to only one class, and it has a value of one when all the classes are in equal number.

– Gini index, expressed as

$$Gini(S) = 1 - \sum_{i=1}^l p_i^2. \quad (5.2)$$

- Compute the information gain on partitioning S into r subsets. This is measured as the impurity of S less the sum of weighted impurity of each subset. For example,

$$Gain(S, S_1, \dots, S_r) = I(S) - \sum_{j=1}^r \frac{|S_j|}{|S|} I(S_j), \quad (5.3)$$

where $|S|$ is the cardinality of S , and $I(S)$ is the intuitive factor defined as either *Entropy*(S) or *Gini*(S) using eqns. (5.1)-(5.2).

- The attribute that provides the largest information gain is chosen to split the node. However, one needs to enumerate all possible splitting points for each attribute.

It has been observed [1] that the Gini index has difficulty when there are a relatively large number of classes. This index emphasizes equal-sized offsprings with purity of all children. Information gain, on the other hand, is biased towards attributes with a large number of possible values. They typically produce trees that are extremely deep and difficult to interpret. However, nothing definite can be said about the consistent superiority of one measure over the other. Measures like information gain and Gini index are all concave (never reporting a worse goodness value after trying a split than before splitting), so that there is no natural way of assessing where to stop further expansion of a node. Techniques like *minimum description length* (Section 5.2.5) are often used to decide which splits to prefer over others, and also for pruning.

A *regression tree* is a decision tree with continuous class labels. It approximates a function with piece-wise constant regions. When computing the split criteria for regression trees, one determines the predicted value for a set S as the average of all values in S . The error is the square root of the sum of square of difference of each member of S from the predicted average. The objective is to pick the smallest average error. The splits are made on categorical attributes.

One of the main difficulties of inducing a recursive partitioning structure is knowing when to stop, with a *right-sized* tree. For moderate-sized problems, the critical issues are generalization accuracy. For very large tree classifiers, on the other hand, the critical issue is optimizing structural properties like height and balance of the tree. The tree quality typically depends more on good stopping rules than on splitting rules [1]. Pruning is a method widely used for obtaining right-sized trees. It proceeds by building a complete tree

(in which splitting no leaf node further will improve the accuracy on the training data) and then removing subtrees that are not contributing significantly towards generalization accuracy. This method is better than stop-splitting rules by partly compensating for the suboptimality of greedy tree induction. For example, consider a very good node T_2 a few levels below a not-so-good node T_1 . Here a stop-splitting rule will stop tree growth at T_1 , whereas pruning may give a high rating for, and retain, the whole subtree at T_1 . Typically, pruning is found to be more beneficial for accuracy with increasing skewness in class distribution and/or increasing sample size.

The advantages of decision trees include reasonable training time, fast application, easy interpretation, easy implementation, and ability to handle large number of features. Since they do not make any assumptions about the underlying data distribution, they are specially suited for exploratory knowledge discovery. Their major demerits include an inability to handle complicated relationships between features, generation of simple axis-parallel decision boundaries, and their problems with lots of missing data. Sample size versus dimensionality of a dataset greatly influences the quality of trees constructed from it. The shortcomings of decision tree models, as well as solutions to alleviate them, have been extensively reported in literature [1].

In the remaining part of this section we describe some decision tree classifier models and discuss issues related to their overfitting, rule extraction, and fusion with neural networks.

5.2.1 ID3

ID3 uses an information theoretic approach. The procedure is that at any point one examines the feature that provides the greatest gain in information or, equivalently, the greatest decrease in entropy. Entropy is measured by Eq. (5.1).

The general case is that of N labeled patterns partitioned into sets belonging to classes C_i , $i = 1, 2, 3, \dots, l$. The population in class C_i is n_i . Each pattern has n input features and each feature can take on two or more values.

5.2.1.1 Algorithm The ID3 prescription for synthesizing an efficient decision tree can be stated as follows:

1. Calculate the initial value of entropy

$$Entropy = \sum_{i=1}^l -(n_i/N) \log_2(n_i/N) = \sum_{i=1}^l -p_i \log_2 p_i, \quad (5.4)$$

where N is the total number of labeled patterns.

2. Select that feature which results in the maximum decrease in entropy or gain in information, according to Eq. (5.3), to serve as the root node of the decision tree.

3. Build the next level of the decision tree providing the greatest decrease in entropy.
4. **repeat** steps 1 through 3. Continue the procedure until all subpopulations are of a single class and the system entropy is zero.

At this stage, one obtains a set of leaf nodes (subpopulation) of the decision tree, where the patterns are of a single class. Note that there can be some nodes which cannot be resolved any further.

Table 5.1 Sample dataset for ID3, along with a split on *height*

Height	Hair	Eyes	Class	Height	Hair	Eyes	Class
tall	blond	brown	C_1	tall	blond	brown	C_1
tall	dark	blue	C_1		dark	blue	C_1
tall	dark	brown	C_1		dark	brown	C_1
short	dark	blue	C_1		red	blue	C_2
short	blond	brown	C_1		blond	blue	C_2
tall	red	blue	C_2	short	dark	blue	C_1
tall	blond	blue	C_2		blond	brown	C_1
short	blond	blue	C_2		blond	blue	C_2

5.2.1.2 Example 1: Let us illustrate the tree formation with a simple example. Table 5.1 provides a sample dataset of eight patterns, containing three attributes (height, hair color, eye color) and two output classes (C_1 , C_2). The initial value of entropy is computed by Eq. (5.4) as

$$-\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8} = 0.954 \text{ bits.}$$

Splitting on the basis of attribute height leads to five samples along the *tall* branch and three along the *short* branch, as depicted in columns 5 to 8 of the table, having corresponding entropies of

$$-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.971 \text{ bits}$$

and

$$-\frac{1}{3} \log_2 \frac{1}{3} - \frac{2}{3} \log_2 \frac{2}{3} = 0.918 \text{ bits,}$$

respectively. The information gain with *height* is evaluated, using Eq. (5.3), as

$$0.954 - \left[\frac{5}{8} * (0.971) + \frac{3}{8} * (0.918) \right] = 0.954 - 0.951 = 0.003 \text{ bits.}$$

Similarly, the corresponding gains based on attributes *hair* and *eyes* are 0.454 and 0.347 bits, respectively. Maximizing, we find that the split with *hair* gives the largest information gain. Hence attribute *hair* is chosen as the root node. This procedure is continued in subsequent levels of the resultant tree for the remaining attributes. It is found at the second level that a split on attribute *eyes* yields a larger information gain and all nodes are resolved (or pure). This completes the tree building procedure.

5.2.2 IBM IntelligentMiner

Here the decision tree uses the Gini index. If a dataset S contains examples from l classes, $gini(S)$ is defined by Eq. (5.2). If a dataset S is split into two subsets S_1 and S_2 , then the Gini index $Gini(S)$ is defined as

$$Gini_{split}(S) = \frac{|S_1|}{|S|} Gini(S_1) + \frac{|S_2|}{|S|} Gini(S_2). \quad (5.5)$$

The attribute that provides the smallest $Gini_{split}(S)$ is chosen to split the node.

5.2.3 Serial PaRallelizable INduction of decision Trees (SPRINT)

SPRINT [14] is a decision-tree classifier for data mining. It is able to handle large disk-resident training sets, with no restrictions on training-set size, and is easily parallelizable. One list is maintained for each attribute in the dataset. The entries in an *attribute list* consist of the attribute value, class value, and record ID (*RID*). The algorithm uses a hash tree proportional to the training set size to store the RIDs.

5.2.3.1 Example 2: Let a sample dataset as in Table 5.2 provide risk factors (*high*, *low*) for the numeric attribute *Age* and the categorical attribute *car type*.

Table 5.2 A sample attribute list for SPRINT

<i>age</i>	<i>car type</i>	<i>risk</i>	<i>RID</i>
23	family	high	0
17	sports	high	1
43	sports	high	2
68	family	low	3
32	truck	low	4
20	family	high	5

Lists for continuous (numeric) attributes are in sorted order and may be disk-resident. Each leaf-node has its own set of attribute lists representing the training examples belonging to that leaf. The Gini index of Eq. (5.2) is used to evaluate the split points, using only the class frequencies in the process. For each attribute the algorithm evaluates splits using the attribute list, and keeps that split with the lowest Gini index.

Let us refer to the data of Table 5.2. The initial attribute lists for the root node are given in Tables 5.3 and 5.4 corresponding to attributes *age* and *car type*, respectively.

Table 5.3 Initial numeric attribute list for root node

<i>age</i>	risk	RID
17	high	1
20	high	5
23	high	0
32	low	4
43	high	2
68	low	3

Table 5.4 Initial categorical attribute list for root node

<i>car type</i>	risk	RID
family	high	0
sports	high	1
sports	high	2
family	low	3
truck	low	4
family	high	5

The split is evaluated for every value in each attribute list, for determining the optimal choice at a given tree-node. While performing the splits, the attribute lists of every node must be divided among the two children. The building phase proceeds by initializing the root node of the tree, while a node α that can be split exists. For each attribute A_j , all possible splits are evaluated on A_j . Then the best split is used to split node α .

In case of continuous attribute A_j , splits of the form $value(A_j) < v$ are considered. From Table 5.3, we find the class frequencies for the root node to be 4, 2 for risk *high*, *low*, respectively. The split $age < 20$ generates class histograms with frequencies of 1, 0 for risk *high*, *low*, respectively, in case of left child [RID = 1], and histogram frequencies 3, 2 for risk *high*, *low*,

respectively, in case of right child [RID = 5, 0, 4, 2, 3]. This results in a Gini value of 0.4. It is found that the lowest Gini value of 0.222 is obtained for the split $age < 32$. This is depicted in Table 5.5 with RID = 1, 5, 0 along the left branch (*high risk*) and RID = 4, 2, 3 along the right (*low risk*).

Since every record is vertically partitioned over all attribute lists, here each attribute list needs to be distributed across the children separately through a hash-join with that of the splitting attribute. The record identifier, which is duplicated into each attribute list, establishes the connection between the vertical parts of the record. Since during the hash-join each attribute list is read and distributed sequentially, the initial sort order of the attribute list is preserved. In the example under consideration the hash table maintains the mapping of RIDs 0, 1, 5 to the left child for the split $age < 32$. The corresponding attribute list for *car type* is given in Table 5.6.

On the other hand, in case of categorical attributes the algorithm considers splits of the form $value(A_j) \in \{v_1, \dots, v_n\}$. Here we have *car type* $\in \{family, sports, truck\}$. Considering Table 5.4, the class frequencies for risk *high*, *low*, corresponding to the root node, are found to be 2,1 [RID = 0, 5, 3]; 2,0 [RID = 1, 2]; and 0,1 [RID = 4] for *car type* v_i in *family*, *sports*, *truck*, respectively. A split on *car type* in *sports* results in class frequencies of 2, 0 [RID = 1, 2] for risk *high*, *low*, respectively, in case of left child, and 2, 2 [RID = 0, 3, 4, 5] for risk *high*, *low*, respectively, in case of right child (not in *sports*). This yields a Gini value of 0.333. The lowest Gini value of 0.267 is obtained for the split on *car type* in *truck*.

Table 5.5 Numeric attribute list for split on $age < 32$

Left child			Right child		
<i>age</i>	risk	RID	<i>age</i>	risk	RID
17	high	1	32	low	4
20	high	5	43	high	2
23	high	0	68	low	3

Table 5.6 Attribute list for *car type* with split on $age < 32$

Left child			Right child		
<i>car type</i>	risk	RID	<i>car type</i>	risk	RID
family	high	0	sports	high	2
sports	high	1	family	low	3
family	high	5	truck	low	4